



LR1120 Transceiver

User Manual

Disclaimers

Long Range-Frequency Hopping Spread Spectrum (LR-FHSS) is a high link-budget, high-performance technology combining the benefits of a modulation employing low energy per bit and advanced frequency hopping schemes to achieve improved coexistence, spectral efficiency and sensitivity. Semtech Corp. holds patents directed to aspects of the LR-FHSS technology.

Your use of LR-FHSS software made available by Semtech Corp. or its affiliates does not grant any rights to their patents for LR-FHSS technology. Rights under Semtech patents may be available via various mechanisms, including by purchasing Semtech SX1261, SX1262, SX1268, LR1110, LR1120 or LR1121 semiconductor devices, or their authorized counterparts from Semtech or its affiliates, or their respective licensees.

Semtech's products are designed to be used in connection with qualified Bluetooth® products and applications but are not certified or qualified Bluetooth® products.

Table of Contents

1. Introduction	18
1.1 Scope	18
1.2 Overview	19
2. System Processes	20
2.1 System Modes	20
2.1.1 Boot	21
2.1.2 Standby.....	21
2.1.3 Calibrations.....	22
2.1.4 Power Down	23
2.1.5 Sleep	24
2.1.6 Reset	25
2.1.7 GNSS Scanning Mode	26
2.1.8 Wi-Fi Passive Scanning Mode	26
2.1.9 DSP Mode	26
2.1.10 RX Mode	26
2.1.11 TX Mode	27
2.1.12 FS Mode.....	27
2.2 Startup Sequence	27
2.3 Firmware Upgrade	28
2.3.1 GetVersion.....	28
2.3.2 EraseFlash.....	28
2.3.3 WriteFlashEncrypted	29
2.4 Mode Transitions & Timings	29
2.5 Info Page	30
2.5.1 EraseInfoPage	30
2.5.2 WriteInfoPage.....	30
2.5.3 ReadInfoPage.....	31
3. Host-Controller Interface	32
3.1 Write Commands	32
3.2 Read Commands	32
3.3 Command Endianness	33
3.4 Status Registers	34
3.4.1 GetStatus	34
3.4.2 Stat1	34
3.4.3 Stat2	35
3.5 BUSY	36
3.6 Errors	37
3.6.1 GetErrors	37
3.6.2 ClearErrors.....	37
3.7 Memory Access	38

3.7.1 WriteRegMem32	38
3.7.2 ReadRegMem32	38
3.7.3 WriteRegMemMask32	39
3.7.4 WriteBuffer8.....	39
3.7.5 ReadBuffer8.....	40
3.7.6 ClearRxBuffer.....	40
3.7.7 GetRandomNumber	40
3.7.8 EnableSpiCrc	41
4. GPIOs	42
4.1 Interrupts	42
4.1.1 SetDioIrqParams.....	44
4.1.2 ClearIrq	44
4.2 RF Switch Control	45
4.2.1 SetDioAsRfSwitch	45
4.2.2 DriveDiosInSleepMode	45
4.3 Temperature Sensor	46
4.3.1 GetTemp.....	46
5. Power Distribution	47
5.1 Selecting DC-DC Converter or LDO Regulation	47
5.2 DC-DC Inductor Selection	47
5.3 Power Modes	48
5.3.1 SetRegMode	48
5.4 VBAT Measurement	49
5.4.1 GetVbat	49
5.5 Power-On-Reset and Brown-Out-Reset	50
5.6 Low Battery Detector	50
5.7 Over Current Protection	50
6. Clock Sources.....	51
6.1 RC Oscillators Clock References	51
6.2 High-Precision Clock References	51
6.2.1 32.768kHz Clock	51
6.2.2 32MHz Crystal	51
6.2.3 32MHz TCXO.....	52
6.3 Commands	53
6.3.1 ConfigLfClock	53
6.3.2 SetTcxoMode.....	54
7. Radio.....	55
7.1 Overview	55
7.2 Commands	56
7.2.1 SetRfFrequency	56
7.2.2 SetRx.....	56
7.2.3 SetTx	57
7.2.4 AutoTxRx.....	58
7.2.5 SetRxTxFallbackMode	59

7.2.6 SetRxDutyCycle.....	60
7.2.7 StopTimeoutOnPreamble	62
7.2.8 GetRssiInst.....	62
7.2.9 GetStats	63
7.2.10 ResetStats.....	63
7.2.11 GetRxBufferStatus	64
7.2.12 SetRxBoosted.....	64
7.2.13 SetLoRaSyncWord	65
7.2.14 GetLoRaRxHeaderInfos.....	65
7.2.15 SetRssiCalibration	66
8. Modems	69
8.1 Modem Configuration	69
8.1.1 SetPacketType	70
8.1.2 GetPacketType	70
8.2 LoRa® Modem	71
8.2.1 LoRa Modulation Principle	71
8.2.2 LoRa Packet Format.....	72
8.2.3 Channel Activity Detection (CAD).....	72
8.3 LoRa Commands	73
8.3.1 SetModulationParams.....	73
8.3.2 SetPacketParams	74
8.3.3 SetCad.....	74
8.3.4 SetCadParams.....	75
8.3.5 SetLoRaSynchTimeout.....	75
8.3.6 SetLoRaPublicNetwork	76
8.3.7 GetPacketStatus	76
8.4 (G)FSK Modem	77
8.4.1 (G)FSK Modulation Principle.....	77
8.4.2 (G)FSK Packet Engine.....	77
8.4.3 (G)FSK Packet Format.....	78
8.4.4 SX128x Compatibility.....	79
8.4.5 Special Settings for BRF= 0.6kbps and 1.2kbps	80
8.5 (G)FSK Commands	81
8.5.1 SetModulationParams.....	81
8.5.2 SetPacketParams	82
8.5.3 SetGfskSyncWord	83
8.5.4 SetPacketAdrs.....	83
8.5.5 SetGfskCrcParams	84
8.5.6 SetGfskWhitParams.....	84
8.5.7 GetPacketStatus	85
8.6 LR-FHSS Modulation	86
8.6.1 LR-FHSS Modulation Principle	86
8.7 LR-FHSS Commands	87
8.7.1 SetModulationParams.....	87

8.7.2 LrFhssBuildFrame	87
8.7.3 LrFhssSetSyncWord	89
8.7.4 Circuit Configuration for Long Range FHSS Transmission.....	89
8.8 Sigfox Modulation	90
8.8.1 SetModulationParams	90
8.8.2 SetPacketParams	90
8.9 Data Buffer	91
8.10 RSSI Functionality	92
9. Power Amplifiers	94
9.1 PA Supply Scheme	95
9.1.1 Low Power PA.....	96
9.1.2 High Power PA.....	97
9.2 PA Output Power	98
9.2.1 Low Power PA.....	98
9.2.2 High Power PA.....	99
9.2.3 High Frequency PA	100
9.3 PA Current Consumption	101
9.3.1 Low Power PA.....	101
9.3.2 High Power PA.....	102
9.3.3 High Frequency PA	104
9.4 Impedance Matching Networks	105
9.4.1 Multi-Band Operation	105
9.4.2 RF Switch Implementation	106
9.4.3 Direct-Tie Implementation	107
9.5 Commands	108
9.5.1 SetPaConfig	108
9.5.2 SetTxParams	109
10. Data Buffer	110
10.1 Data Buffer In Receive Mode	111
10.2 Data Buffer In Transmit Mode	111
10.3 Using the Data Buffer	111
11. Wi-Fi Passive Scanning	112
11.1 Principle of Operation	112
11.1.1 Repetition Schemes.....	113
11.1.2 Preamble Search Step.....	113
11.1.3 Signal Capture Step	113
11.1.4 Signal Demodulation Step	114
11.2 Wi-Fi Commands	115
11.2.1 List of Wi-Fi Commands.....	115
11.2.2 WifiScan.....	116
11.2.3 WifiScanTimeLimit.....	117
11.2.4 WifiCountryCode	118
11.2.5 WifiCountryCodeTimeLimit	119
11.2.6 WifiGetNbResults	120

11.2.7 WifiReadResults.....	121
11.2.8 WifiResetCumulTimings	122
11.2.9 WifiReadCumulTimings	122
11.2.10 WifiGetNbCountryCodeResults	123
11.2.11 WifiReadCountryCodeResults	123
11.2.12 WifiCfgTimestampAPphone	124
11.2.13 WifiReadVersion	124
11.3 Wi-Fi Results Formats	125
11.3.1 Wi-Fi Passive Scanning Result Formats	125
11.3.2 Basic MAC/Type/Channel Result Format	126
11.3.3 Full Result Format.....	127
11.3.4 Extended Complete Result Format.....	130
11.3.5 WifiCountryCode Result Format.....	132
12. GNSS Scanning.....	133
12.1 GNSS Geolocation System Overview	134
12.2 GNSS Principle of Operation	135
12.3 GNSS Commands	136
12.3.1 List of GNSS Commands	136
12.3.2 GnssSetConstellationToUse.....	138
12.3.3 GnssReadConstellationToUse.....	139
12.3.4 GnssReadSupportedConstellations	139
12.3.5 GnssSetMode	140
12.3.6 GnssAutonomous	141
12.3.7 GnssAssisted	142
12.3.8 GnssScan.....	143
12.3.9 GnssSetAssistancePosition	145
12.3.10 GnssReadAssistancePosition	145
12.3.11 GnssGetContextStatus.....	146
12.3.12 GnssReadVersion.....	147
12.3.13 GnssSetAlmanacUpdate	147
12.3.14 GnssReadAlmanacUpdate	148
12.3.15 GnssPushSolverMsg.....	148
12.3.16 GnssPushDmMsg.....	148
12.3.17 GnssConfigDelayResetAP	149
12.3.18 GnssReadLastScanModeLaunched	149
12.3.19 GnssFetchTime	150
12.3.20 GnssReadTime	151
12.3.21 GnssResetTime	151
12.3.22 GnssResetPosition	152
12.3.23 GnssReadWNRollover.....	152
12.3.24 GnssReadDemodStatus	153
12.3.25 GnssReadCumulTiming.....	155
12.3.26 GnssSetTime	157
12.3.27 GnssReadDopplerSolverRes	158

12.3.28 GnssReadDelayResetAP	159
12.3.29 GnssReadWarmStartStatus	159
12.3.30 GnssGetSvWarmStart	160
12.3.31 GnssWriteBitMaskSatActivated	161
12.4 GNSS Scan Abort Procedure	162
12.5 GNSS Scanning Results & Commands	163
12.5.1 GNSS Scan Result Message Description	163
12.5.2 GnssGetResultSize	164
12.5.3 GnssReadResults	164
12.5.4 GnssGetNbSvDetected	165
12.5.5 GnssGetSvDetected	165
12.5.6 GnssGetConsumption	166
12.5.7 GnssGetSvVisible	167
12.6 GNSS Almanac	168
12.6.1 GnssAlmanacUpdateFromSat	168
12.6.2 GnssReadAlmanacStatus	169
12.6.3 GnssConfigAlmanacUpdatePeriod	171
12.6.4 GnssReadAlmanacUpdatePeriod	172
12.6.5 GnssAlmanacFullUpdate	173
13. Ranging by Round Trip Time Of Flight (RTToF)	174
13.1 RTToF Commands API Interface	174
13.2 SetRangingAddr	174
13.3 SetRangingReqAddr	175
13.4 GetRangingResult	175
13.5 SetRangingTxRxDelay	176
13.6 SetRangingParameter	176
14. Bluetooth® Low Energy Beacons Compatibility	177
14.1 ConfigBleBeacon	177
14.2 BleBeaconSend	177
15. Cryptographic Engine	178
15.1 Description	178
15.2 Cryptographic Keys Definition	178
15.3 Commands	180
15.3.1 CEStatus	180
15.3.2 CryptoSetKey	180
15.3.3 CryptoDeriveKey	181
15.3.4 CryptoProcessJoinAccept	182
15.3.5 CryptoComputeAesCmac	183
15.3.6 CryptoVerifyAesCmac	184
15.3.7 CryptoAesEncrypt01	185
15.3.8 CryptoAesEncrypt	186
15.3.9 CryptoAesDecrypt	187
15.3.10 CryptoStoreToFlash	188
15.3.11 CryptoRestoreFromFlash	188

15.3.12 CryptoSetParam	189
15.3.13 CryptoGetParam	189
15.3.14 CryptoCheckEncryptedFirmwareImage.....	190
15.3.15 CryptoCheckEncryptedFirmwareImageResult.....	191
16. LR1120 Provisioning	192
16.1 Description	192
16.2 Provisioning Commands	192
16.2.1 GetChipEui.....	192
16.2.2 GetSemtechJoinEui.....	193
16.2.3 DeriveRootKeysAndGetPin	194
16.3 Crypto Engine Use With LoRaWAN V1.1.x	196
16.4 Crypto Engine Use With LoRaWAN V1.0.x	197
17. Test Commands.....	198
17.1 Regulatory Overview	198
17.1.1 ETSI	198
17.1.2 FCC.....	198
17.2 Commands	199
17.2.1 SetTxCw	199
17.2.2 SetTxInfinitePreamble	199
18. Known Limitations.....	200
19. List Of Commands	201
19.1 Register / Memory Access Operations	201
19.2 System Configuration / Status Operations	202
19.3 Radio Configuration / Status Operations	204
19.4 Wi-Fi Configuration / Status Operations	207
19.5 GNSS Configuration / Status Operations	208
19.6 CryptoElement Configuration / Status Operations	210
19.7 Bootloader Commands	211
20. Revision History	212

List of Figures

Figure 1-1: LR1120 Block Diagram.....	18
Figure 2-1: LR1120 Modes and Transitions	20
Figure 2-2: Bootloader	21
Figure 3-1: Write Command Timing Diagram.....	32
Figure 3-2: Read Command Timing Diagram.....	32
Figure 3-3: GetVersion Write Capture.....	33
Figure 3-4: GetVersion Read Capture.....	33
Figure 3-5: BUSY Timing Diagram	36
Figure 5-1: LR1120 POR and BRN Functions.....	50
Figure 6-1: LR1120 Thermal Insulation on PCB Top Layer.....	52
Figure 6-2: TCXO Circuit Diagram	52
Figure 7-1: LR1120 Radio Block Diagram.....	55
Figure 7-2: LR1120 Current Profile During RX Duty Cycle Operation	61
Figure 7-3: RX Duty Cycle Upon Preamble Detection.....	61
Figure 8-1: Modem Command Order	69
Figure 8-2: LoRa Signal Bandwidth	71
Figure 8-3: LoRa Packet Format	72
Figure 8-4: Fixed-Length Packet	78
Figure 8-5: Variable-Length Packet	78
Figure 8-6: LR-FHSS Spectral Plot Example.....	86
Figure 8-7: RSSI Propagation Time	92
Figure 9-1: LR1120 Power Amplifiers Block Diagram	94
Figure 9-2: PA Supply Scheme	95
Figure 9-3: Low Power PA VR_PA Voltage vs TxPower	96
Figure 9-4: High Power PA VR_PA Voltage vs. TxPower	97
Figure 9-5: Low Power PA Output Power vs. TxPower.....	98
Figure 9-6: HP PA Output Power vs. TxPower	99
Figure 9-7: High Frequency PA Output Power vs. TxPower	100
Figure 9-8: IDDTX vs TxPower, Low Power PA, DC-DC Configuration	101
Figure 9-9: IDDTX vs TxPower, Low Power PA, LDO Configuration	102
Figure 9-10: IDDTX vs TxPower, High Power PA, DC-DC Configuration.....	103
Figure 9-11: IDDTX vs TxPower, High Power PA, LDO Configuration	103
Figure 9-12: IDDTX vs TxPower, High Frequency PA, DC-DC Configuration	104
Figure 9-13: IDDTX vs TxPower, High Frequency PA, LDO Configuration.....	104
Figure 9-14: RF Switch, Double PA Operation	106
Figure 9-15: RF Switch, Single PA Operation (High Power PA Example)	106
Figure 9-16: Single Tie Implementation: Only One PA Used (High Power PA Example)	107
Figure 9-17: Single Tie Implementation: Both PAs Used (High Power PA Example)	107
Figure 10-1: Data buffer operation	110
Figure 11-1: Wi-Fi Passive Scanning Sequence.....	112
Figure 12-1: GNSS System Overview	133
Figure 12-2: GNSS Dual Constellation Timing.....	138
Figure 12-3: LR1120 Power Profile During GnssScan(. . .) Execution	155
Figure 12-4: GNSS Scan Abort Sequence.....	162
Figure 12-5: GNSS Scan Result Message Format.....	163
Figure 16-1: Key Derivation Scheme for LoRaWAN 1.1.x.....	196

Figure 16-2: Key Derivation Scheme for LoRaWAN 1.0.x	197
--	-----

List of Tables

Table 2-1: SetStandby Command	21
Table 2-2: CalibImage Command	22
Table 2-3: ISM Band Values.....	22
Table 2-4: Calibrate Command.....	23
Table 2-5: CalibParams Parameter	23
Table 2-6: SetSleep Command	24
Table 2-7: SleepConfig Parameter.....	24
Table 2-8: Sleep Mode Summary	24
Table 2-9: Reboot Command.....	25
Table 2-10: SetFsCommand.....	27
Table 2-11: GetVersion Command	28
Table 2-12: GetVersion Response	28
Table 2-13: EraseFlash Command	28
Table 2-14: WriteFlashEncrypted Command	29
Table 2-15: Mode Transitions and Timings	29
Table 2-16: EraseInfoPage Command	30
Table 2-17: WriteInfoPage Command	30
Table 2-18: ReadInfoPage Command	31
Table 2-19: ReadInfoPage Response	31
Table 3-1: GetStatus Command	34
Table 3-2: Stat1 Values	34
Table 3-3: Stat2 Values	35
Table 3-4: GetErrors Command.....	37
Table 3-5: GetErrors Response.....	37
Table 3-6: ClearErrors Command	37
Table 3-7: WriteRegMem32 Command.....	38
Table 3-8: ReadRegMem32 Command.....	38
Table 3-9: ReadRegMem32 Response.....	38
Table 3-10: WriteRegMemMask32 Command.....	39
Table 3-11: WriteBuffer8 Command	39
Table 3-12: ReadBuffer8 Command	40
Table 3-13: ReadBuffer8 Response	40
Table 3-14: ClearRxBuffer Command	40
Table 3-15: GetRandomNumber Command.....	40
Table 3-16: GetRandomNumber Response.....	40
Table 3-17: EnableSpiCrc Command	41
Table 4-1: Digital I/Os	42
Table 4-2: IrqToEnable Interruption Mapping.....	43
Table 4-3: SetDioIrqParams Command	44
Table 4-4: ClearIrq Command	44
Table 4-5: SetDioAsRfSwitch Command.....	45
Table 4-6: DriveDiosInSleepMode Command	45
Table 4-7: GetTemp Command	46
Table 4-8: GetTemp Response	46
Table 5-1: Regulation Type Versus Circuit Mode	47
Table 5-2: Recommended Inductors	47

Table 5-3: SetRegMode Command.....	48
Table 5-4: Power Regulation Options.....	48
Table 5-5: GetVbat Command.....	49
Table 5-6: GetVbat Response.....	49
Table 6-1: ConfigLFClock Command.....	53
Table 6-2: SetTcxoMode Command.....	54
Table 6-3: TCXO Supply Voltage Programming Values.....	54
Table 7-1: SetRfFrequency Command.....	56
Table 7-2: SetRx Command.....	56
Table 7-3: SetTx Command.....	57
Table 7-4: AutoTxRx Command.....	58
Table 7-5: SetRxTxFallbackMode Command.....	59
Table 7-6: SetRxDutyCycle Command.....	60
Table 7-7: StopTimeoutOnPreamble Command.....	62
Table 7-8: GetRssiInst Command.....	62
Table 7-9: GetRssiInst Response.....	62
Table 7-10: GetStats Command.....	63
Table 7-11: GetStats Response.....	63
Table 7-12: ResetStats Command.....	63
Table 7-13: GetRxBufferStatus Command.....	64
Table 7-14: GetRxBufferStatus Response.....	64
Table 7-15: SetRxBoosted Command.....	64
Table 7-16: SetLoRaSyncWord Command.....	65
Table 7-17: GetLoRaRxHeaderInfos Command.....	65
Table 7-18: GetLoRaRxHeaderInfos Response.....	65
Table 7-19: SetRssiCalibration Command.....	66
Table 7-20: Gain Tune Values.....	66
Table 7-21: Recommended Values for Reference EVK.....	66
Table 8-1: SetPacketType Command.....	70
Table 8-2: GetPacketType Command.....	70
Table 8-3: GetPacketType Response.....	70
Table 8-4: SetModulationParams Command.....	73
Table 8-5: SetPacketParams Command.....	74
Table 8-6: SetCad Command.....	74
Table 8-7: SetCadParams Command.....	75
Table 8-8: CadExitMode Parameter.....	75
Table 8-9: SetLoRaSynchTimeout Command.....	75
Table 8-10: SetLoRaPublicNetwork Command.....	76
Table 8-11: GetPacketStatus Command.....	76
Table 8-12: GetPacketStatus Response.....	76
Table 8-13: SetModulationParams Command.....	81
Table 8-14: Bandwidth Parameter.....	81
Table 8-15: SetPacketParams Command.....	82
Table 8-16: SetGfskSyncWord Command.....	83
Table 8-17: SetPacketAdrs Command.....	83
Table 8-18: SetGfskCrcParams Command.....	84
Table 8-19: SetGfskWhitParams Command.....	84
Table 8-20: GetPacketStatus Command.....	85
Table 8-21: GetPacketStatus Response.....	85
Table 8-22: SetModulationParams Command.....	87
Table 8-23: LrFhssBuildFrame Command.....	87

Table 8-24: Maximum user Payload Length (in Bytes)	88
Table 8-25: LrFhssSetSyncWord Command.....	89
Table 8-26: SetModulationParams Command	90
Table 8-27: SetPacketParams Command	90
Table 8-28: RSSI Information Origin and Meaning	92
Table 8-29: Twait & Trssi Values for GFSK Bandwidths.....	93
Table 9-1: Optimized Settings for LP PA with Same Matching Network	105
Table 9-2: Optimized Settings for HP PA with Same Matching Network	105
Table 9-3: Optimized Settings for HF PA with the Same Matching Network	105
Table 9-4: SetPaConfig Command	108
Table 9-5: PaDutyCycle Parameter Allowed Ranges	108
Table 9-6: SetTxParams Command.....	109
Table 9-7: RampTime Values	109
Table 11-1: Summary Of Available Wi-Fi Commands.....	115
Table 11-2: WifiScan Command	116
Table 11-3: WifiScanTimeLimit Command	117
Table 11-4: WifiCountryCode Command.....	118
Table 11-5: WifiCountryCode Example	118
Table 11-6: WifiCountryCodeTimeLimit Command.....	119
Table 11-7: WifiGetNbResults Command.....	120
Table 11-8: WifiGetNbResults Response.....	120
Table 11-9: WifiReadResults Command	121
Table 11-10: WifiReadResults Response	121
Table 11-11: Example to Read Basic Results of Passive Scan	121
Table 11-12: WifiResetCumulTimings Command	122
Table 11-13: Wi-Fi Cumulative Timings Description	122
Table 11-14: WifiReadCumulTimings Command.....	122
Table 11-15: WifiReadCumulTimings Response.....	122
Table 11-16: WifiGetNbCountryCodeResults Command	123
Table 11-17: WifiGetNbCountryCodeResults Response.....	123
Table 11-18: WifiReadCountryCodeResults Command.....	123
Table 11-19: WifiReadCountryCodeResults Response	123
Table 11-20: WifiCfgTimestampAPphone Command	124
Table 11-21: WifiReadVersion Command	124
Table 11-22: WifiReadVersion Response	124
Table 11-23: Wi-Fi Result Formats and Wi-Fi Scan Mode Relationship	125
Table 11-24: Basic Results Format per MAC Address.....	126
Table 11-25: Basic Complete Results Format per MAC Address	127
Table 11-26: Wi-Fi DatarateID Field	128
Table 11-27: Wi-Fi ChannelID Field	128
Table 11-28: Wi-Fi MacOrigin Field	128
Table 11-30: FrameCtl SubType Values.....	129
Table 11-29: Wi-Fi FrameCtl Field	129
Table 11-31: Extended Basic Complete Results Format per MAC Address	130
Table 11-32: WifiCountryCode Result Format Sent Over SPI (12 bytes)	132
Table 12-1: Summary of Available GNSS Commands	136
Table 12-2: GnssSetConstellationToUse Command	138
Table 12-3: GnssReadConstellationToUse Command	139
Table 12-4: GnssReadConstellationToUse Response	139
Table 12-5: GnssReadSupportedConstellations Command.....	139
Table 12-6: GnssReadSupportedConstellations Response.....	139

Table 12-7: GnssSetMode Command	140
Table 12-8: GnssAutonomous Command	141
Table 12-9: GnssAssisted Command	142
Table 12-10: GnssScan Command	143
Table 12-11: Effort Mode Summary	143
Table 12-12: ResultMask Field Definition	144
Table 12-13: Number of Doppler in NAV Message Configuration	144
Table 12-14: GnssSetAssistancePosition Command	145
Table 12-15: GnssReadAssistancePosition Command	145
Table 12-16: GnssReadAssistancePosition Response	145
Table 12-17: GnssGetContextStatus Command	146
Table 12-18: GnssGetContextStatus Response	146
Table 12-19: GnssReadVersion Command	147
Table 12-20: GnssReadVersion Response	147
Table 12-21: GnssSetAlmanacUpdate Command	147
Table 12-22: GnssReadAlmanacUpdate Command	148
Table 12-23: GnssReadAlmanacUpdate Response	148
Table 12-24: GnssPushSolverMsg Command	148
Table 12-25: GnssPushDmMsg Command	148
Table 12-26: GnssConfigDelayResetAP Command	149
Table 12-27: GnssReadLastScanModeLaunched Command	149
Table 12-28: GnssReadLastScanModeLaunched Response	149
Table 12-29: GnssFetchTime Command	150
Table 12-30: GnssReadTime Command	151
Table 12-31: GnssReadTime Response	151
Table 12-32: GnssResetTime Command	151
Table 12-33: GnssResetPosition Command	152
Table 12-34: GnssReadWNRollover Command	152
Table 12-35: GnssReadWNRollover Response	152
Table 12-36: GnssReadDemodStatus Command	153
Table 12-37: GnssReadDemodStatus Response	153
Table 12-38: GnssReadCumulTiming Command	155
Table 12-39: GnssReadCumulTiming Response	155
Table 12-40: GnssSetTime Command	157
Table 12-41: GnssReadDopplerSolverRes Command	158
Table 12-42: GnssReadDopplerSolverRes Response	158
Table 12-43: GnssReadDelayResetAP Command	159
Table 12-44: GnssReadDelayResetAP Response	159
Table 12-45: GnssReadWarmStartStatus Command	159
Table 12-46: GnssReadWarmStartStatus Response	159
Table 12-47: GnssGetSvWarmStart Command	160
Table 12-48: GnssGetSvWarmStart Response	160
Table 12-49: GnssWriteBitMaskSatActivated Command For GPS	161
Table 12-50: GnssWriteBitMaskSatActivated Command For Beidou	161
Table 12-51: GnssGetResultSize Command	164
Table 12-52: GnssGetResultSize Response	164
Table 12-53: GnssReadResults Command	164
Table 12-54: GnssReadResults Response	164
Table 12-55: GnssGetNbSvDetected Command	165
Table 12-56: GnssGetNbSvDetected Response	165
Table 12-57: GnssGetSvDetected Command	165

Table 12-58: GnssGetSvDetected Response	165
Table 12-59: GnssGetConsumption Command	166
Table 12-60: GnssGetConsumption Response	166
Table 12-61: GnssGetSvVisible Command	167
Table 12-62: GnssGetSvVisible Response	167
Table 12-63: GnssAlmanacUpdateFromSat Command	168
Table 12-64: GnssReadAlmanacStatus Command	169
Table 12-65: GnssReadAlmanacStatus Response	169
Table 12-66: GPS Bit Mapping	170
Table 12-67: Beidou Bit Mapping (0)	170
Table 12-68: Beidou Bit Mapping (1)	170
Table 12-69: GnssConfigAlmanacUpdatePeriod Command	171
Table 12-70: GnssReadAlmanacUpdatePeriod Command	172
Table 12-71: GnssReadAlmanacUpdatePeriod Response	172
Table 12-72: GnssAlmanacFullUpdate Command	173
Table 12-73: AlmanacFullUpdatePayload Parameter Format	173
Table 12-74: AlmanacHeader Parameter Format	173
Table 12-75: SVn Almanac Parameter Format	173
Table 13-1: Ranging Commands	174
Table 13-2: SetRangingAddr Command	174
Table 13-3: SetRangingReqAddr Command	175
Table 13-4: GetRangingResult Request	175
Table 13-5: GetRangingResult Response	175
Table 13-6: SetRangingTxRxDelay Command	176
Table 13-7: Typical Delay Values for Semtech EVKs	176
Table 13-8: SetRangingParameter Command	176
Table 14-1: ConfigBleBeacon Command	177
Table 14-2: BleBeaconSend Command	177
Table 15-1: Cryptographic Keys Usage and Derivation	178
Table 15-2: CryptoSetKey Command	180
Table 15-3: CryptoSetKey Response	180
Table 15-4: CryptoDeriveKey Command	181
Table 15-5: CryptoDeriveKey Response	181
Table 15-6: CryptoProcessJoinAccept Command	182
Table 15-7: CryptoProcessJoinAccept Response	182
Table 15-8: CryptoComputeAesCmac Command	183
Table 15-9: CryptoComputeAesCmac Response	183
Table 15-10: CryptoComputeAesCmac Command Example	183
Table 15-11: CryptoComputeAesCmac Response Example	183
Table 15-12: CryptoVerifyAesCmac Command	184
Table 15-13: CryptoVerifyAesCmac Response	184
Table 15-14: CryptoAesEncrypt01 Command	185
Table 15-15: CryptoAesEncrypt01 Response	185
Table 15-16: CryptoAesEncrypt Command	186
Table 15-17: CryptoAesEncrypt Response	186
Table 15-18: CryptoAesDecrypt Command	187
Table 15-19: CryptoAesDecrypt Response	187
Table 15-20: CryptoStoreToFlash Command	188
Table 15-21: CryptoAesDecrypt Response	188
Table 15-22: CryptoRestoreFromFlash Command	188
Table 15-23: CryptoRestoreFromFlash Response	188

Table 15-24: CryptoSetParam Command	189
Table 15-25: CryptoSetParam Response	189
Table 15-26: CryptoGetParam Command	189
Table 15-27: CryptoGetParam Response	189
Table 15-28: CryptoCheckEncryptedFirmwareImage Command	190
Table 15-29: CryptoCheckEncryptedFirmwareImageResult Command	191
Table 15-30: CryptoCheckEncryptedFirmwareImageResult Response	191
Table 16-1: GetChipEui Command	192
Table 16-2: GetChipEui Response	192
Table 16-3: GetSemtechJoinEui Command	193
Table 16-4: GetSemtechJoinEui Response	193
Table 16-5: DeriveRootKeysAndGetPin Command (Standard)	194
Table 16-6: DeriveRootKeysAndGetPin Response (Standard).....	194
Table 16-7: DeriveRootKeysAndGetPin Command (Advanced)	195
Table 16-8: DeriveRootKeysAndGetPin Response (Advanced)	195
Table 16-9: LoRaWAN 1.0.x vs. 1.1.x Security Correspondence Table	197
Table 17-1: ETSI Test Signals	198
Table 17-2: SetTxCw Command.....	199
Table 17-3: SetTxInfinitePreamble Command	199
Table 19-1: Register / Memory Access Operations	201
Table 19-2: System Configuration / Status Operations	202
Table 19-3: Radio Configuration / Status Operations	204
Table 19-4: Wi-Fi Scanning Configuration / Status Operations.....	207
Table 19-5: GNSS Scanning Configuration / Status Operations.....	208
Table 19-6: CryptoElement Configuration / Status Operations	210
Table 19-7: Bootloader Commands.....	211
Table 20-1: Revision History.....	212

1. Introduction

1.1 Scope

This document provides complete information on how to use the LR1120 transceiver in an application. It covers both hardware and software aspects. For LR1120 functionalities and circuit specifications, refer to the LR1120 Datasheet.

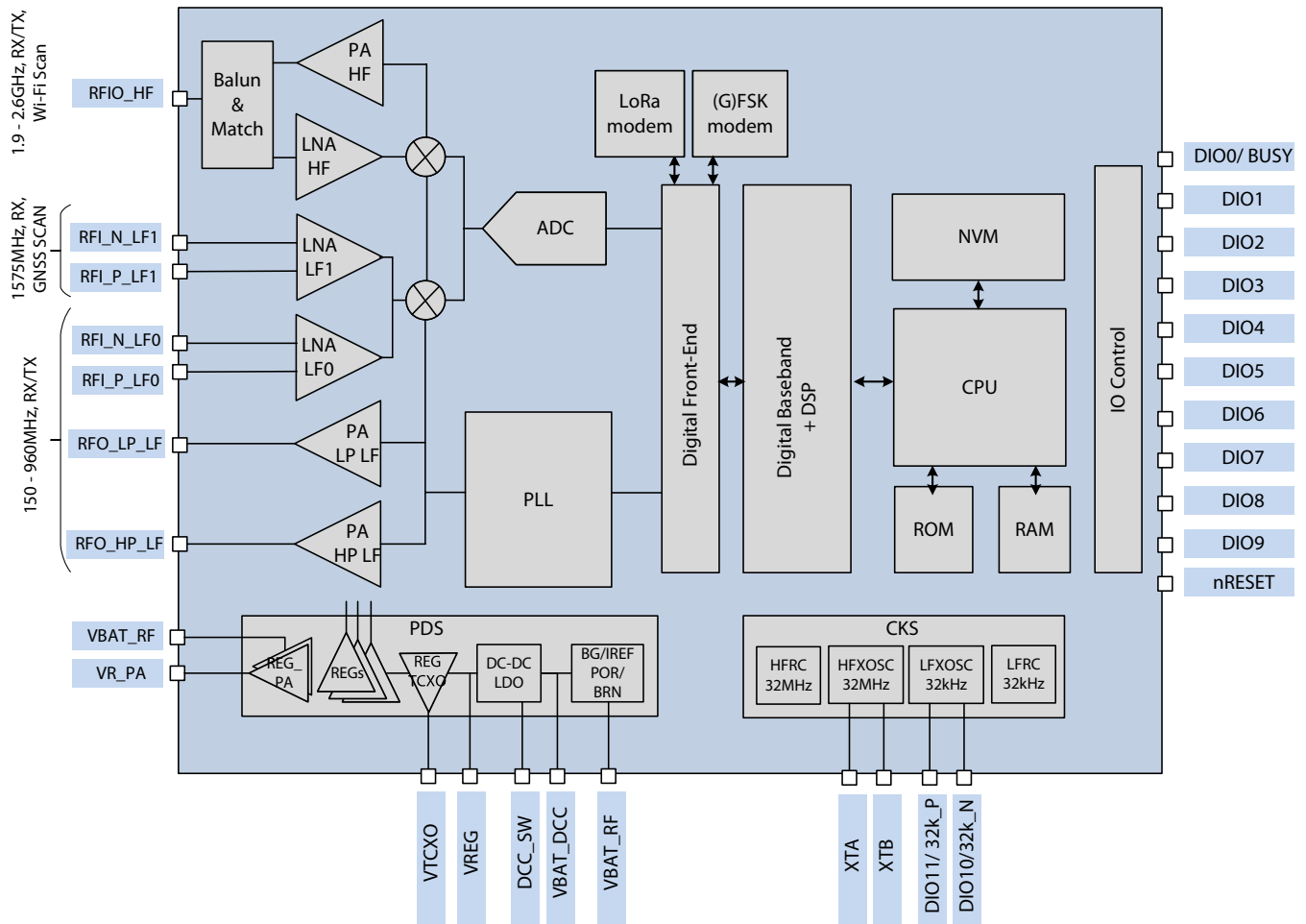


Figure 1-1: LR1120 Block Diagram

1.2 Overview

The LR1120 chip is a long range, ultra-low power transceiver that enhances LoRa®-enabled geolocation applications. For LPWAN use cases, the LR1120 supports LoRa and (G)FSK modulation on both sub-GHz and 2.4GHz bands, as well as Sigfox® modulation on sub-GHz bands, and Long Range Frequency Hopping Spread Spectrum (LR-FHSS) on sub-GHz, 1.9-2.1GHz Satellite, and 2.4GHz ISM bands. The device is highly configurable over the 150-960MHz, 1.9-2.6GHz Satellite band and 2.4GHz ISM bands to meet different application requirements utilizing the global LoRaWAN® Standard or proprietary protocols.

Besides the world-wide sub-GHz, 1.9-2.6GHz Satellite band and 2.4GHz transceiver capabilities, the LR1120 features a very low power multi-band front-end that can acquire several signals of opportunity for geolocation purposes (802.11b/g/n Wi-Fi® AP MAC addresses, GNSS (GPS, BeiDou) satellite signals). The acquired information is transmitted using an LPWAN network to a geolocation server, which computes the position of the object.

The LR1120 is optimized for low power and long battery life applications requiring indoor and outdoor geolocation. Its efficient Wi-Fi and GNSS geolocation capabilities, coupled with highly optimized detection algorithms, allow to achieve a geolocation at a fraction of the power needed by existing solutions on the market.

The LR1120 geolocation capability is further increased by Bluetooth® Low Energy beaconing compatibility to allow indoor and outdoor geolocation use cases, as well as a powerful Ranging by Round Trip Time Of Flight (RTTOf) engine for localization and recovery of lost assets in logistics applications.

2. System Processes

2.1 System Modes

The LR1120 operating modes are shown in [Figure 2-1: LR1120 Modes and Transitions](#):

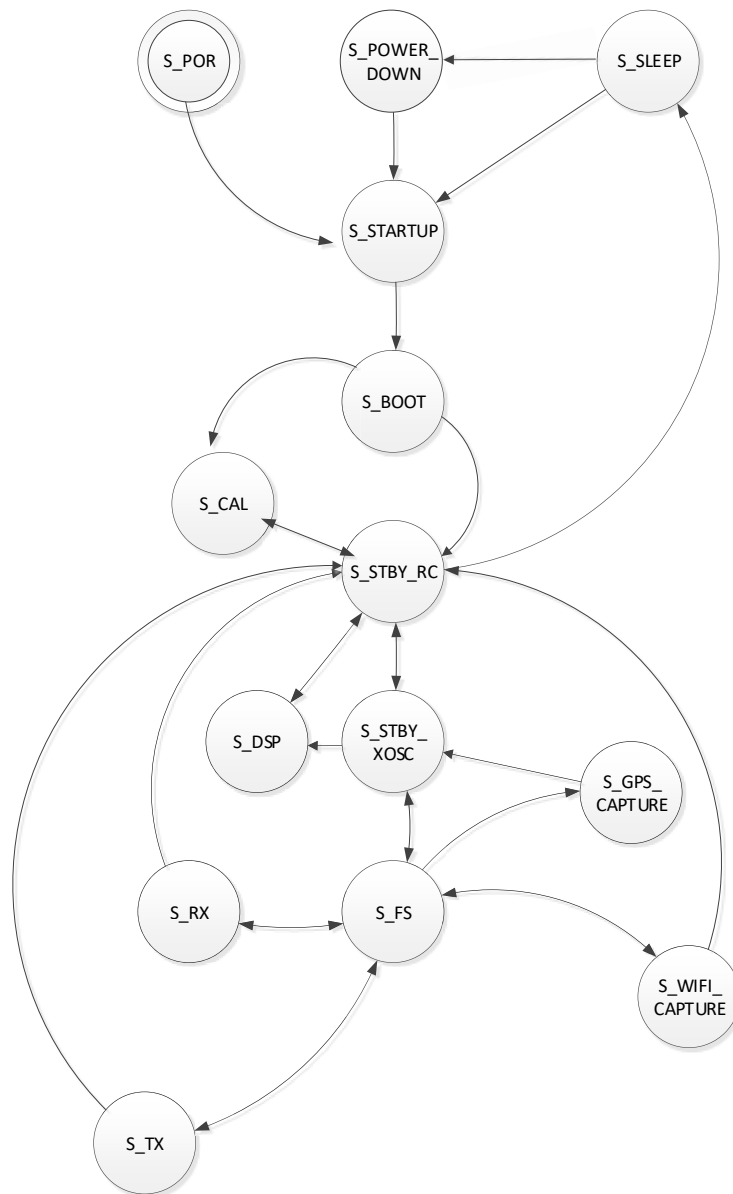


Figure 2-1: LR1120 Modes and Transitions

2.1.1 Boot

The bootloader is the first piece of software executed after power-on reset, and is located at the beginning of flash memory. The detailed description in AN1200.57 LR1110 Program Memory Update applies to all LR11xx devices.

Two main tasks are assigned to the bootloader:

- The first task checks if a valid firmware is loaded before jumping there.
- The second task loads a firmware: it receives encrypted chunks of data sent through the SPI and performs an on-the-fly decryption before writing data in flash.

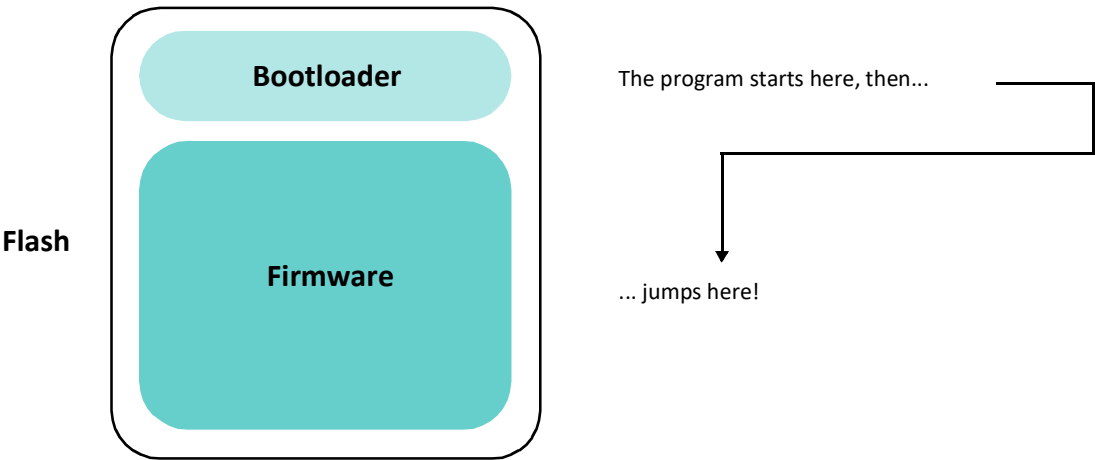


Figure 2-2: Bootloader

2.1.2 Standby

This is the default mode of the LR1120. It is the return state from all other modes (except for specific fall-back options), and the mode from which transitions to other modes are possible. All commands to configure the device should be issued in this mode.

Two clocks are available: either the internal 32MHz RC oscillator (Standby RC mode), or an external 32MHz crystal/TCXO (Standby Xosc mode). The RC clock is used by default for all automatic mode transitions. The crystal/TCXO clock allows faster transitions to other modes at the expense of a higher power consumption.

2.1.2.1 SetStandby

Command `SetStandby(...)` sets the device in standby mode with the chosen 32MHz oscillator.

Table 2-1: SetStandby Command

Byte	0	1	2
Data from Host	0x01	0x1C	StdbbyConfig
Data to Host	Stat1	Stat2	IrqStatus(31:24)

StdbbyConfig selects the oscillator used in standby mode:

- 0x00: Selects internal RC oscillator (Standby RC mode)
- 0x01: Selects external Xtal/TCXO oscillator (Standby Xosc mode)

2.1.3 Calibrations

During the startup sequence, the device firmware calibrates the low and high frequency RC oscillators, the PLL, the ADC, and the image rejection of the mixer at 915MHz. After the calibration procedure the device is set in Standby RC mode.

If operating at another frequency, the image calibration procedure must be restarted using command `CalibImage(...)`.

An image calibration is advised after large temperature variations and optimal image rejection using command `CalibImage(...)`.

Image calibration is necessary if there is a frequency change > 10MHz, or a temperature change > 10°C.

2.1.3.1 CalibImage

The `CalibImage(...)` command launches an image calibration for the given range of frequencies *Freq1* and *Freq2* on the RFI_N/P_LF sub-GHz path.

Table 2-2: CalibImage Command

Byte	0	1	2	3
Data from Host	0x01	0x11	Freq1	Freq2
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)

By default, the image calibration is made in the band 902 - 928MHz. Nevertheless, it is possible to request the device to perform a new image calibration at other frequencies. The frequencies are given in 4MHz steps (Ex: 900MHz -> 0xE1).

The calibration is valid for all frequencies between the two parameters. Typically, the user selects the parameters *Freq1* and *Freq2* from [Table 2-3: ISM Band Values](#). The same frequency may be provided as *Freq1* and *Freq2* to perform a single calibration for the *Freq1* / *Freq2* value.

Table 2-3: ISM Band Values

Frequency Band [MHz]	Freq1	Freq2
430-440	0x6B	0x6E
470-510	0x75	0x81
779-787	0xC1	0xC5
863-870	0xD7	0xDB
902-928	0xE1 (default)	0xE9 (default)

The command operates in any mode. At the end of the calibration procedure, the device returns to Standby RC.

For other frequency bands, *Freq1* and *Freq2* values can be obtained using the following formula:

- $Freq1 = \text{floor}((fmin_mhz - 1)/4)$
- $Freq2 = \text{ceiling}((fmax_mhz + 1)/4)$

In the case of POR, or when the device is recovering from power-down or sleep mode without retention, the image calibration is performed as part of the initial calibration process and for optimal image rejection in the band 902 - 928MHz. If a TCXO is fitted, the calibration fails.

2.1.3.2 Calibrate

The `Calibrate(...)` command calibrates the requested blocks defined by the *CalibParams* parameter.

Table 2-4: Calibrate Command

Byte	0	1	2
Data from Host	0x01	0x0F	CalibParams
Data to Host	Stat1	Stat2	IrqStatus(31:24)

Table 2-5: CalibParams Parameter

Bits	(7:6)	5	4	3	2	1	0
Name	RFU	PLL_TX	IMG	ADC	PLL	HF_RC	LF_RC

Command operates in any mode. At the end of the calibration procedure, the device returns to Standby RC.

Note: For Bluetooth® Low Energy beaconing compatibility the LR1120 requires a PLL_TX calibration sequence to be performed prior to the first Bluetooth® Low Energy beaconing compatible transmission. Refer to [Section 14](#) for additional information.

2.1.4 Power Down

This is the lowest power consumption mode of the device. In this mode:

- All clocks are stopped, therefore no RTC is available
- There is no data retention, so device reconfiguration is necessary when leaving power down mode
- The BUSY signal is set to high, indicating to the host that the device is not ready to accept a command
- The device is put in power down mode with the `SetSleep(...)` command (refer to sleep mode description)

The device can exit this mode if NSS activity is detected.

Exiting this mode, the device performs a firmware restart, and sets the BUSY signal to low, indicating that the startup phase has been performed successfully and that the device is ready to accept a command.

2.1.5 Sleep

Sleep mode configures the LR1120 into a low power consumption mode between radio or geolocation operations, while retaining the configuration register values and storing the firmware data in RAM. When the LR1120 is in sleep mode, the BUSY signal is set to 1 with a pull-up, all MISO SPI signals are high-Z, and all DIOs are in Hi-Z mode (see [DriveDiosInSleepMode](#)).

An optional 32kHz source can run either on the internal RC oscillator, or on the internal 32.768kHz oscillator driving an external crystal. The 32.768kHz crystal oscillator allows a faster transition to standby mode, at the expense of higher power consumption. In both cases, the RTC uses the 32kHz clock source to allow an automatic wake-up from Sleep mode.

2.1.5.1 SetSleep

SetSleep(. . .) puts the device in Powerdown or Sleep mode, and configures the timeout for automatic wake-up.

Table 2-6: SetSleep Command

Byte	0	1	2	3	4	5	6
Data from Host	0x01	0x1B	SleepConfig	SleepTime(31:24)	SleepTime(23:16)	SleepTime(15:8)	SleepTime(7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00

Table 2-7: SleepConfig Parameter

SleepConfig bit	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
Definition	RFU	RFU	RFU	RFU	RFU	RFU	Wakeup	Retention

- *SleepConfig* defines in which sleep mode the device is put, and if it wakes up after a given time on the RTC event:
 - ♦ *Retention* (bit 0) defines if the device configuration and firmware data are retained:
 - ♦ 1: 8kB of memory used for device state and firmware data retention
 - ♦ 0: No data retention (Power Down mode)
 - ♦ *Wakeup* (bit 1) determines if the device wakes up after a given time on the RTC event:
 - ♦ 1: Automatic wake-up enabled. Device automatically goes in Standby mode with RC oscillator, at end of *SleepTime* timer. Configure the 32kHz clock source using command `ConfigLFClock( . . . )` for modem applications
 - ♦ 0: Automatic wake-up disabled
 - ♦ Other bits are RFU and should be set to 0
- *SleepTime*: sleep time in number of 32.768kHz clock cycles, prior to automatic wake-up. Therefore, the sleep time can vary from 0ms to 36.4 hours in steps of 30.52us

The device exits this mode upon the falling edge on the NSS signal even when automatic wakeup is enabled. Exiting this mode, the device performs a firmware restart. When the BUSY signal is set to low, it indicates that the startup phase has been performed successfully, and that the device is ready to accept a command.

The following table summarizes the sleep modes according to Retention and Wakeup bits configuration, with their current consumption (RC /XTAL) and Standby transitions times (indicative values, for comparison only).

Table 2-8: Sleep Mode Summary

Retention	Wakeup	Datasheet	Indicative Consumption (uA)	Indicative Transition to Standby (ms)
0	0	Powerdown	IDDPDN	30
0	1	Sleep	IDDSL1 / IDDSL2	30
1	0	Sleep w/ 8kB retention, no RTC	IDDSL3	<1
1	1	Sleep w/ 8kB retention	IDDSL3A / IDDSL4A	<1

2.1.6 Reset

Four reset sources are available to trigger an LR1120 restart and execute the startup sequence: Power-On-Reset / Brown-Out Reset (POR/BRN), NRESET, and Reboot(. . .) command.

The BUSY signal is kept high during each one of the reset procedures, and returns to low when the restart procedure is finished. At the end of the restart procedure, the device is ready to accept commands, it goes into Standby mode with RC oscillator on (STBY_RC). All device context is lost during this operation, so the device must be re-configured and recalibrated. POR/BRN and NRESET also trigger an authentication of the internal firmware.

2.1.6.1 Power-On-Reset and Brown-Out Reset

The LR1120 performs a restart if either of the following occurs:

- The battery voltage rises above the Power-On-Reset (POR) level
- The battery voltage temporarily drops below the Brown-Out Reset (BRN) level

Both POR and BRN trigger a full restart of the internal firmware. The *Status* field of the *Stat2* status variable indicates if a POR or BRN occurred.

Please refer to [5.5 Power-On-Reset and Brown-Out-Reset](#) for addition information on the POR and BRN.

2.1.6.2 NRESET

Putting the NRESET signal to low for at least 100µs restarts the LR1120. The restart is equivalent to a Power-On Reset, and the device follows the same restart sequence.

2.1.6.3 Reboot

Command Reboot(. . .) triggers a restart of the LR1120 firmware.

Table 2-9: Reboot Command

Byte	0	1	2
Data from Host	0x01	0x18	StayInBootLoader
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *StayInBootLoader* determines the type of reboot:
 - ♦ 0: Performs a software restart
 - ♦ 3: The boot-loader does not execute the firmware in flash, but allows firmware upgrades
 - ♦ Other values are RFU

The 32kHz clock's configuration is kept on a reboot. Command ConfigLFClock(. . .) modifies the 32kHz clock configuration.

2.1.7 GNSS Scanning Mode

GNSS scanning mode detects GPS and BeiDou signals on the RFI_N_LF1 and RFI_P_LF1 pins for outdoor geolocation. Satellite signals are digitized and processed by the integrated DSP. At the end of the satellite signal processing, the BUSY signal returns to low, indicating to the host controller that the GNSS scanning data is available. The result can then be transmitted to a geolocation server using an LPWAN network to compute the device position.

Different GNSS scanning sub-modes are available, depending on the availability of almanac data and assistance information. Refer to the section 12. [GNSS Scanning](#) for more details.

2.1.8 Wi-Fi Passive Scanning Mode

Wi-Fi Passive Scanning mode detects and demodulates Wi-Fi signals (802.11b, g, or n) from access points in the proximity of the device on the RFIO_HF pin. The Wi-Fi signal is processed by the integrated DSP, and the available MAC addresses are extracted. At the end of the Wi-Fi signal processing, the BUSY signal returns to low, indicating that the MAC addresses are available to the host controller and ready to be sent to a geolocation server using an LPWAN network to compute the device position.

2.1.9 DSP Mode

The LR1120 geolocation functions need to process the Wi-Fi or GNSS environment captures. In this mode, only the DSP and the associated regulators are kept active in order to minimize the power consumption. The BUSY signal is high.

This mode is activated automatically by the LR1120 during the GNSS and Wi-Fi scanning processes (IDDRXGPS2 and IDDRXWIFI3 respectively), and is not actionable by the user.

2.1.10 RX Mode

RX mode receives incoming RF packets on the RFI_N_LF0/RFI_P_LF0 pins in the sub-GHz band (150-960MHz), and the RFIO_HF pin in the 1.9-2.1GHz Satellite band and 2.4GHz band (2400-2500MHz), using one of the modems (LoRa or (G)FSK). The device enters RX mode using command `SetRx( . . . )`. At packet reception, an RX_DONE interrupt is generated, and the received data is stored in the device data buffer. The RX operation can be automatically terminated after a packet reception, duty-cycled or infinite, based on the application requirements.

While in RX mode, the LR1120 operates in different sub-modes:

- Continuous mode, the device remains in RX mode and looks for incoming packets until the host requests a different mode.
- Single mode, the device automatically returns to a configured mode (Standby RC by default) after a packet reception.
- Single with timeout mode, the device automatically returns to a configured mode (Standby RC by default) after a packet reception or after the given timeout. If a sync word (G)FSK or a LoRa header is detected, the timeout is stopped.
- RX Duty Cycle mode, the device goes periodically into RX mode to receive a packet before going back to Sleep mode, until a packet is received.
- AutoTx mode (auto transmits a packet a given time after packet reception), the device goes into an intermediary mode for the requested time after a packet reception, before entering TX mode to transmit the packet.

2.1.11 TX Mode

TX mode uses the supported modems to transmit RF packets using the selected sub-GHz PA on the RFO_LP_LF or RFO_HP_LF pins in the sub-GHz band (150-960MHz), and the RFIO_HF pin in the 1.9-2.1GHz Satellite band and 2.4GHz band (2400-2500MHz).

After ramping-up the PA, the LR1120 transmits the data buffer at the given frequency, PA, output power and packet and modulation configurations. When the last bit of the packet has been sent, a TX_DONE interrupt is generated, the PA regulator is ramped down, the selected PA is switched OFF and the device goes back to Standby RC or Xosc mode, depending on the *FallBackMode* configuration.

In TX mode, the BUSY signal goes low as soon as the PA has ramped-up and transmission of the preamble starts.

While in TX mode, the LR1120 operates in different sub-modes:

- Single mode, the device automatically returns to a configured mode (Standby RC by default) after a packet transmission.
- Single mode with timeout, the device automatically returns to a configured mode (Standby RC by default) after a packet transmission or after the given timeout.
- AutoRX mode, (automatically goes into RX mode a given time after transmitting a packet) the device goes into an intermediary mode for the requested time after a packet transmission, before entering RX mode for reception of a packet or until the configured timeout.
- Continuous Wave mode (CW mode), the device indefinitely transmits an unmodulated carrier at the predefined frequency until another command is issued to change the mode.
- Infinite preamble mode: the device indefinitely transmits an infinite preamble of the configured modulation.

2.1.12 FS Mode

Frequency Synthesis (FS) mode is an intermediate mode between standby mode and the RX or TX modes, where the PLL and the associated regulators are switched on. The BUSY signal goes low as soon as the PLL is locked.

2.1.12.1 SetFs

Command SetFs(. . .) sets the device in Frequency Synthesis mode.

Table 2-10: SetFsCommand

Byte	0	1
Data from Host	0x01	0x1D
Data to Host	Stat1	Stat2

2.2 Startup Sequence

At power-up or after a reset, the device initiates its startup phase.

- The BUSY signal is set to high, indicating that the device is busy and cannot accept a command.
- When the power management unit and RC oscillator become available, the embedded CPU starts and executes the internal firmware.
- At the end of the startup sequence, the device is set in Standby RC mode, the BUSY signal goes low and the device accepts commands.

2.3 Firmware Upgrade

The LR1120 can be upgraded with a new firmware image, supplied by Semtech. Complete details are provided in the Application note AN1200.57 “LR1110: Upgrade of the Program Memory”, available from the Semtech website. The related commands are described hereafter.

2.3.1 GetVersion

Command `GetVersion()` returns the version of the LR1120.

Table 2-11: GetVersion Command

Byte	0	1
Data from Host	0x01	0x01
Data to Host	Stat1	Stat2

Table 2-12: GetVersion Response

Byte	0	1	2	3	4
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	HW Version	Use Case	FW Major	FW Minor

- *HW Version* is the version of the LR1120 hardware
- *Use Case* specifies the usage of the device:
 - ♦ 0x01: LR1110
 - ♦ 0x02: LR1120
 - ♦ 0x03: LR1121
 - ♦ 0xDF: Bootloader mode
- *FW Major + FW Minor* is the version of the LR1120 internal firmware stored in flash memory. Shipping versions of the LR1120 after production test is rev 01.01. It is advised to update the firmware with the latest firmware.

2.3.2 EraseFlash

In bootloader mode **only**, the `EraseFlash(...)` command must be used before a new image is written to the device.

If executed in another mode, the command status in *Stat1* is set to *P_ERR*.

Table 2-13: EraseFlash Command

Byte	0	1
Data from Host	0x80	0x00
Data to Host	Stat1	Stat2

2.3.3 WriteFlashEncrypted

The WriteFlashEncrypted() command writes a new firmware image to the LR1120. More details are supplied in the AN1200.57 Application Note. Command [CryptoCheckEncryptedFirmwareImage](#) can be used prior to writing the image.

Table 2-14: WriteFlashEncrypted Command

Byte	0	1	2	3	4	5	6	7
Data from Host	0x80	0x03	Offset (31:24)	Offset (23:16)	Offset (15:8)	Offset (7:0)	Data1 (31:24)	Data1 (23:16)
Data to Host	Stat1	Stat2	0x00	0x00	0x00	0x00	0x00	0x00

Byte	8	9	10	11			...	4*N+5
Data from Host	Data1 (15:8)	Data1 (7:0)	Data2 (31:24)	Data2 (23:16)	Data2 (15:8)	Data2 (7:0)	...	DataN (7:0)
Data to Host	0x00	0x00	0x00	0x00	0x00	0x00	...	0x00

Where N must range from 1 to 32 inclusive.

2.4 Mode Transitions & Timings

Table 2-15: [Mode Transitions and Timings](#) lists the main mode transitions of the LR1120. Please refer to [Figure 2-1: LR1120 Modes and Transitions](#) for a representation of the LR1120 modes and mode transitions.

Table 2-15: Mode Transitions and Timings

Transition	T _{SW} Mode Typical value (μs)
POR to STBY_RC	237e3
SLEEP to STBY_RC (no data retention)	52e3
if XOSC = XTAL	10e3
if XOSC = TCXO	
SLEEP to STBY_RC (with data retention)	<1000
STBY_RC to STBY_XOSC	43
STBY_XOSC to FS for sub-GHz	50
STBY_XOSC to FS for 2.4 GHz band	80
STBY_XOSC to TX for sub-GHz	142
STBY_XOSC to TX for 2.4 GHz band	172
FS to RX (LoRa®, (G)FSK)	39
FS to TX	102
RX to FS	25
RX to TX	118

2.5 Info Page

The LR11xx flash memory contains an Info Page of 512 32-bit words, on which the host controller can store user data.

Note: the maximum number of LR11x flash memory R/W accesses is 10k cycles.

2.5.1 EraseInfoPage

The command `EraseInfoPage(...)` erases the requested Info Page in the LR11xx flash memory.

Table 2-16: EraseInfoPage Command

Byte	0	1	2
Data from Host	0x01	0x21	InfoPage
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *InfoPage*: can only be 1. Other values are RFU

2.5.2 WriteInfoPage

The command `WriteInfoPage(...)` writes a block of 32-bit words in the Info Page flash memory space starting at a specific word address.

Table 2-17: WriteInfoPage Command

Byte	0	1	2	3	4	5	6	7
Data from Host	0x01	0x22	InfoPage	InfoWordAddr (15:8)	InfoWordAddr (7:0)	data1 (31:24)	data1 (23:16)	data1 (15:8)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0	0

Byte	8	9	10	...	N*4+4
Data from Host	data1 (7:0)	data2 (31:24)	data2 (23:16)	...	dataN (7:0)
Data to Host	0	0	0	...	0

- *InfoPage*: Can only be 1. Other values are RFU.
- *InfoWordAddr*: Starting address of the data block in the Info Page. The address is auto incremented after each data word so that data is stored in contiguous memory locations.
- *data*: Data block to be written in the Info Page. Data length should be a multiple of 4. The maximum value for N is 64.

2.5.3 ReadInfoPage

The command `ReadInfoPage(...)` reads a block of 32-bit words in Info Page flash memory space starting at a specific word address.

Table 2-18: ReadInfoPage Command

Byte	0	1	2	3	4	5
Data from Host	0x01	0x23	InfoPage	InfoWordAddr (15:8)	InfoWordAddr (7:0)	InfoReadLen
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

Table 2-19: ReadInfoPage Response

Byte	0	1	2	3	4	5	6	...	N*4
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00	0x00	...	0x00
Data to Host	Stat1	data1 (31:24)	data1 (23:16)	data1 (15:8)	data1 (7:0)	data2 (31:24)	data2 (23:16)	...	data2 (7:0)

- *InfoPage*: Can only be 1. Other values are RFU.
- *InfoWordAddr*: Starting address of the data block in the Info Page. The address is auto incremented after each data word so that data is stored in contiguous memory locations.
- *InfoReadLen*: Number of words to read. The maximum value for *InfoReadLen* is 64.

Returns CMD_PERR if the user tries to read data after the end of page.

3. Host-Controller Interface

The LR1120 exposes an API which allows the Host controller to communicate with the LR1120 through a set of SPI commands / responses. The BUSY signal is used as a handshake to indicate if the LR1120 is ready to accept a command. Therefore, it is necessary to check the status of BUSY prior to sending a command.

3.1 Write Commands

During write commands, the LR1120 returns the status registers and the interrupt registers to the host on the MOSI pin, depending on the length of the command opcode and arguments.

The host sends a 16-bit opcode followed by the required arguments.

The BUSY signal is automatically asserted on the falling edge of the NSS.

Once the LR1120 finishes processing the command, the BUSY signal is de-asserted to indicate that the device is ready to accept another command.

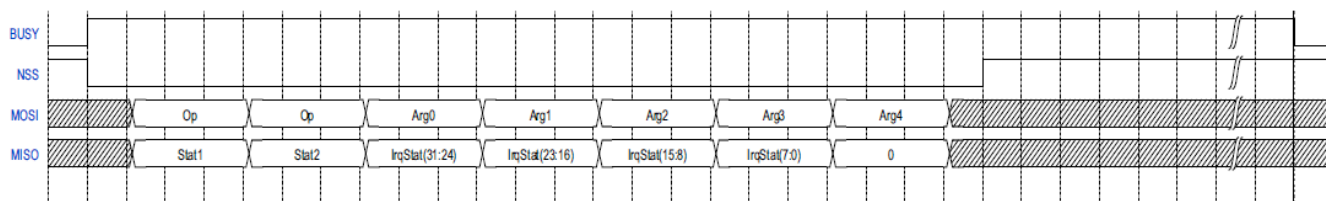


Figure 3-1: Write Command Timing Diagram

3.2 Read Commands

Specific Read commands retrieve data from LR1120, such as internal status or geolocation results.

The host sends a 16-bit opcode, followed by arguments if required.

The BUSY signal is automatically asserted on the falling edge of the NSS.

Once the LR1120 has finished preparing the requested data, the BUSY signal is de-asserted.

The host can then read back the data by sending NOPs (0x00 bytes) to shift out the data on the SPI.

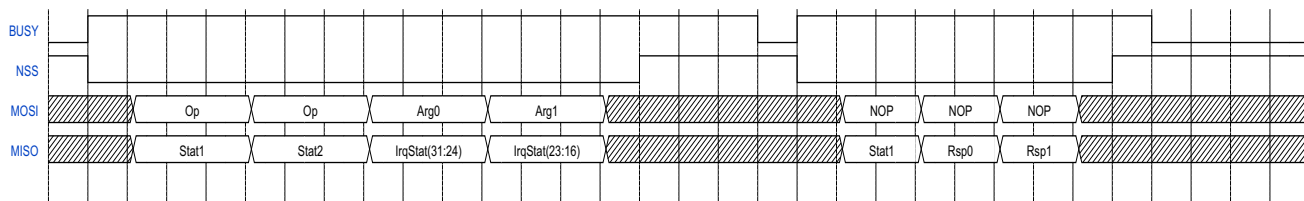


Figure 3-2: Read Command Timing Diagram

3.3 Command Endianness

The following figures are examples of an SPI transaction for command `GetVersion(...)`.

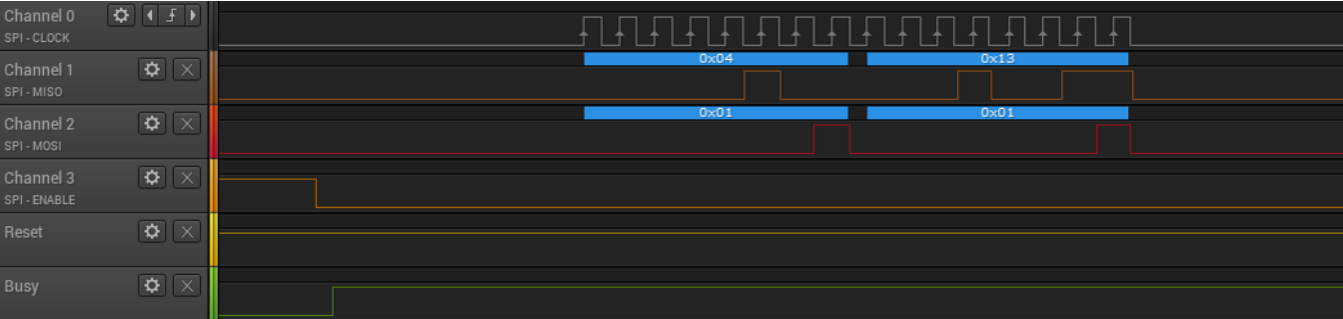


Figure 3-3: GetVersion Write Capture

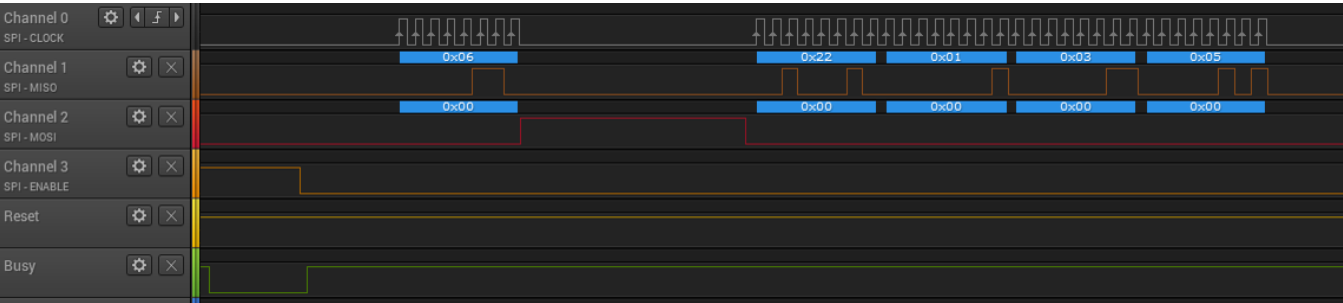


Figure 3-4: GetVersion Read Capture

3.4 Status Registers

The LR1120 features 2 status variables *Stat1* and *Stat2*, which determine the status of the LR1120 (the last command sent, of the device interrupts, of the device operating mode, and of the bootloader) without the need for the host to send a specific command. Command `GetStatus(...)` returns the status registers.

Stat1 and *Stat2* are always sent when the host issues a command. Only *Stat1* is sent back when retrieving data from the LR1120.

3.4.1 GetStatus

Command `GetStatus(...)` returns the LR1120 status flags *stat1* and *stat2*, and the LR1120 interrupt flags. It then clears the *stat2* *ResetStatus* field.

Note that there is an alternate method for retrieving this status information: If a sequence of zeros (NOP) is written to the MOSI signal, the LR1120 clocks out either the response to the last command, or the status information if no response is pending. If the SPI read command is designed to write zero /NOP on the MOSI signal, then this provides a way to obtain the status information by using an ordinary SPI read command. Note, however, that this method does not clear the *ResetStatus* field. See *stat1/CMD_DAT* for more information.

Table 3-1: GetStatus Command

Byte	0	1	2	3	4	5
Data from Host	0x01	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

3.4.2 Stat1

Table 3-2: Stat1 Values

Bits	(7:4)	(3:1)	(0)
Name	RFU	Command Status	Interrupt Status

- *Command Status* indicates the status of the last command sent by the host:
 - ♦ 0: CMD_FAIL: The last command could not be executed
 - ♦ 1: CMD_PERR: The last command could not be processed (wrong opcode, arguments). It is possible to generate an interrupt on DIO if a command error occurred
 - ♦ 2: CMD_OK: The last command was processed successfully
 - ♦ 3: CMD_DAT: The last command was successfully processed, and data is currently transmitted instead of IRQ status
 - ♦ 4-7: RFU
- *Interrupt Status* indicates if an LR1120 system interrupt was raised:
 - ♦ 0: No interrupt active
 - ♦ 1: At least 1 interrupt active

3.4.3 Stat2

Table 3-3: Stat2 Values

Bits	(7:4)	(3:1)	(0)
Name	Reset Status	Chip Mode	Bootloader

- *Reset Status* indicates the origin of an LR1120 reset:
 - ♦ 0: Cleared (no active reset)
 - ♦ 1: Analog reset (Power On Reset, Brown-Out Reset)
 - ♦ 2: External reset (NRESET pin)
 - ♦ 3: System reset
 - ♦ 4: Watchdog reset
 - ♦ 5: Wakeup NSS toggling
 - ♦ 6: RTC restart
 - ♦ 7: RFU
- *Chip Mode* indicates the current mode of the LR1120:
 - ♦ 0: Sleep
 - ♦ 1: Standby with RC Oscillator
 - ♦ 2: Standby with external Oscillator
 - ♦ 3: FS
 - ♦ 4: RX
 - ♦ 5: TX
 - ♦ 6: Wi-Fi or GNSS geolocation
 - ♦ 7: RFU
- *Bootloader:*
 - ♦ 0: currently executes from boot-loader
 - ♦ 1: currently executes from flash. The *ResetStatus* field is cleared on the first `GetStatus()` command after a reset. It is not cleared by any other command

3.5 BUSY

DIO0 is used for the BUSY signal: the BUSY signal is set high when a command is being processed, and when the device is not ready to accept a new command. The timing diagram of the BUSY signal is as follows:

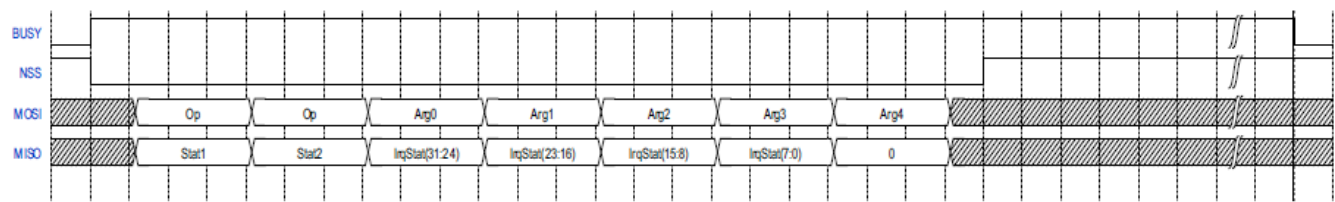


Figure 3-5: BUSY Timing Diagram

The amount of time the BUSY signal stays high after the end of rising edge of NSS ($T_{SW Mode}$) depends on the nature of the command. The most common switching times $T_{SW Mode}$ are indicated in [Section 2.4 "Mode Transitions & Timings" on page 29](#).

3.6 Errors

3.6.1 GetErrors

Command `GetErrors(...)` returns the pending errors that occurred since the last `ClearErrors(...)`, or the circuit startup.

It is possible to generate an interrupt on DIO9 or DIO11 when an error occurs. The error cannot be masked.

Table 3-4: GetErrors Command

Byte	0	1
Data from Host	0x01	0x0D
Data to Host	Stat1	Stat2

Table 3-5: GetErrors Response

Byte	0	1	2
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	ErrorStat(15:8)	ErrorStat(7:0)

ErrorStat contains all the possible error flags that could occur during chip operations:

- Bit 0: `LF_RC_CALIB_ERR`. Calibration of low frequency RC was not done. To fix it redo a calibration.
- Bit 1: `HF_RC_CALIB_ERR`. Calibration of high frequency RC was not done. To fix it redo a calibration.
- Bit 2: `ADC_CALIB_ERR`. Calibration of ADC was not done. To fix it redo a calibration.
- Bit 3: `PLL_CALIB_ERR`. Calibration of maximum and minimum frequencies was not done. To fix it redo the PLL calibration.
- Bit 4: `IMG_CALIB_ERR`. Calibration of the image rejection was not done. To fix it redo the image calibration.
- Bit 5: `HF_XOSC_START_ERR`. High frequency XOSC did not start correctly. To fix it redo a reset, or send `SetTcxoCmd(...)` if a TCXO is connected and redo calibrations.
- Bit 6: `LF_XOSC_START_ERR`. Low frequency XOSC did not start correctly. To fix it redo a reset.
- Bit 7: `PLL_LOCK_ERR`. The PLL did not lock. This can come from too high or too low frequency configuration, or if the PLL was not calibrated. To fix it redo a PLL calibration, or use other frequencies.
- Bit 8: `RX_ADC_OFFSET_ERR`. Calibration of ADC offset was not done. To fix it redo a calibration.
- Bit 9-15: RFU.

3.6.2 ClearErrors

Command `ClearErrors(...)` clears all errors flags pending in the device. The error flags cannot be cleared individually.

Table 3-6: ClearErrors Command

Byte	0	1
Data from Host	0x01	0x0E
Data to Host	Stat1	Stat2

3.7 Memory Access

3.7.1 WriteRegMem32

Command `WriteRegMem32(...)` writes blocks of 32-bit words in register/memory space starting at a specific address.

Table 3-7: WriteRegMem32 Command

Byte	0	1	2	3	4	5	6	7
Data from Host	0x01	0x05	Addr (31:24)	Addr (23:16)	Addr (15:8)	Addr (7:0)	Data1 (31:24)	Data1 (23:16)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00

Byte	8	9	10	...	4*N +5
Data from Host	Data1 (15:8)	Data1 (7:0)	Data2 (31:24)	...	DataN (7:0)
Data to Host	0x00	0x00	0x00	...	0x00

- The address is auto incremented after each data byte so that data is stored in contiguous register/memory locations.
- The value of N is maximum 64.

3.7.2 ReadRegMem32

Command `ReadRegMem32(...)` reads blocks of 32-bit words in register/memory space starting at a specific address.

Table 3-8: ReadRegMem32 Command

Byte	0	1	2	3	4	5	6
Data from Host	0x01	0x06	Addr (31:24)	Addr (23:16)	Addr (15:8)	Addr (7:0)	Len
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00

Table 3-9: ReadRegMem32 Response

Byte	0	1	2	3	4	5	...	4*N
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	Data1 (31:24)	Data1 (23:16)	Data1 (15:8)	Data1 (7:0)	Data2 (31:24)	...	DataN (7:0)

- The address is auto incremented after each data byte so that data is read from contiguous register locations
- *Len* is the number of words to read, and is maximum 64

3.7.3 WriteRegMemMask32

Command `WriteRegMemMask32(...)` reads/modifies/writes the masked bits (Mask bits = 1) of a single 32-bit word in register/memory space at the specified address.

Table 3-10: WriteRegMemMask32 Command

Byte	0	1	2	3	4	5	6	7
Data from Host	0x01	0x0C	Addr (31:24)	Addr (23:16)	Addr (15:8)	Addr (7:0)	Mask (31:24)	Mask (23:16)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00

Byte	8	9	10	11	12	13
Data from Host	Mask (15:8)	Mask (7:0)	Data (31:24)	Data (23:16)	Data (15:8)	Data (7:0)
Data to Host	0x00	0x00	0x00	0x00	0x00	0x00

3.7.4 WriteBuffer8

Command `WriteBuffer8(...)` writes a block of bytes into the radio TX buffer.

Table 3-11: WriteBuffer8 Command

Byte	0	1	2	3	4	5	6	...	N+1
Data from Host	0x01	0x09	Data1	Data2	Data3	Data4	Data5	...	DataN
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	...	0x00

- *Data*: N bytes of data. The value of N is maximum 255

3.7.5 ReadBuffer8

Command `ReadBuffer8(...)` reads a block of *Len* bytes in the radio RX buffer starting at a specific *Offset*. RX buffer must be implemented as a ring buffer.

Table 3-12: ReadBuffer8 Command

Byte	0	1	2	3
Data from Host	0x01	0x0A	Offset (7:0)	Len (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)

Table 3-13: ReadBuffer8 Response

Byte	0	1	2	3	...	N
Data from Host	0x00	0x00	0x00	0x00	...	0x00
Data to Host	Stat1	Data1	Data2	Data3	...	DataN

3.7.6 ClearRxBuffer

Command `ClearRxBuffer(...)` clears all data in the radio RX buffer. It writes '0' over the whole Rx buffer. It is mainly used for debug purposes to ensure the data in the RX buffer is not from the previous packet.

Table 3-14: ClearRxBuffer Command

Byte	0	1
Data from Host	0x01	0x0B
Data to Host	Stat1	Stat2

3.7.7 GetRandomNumber

This command gets a 32-bit random number. This is not used for security purposes.

Table 3-15: GetRandomNumber Command

Byte	0	1
Data from Host	0x01	0x20
Data to Host	Stat1	Stat2

Table 3-16: GetRandomNumber Response

Byte	0	1	2	3	4
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	RandomNo(31:24)	RandomNo(23:16)	RandomNo(15:8)	RandomNo(7:0)

3.7.8 EnableSpiCrc

This command enables / disables an 8-bit CRC on the Serial Peripheral Interface.

CRC generation uses a polynomial generator 0x65 (reversed reciprocal), initial value 0xFF. The CRC is calculated on all data received on the MOSI signal (including Opcode) and on all data sent on the MISO signal (including all status).

This command is always protected by the CRC:

- To enable the CRC, the CRC must already be appended in this command. As an example: the whole command to enable is 0x01 0x28 0x01 0x20.
- To disable the CRC, the CRC must be appended to the command since it is still enabled. As an example: the whole command to disable is 0x01 0x28 0x00 0x1C.

Table 3-17: EnableSpiCrc Command

Byte	0	1	2	3
Data from Host	0x01	0x28	Enable	CRC
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)

- *Enable*: enables / disables an 8-bit CRC on the SPI interface:
 - ♦ 0: Disabled. No CRC is expected or sent on the SPI (default)
 - ♦ 1: Enabled. A CRC is expected and sent on the SP.
 - ♦ Other values are RFU
- *CRC*: cyclic redundancy check

4. GPIOs

The LR1120 features 13 digital I/Os.

Table 4-1: Digital I/Os

I/O	Description
DIO0	Used for BUSY signalling, and is mandatory to properly handle the host controller interface.
DIO1 to DIO4	Dedicated to the SPI interface signals NSS, SCK, MOSI and MISO respectively.
DIO5, DIO6, DIO7	Can control external RF switches or LNAs on the Wi-Fi, GNSS, RFIO-HF, and sub-GHz RF paths.
DIO8	Can control external RF switches or LNAs on the Wi-Fi, GNSS, RFIO-HF, and sub-GHz RF paths. Can be used as a 32.768kHz source to host controller if a 32.768kHz crystal oscillator is connected to DIO10 and DIO11.
DIO9	Dedicated to LR1120 interrupts. It is recommended to connect DIO9 to the host controller for the lowest-power applications. DIO11 can be used as another interrupt pin if no 32.768kHz crystal oscillator is used.
DIO10	Can control external RF switches or LNAs on the Wi-Fi, GNSS, RFIO-HF, and sub-GHz RF paths. Can be used as connection pins for an external 32.768kHz crystal oscillator as an RTC source.
DIO11	Can be used as connection pins for an external 32.768kHz crystal oscillator as an RTC source. Can be used as an input pin if the 32.768kHz signal is fed by the host controller. In this case DIO10 must be left unconnected. Can be used as another interrupt pin if no 32.768kHz crystal oscillator is used.
NRESET	Can cancel on-going functions of the LR1120, and reset all HW and FW. Although a device restart is also possible through host controller commands, it is recommended to allow the host controller to control this signal.

4.1 Interrupts

The LR1120 features numerous interrupt sources, allowing the host to react to a large variety of events in the LR1120 system without the need to poll registers, therefore allowing power-optimized applications.

The LR1120 interrupts are multiplexed on the DIO9 and/or DIO11 pin. When the application receives an interrupt, it can determine the source by using command `GetStatus(...)`. The interrupt can then be cleared using the `ClearIrq(...)` command.

Command `SetDioIrqParams(...)` configures which interrupt signal should be activated on the DIO9 and/or DIO11 interrupt pins.

The status of the LR1120 interrupts can be read using command `GetStatus(...)`, but they are also returned by the LR1120 to the host on the MISO signal during the SPI transactions via the *IrqStatus* bytes, simultaneously with the command arguments. Therefore, the number of *IrqStatus* sent by the LR1120 during the SPI commands depends on the number of arguments. Refer to Section 3. [Host-Controller Interface](#) for additional information.

The interrupts mapping table *IrqToEnable* is as follows:

Table 4-2: IrqToEnable Interruption Mapping

Bit	Interrupt	Description
0	RFU	RFU
1	RFU	RFU
2	TxDone	Packet transmission completed
3	RxDone	Packet received
4	PreambleDetected	Preamble detected
5	SyncWordValid / HeaderValid	Valid sync word / LoRa® header detected
6	HeaderErr	LoRa header CRC error
7	Err	Packet received with error. LoRa: Wrong CRC received (G)FSK: CRC error
8	CadDone	LoRa Channel activity detection finished
9	CadDetected	LoRa Channel activity detected
10	Timeout	RX or TX timeout
11	LrFhssHop	LR-FHSS intra-packet hopping
12-18	RFU	RFU
19	GNSSDone	GNSS Scan finished
20	WifiDone	Wi-Fi Scan finished
21	LBD	Low Battery Detection
22	CmdError	Host command error
23	Error	An error other than a command error occurred (see GetErrors)
24	FskLenError	IRQ raised if the packet was received with a length error
25	FskAddrError	IRQ raised if the packet was received with an address error
26	-	RFU
27	LoRaRxTimestamp	Last LoRa symbol received. To be used for time-stamping the received packet. The device is still in RX mode.
28	GnssAbort	Command GnssScan, GnssFetchTime, or GnssAlmanacUpdateFromSat aborted
29-31	-	RFU

4.1.1 SetDioIrqParams

Command `SetDioIrqParams(...)` configures which interrupt signal should be activated on the DIO9 and/or DIO11 interrupt pin (referred to as IRQ pin 1 and/or 2).

Table 4-3: SetDioIrqParams Command

Byte	0	1	2	3	4	5
Data from Host	0x01	0x13	Irq1ToEnable (31:24)	Irq1ToEnable (23:16)	Irq1ToEnable (15:8)	Irq1ToEnable (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

Byte	6	7	8	9
Data from Host	Irq2ToEnable (31:24)	Irq2ToEnable (23:16)	Irq2ToEnable (15:8)	Irq2ToEnable (7:0)
Data to Host	0x00	0x00	0x00	0x00

4.1.2 ClearIrq

The `ClearIrq(...)` command clears the selected interrupt signals by writing a 1 in the respective bit.

Table 4-4: ClearIrq Command

Byte	0	1	2	3	4	5
Data from Host	0x01	0x14	IrqToClear (31:24)	IrqToClear (23:16)	IrqToClear (15:8)	IrqToClear (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

The *IrqToClear* is identical to *IrqToEnable* assignment.

4.2 RF Switch Control

4.2.1 SetDioAsRfSwitch

DIO5, DIO6, DIO7, DIO8 and DIO10 can control external RF switches or LNAs on the Sub-GHz, GNSS, Wi-Fi and RFIO_HF RF paths using the `SetDioAsRfSwitch(...)` command. Only the lowest 5 bits of all the configurations as well as the enable are taken into account.

Each Cfg bit corresponds to the state of the RFSW output for that particular mode:

Table 4-5: SetDioAsRfSwitch Command

Byte	0	1	2	3	4	5	6	7	8	9
Data from Host	0x01	0x12	RfSw Enable	RfSw StbyCfg	RfSw RxCfg	RfSw TxCfg	RfSw TxHPCfg	RfSw TxHfCfg	RfSw GnssCfg	RfSw WifiCfg
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00	0x00

- *RfswEnable* value indicates which switch is used (1) and which is not (0):
 - ♦ Bit 0 - RFSW0 Enabled (DIO5 pin)
 - ♦ Bit 1 - RFSW1 Enabled (DIO6 pin)
 - ♦ Bit 2 - RFSW2 Enabled (DIO7 pin)
 - ♦ Bit 3 - RFSW3 Enabled (DIO8 pin)
 - ♦ Bit 4 - RFSW4 Enabled (DIO10 pin)
 - *RfSwStbyCfg*: Each bit indicates the state of the relevant RFSW DIO when in standby mode (bits 5:7 RFU)
 - *RfSwRxCfg*: Each bit indicates the state of the relevant RFSW DIO when in RX mode
 - *RfSwTxCfg*: Each bit indicates the state of the relevant RFSW DIO when in low power TX mode
 - *RfSwTxHPCfg*: Each bit indicates the state of the relevant RFSW DIO when in high power TX mode
 - *RfSwTxHfCfg*: Each bit indicates the state of the relevant RFSW DIO when in radio high frequency TX mode
 - *RfSwGnssCfg*: Each bit indicates the state of the relevant RFSW DIO when in GNSS scanning mode
 - *RfSwWifiCfg*: Each bit indicates the state of the relevant RFSW DIO when in Wi-Fi scanning mode or high frequency RX mode
- By default, no DIO is used as RF switch: all RFSW outputs are in High-Z state.

This command only works with the chip in Standby RC mode, otherwise it returns a `CMD_FAIL` on the next `GetStatus` command.

4.2.2 DriveDiosInSleepMode

Enables or disables the addition of pull ups or pull downs resistors on the configured RF switch and IRQ line DIOs. This command allows to save power consumption in an application where the RF switches are supplied by the LR1120 DIOs, when the LR1120 is in sleep mode.

Table 4-6: DriveDiosInSleepMode Command

Byte	0	1	2
Data from Host	0x01	0x2A	Enable
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *Enable* value indicates which switch is used for pull-up/down (1) and which is not (0):
 - ♦ 0: No pull-up or pull-down is configured by the FW (default).
 - ♦ 1: A pull-up or pull-down is added by the FW on the configured RF switch and IRQ DIOs when going to sleep modes (all sleep modes), depending on the state of the DIO in config RC mode.

On wake-up from sleep mode, the according DIOs are re-configured in push/pull mode, and pull-up/down are removed.

Note: When going to sleep mode without retention (retention = 0), all pending IRQs are cleared before going to sleep mode.

4.3 Temperature Sensor

The LR1120 has a built-in temperature sensor which gives an indication of the internal device temperature.

4.3.1 GetTemp

The temperature measurement can be triggered using command `GetTemp( . . . )`.

Table 4-7: GetTemp Command

Byte	0	1
Data from Host	0x01	0x1A
Data to Host	Stat1	Stat2

Table 4-8: GetTemp Response

Byte	0	1	2
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	Temp(15:8)	Temp(7:0)

The temperature value is a function of an internal reference voltage (typ. 1.35V), and a typical temperature characteristic (typ -1.7mV/°C), and can be approximated using the following formula:

$$\text{Temperature(degC)} \sim 25 + \frac{1000}{-1.7\text{mV/}^{\circ}\text{C}} \times (\text{Temp}(10:0)/2047 * 1.35 - 0.7295)$$

NOTE: `GetTemp( )` uses XOSC mode to get the temperature, so if a TCXO is connected to the LR1120, it must be configured using `SetTcxoMode` before calling `GetTemp()`.

5. Power Distribution

5.1 Selecting DC-DC Converter or LDO Regulation

Two forms of voltage regulation (DC-DC buck converter or linear LDO regulator) are available depending upon the design priorities of the application. The linear LDO regulator is always present in all modes but the transceiver uses DC-DC when selected. Alternatively a high efficiency DC to DC buck converter can be enabled in FS, Rx and Tx modes.

The DC-DC can be driven by two clock sources:

- In STDBY_XOSC: RC13M supplies the clock and the frequency is $RC13M / 4$, the switching frequency of the DC-DC converter is 3.25MHz
- In FS, Rx, Tx: the PLL supplies the clock and the frequency is ~5MHz

For applications where cost and size are constrained, LDO-only operation is possible which negates the need for the 15μH inductor, conferring the benefits of a reduced bill of materials and reduced board space. at the expense of power consumption.

The following table illustrates the power regulation options for different modes and user settings.

Table 5-1: Regulation Type Versus Circuit Mode

Circuit Mode	Sleep	STDBY_RC	STDBY_XOSC	FS	Rx	Tx
Regulator Type = 0	-	LDO	LDO	LDO	LDO	LDO
Regulator Type = 1	-	LDO	DC-DC + LDO	DC-DC + LDO	DC-DC + LDO	DC-DC + LDO

The user can specify the use of DC-DC by using the command `SetRegulatorMode( . . . )`. This operation must be carried out in STDBY_RC mode only.

When the DC-DC is enabled, the LDO remains ON and its target voltage is set 50mV below the DC-DC voltage to ensure voltage stability for high current peaks. If the DC-DC voltage drops to this level due to a high current peak, the LDO covers for the current need at the expense of the energy consumption of the radio, which is increased.

5.2 DC-DC Inductor Selection

The inductor is essential to ensure optimal performance of the DC-DC internal block. An incorrect inductor could cause unwanted effects ranging from ripple currents to early aging of the device, and a degradation of DC-DC regulator efficiency.

The preferred inductor is shielded, presenting a low internal series resistance and a resonance frequency much higher than the DC-DC switching frequency. When selecting the 15μH inductor, the user should therefore select a part with the following considerations:

- DCR (max) = 2ohms
- Idc (min) = 100mA
- Freq (min) = 20MHz

Table 5-2: Recommended Inductors

Reference	Size	Value	Description	Manufacturer
MLZ1608N100LT000	0603	10uH	MLZ1608 Multilayer Shielded Inductor ±20%	TDK
MLZ1608N150LT000	0603	15uH	MLZ1608 Multilayer Shielded Inductor ±20%	TDK
MLZ2012M150W	0805	15uH	MLZ2012 Multilayer Shielded Inductor ±5%	TDK

5.3 Power Modes

Two power modes are available:

- DC-DC converter for low power applications
- LDO for low-cost or small size applications

5.3.1 SetRegMode

Command `SetRegMode( . . . )` defines which regulator should be used.

Table 5-3: SetRegMode Command

Byte	0	1	2
Data from Host	0x01	0x10	RegMode
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *RegMode* defines if the DC-DC converter has to be switched ON:
 - ♦ 0: Do not switch on the DC-DC converter in any mode (Default)
 - ♦ 1: Automatically switch on the DC-DC converter, depending on the mode as per [Table 5-4](#)
 - ♦ Other values are RFU

This command only works with the device in Standby RC mode, otherwise it returns `CMD_FAIL` on the next `GetStatus` command.

The following table illustrates the power regulation options for different modes and user settings.

Table 5-4: Power Regulation Options

Circuit Mode	Sleep	STDBY_RC	STDBY_XOSC	FS	RX	TX
Regulator Type = 0	-	LDO	LDO	LDO	LDO	LDO
Regulator Type = 1	-	LDO	DC-DC + LDO	DC-DC + LDO	DC-DC + LDO	DC-DC + LDO

5.4 VBAT Measurement

5.4.1 GetVbat

GetVbat (. . .) command monitors the battery supply voltage, it returns the Vbat voltage as a function of a reference voltage:

Table 5-5: GetVbat Command

Byte	0	1
Data from Host	0x01	0x19
Data to Host	Stat1	Stat2

Table 5-6: GetVbat Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	Vbat(7:0)

$$VBAT(V) = \left(\left(\frac{5 \times VBat(7:0)}{255} \right) - 1 \right) 1.35$$

5.5 Power-On-Reset and Brown-Out-Reset

The LR1120 features both POR and BRN features.

- POR and BRN ensure a proper startup of the circuit, maintaining the internal blocks reset until a safe battery voltage level is reached, for example at battery insertion.
- The BRN triggers a device reset if the battery voltage goes below the safe operation threshold of 1.7V (typically)
- The POR/BRN detector has a 50mV hysteresis
- A POR resets the statistics

Refer to [Figure 5-1: LR1120 POR and BRN Functions](#) for an illustration of the POR and BRN functions.

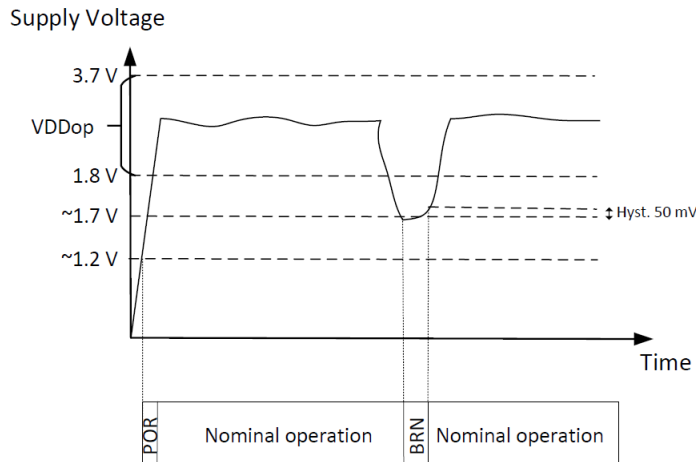


Figure 5-1: LR1120 POR and BRN Functions

5.6 Low Battery Detector

The Low Battery Detector (LBD) detects when the supply voltage VBAT drops below 1.88V (typ). The LBD indication is given through an interrupt signal, hence minimizing the host activity in critical supply voltage conditions. The LBD IRQ is activated through command `SetDioIrqParams()`.

5.7 Over Current Protection

The LR1120 has a built-in Over Current Protection (OCP) block which prevents surge currents when the device is used at its highest power levels, thus protecting the battery that may power the application. The current clamping values are trimmable by register access. The OCP threshold should be configured just below the battery requirement but at least 25% higher than the PA output stage current.

The default OCP values are:

- 60mA for the low power PA (LP PA)
- 150mA for the high power PA (HP PA)
- 50mA for the high frequency PA (HF PA)

6. Clock Sources

The LR1120 uses both low frequency (32kHz) and high frequency (32MHz) clock sources. For each frequency, the clock signal can be obtained by either an RC oscillator, or a crystal oscillator. RC oscillators allow optimized power consumption and faster switching times. Crystal oscillators provide a more precise frequency, in cases when frequency accuracy is needed. RF operations require a 32MHz high precision clock reference, which can be provided by either an external crystal oscillator or by a TCXO.

The clock requirements for LoRa Edge™ devices are summarized in AN1200.74 LoRa Edge Clock Requirements.

6.1 RC Oscillators Clock References

Two RC oscillators are available:

- The 32.768kHz RC oscillator (RTC) can be used by the circuit in Sleep mode to wake-up the device when performing periodic or duty cycled operations. Several commands make use of this RTC to generate time-based events.
- The 32MHz RC oscillator is enabled for all SPI communication to permit configuration of the device without the need to start the 32MHz crystal oscillator.

6.2 High-Precision Clock References

This section provides relevant information about the reference clocks. For additional guidance on clock external clock sources (crystal and TCXO) refer to AN1200.59 Selecting the Optimal Reference Clock.

6.2.1 32.768kHz Clock

A 32.768kHz clock source can be used instead of the (default) 32.768kHz RC oscillator as a low frequency clock source using command `ConfigLfClock(...)`. This clock source can be provided either by a 32.768kHz crystal attached to the LR1120 chip, or by an external 32.768kHz signal fed to the DIO11 pin. The amplitude of the external 32.768kHz signal fed to the DIO11 pin should be a full swing rail-to-rail clock signal. Smaller or much larger amplitudes could cause extra consumption.

A 32.768kHz clock source is mandatory for GNSS Dual Constellation and for GNSS Advanced scan mode. Refer to sections [12.3.2 GnssSetConstellationToUse](#) and [12.3.5 GnssSetMode](#) for details.

6.2.2 32MHz Crystal

A 32MHz crystal oscillator is the cheapest and lowest power consuming approach to provide the 32MHz clock reference to the LR1120. The crystal loading capacitance are integrated, minimizing the overall BOM cost and optimizing the PCB space. In case of crystal operation, the VTCXO pin should be left unconnected.

6.2.2.1 Frequency Drift During Packet Transmission and Thermal Insulation

The transmission of RF packets at high RF power by the LR1120 generates significant heat which may transfer to the 32MHz crystal through the PCB. For long packet durations, this generates a frequency drift which may induce receive errors in the peer device if no precautions are taken during PCB design. This effect is described in more detail in the SX1261/62 application note AN1200.37 Recommendations for Best Performance.

For example, in LoRa modulation when the Low Data Rate (LDRO) is used, a frequency drift rate of the transmitted packet of 120Hz/s typically implies a 3dB sensitivity reduction in the LR1120 receiver for all SF and BW. This maximum drift rate is 110Hz/s when the LDRO is not used. Therefore, the frequency drift during packet transmission has to be kept below this maximum value in order to ensure best packet reception. A design using a TCXO is not subject to frequency drift during the packet transmission.

Implementing cuts in the PCB's ground plane layers reduces heat transfer between the LR1120 and the 32MHz crystal, as shown in [Figure 6-1: LR1120 Thermal Insulation on PCB Top Layer](#).

Refer to the LR1120 PCB reference design on www.semtech.com for an implementation example in PCB design.

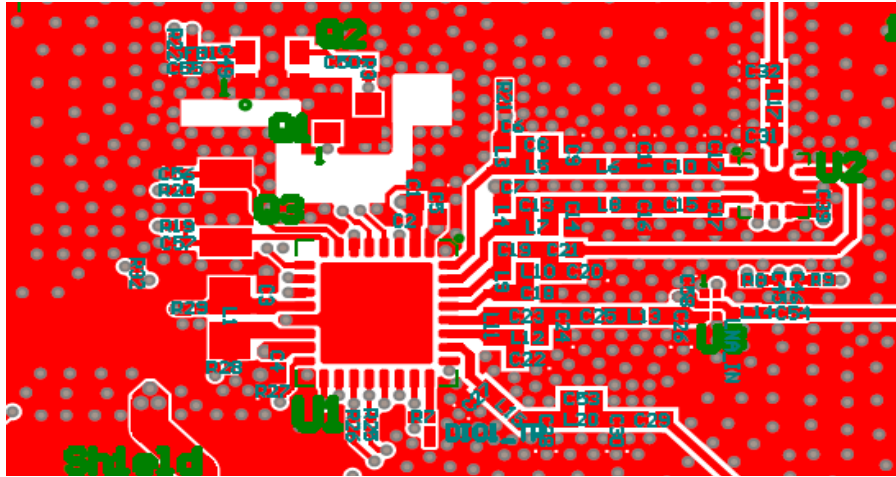


Figure 6-1: LR1120 Thermal Insulation on PCB Top Layer

6.2.3 32MHz TCXO

In environments with extreme temperature variation, it may be required to use a TCXO (Temperature Compensated Crystal Oscillator) to achieve better frequency accuracy. A TCXO is required by the LR1120 GNSS features in order to minimize the power consumption required to perform an outdoor geolocation. When a TCXO is used:

- The TCXO should be connected to pin XTA, through a 220Ω resistor and a 10pF DC-cut capacitor. AN1200.59 provides guidelines for the selection of 32MHz TCXOs and associated components.
- Pin XTB should be left open.
- The TCXO is supplied by the internal regulator on the VTCXO pin.
- The regulated VTCXO is programmable from 1.6 to 3.3V using command `SetTcxoMode()`.
- VBAT should always be 200mV higher than the programmed voltage to ensure proper operation.
- The nominal current drain is 1.5mA, but the regulator can support up to 4mA of load.
- Clipped-sine output TCXO are required, with the output amplitude not exceeding 1.2V peak-to-peak.

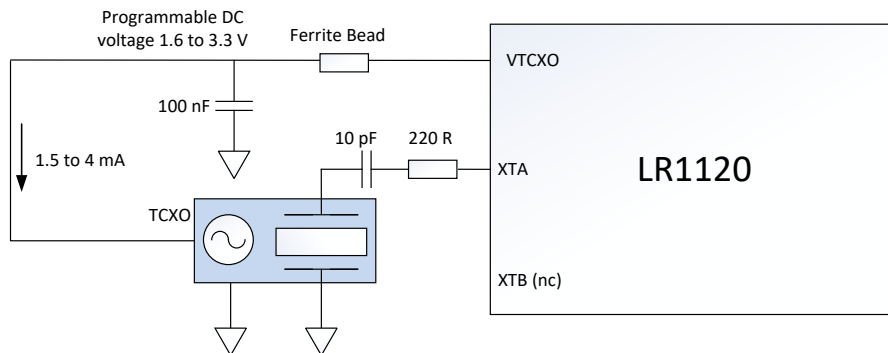


Figure 6-2: TCXO Circuit Diagram

6.3 Commands

6.3.1 ConfigLfClock

Configures the 32kHz source.

Table 6-1: ConfigLfClock Command

Byte	0	1	2
Data from Host	0x01	0x16	LfClkConfig
Data to Host	Stat1	Stat2	IrqStatus (31:24)

LfClkConfig parameter:

- bits 0-1: LF clock selection:
 - ♦ 0: Use 32.768kHz RC oscillator
 - ♦ 1: Use 32.768kHz crystal oscillator
 - ♦ 2: Use externally provided 32.768kHz signal on DIO11 pin
 - ♦ 3: RFU
- bit 2: When to release BUSY signal:
 - ♦ 0: Wait for Xtal 32k ready
 - ♦ 1: Wait for Xtal 32k to be ready before releasing the BUSY signal
- bits 3-7: RFU.

6.3.2 SetTcxoMode

Configures the chip for a connected TCXO.

The TCXO must be configured using `SetTcxoMode( . . . )` before calling `GetTemp( )`.

Table 6-2: SetTcxoMode Command

Byte	0	1	2	3	4	5
Data from Host	0x01	0x17	RegTcxoTune	Delay (23:16)	Delay (15:8)	Delay (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

- *RegTcxoTune* tunes the output voltage on the TCXO supply pin VTCXO, according to [Table 6-3: TCXO Supply Voltage Programming Values](#). The TCXO supply voltage precision varies with *RegTcxoTune* setting. It is typically +/-50mV for VTCXO = 1.8V setting.

Table 6-3: TCXO Supply Voltage Programming Values

RegTcxoTune	TCXO Supply Voltage (typical)
0x00	1.6V
0x01	1.7V
0x02	1.8V
0x03	2.2V
0x04	2.4V
0x05	2.7V
0x06	3.0V
0x07	3.3V

- *Delay* represents the maximum duration for the 32MHz oscillator to start and stabilize (in 30.52us steps). If the 32MHz oscillator from the TCXO is not detected internally at the end the delay period, the device internal firmware triggers a HF_XOSC_START_ERR error.
 - ♦ 0: Disables TCXO mode (default value).
 - ♦ 1: Sets TCXO mode. A complete Reset of the chip is required to get back to normal XOSC operation.

Command only operates in Standby RC mode, otherwise it returns CMD_FAIL on the next `GetStatus( )` command.

7. Radio

7.1 Overview

The LR1120 is a half-duplex RF transceiver capable of handling constant envelope modulation schemes such as LoRa, (G)FSK, Sigfox, and LR-FHSS. The LoRa, Sigfox and (G)FSK modulations are fully compatible with the SX1261/62/68 family for sub-GHz, and can be configured to be compatible with the SX1280/81 family for 2.4GHz.

The radio system is shown in [Figure 7-1: LR1120 Radio Block Diagram](#) below. It is composed of the frequency synthesizer (also referred as PLL), three TX paths (High Power, Low Power and High Frequency), and two RX paths, followed by a high-bandwidth ADC. Both the ADC and the PLL are tied to the digital subsystem, the supported modems, and the LR-FHSS modulator.

The PAs, supported modems, and LR-FHSS modulation are described in dedicated sections.

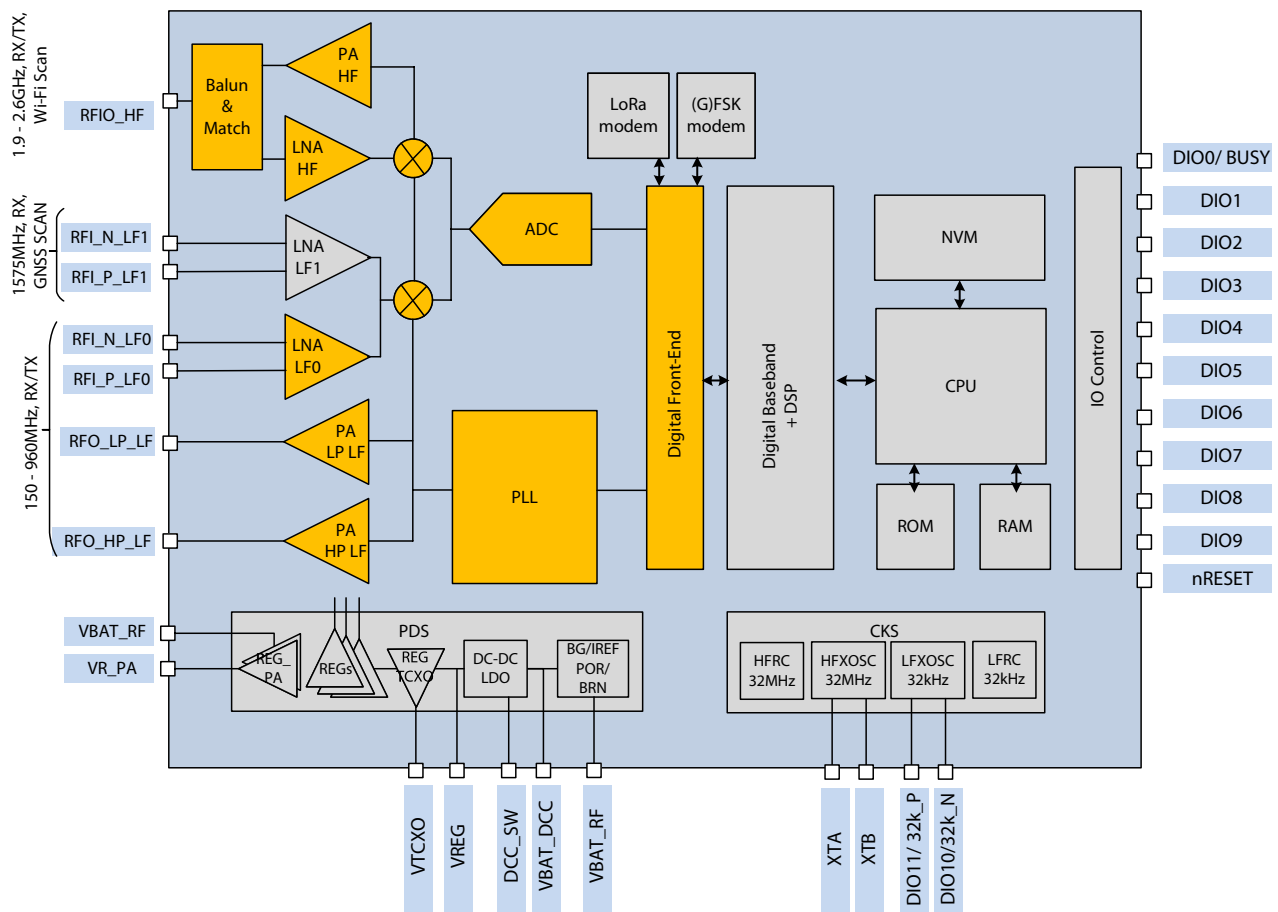


Figure 7-1: LR1120 Radio Block Diagram

The LR1120 frequency synthesizer allows a continuous operation in the 150MHz-2500MHz frequency range. It is shared between the GNSS and Wi-Fi scanning engines, and the sub-GHz radio and HF radio, therefore no simultaneous GNSS scanning, Wi-Fi scanning, or sub-GHz /HF radio operation is possible.

The LR1120 frequency synthesizer is clocked by a 32MHz reference, provided by either a crystal oscillator, or a TCXO. Refer to [Section 6. "Clock Sources" on page 51](#) for details.

7.2 Commands

7.2.1 SetRfFrequency

Command `SetRfFrequency(...)` sets the RF (PLL) frequency of the radio. In RX mode, the frequency is internally down-converted to IF Frequency.

Table 7-1: SetRfFrequency Command

Byte	0	1	2	3	4	5
Data from Host	0x02	0x0B	RfFreq (31:24)	RfFreq (23:16)	RfFreq (15:8)	RfFreq (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

- *RfFreq*: RF Frequency of the radio in Hz. All frequency dependent parameters are automatically recomputed by the LR1120 firmware when processing this command.

7.2.2 SetRx

Command `SetRx(...)` sets the sub-GHz radio in RX mode. The sub-GHz path is selected for *RfFreq* frequencies below or equal to 1.50GHz. Above this value the HF path is selected. If no packet is received after the defined *RxTimeout*, the device goes back to Standby RC mode.

Table 7-2: SetRx Command

Byte	0	1	2	3	4
Data from Host	0x02	0x09	RxTimeout (23:16)	RxTimeout (15:8)	RxTimeout (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)

- *RxTimeout* is expressed in periods of the 32.768kHz RTC. The maximum timeout value corresponds to 512s. Values 0x000000 and 0xFFFFFFFF disable the timeout function.
 - 0x000000 sets the device in RX mode until a reception occurs. After packet reception, the device returns to Standby mode.
 - 0xFFFFFFFF sets the device in RX mode until the host sends a command to change the mode. The device can receive several packets. Each time a packet is received, a packet done indication is given to the host and the device automatically searches for a new packet.

If the timer is active, the radio stops the reception at the end of the timeout period, unless a preamble or a Header has been detected as defined by the *StopTimeoutOnPreamble* configuration.

If no packet type was configured, or the packet type does not allow RX operations, this command fails.

The BUSY signal goes low after the device is set into RX mode.

7.2.3 SetTx

Command SetTx(. . .) sets the sub-GHz radio or the HF radio in TX mode, triggering RF packet transmission, and starting the RTC with the given *TxTimeout* value. There is no PA selection based on the RfFreq configuration. For details on PA choice and operation refer to [Section 9](#).

If the RTC event fires before the end of transmission, it triggers a TIMEOUT IRQ, and stops the transmission. Otherwise, at the end of the packet transmission, a TX_DONE interrupt is generated.

After a TIMEOUT IRQ or TX_DONE IRQ, the device goes back to STBY_RC (default), STBY_XOSC or FS depending on the *FallBackMode* configuration.

Table 7-3: SetTx Command

Byte	0	1	2	3	4
Data from Host	0x02	0x0A	TxTimeout (23:16)	TxTimeout (15:8)	TxTimeout (7:0)
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:16)	IrqStatus(15:8)

- *TxTimeout* is expressed in periods of the 32.768kHz RTC. The maximum value corresponds to 512 s. 0x000000 disables the timeout function.

If no packet type was configured, or the packet type does not allow Tx operations, the command fails.

The BUSY signal goes to 0 after the device is set into TX mode.

7.2.4 AutoTxRx

Command `AutoTxRx(...)` automatically performs the transition to RX mode after a packet transmission, or to TX mode after a packet reception. After the second mode, the device goes back to Standby RC mode.

If AutoTxRx mode is enabled, and a:

- `SetTx(...)` command is sent to the device, the device goes to RX mode after TX_DONE and the given delay.
Timeout is used as the *RxTimeout* of the auto RX.
- `SetRx(...)` command is sent to the chip, the chip goes to TX mode after RX_DONE and the given delay.
Timeout is used as the *TxTimeout* of the auto TX.

If an Rx Duty Cycle is started, this mode is not used.

Table 7-4: AutoTxRx Command

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x02	0x0C	Delay (23:16)	Delay (15:8)	Delay (7:0)	Intermediary Mode	Timeout (23:16)	Timeout (15:8)	Timeout (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00

- *Delay* defines the transition time between the TX and RX mode, expressed in periods of the 32.768kHz RTC. The maximum Delay value corresponds to 512s.
 - ♦ 0x000000: Performs a direct TX to RX or RX to TX transition, without going through the IntermediaryMode.
 - ♦ 0xFFFFF: Disables the AutoTxRx function. The AutoTxRx function is disabled by default.
- *IntermediaryMode*: device mode inbetween TX and RX modes.
 - ♦ 0x00: Sleep mode.
 - ♦ 0x01: Standby RC mode.
 - ♦ 0x02: Standby Xosc mode.
 - ♦ 0x03: FS mode.
- *Timeout* defines the timeout of the second mode, after the automatic transition. It is expressed in periods of the 32.768kHz RTC. The maximum timeout value corresponds to 512s.
 - ♦ 0x000000: Disables the timeout function.

7.2.5 SetRxTxFallbackMode

Command `SetRxTxFallbackMode(...)` defines what mode the device goes into after a packet transmission or a packet reception. If an *Rx Duty Cycle* is started or an *AutoRxTx* is configured, this mode is not used.

Table 7-5: SetRxTxFallbackMode Command

Byte	0	1	2
Data from Host	0x02	0x13	FallbackMode
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *FallbackMode* values:
 - ♦ 0x01: Standby RC mode (default value).
 - ♦ 0x02: Standby Xosc mode.
 - ♦ 0x03: FS mode.
 - ♦ Other values are RFU.

The fallback mode is also used for an Rx Duty Cycle after the `RX_DONE` interrupt, or for an *AutoRxTx* after the RX to TX, or when the TX to RX sequence is completed.

7.2.6 SetRxDutyCycle

Command `SetRxDutyCycle(...)` periodically opens RX windows. Between RX windows, the device goes in Sleep mode (with retention). The clock source for the RTC has to be configured with a command before entering Duty Cycle mode.

Table 7-6: SetRxDutyCycle Command

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x02	0x14	RxPeriod (23:16)	RxPeriod (15:8)	RxPeriod (7:0)	Sleep Period (23:16)	Sleep Period (15:8)	Sleep Period (7:0)	Mode
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00

- *RxPeriod* defines the maximum RX window duration, expressed in periods of the 32.768kHz RTC. The maximum Delay value corresponds to 512s.
- *SleepPeriod* defines the duration of the Sleep period between the RX windows. It is expressed in periods of the 32.768kHz RTC. The maximum Delay value corresponds to 512s.
- *Mode* selects the device mode during the RX windows:
 - 0: Configures the device in RX mode during RX windows. Available for (G)FSK and LoRa packet types.
 - 1: Configures the device in CAD mode during RX windows. Available only for LoRa packet types. Returns CMD_FAIL for (G)FSK packet types.

The *Mode* parameter is optional, and is set to 0 if not sent.

When this command is sent in Standby mode, the context (device configuration) is saved and the device enters in a loop defined by the following steps, and depicted in [Figure 7-2: LR1120 Current Profile During RX Duty Cycle Operation](#).

1. The device enters RX and listens for an incoming RF packet for a period of time defined by *RxPeriod*.
2. Upon preamble detection, the timeout is stopped and restarted with the value $2 * RxPeriod + SleepPeriod$, as shown in [Figure 7-3: RX Duty Cycle Upon Preamble Detection](#).
3. If no packet is received during the RX window, the device goes into Sleep mode with context saved for a period of time defined by *SleepPeriod*.
4. At the end of the Sleep window, the device automatically restarts the process of restoring context and enters the RX mode, and so on. At any time, the host can stop the procedure.

The loop is terminated if either:

- A packet is detected during the RX window, at which moment the chip interrupts the host via the RX_DONE flag and returns to the configured Fallback mode (refer to [Section 7.2.5 "SetRxTxFallbackMode" on page 59](#)).
- The host issues a `SetStandby(...)` command during the RX window.
- The device is woken up from Sleep mode with a falling edge of NSS. In this case, the user should send the `SetStandby(...)` command to avoid race conditions if the NSS falling edge was issued during the boot phase.

If an `RxDutyCycle(...)` is started, *AutoRxTx* or *SetRxTxFallback* modes are not used.

`StopTimeoutOnPreamble(...)` has no effect on this mode.

Note: The `RxDutyCycle(...)` command returns CMD_FAIL in the next command status if the packet type has not been set.

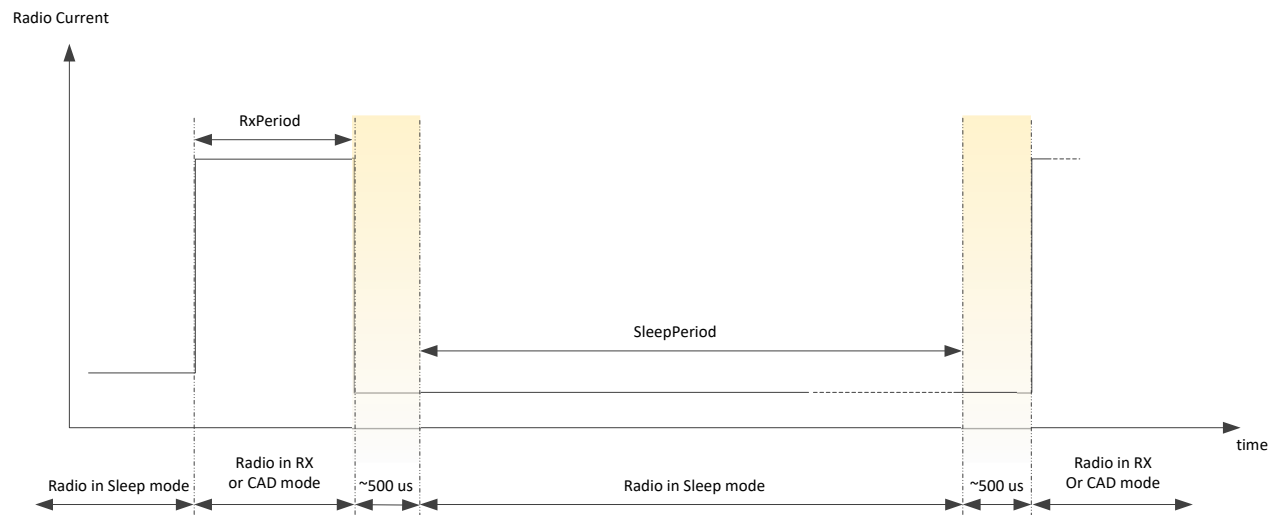


Figure 7-2: LR1120 Current Profile During RX Duty Cycle Operation

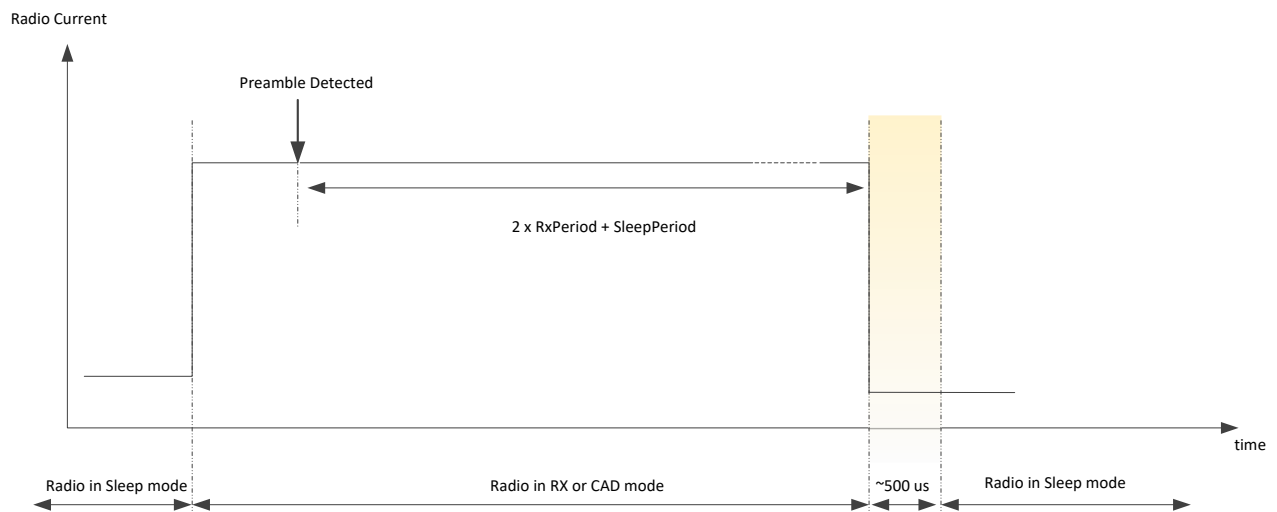


Figure 7-3: RX Duty Cycle Upon Preamble Detection

7.2.7 StopTimeoutOnPreamble

Command `StopTimeoutOnPreamble(...)` defines if the RX timeout should be stopped on Syncword / Header detection or on PreambleDetection.

Table 7-7: StopTimeoutOnPreamble Command

Byte	0	1	2
Data from Host	0x02	0x17	StopOnPreamble
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *StopOnPreamble* values:
 - ♦ 0x00: Stop on Syncword/Header detection (default value).
 - ♦ 0x01: Stop on Preamble detection.

7.2.8 GetRssiInst

Command `GetRssiInst(...)` returns the instantaneous RSSI value at the precise time when the command is sent. Therefore if no RF packet is present, the RSSI value returned by command `GetRssiInst(...)` corresponds to the RF noise.

Table 7-8: GetRssiInst Command

Byte	0	1
Data from Host	0x02	0x05
Data to Host	Stat1	Stat2

Table 7-9: GetRssiInst Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	Rssi

The RSSI is calculated using the following formula: $RSSI\ (dBm) = -Rssi/2$.

7.2.9 GetStats

Command `GetStats(...)` returns the internal statistics of the received RF packets:

Table 7-10: GetStats Command

Byte	0	1
Data from Host	0x02	0x01
Data to Host	Stat1	Stat2

Table 7-11: GetStats Response

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	NbPkt Received (15:8)	NbPkt Received (7:0)	NbPkt CrcError (15:8)	NbPkt CrcError (7:0)	Data1 (15:8)	Data1 (7:0)	Data2 (15:8)	Data2 (7:0)

- *NbPktReceived* is the total number of received packets.
- *NbPktCrcError* is the total number of received packets with a CRC error.
- *Data1* is PacketType dependant:
 - ♦ (G)FSK mode: *Data1* = *NbPacketLengthError(15:0)*: number of packet with a length error.
 - ♦ LoRa mode: *Data1* = *NbPktHeaderErr(15:0)*: number of packets with a Header error.
- *Data2* is PacketType dependant:
 - ♦ (G)FSK mode: *Data2* = 0x00.
 - ♦ LoRa mode: *Data2* = *NbPktFalseSync(15:0)*: number of false synchronizations.

Statistics are reset on a Power On Reset, power down, or by command `ResetStats(...)`.

7.2.10 ResetStats

Command `ResetStats(...)` resets the internal statistics of the received RF packets:

Table 7-12: ResetStats Command

Byte	0	1
Data from Host	0x02	0x00
Data to Host	Stat1	Stat2

7.2.11 GetRxBufferStatus

Command `GetRxBufferStatus(...)` returns the length of the last packet received and the offset in the RX buffer of the first byte received:

Table 7-13: GetRxBufferStatus Command

Byte	0	1
Data from Host	0x02	0x03
Data to Host	Stat1	Stat2

Table 7-14: GetRxBufferStatus Response

Byte	0	1	2
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	PayloadLengthRX	RxStartBufferPointer

- *PayloadLengthRX* is the Payload length of the last packet received, in bytes.
- *RxStartBufferPointer* is the offset in the RX buffer of the first byte received.

7.2.12 SetRxBoosted

Command `SetRxBoosted(...)` sets the device in RX Boosted mode, allowing a ~2dB increased sensitivity, at the expense of a ~2mA higher current consumption in RX mode.

Table 7-15: SetRxBoosted Command

Byte	0	1	2
Data from Host	0x02	0x27	RxBoosted
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *RxBoosted*: Sets the Rx Boosted mode.
 - ♦ 0: RX Boosted mode deactivated.
 - ♦ 1: RX Boosted mode activated.
 - ♦ Other values are RFU.

7.2.13 SetLoRaSyncWord

This command sets the SetLoRaSyncWord.

Table 7-16: SetLoRaSyncWord Command

Byte	0	1	2
Data from Host	0x02	0x2B	Syncword
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *Syncword*: Sets the SetLoRaSyncWord. Valid for all SFs. Example values are:
 - ♦ 0x12: Private Network (default).
 - ♦ 0x34: Public Network.

7.2.14 GetLoRaRxHeaderInfos

Command `GetLoRaRxHeaderInfos(...)` returns the information coded in the last received packet header (explicit header mode), or the configured *coding_rate* and *crc_type* settings:

Table 7-17: GetLoRaRxHeaderInfos Command

Byte	0	1
Data from Host	0x02	0x30
Data to Host	Stat1	Stat2

Table 7-18: GetLoRaRxHeaderInfos Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	Infos

- *Infos*:
 - ♦ bits 7-5: 0 RFU
 - ♦ bits 4: 1 = CRC ON, 0 = CRC OFF
 - ♦ bit 3: 0 RFU
 - ♦ bits 2-0: coding rate (see [8.3.1 SetModulationParams](#) command for values)

7.2.15 SetRssiCalibration

Command `SetRssiCalibration(...)` sets the gain offset for the on-chip power estimation. The LR1120 internal LNA has a set of predefined gains (G4, G5, ... G13 (split into sub-gains G13 hp1, G13hp2, ... G13 hp7)), used to amplify the RF power at the correct level, depending on the RF input power. In normal usage of the chip, the LNA gain control is automatic.

The power seen by the LR1120 analog front-end is affected by external components such as the matching network, or RF switches.

- An incorrect RSSI results in a sensitivity degradation in (G)FSK mode and an incorrect gain selection in LoRa and GFSK mode.
- An incorrect gain can result in a missed detection (packet loss) or decreased resistance to interference.

By default, the chip is calibrated for the 868-915MHz band on the LR1120 EVK. Therefore, in order to have a correct reading of the RSSI value of the chip integrated in the application PCB, the RSSI must be calibrated. The RSSI must be calibrated for each hardware type, but not for each device individually: the RSSI offset must be determined once during the hardware debug or evaluation phase, and then applied to all the devices sharing exactly the same hardware (identical schematic, layout and BOM).

Table 7-19: SetRssiCalibration Command

Byte	0	1	2	3	4	5	6
Data from Host	0x02	0x29	Tune G5(7:4) G4(3:0)	Tune G7(7:4) G6(3:0)	Tune G9(7:4) G8(3:0)	Tune G11(7:4) G10(3:0)	Tune G13(7:4) G12(3:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0

Byte	7	8	9	10	11	12
Data from Host	Tune G13hp2(7:4) G13hp1(3:0)	Tune G13hp4(7:4) G13hp3(3:0)	Tune G13hp6(7:4) G13hp5(3:0)	Tune G13hp7(3:0)	GainOffset (15:8)	GainOffset (7:0)
Data to Host	0	0				

- `TuneGx(...)` (arguments 2 to 10): RSSI Gain Tune values. A tune is a 4-bit signed value, where 1lsb = 0.5dB.

Table 7-20: Gain Tune Values

Tune	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Value (dBm)	0	0.5	1	1.5	2	2.5	3	3.5	-4	-3.5	-3	-2.5	-2	-1.5	-1	-0.5

- *GainOffset*: Global offset added to the Gain Tune values. The offset is a 12-bit signed value, where 1lsb = 0.5dB.

Table 7-21: Recommended Values for Reference EVK

Frequency	Gain Offset	G4 Tune	G5 Tune	G6 Tune	G7 Tune	G8 Tune	G9 Tune	G10 Tune	G11 Tune	G12 Tune	G13 Tune	G13hp1 Tune	G13hp2 Tune	G13hp3 Tune	G13hp4 Tune	G13hp5 Tune	G13hp6 Tune	G13hp7 Tune
Below 600MHz	0	12	12	14	0	1	3	4	4	3	6	6	6	6	6	6	6	6
From 600MHz to 2GHz	0	2	2	2	3	3	4	5	4	4	6	5	5	6	6	6	7	6
Above 2GHz	2030	6	7	6	4	3	4	14	12	14	12	12	12	12	8	8	9	9

Gain Tune and Gain Offset values can be determined using an RF generator with the LR1120 hardware implementation as follows:

1. Setup the device in GFSK mode and set the desired RF frequency.
2. Setup the device in manual gain mode (disable the AGC (Auto Gain Control)).
3. Use SetRssiCalibration API to set all tunes and offsets to 0.
4. Loop through all the different gains, including LNA boost mode (high power mode):
 - a. Use the generator to emit a continuous tone at a specific output power `gen_pwr` (gain dependant), and account for all cable losses. A direct connection is recommended.
 - b. Read the instantaneous RSSI (`rssi_inst`).
 - c. Calculate the `rssi_error = rssi_inst - gen_pwr`.
 - d. Log the `rssi_error` in a file.
5. Find the common offset between all errors, and use it as the new global offset.
6. Find the individual errors (relative to the new global offset) and use them as the tunes.
7. Call SetRssiCalibration(), and go through the main loop again to check that the instantaneous RSSI is now calibrated.

Some pseudo-code can be found hereafter:

```
uint8_t gain[] = { 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 }; // NB: for
the 2G4 path, the max gain is 16 - 17-20 can be ignored.

float power[] = { -35.0, -41.0, -45.0, -50.0, -53.5, -60.0, -65.0, -69.5, -75.0, -81.0,
-82.5, -83.5, -84.0, -85.0, -86.0, -86.5, -87.0 };

/* Configure the chip at the system level */
lr11xx_system_reset( context );
lr11xx_system_set_reg_mode( context, reg_mode );
lr11xx_system_set_dio_as_rf_switch( context, rf_switch_cfg );
lr11xx_system_set_tcxo_mode( context, tune, timeout ); // Optional - only if there is a TCXO
lr11xx_system_clear_errors( context );
lr11xx_system_calibrate( context, 0x3F ); // 0x3F to enable all fields

/* Configure the chip at the modem level */
lr11xx_radio_set_pkt_type( context, LR11XX_RADIO_PKT_TYPE_GFSK );
lr11xx_radio_set_rf_freq( context, freq_in_hz );

lr11xx_system_calibrate_image_in_mhz( context, freq1_in_mhz, freq2_in_mhz );

lr11xx_radio_set_gfsk_mod_params( context, mod_params ); // Rx BW has to be set to
LR11XX_RADIO_GFSK_BW_234300 - other modulation parameters can be anything
lr11xx_radio_set_gfsk_pkt_params( context, pkt_params ); // Packet parameters can be anything

/* Configure the chip to be controlled manually */
lr11xx_regmem_write_regmem32_mask( context, 0x00F20214, 0x00080000, 0x00080000 );
lr11xx_regmem_write_regmem32_mask( context, 0x00F20230, 0x71110000, 0x71100000 );

lr11xx_radio_set_rssi_calibration( context, rssi_cal_table ); // All parameters of rssi_cal_table
set to 0
lr11xx_radio_set_rx_with_timeout_in_rtc_step( context, 0xFFFFF );
```

```

for( int i = 0; i++; i < 17 ) // 17 is the number of elements in gain array
{
    const uint8_t gain_step = MIN( gain[i], 13 );
    const uint8_t lna_boost = ( gain > 13 ) ? gain - 13 : 0;

    lr11xx_regmem_write_regmem32_mask( context, 0x00F20214, 0x00F00000, gain_step << 20 );
    lr11xx_regmem_write_regmem32_mask( context, 0x00F3008C, 0x00070000, lna_boost << 16 );

/* Wait for 1 ms */

/* Insert here a control for your test equipment to generate a tone at RF frequency set to freq_in_hz
with an output power set to power[i] dBm */

    lr11xx_radio_get_rssi_inst( context, rssi_in_dbm );

/* Add a way to log (gain[i], power[i], rssi_in_dbm) triplet to be able to compute offset and tunes for
the RSSI calibration */
}

```


8. Modems

8.1 Modem Configuration

The LR1120 contains different modems capable of handling different constant envelope modulations. For LoRa®/(G)FSK modems, LR-FHSS and Sigfox modulations, Bluetooth® Low Energy beaconing compatibility, and Ranging by Round Trip Time Of Flight (RTTof), the suitable command order is the following:

- The user must specify the modem to be used in command `SetPacketType(...)`
- `SetModulationParams(...)` configures the modem parameters (SF, BW, CR and LDRO)
- `SetPacketParams(...)` defines the RF packet parameters (Payload length, Implicit/explicit mode, ...). This command is not needed for LR-FHSS
- `SetPaConfig(...)` configures the PA settings used for RF packet transmission (which PA, supply mode...)
- `SetTxParams(...)` defines the PA parameters (output power, ramp time).

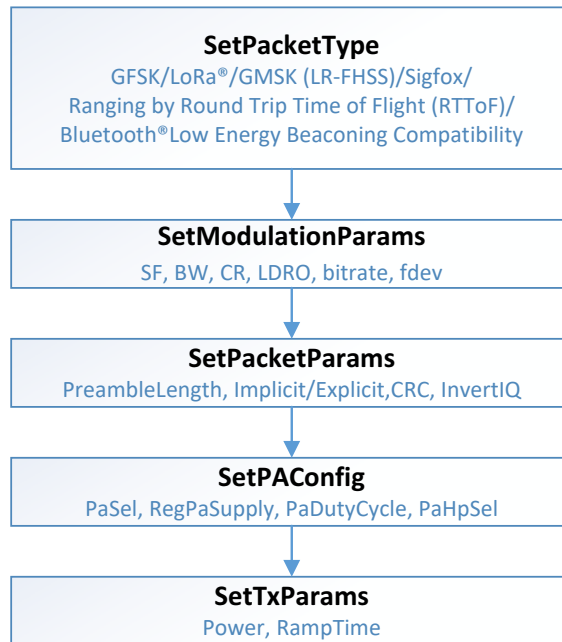


Figure 8-1: Modem Command Order

8.1.1 SetPacketType

The `SetPacketType(...)` command defines which modem is to be used.

Table 8-1: SetPacketType Command

Byte	0	1	2
Data from Host	0x02	0x0E	PacketType
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *PacketType* defines the modem to be used for the next RF transactions:
 - ♦ 0x00: None (default)
 - ♦ 0x01: (G)FSK
 - ♦ 0x02: LoRa
 - ♦ 0x03: Sigfox Uplink¹
 - ♦ 0x04: GMSK (LR-FHSS)
 - ♦ 0x05: Ranging
 - ♦ 0x06: Bluetooth Low Energy beaconing compatible
 - ♦ Other values are RFU

This command is the first one to be called before going to RX or TX and before defining modulation and packet parameters. It only works with the device in Standby RC, Standby Xosc or Fs mode, otherwise it returns CMD_FAIL in the status of the next command.

8.1.2 GetPacketType

Command `GetPacketType(...)` returns the current protocol of the radio.

Table 8-2: GetPacketType Command

Byte	0	1
Data from Host	0x02	0x02
Data to Host	Stat1	Stat2

Table 8-3: GetPacketType Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	PacketType

- *PacketType* corresponds to the modem used for the following RF transactions:
 - ♦ 0: None
 - ♦ 1: (G)FSK
 - ♦ 2: LoRa
 - ♦ 3: Sigfox Uplink¹
 - ♦ 4: LR-FHSS
 - ♦ 5: Ranging
 - ♦ 6: Bluetooth Low Energy beaconing compatible
 - ♦ Other values are RFU

1. Sigfox downlink is performed using *PacketType* 0x01. See AN1200.73 for details.

8.2 LoRa® Modem

8.2.1 LoRa Modulation Principle

The LoRa modem uses a proprietary spread spectrum modulation, which permits an increased link budget and increased immunity to in-band interference compared to legacy modulation techniques. It has the capability to receive signals with negative SNR that increases sensitivity as well as link budget and range of the LoRa receiver.

8.2.1.1 Spreading Factor (SF)

The spread spectrum LoRa modulation is performed by representing each bit of payload information by multiple chips of information. The rate at which a spread symbol (containing 2^{SF} chips) is sent is referred to as the symbol rate (R_s). The ratio between the nominal symbol rate and chip rate is the Spreading Factor and it defines the number of bits sent per symbol.

Note: The spreading factor must be known in advance on both transmit and receive sides of the link as different spreading factors are orthogonal to each other.

8.2.1.2 LoRa Bandwidth (BWL)

The LoRa modem operates at a programmable bandwidth (BWL) around a programmable central frequency f_{RF} . The LoRa modem bandwidth always refers to the double side band (DSB), as shown in [Figure 8-2: LoRa Signal Bandwidth](#).

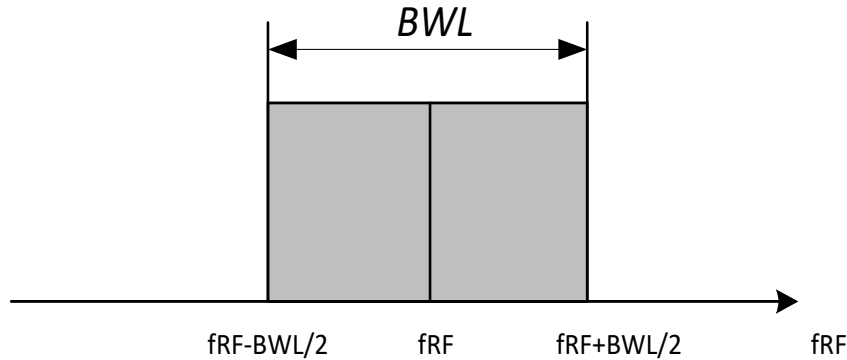


Figure 8-2: LoRa Signal Bandwidth

An increase in signal bandwidth permits the use of a higher effective data rate, thus reducing transmission time at the expense of reduced sensitivity.

Note: Most countries have regulatory constraints on the permissible occupied bandwidth that only allow usage of a subset of BWL.

8.2.1.3 Coding Rate (CR)

To further improve the robustness of the link, the LoRa modem employs cyclic error coding to perform forward error detection and correction. Such error coding incurs a transmission overhead.

8.2.1.4 Low Data Rate Optimization (LDRO)

LDRO increases the robustness of the LoRa link at low effective data rates, improving the sensitivity level and increasing the robustness towards frequency drift and Doppler events. Its use is mandated with spreading factors of 11 and 12 at 125kHz bandwidth, and SF12 at 250kHz BW.

8.2.1.5 LoRa Symbol Rate (R_s)

With a knowledge of the key parameters that can be controlled by the user we define the LoRa symbol rate as:

$$R_s = \frac{BWL}{2^{SF}}$$

where BWL is the programmed bandwidth and SF is the spreading factor. The transmitted signal is a constant envelope signal. Equivalently, one chip is sent per second per Hz of bandwidth.

8.2.2 LoRa Packet Format

The LoRa modem employs two packet formats: explicit and implicit. The explicit packet contains additional information, it includes a short header that contains information about the number of bytes, coding rate and whether a CRC is used in the packet.

In certain scenarios, where the payload, coding rate and CRC presence are fixed or known in advance, it may be advantageous to reduce transmission time by invoking implicit header mode. In this mode the header is removed from the packet. In this case the payload length, error coding rate and presence of the payload CRC must be manually configured identically on both sides of the radio link.

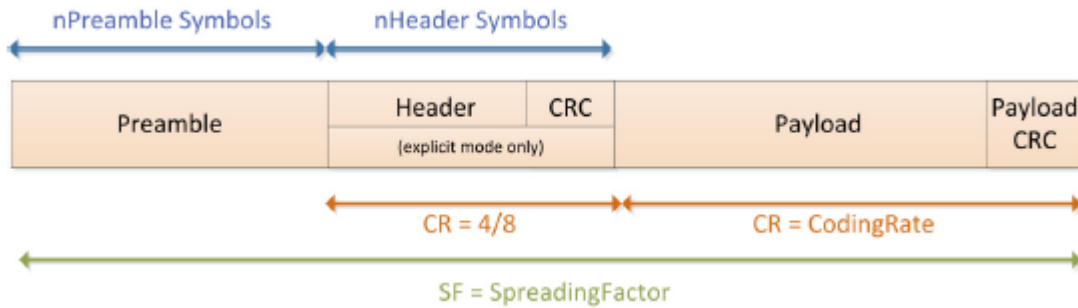


Figure 8-3: LoRa Packet Format

8.2.2.1 Preamble

The LoRa packet starts with a preamble sequence, used to synchronize the receiver with the incoming signal. The transmitted preamble length may vary from 1 to 65535 symbols. This permits the transmission of near arbitrarily long preamble sequences. In order to optimize the packet reception, it is advised to use a minimum preamble length of 12 with SF5 and SF6, and of 8 for other SF.

The receiver undertakes a preamble detection process that periodically restarts. For this reason the preamble length should be configured as identical to the transmitter preamble length. Where the preamble length is not known, or can vary, the maximum preamble length should be programmed on the receiver side.

8.2.2.2 Header (explicit packets only)

The header provides payload information:

- The payload length in bytes
- The forward error correction coding rate
- An optional 16-bit CRC for the payload

The header is transmitted with maximum error correction code (4/8). It also has its own CRC to allow the receiver to discard invalid headers.

8.2.2.3 Payload

The packet payload is a variable-length field that contains the actual data coded at a rate specified in the header in explicit mode or in the register settings in implicit mode. An optional CRC may be appended.

8.2.2.4 LoRa Packet Time On Air

The Time On Air of the LoRa packet is shown in the LR11xx drivers.

8.2.3 Channel Activity Detection (CAD)

Used only in LoRa packets, the Channel Activity Detection (CAD) is a LoRa specific mode of operation where the device searches for the presence of a LoRa preamble signal.

At the end of the search period, the device triggers the IRQ CADdone. If a valid signal has been detected it also generates the IRQ CadDetected. A minimum of 2 symbols is recommended to perform a CAD.

After the search has completed, the device returns to STDBY_RC mode. The length of the search is configured via command `SetCadParams(...)`.

8.3 LoRa Commands

8.3.1 SetModulationParams

Command `SetModulationParams(...)` configures the modulation parameters for the selected modem. Since the parameters are modem dependent, the description hereafter is valid only for the LoRa modem.

Table 8-4: SetModulationParams Command

Byte	0	1	2	3	4	5
Data from Host	0x02	0x0F	SF	BWL	CR	LowDataRateOptimize
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:16)	IrqStatus(15:8)	IrqStatus (7:0)

- *SF* defines the spreading factor (values other than those below are RFU). SF5 and SF6 are compatible with the SX126x device family. SF6 can be made compatible with the SX127x family in implicit mode via a register setting¹.
 - ♦ 0x05: SF5
 - ♦ 0x06: SF6
 - ♦ 0x07: SF7
 - ♦ 0x08: SF8
 - ♦ 0x09: SF9
 - ♦ 0x0A: SF10
 - ♦ 0x0B: SF11
 - ♦ 0x0C: SF12
- *BWL* defines the LoRa modulation bandwidth (values other than those below are RFU):
 - ♦ 0x03: LoRa_BW_62, LoRa Bandwidth 62.5kHz
 - ♦ 0x04: LoRa_BW_125, LoRa Bandwidth 125kHz
 - ♦ 0x05: LoRa_BW_250, LoRa Bandwidth 250kHz
 - ♦ 0x06: LoRa_BW_500, LoRa Bandwidth 500kHz
 - ♦ 0x0D: BW203 (2.4GHz band only)
 - ♦ 0x0E: BW406 (2.4GHz band only)
 - ♦ 0x0F: BW812 (2.4GHz band only)
- *CR* configures the Coding Rate. Long interleaver increases robustness against interferers. (values other than those below are RFU):
 - ♦ 0x01: Short Interleaver CR = 4/5 Overhead Ratio 1.25
 - ♦ 0x02: Short Interleaver CR = 4/6 Overhead Ratio 1.5
 - ♦ 0x03: Short Interleaver CR = 4/7 Overhead Ratio 1.75
 - ♦ 0x04: Short Interleaver CR = 4/8 Overhead Ratio 2
 - ♦ 0x05: Long Interleaver CR = 4/5² Overhead Ratio 1.25
 - ♦ 0x06: Long Interleaver CR = 4/6² Overhead Ratio 1.5
 - ♦ 0x07: Long Interleaver CR = 4/8² Overhead Ratio 2
- *LowDataRateOptimize* reduces the number of bits per symbol:
 - ♦ 0x00: LowDataRateOptimize off
 - ♦ 0x01: LowDataRateOptimize on

1.The LR1120 must set bit 18 to 1 and bit 23 to 0 of the register at address 0xF20414.

2.Long Interleaver (CR = 4/5, 4/6 and 4/8) is supported for packets of minimum Payload length of 8 bytes, and maximum Payload length of 253 bytes if the CRC is activated (255 bytes if the CRC is deactivated).

8.3.2 SetPacketParams

Command `SetPacketParams(...)` configures the parameters of the RF packet for the selected modem. Since the parameters are modem dependent, the description hereafter is valid only for the LoRa modem.

Table 8-5: SetPacketParams Command

Byte	0	1	2	3	4	5	6	7
Data from Host	0x02	0x10	PbLengthTX (15:8)	PbLengthTX (7:0)	HeaderType	PayloadLen	CRC	InvertIQ
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00

- *PbLengthTX* defines the length of the LoRa packet preamble.
 - ♦ Coded on 2 bytes, from 0x0001 (1) to 0xFFFF (65535). Minimum of 12 with SF5 and SF6, and of 8 for other SF advised.
- *HeaderType* defines if the header is explicit or implicit:
 - ♦ 0x00: Explicit header (default)
 - ♦ 0x01: Implicit header
- *PayloadLen* defines the size of the payload (bytes) to transmit or the maximum size of the payload that the receiver can accept.
 - ♦ In explicit header mode:
 - 0: Reception of any payload length between 0 and 255 bytes allowed.
 - N: Reception of any payload length between 1 and N bytes accepted. Payload lengths of 0 or > N are rejected and result in a HeaderErr IRQ.
 - ♦ In implicit header mode, *PayloadLen* configures the exact length of the payload to be transmitted or received.
- *CRC* defines if the CRC is OFF or ON:
 - ♦ 0x00: OFF
 - ♦ 0x01: ON
- *InvertIQ* defines if the I and Q signals are inverted.
 - ♦ 0x00: Not inverted
 - ♦ 0x01: Inverted

This command fails if no packet type has been set.

8.3.3 SetCad

Command `SetCad(...)` activates the CAD feature.

Table 8-6: SetCad Command

Byte	0	1
Data from Host	0x02	0x18
Data to Host	Stat1	Stat2

8.3.4 SetCadParams

Command `SetCadParams(...)` defines the LoRa CAD parameters.

Table 8-7: SetCadParams Command

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x02	0x0D	Symbol Num	DetPeak	DetMin	CadExit Mode	Timeout (23:16)	Timeout (15:8)	Timeout (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00

- *SymbolNum* defines the number of symbols used for the CAD detection.
- *DetPeak* and *DetMin* define the sensitivity of the LoRa modem when trying to correlate to actual LoRa preamble symbols. These two settings depend on the LoRa spreading factor, the Bandwidth, and the number of symbols used to validate the detection. Choosing the right value must be carefully tested to ensure a good detection at sensitivity level, and also to limit the number of false detections.
- *CadExitMode* defines the action to be performed after a CAD operation.

Table 8-8: CadExitMode Parameter

Value	CadExitMode	Operation
0x00	CAD_ONLY	The chip performs a CAD operation in LoRa. Once done and whatever the activity on the channel, the device goes back to STBY_RC mode.
0x01	CAD_RX	The device performs a CAD operation and if an activity is detected, it stays in RX until a packet is detected or the timer reaches the timeout defined by <i>Timeout</i> * 31.25us
0x10	CAD_LBT	The device performs a CAD operation and if no activity is detected, it goes into TX mode with the defined <i>Timeout</i> as timeout parameter.

- *Timeout* is only used when the CAD is performed with *cadExitMode* = CAD_RX or CAD_LBT.
 - ♦ If *cadExitMode* = CAD_RX, see [7.2.2 SetRx](#) for *Timeout* definition
 - ♦ If *cadExitMode* = CAD_LBT, see [7.2.3 SetTx](#) for *Timeout* definition

8.3.5 SetLoRaSynchTimeout

Command `SetLoRaSynchTimeout(...)` configures the LoRa modem to issue an RX timeout after exactly *SymbolNum* symbols if no packet was detected by then.

Table 8-9: SetLoRaSynchTimeout Command

Byte	0	1	2
Data from Host	0x02	0x1B	SymbolNum
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *SymbolNum*: 0x00: No timeout (default value)

8.3.6 SetLoRaPublicNetwork

Command `SetLoRaPublicNetwork(...)` sets the LoRa modem Syncword to public or private.

Table 8-10: SetLoRaPublicNetwork Command

Byte	0	1	2
Data from Host	0x02	0x08	PublicNetwork
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *PublicNetwork:*
 - ♦ 0x00: Private network (default)
 - ♦ 0x01: Public network
 - ♦ Other values are RFU

8.3.7 GetPacketStatus

Command `GetPacketStatus(...)` gets the status of the last received packet. Since the returned values are modem dependent, the description hereafter is valid only for the LoRa modem.

Table 8-11: GetPacketStatus Command

Byte	0	1
Data from Host	0x02	0x04
Data to Host	Stat1	Stat2

Table 8-12: GetPacketStatus Response

Byte	0	1	2	3
Data from Host	0x00	0x00	0x00	0x00
Data to Host	Stat1	RssiPkt	SnrPkt	SignalRssiPkt

- *RssiPkt* defines the average RSSI over the last packet received. RSSI value in dBm is $-RssiPkt/2$.
- *SnrPkt* is an estimation of SNR on last packet received. Expressed in two's complement format multiplied by 4. Actual SNR in dB is $SnrPkt/4$. The SNR value is an unbiased scaled value.
If the division by 4 is performed in floating-point, standard floating-point rounding provides the correct value.
If the division by 4 is performed in fixed point, it is recommended to round towards the nearest integer. In our drivers coded in C, this can be done by first adding 2, and then performing a fixed-point truncated division by 4.

```
pkt_status->snr_pkt_in_db = ( ( ( int8_t ) rbuffer[1] ) + 2 ) >> 2;
```
- *SignalRssiPkt* is an estimation of RSSI of the LoRa signal (after despreading) on last packet received, in two's complement format [negated, dBm, fixdt(0,8,1)]. Actual RSSI in dB is $-SignalRssiPkt/2$.

Additional information on the RSSI can be found in section [Section 8.10 "RSSI Functionality" on page 92](#).

8.4 (G)FSK Modem

8.4.1 (G)FSK Modulation Principle

The (G)FSK modem can transmit and receive 2-FSK modulated packets over data rates ranging from 0.6kbps to 300kbps.

Both the bit rate (*Bitrate*) and frequency deviation (*Fdev*) are directly configured using command *SetModulationParams()*.

Additionally, in transmission mode, several shaping filters can be applied to the signal in packet mode or in continuous mode. In reception mode, the user needs to select the best reception bandwidth depending on its conditions. To ensure correct demodulation, the following limit must be respected for the bandwidth:

$$(2 \times Fdev + BR) < BWF$$

where the bandwidth BWF ranges from 4.8kHz to 467kHz.

The bandwidth must be chosen so that:

$$\text{Bandwidth [DSB]} \geq BR + 2 \times \text{frequency deviation} + \text{frequency error}$$

where the frequency error is twice the frequency error of the crystal oscillator used.

8.4.2 (G)FSK Packet Engine

The LR1120 is designed for packet-based communication. The packet controller only works in half-duplex mode i.e. either in transmit or receive at a time. It is configured using command *SetPacketParams(...)*. This function can be called only after defining the protocol.

8.4.2.1 Reception

The packet controller block is responsible for assembling received data bit-streams into packets and storing them in the data buffer. It also performs bit-stream decoding operations such as de-whitening & CRC-checks on the received bit-stream.

Preamble Detection in Receiver Mode

The LR1120 can gate the reception of a packet if an insufficient number of alternating preamble symbols (usually referred to 0x55 or 0xAA in hexadecimal form) has been detected. The parameter *PbIDetect* in command *SetPacketParams(...)* defines the preamble detector length and allows the user to select a value ranging from:

- Off: where the radio performs no gating and locks directly on the following Syncword, to
- 32 bits: where the radio expects to receive 32 bits of preamble before the following Syncword. In this case, if the 32 bits of preamble are not detected, the radio either drops the reception in RxSingle mode, or restarts its tracking loop in RxContinuous mode.

To achieve the best performance, it is recommended to set *PbIDetect* to 8 or 16 bits, depending on the complete size of preamble sent by the transmitter.

Note: To detect packets correctly, *PbIDetect* must always be smaller than the size of the following Syncword. If *PbIDetect* is greater than the following Syncword length (typically when no Syncword is used) the user should fill some of the Syncword bytes with 0x55.

8.4.2.2 Transmission

On the transmit side, the packet handler can construct a packet and send it bit by bit to the modulator for transmission. It can whiten the payload and append the CRC-checksum to the end of the packet.

8.4.3 (G)FSK Packet Format

The (G)FSK packet format provides a conventional packet format for application in proprietary NRZ coded, low energy communication links. The packet format has built in facilities for CRC checking of the payload, dynamic payload size and packet acknowledgement. Whitening based upon pseudo random number generation can be enabled. Two principle packet formats are available in the (G)FSK protocol: fixed-length and variable-length.

8.4.3.1 Fixed-Length Packet

If the packet length is fixed and known on both sides of the link then knowledge of the packet length does not need to be transmitted over the air. Instead the packet length can be written to the parameter *PacketLength* which determines the packet length in bytes:

- 0 to 255 if address filtering is not activated
- 0 to 254 if address filtering is activated

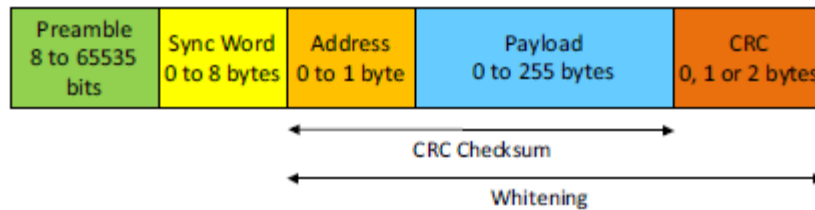


Figure 8-4: Fixed-Length Packet

It is usually recommended to use a minimum of 16 bits for the preamble to guarantee a valid reception of the packet on the receiver side. The CRC operation, packet length and preamble length are defined in command *SetPacketParams(...)*.

8.4.3.2 Variable-Length Packet

This field encodes the payload length in bytes.

If the packet is of uncertain or variable size, information about the payload length must be transmitted within the packet.

The format of the variable-length packet is shown in the figure below:

- the packet length is 0 to 254 bytes if address filtering is activated
- the Length field is 1 byte long, except for SX128x compatibility, when it is 9 bits (see 8.4.4 SX128x Compatibility)

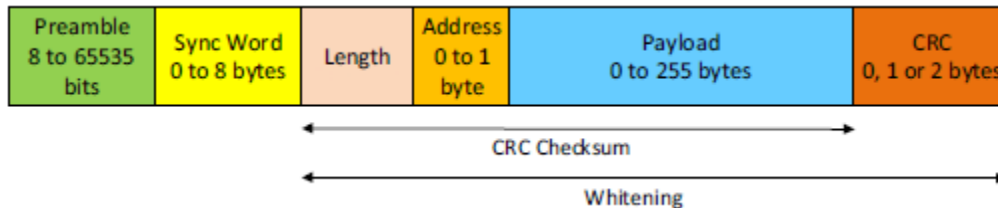


Figure 8-5: Variable-Length Packet

8.4.3.3 Setting The Packet Length Or Node Address

The packet length and Node or Broadcast address are not considered part of the payload and are added automatically in hardware. The packet length is added automatically in the packet when the *PacketType* field is set to variable size in command *SetPacketParams(...)*.

The node or broadcast address can be enabled by the *AddrComp* field in command *SetPacketParams(...)*. This field allows the user to enable and select an additional packet filtering at the payload level.

8.4.3.4 Whitening

The whitening is based around the 9-bit LFSR polynomial $x^9 + x^5 + 1$ for sub-GHz, and $x^7 + x^4 + 1$ for HF, which generates a random sequence. With this structure, the LSB at the output of the LFSR is XORed with the MSB of the data. At the initial stage, command *SetGfskWhitParams(...)* sets the whitening Seed.

The payload (including the payload length, the Node or Broadcast address and CRC checksum when needed) is then XORed with this random sequence to generate the whitened payload.

The data is de-whitened on the receiver side by XORing with the same random sequence. This process limits the number of consecutive 1's or 0's to 9.

Note that data whitening is only required when the user data has high correlation with long strings of 0's and 1's. If the data is already random then the whitening is not required.

8.4.3.5 CRC

The LR1120 offers full flexibility to select the CRC polynomial and initial value of the selected polynomial. The user can also select a complete inversion of the computed CRC to comply with some international standards.

The CRC can be enabled and configured in the *CrcType* field in command *SetPacketParams(...)*. This field allows the user to enable and select the length and configuration of the CRC.

Command *SetGfskCrcParams(...)* configures the CRC polynomial and initial value.

8.4.4 SX128x Compatibility

The following limitations apply when using the SX128x 9-bit header *PacketType* 0x02:

- Payload length: payload length is limited to max. 254 - CrcLength (Ex: 252 bytes with a 16-bit crc)
- CRC length: 0,1 or 2 bytes
- Syncword length: only allowed syncword length values are 8,16,24, 32 or 40 bits
- AddrComp: has to be disabled (feature not present on SX128x)
- DcFree has to be either disabled (0x00) or SX128x compatible (0x03). 0x01 is not accepted

In order to make the LR1120 air-compatible with the SX128x:

- Set the whitening seed to 0x0001 with the *SetGfskWhitParams* command (default SX128x value)
- *SetGfskCrcParams*: (default SX128x values):
 - ♦ 1 byte CRC: seed = 0xFF, poly = 0x07
 - ♦ 2 byte CRC: seed = 0xFFFF, poly = 0x1021
- Note that:
 - ♦ for transmitted frames, 7 additional bits are transmitted
 - ♦ for received frames 23 additional bits are received (delaying the RX_DONE IRQ).

8.4.5 Special Settings for BRF= 0.6kbps and 1.2kbps

Additional radio settings are necessary for bit rates 0.6kbps and 1.2kbps, specifically:

- BRF= 0.6kbps: Write 0 inside bit 0 at address 0x08b8 (read-modify-write instruction).
- BRF= 1.2kbps,: Write the following bits (read-modify-write):
 - ◆bits 4:3 @ 0x06D1 to 0x03
 - ◆bits 4:2 @ 0x089B to 1
 - ◆bit 4 @0x08B8 to 0
 - ◆bit 6 @0x06AC to 1
 - ◆bit 5 @ 0x06AC to 0
 - ◆bit 4 @ 0x06AC to 1

The user can read the default values of registers 0x6D1, 0x89B, 0x6AC, 0x8B8 at any time, and use those values to restore these registers in case of need. These registers are not restored to their default value by the commands SetPacketParams and SetModulationParams.

8.5 (G)FSK Commands

8.5.1 SetModulationParams

Command `SetModulationParams(...)` configures the modulation parameters for the selected modem. Since the parameters are modem dependent, the description hereafter is valid only for the (G)FSK modem.

Table 8-13: SetModulationParams Command

Byte	0	1	2	3	4	5	6	7	8	9	10	11
Data from Host	0x02	0x0F	Bitrate (31:24)	Bitrate (23:16)	Bitrate (15:8)	Bitrate (7:0)	Pulse Shape	BWF	Fdev (31:24)	Fdev (23:16)	Fdev (15:8)	Fdev (7:0)
Data to Host	Stat1	Stat2	Irq Status (31:24)	Irq Status (23:16)	Irq Status (15:8)	Irq Status (7:0)	0x00	0x00	0x00	0x00	0x00	0x00

- *BitRate*: (G)FSK bit rate interpretation depends on the *BitRate*(31). It is either bits/s or the 8 LSBs are a fractional part:
 - ♦ *BitRate*(31) = 0: GFSK Bitrate in bits/s is *BitRate*(30:0)
 - ♦ *BitRate*(31) = 1: GFSK Bitrate in bits/s is *BitRate*(30:0)/256
- *PulseShape* defines the filtering applied to the (G)FSK packet:
 - ♦ 0x00: No filter applied
 - ♦ 0x08: Gaussian BT 0.3
 - ♦ 0x09: Gaussian BT 0.5
 - ♦ 0x0A: Gaussian BT 0.7
 - ♦ 0x0B: Gaussian BT 1
 - ♦ 0x16: Raise Cosine BPSK BT 0.7
- *BWF* defines the bandwidth

Table 8-14: Bandwidth Parameter

BWF	Description	BWF	Description
0x1F	RX_BW_4800 (4.8kHz DSB)	0x0C	RX_BW_58600 (58.6kHz DSB)
0x17	RX_BW_5800 (5.8kHz DSB)	0x1B	RX_BW_78200 (78.2kHz DSB)
0x0F	RX_BW_7300 (7.3kHz DSB)	0x13	RX_BW_93800 (93.8 kHz DSB)
0x1E	RX_BW_9700 (9.7kHz DSB)	0x0B	RX_BW_117300 (117.3kHz DSB)
0x16	RX_BW_11700 (11.7kHz DSB)	0x1A	RX_BW_156200 (156.2kHz DSB)
0x0E	RX_BW_14600 (14.6kHz DSB)	0x12	RX_BW_187200 (187.2kHz DSB)
0x1D	RX_BW_19500 (19.5kHz DSB)	0x0A	RX_BW_234300 (234.3kHz DSB)
0x15	RX_BW_23400 (23.4kHz DSB)	0x19	RX_BW_312000 (312kHz DSB)
0x0D	RX_BW_29300 (29.3kHz DSB)	0x11	RX_BW_373600 (373.6kHz DSB)
0x1C	RX_BW_39000 (39kHz DSB)	0x09	RX_BW_467000 (467kHz DSB)
0x14	RX_BW_46900 (46.9kHz DSB)	-	-

- *Fdev* defines the frequency deviation (Hz)

8.5.2 SetPacketParams

Command `SetPacketParams(...)` configures the parameters of the RF packet for the selected modem. Since the parameters are modem dependent, the description hereafter is valid only for the (G)FSK modem.

Table 8-15: SetPacketParams Command

Byte	0	1	2	3	4	5	6	7	8	9	10
Data from Host	0x02	0x10	PblLength TX(15:8)	PblLength TX(7:0)	Pbl Detect	Sync WordLen	Addr Comp	Packet Type	Payload Len	Crc Type	DcFree
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00	0x00	0x00

- *PblLengthTX* defines the (G)FSK packet preamble length in bits. Coded on 2 bytes, from 0x0008 (8 bits) to 0xFFFF (65535 bits).
- *PblDetect* defines the preamble detector length. The preamble detector acts as a gate to the packet controller, when not 0x00 (preamble detector length off), the packet controller only becomes active if a certain number of preamble bits have been successfully detected by the radio.
 - ♦ 0x00: Preamble detector length off
 - ♦ 0x04: Preamble detector length 8 bits
 - ♦ 0x05: Preamble detector length 16 bits
 - ♦ 0x06: Preamble detector length 24 bits
 - ♦ 0x07: Preamble detector length 32 bits
- *SyncWordLen* defines the length of the Syncword in bits. The Syncword is programmed by *SetGfskSyncWord(...)*. The *SyncWordLen* must be byte-aligned. For example, up to 7 bits of the Syncword will go into the payload if the *SyncWordLen* is not aligned on 8 bits.
- *AddrComp* allows conditioning the packet reception to a predefined peer device address. Node address and broadcast address can be set with the *SetPacketAdrs(...)* command. If the address comparison fails then the packet reception is aborted and the *adrsErr* flag is set.
 - ♦ 0x00: Address Filtering Disabled
 - ♦ 0x01: Rx & Tx: Address Filtering activated on Node address
 - ♦ 0x02: Address Filtering activated for Rx on Node & broadcast addresses, for Tx on Node address
- *PacketType* defines the length of the incoming packet.
 - ♦ 0x00: Packet length is known on both sides, the size of the payload is not added to the packet
 - ♦ 0x01: The packet is of variable size, *PayloadLen* is the size of the packet (with header encoded on 8 bits)
 - ♦ 0x02: The packet is of variable size, *PayloadLen* is the size of the packet - compatible with SX128x chip (with header encoded on 9 bits)
- *PayloadLen* defines the size of the payload (bytes) to transmit or the maximum payload size the receiver can accept.
 - ♦ In explicit header mode:
 - ♦ 0: Reception of any payload length between 0 and 255 bytes allowed
 - ♦ N: Reception of any payload length between 1 and N bytes accepted. Payload lengths of 0 or > N are rejected and result in a *HeaderErr* IRQ
 - ♦ In implicit header mode, *PayloadLen* configures the exact length of the payload to be transmitted or received.
- *CrcType* defines the packet CRC. The CRC can be fully configured and the polynomial used, as well as the initial values can be entered directly through command *SetGfskCrcParams(...)*:
 - ♦ 0x01: CRC_OFF (No CRC)
 - ♦ 0x00: CRC_1_BYTE (CRC computed on 1 byte)
 - ♦ 0x02: CRC_2_BYTE (CRC computed on 2 bytes)
 - ♦ 0x04: CRC_1_BYTE_INV (CRC computed on 1 byte and inverted)
 - ♦ 0x06: CRC_2_BYTE_INV (CRC computed on 2 bytes and inverted)
- *DcFree* enables whitening on the RF packet:
 - ♦ 0x00: No encoding
 - ♦ 0x01: SX127x/SX126x/LR11xx compatible whitening enable
 - ♦ 0x03: SX128x compatible whitening enable

8.5.3 SetGfskSyncWord

Command `SetGfskSyncWord( . . . )` configures the Syncword of the (G)FSK packet.

Table 8-16: SetGfskSyncWord Command

Byte	0	1	2	3	4	5	6	7	8	9
Data from Host	0x02	0x06	Syncword (63:56)	Syncword (55:48)	Syncword (47:40)	Syncword (39:32)	Syncword (31:24)	Syncword (23:16)	Syncword (15:8)	Syncword (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00	0x00

- *Syncword*: Default 0x9723522556536564. For reception only, the GFSK *Syncword* must be a multiple of 8 bits. If it is not, the *Syncword* length must still be configured as a multiple of 8 bits, and the *Syncword* must have filler added at the beginning. For example, if the *Syncword* length was 30 bits, the syncword length should be configured to 32 bits, and "01b" or "10b" should be added to the beginning of the *Syncword*.

8.5.4 SetPacketAdrs

Command `SetPacketAdrs( . . . )` sets the Node address and Broadcast address used for (G)FSK packet reception/transmission when filtering is enabled (AddrComp 0x01, or 0x02).

Table 8-17: SetPacketAdrs Command

Byte	0	1	2	3
Data from Host	0x02	0x12	NodeAddr	BroadcastAddr
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)

- *NodeAddr*: Default 0x00
- *BroadcastAddr*: Default 0x00

If the address comparison fails then the packet reception is aborted and the `adrsErr` flag is set.

8.5.5 SetGfskCrcParams

Command `SetGfskCrcParams(...)` configures the CRC polynomial and initial value, so they do not need to be controlled manually.

Table 8-18: SetGfskCrcParams Command

Byte	0	1	2	3	4	5	6	7	8	9
Data from Host	0x02	0x24	InitValue (31:24)	InitValue (23:16)	InitValue (15:8)	InitValue (7:0)	Poly (31:24)	Poly (23:16)	Poly (15:8)	Poly (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00	0x00

- *InitValue*: Initial value of the configured CRC polynomial (default 0x1D0F)
- *Poly*: CRC polynomial (default 0x1021)

This flexibility permits the user to select any standard CRC or to use their own CRC, allowing a specific detection of a given packet. Examples:

To use the IBM CRC configuration, the user must select:

- 0xFFFF for the initial value
- 0x8005 for the CRC polynomial
- 0x02 (CRC_2_BYTE) for the field *CrcType* in command `SetPacketParams(...)`

For the CCITT CRC configuration the user must select:

- 0x1D0F for the initial value
- 0x1021 for the CRC polynomial
- 0x06 (CRC_2_BYTE_INV) for the field *CrcType* in command `SetPacketParams(...)`

8.5.6 SetGfskWhitParams

Command `SetGfskWhitParams(...)` sets the whitening Seed:

Table 8-19: SetGfskWhitParams Command

Byte	0	1	2	3
Data from Host	0x02	0x025	Seed (15:8)	Seed (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)

- *Seed* default value is 0x0100

8.5.7 GetPacketStatus

Command `GetPacketStatus(...)` gets the status of the last received packet. Since the returned values are modem dependent, the description hereafter is valid only for the (G)FSK modem.

Table 8-20: GetPacketStatus Command

Byte	0	1
Data from Host	0x02	0x04
Data to Host	Stat1	Stat2

Table 8-21: GetPacketStatus Response

Byte	0	1	2	3	4
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	RxStatus (31:24)	RxStatus (23:16)	RxStatus (15:8)	RxStatus (7:0)

- *RxStatus* bits are as follows:
 - ♦ Bits 31:24: *RssiSync*, RSSI values latched upon the detection of sync address. Negated, dBm, ufix(8,1).
 - ♦ Bits 23:16 *RssiAvg*, RSSI average over the payload of the received packet. Latched upon the packet_done IRQ. Negated, dBm, ufix(8,1).
 - ♦ Bits 15:8 *RxLen*, Length of the received packet
 - ♦ Bit 7: RFU
 - ♦ Bit 6: RFU
 - ♦ Bit 5: *Adrserr*, Address filtering status of current packet. Asserted when the received packet address byte doesn't match the configured node_adrs(7:0) or broadcast(7:0) address according to the adrs_comp(1:0) configuration.
 - ♦ Bit 4: *Crcerr*, CRC check status of the current packet. Only applicable in Rx when the CRC check is not disabled. The packet is available in FIFO.
 - ♦ Bit 3: *Lenerr*, Length filtering status of the current packet. Asserted when the length of the received packet is greater than the Max length defined in the payload_len(7:0) field. Only applicable in Rx for variable length packets.
 - ♦ Bit 2: *Aborterr*, Abort status of the current packet. Asserted when the current packet is aborted with the pkt_abort_p register field. Applicable in both Rx and Tx.
 - ♦ Bit 1: *PktRcvd*, Packet reception status. Indicates that the packet reception is done. Only show the completion of the receive process and not the packet validity. Only applicable in Rx.
 - ♦ Bit 0: *PktSent*, Packet transmission status. Indicates that the packet transmission is done. Only applicable in Tx.

8.6 LR-FHSS Modulation

LR-FHSS is a frequency hopping spread spectrum modulation, perfectly suited for situations where long-range communication is required for a large number of devices in the same area. LR-FHSS can nearly equate the link-budget of LoRa SF12 BW125kHz, allowing many devices to coexist on the same spectral allocation. LR-FHSS implementation in the LR1120 is transmit only. The LR1120 fully controls the following functionalities:

- constructing frames, including preamble, sync headers, payload, integrity check, whitening, interleaving and forward error correction calculations
- calculation of hopping sequences
- handling of the whole frequency hopping mechanism

8.6.1 LR-FHSS Modulation Principle

The LR-FHSS is a low data rate modulation derived from a GMSK modulation, with intra-packet frequency hopping sequences configurable for each device. Only 488.28215Hz data rate is currently supported. The payload uses convolution encoding to perform forward error detection and correction, implying a transmission overhead.

Spectrally speaking, the LR-FHSS packet is split into several frames, sent over pseudo-randomly distributed frequencies spaced by a configurable step size (Grid). The total spectral occupation of the LR-FHSS packet corresponds to the Bandwidth parameter. LR-FHSS modulation is represented in [Figure 8-6: LR-FHSS Spectral Plot Example](#).

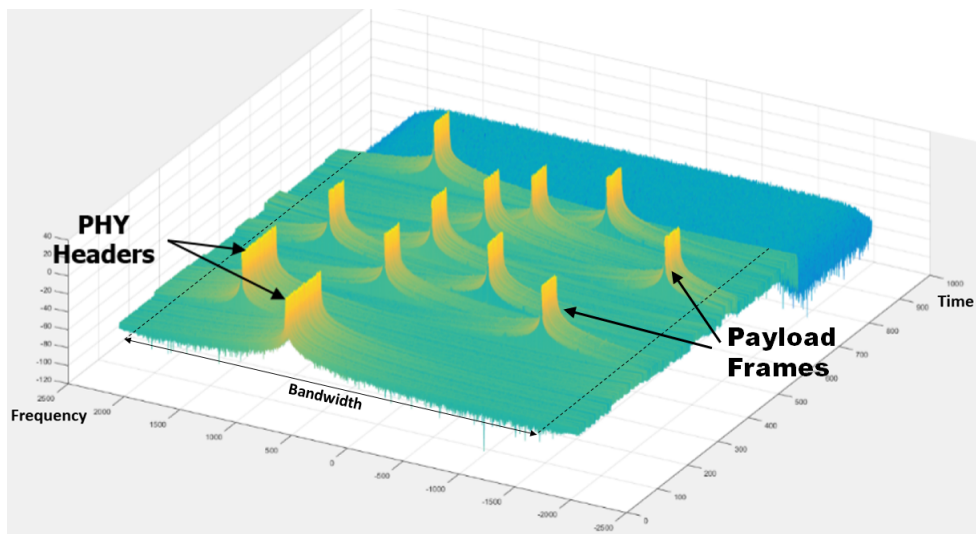


Figure 8-6: LR-FHSS Spectral Plot Example

Each payload frame, except the last frame, contains exactly 50 encoded bits, and are then of equal duration in time. The last frame may contain less than 50 bits. The number of frames varies therefore with the payload size and the Coding Rate.

The Payload frames are preceded with one to four synchronization sequences (Headers) to allow robust frequency and timing synchronization. Even if receiving a single Header is sufficient to be able to decode the payload, a minimum of 2 Headers improves immunity against radio interference. If multiple headers are used, they are sent on different frequencies. Each header contains exactly 114 encoded bits, and are then of equal duration in time.

Additional information is provided in AN1200.58 Long Range FHSS Demo and AN1200.64 Long Range FHSS System Performance.

8.7 LR-FHSS Commands

8.7.1 SetModulationParams

Command `SetModulationParams(...)` configures the modulation parameters for the selected modem. Since the parameters are modem dependent, the description hereafter is valid only for the LR-FHSS modulation.

Table 8-22: SetModulationParams Command

Byte	0	1	2	3	4	5	6
Data from Host	0x02	0x0F	BitRate (31:24)	BitRate (23:16)	BitRate (15:8)	BitRate (7:0)	PulseShape
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0

- *BitRate* defines the LR-FHSS Bit rate in bits per second. Bitrate in bits/s is $Bitrate(30:0)/256$, with $Bitrate(31) = 1$. For example, 488.28215Hz bit rate will be coded $Bitrate = 125000$ with $Bitrate(31) = 1$.
In the current LR-FHSS implementation, only 488.28215 Hz is supported, other values are RFU.
- *PulseShape* defines the filtering applied to the LR-FHSS packet:
 - ♦ 0x0B: Gaussian BT 1. Other values are RFU

8.7.2 LrFhssBuildFrame

Command `LrFhssBuildFrame(...)` encodes the given payload and configures the internal hopping table

Table 8-23: LrFhssBuildFrame Command

Byte	0	1	2	3	4	5	6	7
Data from Host	0x02	0x2C	HeaderCount	CR	ModType	Grid	Hopping	BW
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0	0

Byte	8	9	10	11	12	...	...
Data from Host	HopSequence (8)	HopSequence (7:0)	DeviceOffset	Payload[0]	Payload[1]	...	Payload[N]
Data to Host	0	0	0	0	0	...	0

- *HeaderCount*: number of Headers for the LR-FHSS packet: 1, 2, 3 or 4
- *CR*: configures the Coding Rate:
 - ♦ 0x00: CR = 5/6
 - ♦ 0x01: CR = 2/3
 - ♦ 0x02: CR = 1/2
 - ♦ 0x03: CR = 1/3
- *ModType*: 0 = GMSK modulation, 488.28125bps data rate
- *Grid*: frequency hopping grid step size:
 - ♦ 0x00: 25.390625kHz for FCC use case
 - ♦ 0x01: 3.90625kHz for non-FCC use case

- *Hopping*: configures intra-packet hopping:
 - ♦ 0x00: no hopping (test purpose)
 - ♦ 0x01: hopping
- *BW*: bandwidth occupied by the frequency hopping pattern:
 - ♦ 0x00: 39.06kHz
 - ♦ 0x01: 85.94kHz
 - ♦ 0x02: 136.72kHz
 - ♦ 0x03: 183.59kHz
 - ♦ 0x04: 335.94kHz
 - ♦ 0x05: 386.72kHz
 - ♦ 0x06: 722.66kHz
 - ♦ 0x07: 773.44kHz
 - ♦ 0x08: 1523.4kHz
 - ♦ 0x09: 1574.2kHz
 - ♦ Other values are RFU
- *HopSequence*: 9-bit integer, seed of the pseudo-random frequency hopping sequence:
 - ♦ value in [0; 383] for Grid = 0x00, BW in {0x06, 0x07, 0x08, 0x09}
 - ♦ value in [0; 383] for Grid = 0x01, BW in {0x00, 0x01, 0x02, 0x03}
 - ♦ value in [0; 511] for Grid = 0x01, BW in {0x04, 0x05, 0x06, 0x07, 0x08, 0x09}
- *DeviceOffset*: per device frequency offset to reduce the risk of packet collision. Frequency offset (Hz) = DeviceOffset * 488.28125Hz:
 - ♦ signed value in [-26; 25] for Grid = 25.390625kHz
 - ♦ signed values in [-4; 3] for Grid = 3.90625kHz

For a FCC use case, *DeviceOffset* shall not be changed once configured.
- *Payload*: actual payload to encode. Returns CMD_OK if the parameters are valid and if the payload can be encoded, CMD_PERR otherwise.

The maximum supported packet length is 255 coded bytes. The maximum user payload length is given by the [Table 8-24: Maximum user Payload Length \(in Bytes\)](#).

Table 8-24: Maximum user Payload Length (in Bytes)

CR	HeaderCount = 1	HeaderCount = 2	HeaderCount = 3	HeaderCount = 4
CR = 5/6	189	178	167	155
CR = 2/3	151	142	133	123
CR = 1/2	112	105	99	92
CR = 1/3	74	69	65	60

A FCC use case corresponds to a use case where BW = 0x08/0x09, Hopping = 0x01, Grid = 0x00.

Note: This command does not send the LR-FHSS packet. The transmission will be triggered using SetTx(. . .) command.

8.7.3 LrFhssSetSyncWord

Command `LrFhssSetSyncWord(...)` sets the LR-FHSS Syncword.

Table 8-25: LrFhssSetSyncWord Command

Byte	0	1	2	3	4	5
Data from Host	0x02	0x2D	Syncword[0]	Syncword[1]	Syncword[2]	Syncword[3]
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

Syncword: LR-FHSS Syncword. Default value is {0x2C, 0x0F, 0x79, 0x95}.

8.7.4 Circuit Configuration for Long Range FHSS Transmission

After power up (battery insertion or hard reset) the LR1120 runs automatically a calibration procedure and goes to STDBY_RC mode. This is indicated by a low state on BUSY pin.

From STDBY_RC, the sequence of operations needed to transmit a Long Range FHSS packet are:

1. If not in STDBY_RC mode, then set the circuit in this mode with the command `SetStandby(...)`.
2. Set the basic configuration parameters such as RF switch, PA configuration, OCP, TxParams, RfFrequency...
3. Define the modulation (LR-FHSS) with the command `SetPacketType(...)`.
4. Set the modulation parameters (`SetModulationParams(...)` command) with the proper Bit Rate (488.28125 bps), the proper Pulse Shape.
5. Call the command `LrFhssBuildFrame(...)`.
6. Activate the IRQ: *TxDone*.
7. Set the circuit in transmitter mode to start transmission with the command `SetTx(...)`.
8. Wait for *TxDone* interrupt.

If configured, *LrFhssHop* IRQ is asserted at each frequency hop inside the LR-FHSS packet, after each PA ramp-up.

8.8 Sigfox Modulation

The LR1120 can transmit Sigfox packets at both 100 and 600bps. This allows support of all the Sigfox Radio Configurations from RC1 to RC7. The LR1120 is also able to receive Sigfox downlinks at 600bps in the selected Sigfox Radio Configuration. Please contact a Semtech representative for additional information.

No Sigfox protocol stack is embedded inside the LR1120. The user must configure and transmit the Sigfox packets according to the command order depicted in [Figure 8-1: Modem Command Order](#).

Since commands `SetModulationParams(...)` and `SetPacketParams(...)` are modem specific, the description of those commands for Sigfox support is described below.

8.8.1 SetModulationParams

Command `SetModulationParams(...)` configures the modulation parameters for the selected modem. Since the parameters are modem dependent, the description hereafter is valid only for the Sigfox modulation.

Table 8-26: SetModulationParams Command

Byte	0	1	2	3	4	5	6
Data from Host	0x02	0x0F	BitRate (31:24)	BitRate (23:16)	BitRate (15:8)	BitRate (7:0)	PulseShape
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0

- *BitRate*: see [Section 8.5.1](#). Only supports 100bps (RC1/RC3/RC5/RC6 and RC7) and 600bps (RC2 and RC4).
- *PulseShape*: refer to the description in [Section 8.5.1](#). Only BT 0.7 supported (0x16).

8.8.2 SetPacketParams

Command `SetPacketParams(...)` configures the parameters of the RF packet for the selected modem. Since the parameters are modem dependent, the description hereafter is valid only for the Sigfox modulation..

Table 8-27: SetPacketParams Command

Byte	0	1	2	3	4	5	6	5	5
Data from Host	0x02	0x10	Payloadlen	RampUp Delay (15:8)	RampUp Delay (7:0)	RampDown Delay (15:8)	RampDown Delay (7:0)	BitNum (15:8)	BitNum (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0	0	0

- *PayloadLen*: in bytes. To configure the packet controller.
- *RampUpDelay*: in us. Delay to fine tune ramp up time.
- *RampDownDelay*: in us. Delay to fine tune ramp down time.
- *BitNum*: in bits. Used to ramp down the PA before the end of payload length, in case actual length is not a multiple of 8.

8.9 Data Buffer

The LR1120 is equipped with two 255 byte RAM data buffers which are accessible in all modes except sleep mode. One buffer stores the received payload data, while the other contains the payload data to be transmitted.

The LR1120 automatically controls the data pointers, which means that no base address handling by the user is necessary.

- Data Buffer in Receive Mode:
 - ♦ The received payload data are stored in the RX buffer:
 - ♦ They can be read back using command *ReadBuffer8(...)* (see [3.7.5 ReadBuffer8](#))
 - ♦ *GetRxbufferStatus(...)* reads the pointer to the first byte of the last packet received and the packet length (see [7.2.11 GetRxBufferStatus](#)).
 - ♦ *ClearRxBuffer(...)* clears all the data in the LR1120 RX buffer (see [3.7.6 ClearRxBuffer](#))
- Data Buffer in Transmit Mode:
 - ♦ The payload data to be transmitted shall be written the Tx Buffer using command *WriteBuffer8(...)* (see [3.7.4 WriteBuffer8](#))

8.10 RSSI Functionality

The RSSI information of the LR1120 is available either in the radio chain, or after demodulation at the end of the reception stage. A summary of the RSSI information and their meaning is summarized in table [Table 8-28: RSSI Information Origin and Meaning](#) below:

Table 8-28: RSSI Information Origin and Meaning

Command	Modem	Name	Description
<i>GetRssiInst(...)</i>	All	<i>RssiInst</i>	Instantaneous RSSI
<i>GetPacketStatus(...)</i>	(G)FSK	<i>RssiSync</i>	Instantaneous RSSI value latched in the (G)FSK demodulator, upon detection of sync address
		<i>RssiAvg</i>	Average RSSI value over the whole payload of the received packet, determined in the (G)FSK demodulator.
	LoRa®	<i>RssiPkt</i>	Measurement of the mean energy at the input of the modem over the last packet received.
		<i>SignalRssiPkt</i>	Estimation of the mean energy of the LoRa signal over the last packet received. Equivalent to RssiPkt - environment noise

Refer to each command description for implementation details on the various RSSI fields.

The RSSI information has a propagation time in the digital chain of the receiver. As depicted in [Figure 8-7](#) hereafter, a longer time is typically necessary for the first RSSI value to be available (*Twait*) than for the following RSSI samples (*Trssi*).

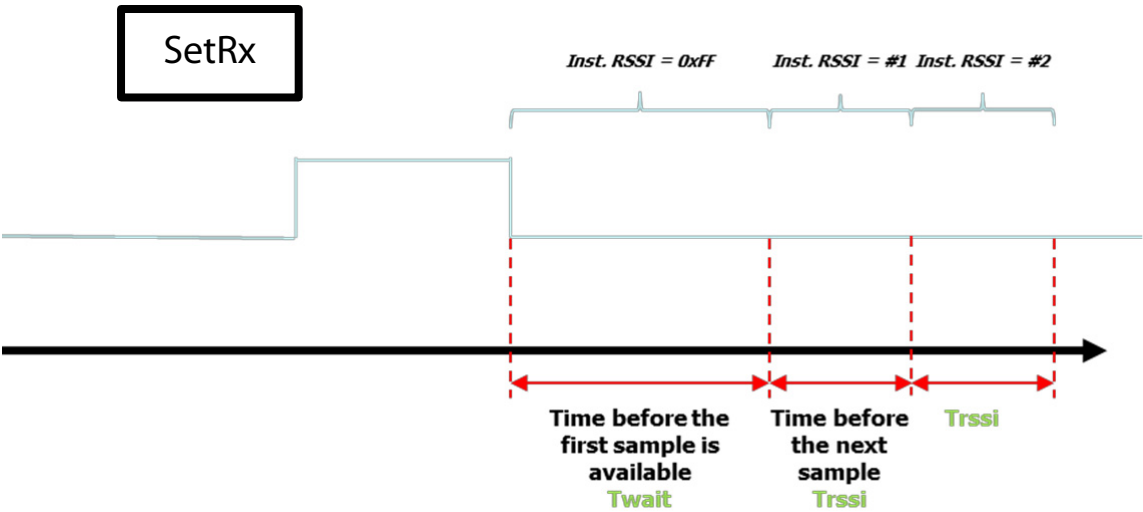


Figure 8-7: RSSI Propagation Time

For GFSK, Twait and Trssi depend on the selected bandwidth, as shown in [Table 8-29](#) below:

Table 8-29: Twait & Trssi Values for GFSK Bandwidths

BWF kHz	Twait us	Trssi us
467	30.68	2.00
373.6	37.78	2.50
312	44.89	3.00
234.3	54.61	4.00
187.2	67.69	5.00
156.2	80.76	6.00
117.3	102.30	8.00
93.8	127.28	10.00
78.2	152.26	12.00
58.6	197.61	16.00
46.9	246.41	20.00
39	295.20	24.00
29.3	388.18	32.00
23.4	484.59	40.00
19.5	581.01	48.00
14.6	769.24	64.00
11.7	960.91	80.00
9.7	1152.58	96.00
7.3	1531.30	128.00
5.8	1913.47	160.00
4.8	2295.64	192.00

For LoRa modulation, Twait and Rssi are identical, and equal to the LoRa Symbol time.

9. Power Amplifiers

The LR1120 features 2 power amplifiers for sub-GHz operation, and one power amplifier for HF (2.4GHz) operation:

- A high power PA (HP PA), optimized for +22dBm operation
- A low power PA (LP PA), optimized for +14dBm operation, capable of +15dBm output power
- A high frequency PA (HF PA), optimized for +13dBm operation in the 2.4GHz band

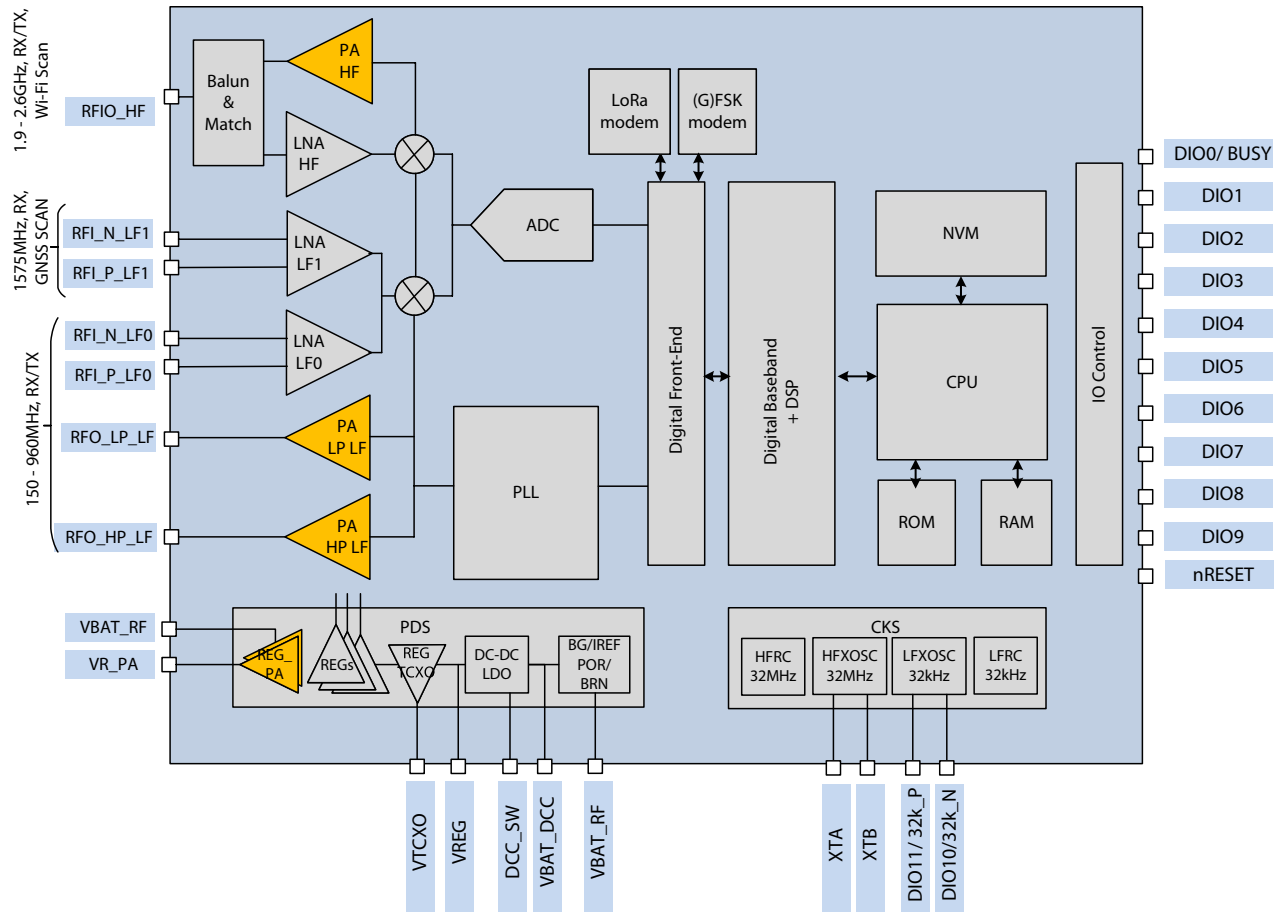


Figure 9-1: LR1120 Power Amplifiers Block Diagram

The PA is configured using two commands: `SetPaConfig(...)` and `SetTxParams(...)`.

The `SetPaConfig(...)` command is used to:

- Select the PA to be used (HP, LP or HF)
- Select the supply of the PA (VBAT or VREG)
- Select the duty cycle of either PA
- Select the size of the PA (only applicable to the high power PA)

The `SetTxParams(...)` command is used to:

- Control the supply voltage of the PA (VR_PA) and output power
- Choose the ramp time at the start / stop of TX

The user should configure the PA corresponding to the output power and current consumption requirements of the application. No automatic frequency limitation is performed during the PA selection: the frequency selection has to be performed according to the capability offered by the external matching network component values.

9.1 PA Supply Scheme

The PA supply scheme is depicted in [Figure 9-2: PA Supply Scheme](#).

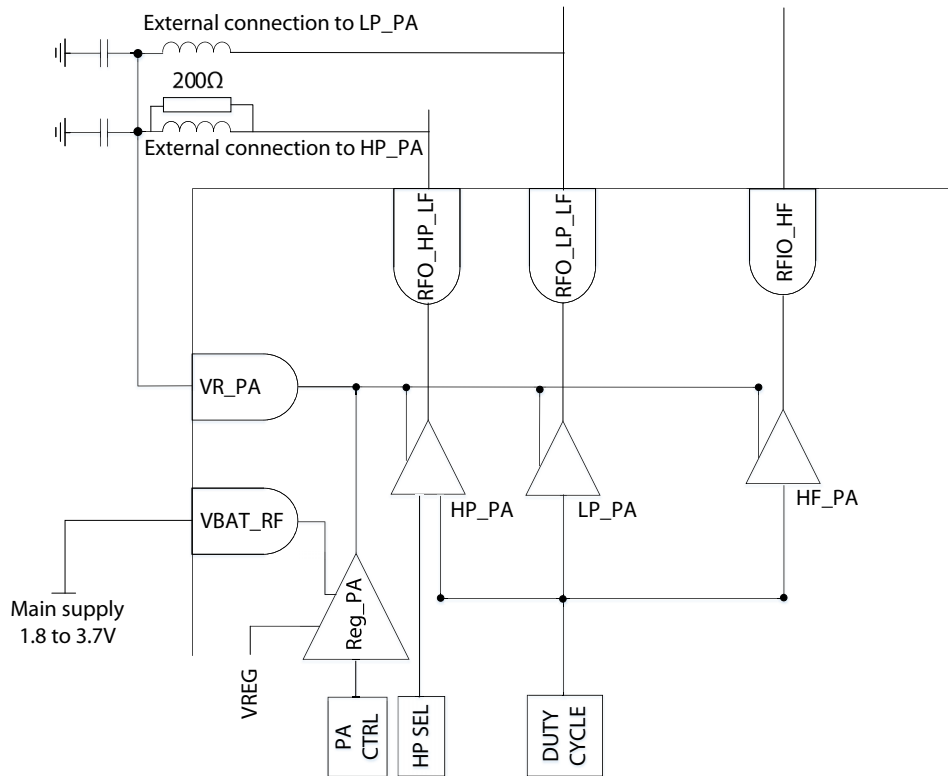


Figure 9-2: PA Supply Scheme

The PA regulator (Reg_PA) supplies the high power PA (HP PA), the low power PA (LP PA), and the high-frequency PA (HF PA) through the VR_PA pin. Both the HP PA and the LP PA require a high-Q choke inductor connected externally between their respective outputs, and VR_PA to provide the bias and control the output power. No external choke inductor is necessary for the HF PA. It is best practice to place a 200 Ohm resistor in parallel to the HP PA choke inductor.

The PA regulator is internally connected to the DC-DC /LDO output for the low power PA and high frequency PA, allowing a +15dBm and +13dBm operation in both DCDC or LDO configurations. For the high power PA, it is connected to the VBAT_RF pin, therefore to the main supply voltage. This means that the maximum output power generated by the high power PA depends on the VBAT voltage.

The TX main supply can switch between the battery VBAT and the internal regulator VREG, according to the PA use case. When operating with VR_PA above 1.35V (e.g. in the case of high power), the battery supply VBAT must be chosen. When operating with VR_PA below 1.35V (e.g. in the case of low power PA), either supply can be chosen. However, it is better to choose the internal regulator VREG whenever the required VR_PA is 1.35V or below, in order to benefit from the Buck converter.

The LR1120 incorporates a precise duty cycle trimmer shared between the three power amplifiers. This duty cycle trimmer can be used to trade-off the output power, efficiency, and harmonic emission to address the different regional standard requirements. The duty cycle trimmer is dependent on the impedance presented to the PA, and thus might vary from one PCB design to another.

9.1.1 Low Power PA

For maximum efficiency, the low power PA should be operated with a maximum VR_PA near 1.35V.

To get VR_PA = 1.35V the user should set TxPower = 14. At this setting:

- The PA can deliver up to 15dBm output power, constant over the specified battery supply range
- The actual maximum output power can be set according to the duty cycle setting (*PaDutyCycle*)
- If needed, the output power can be decremented in steps of 1dB from maximum by using *TxPower* < 14

The VR_PA variation over the programmed power *TxPower* for different supply voltages and *PaDutyCycle* conditions is depicted in [Figure 9-3: Low Power PA VR_PA Voltage vs TxPower](#) below (valid for VBAT and VREG supplies, in both LDO or DCDC configurations).

Note: All figures in this chapter are indicative and typical, and are not a specification. These figures only highlight the behavior of the PA over the various parameters and conditions.

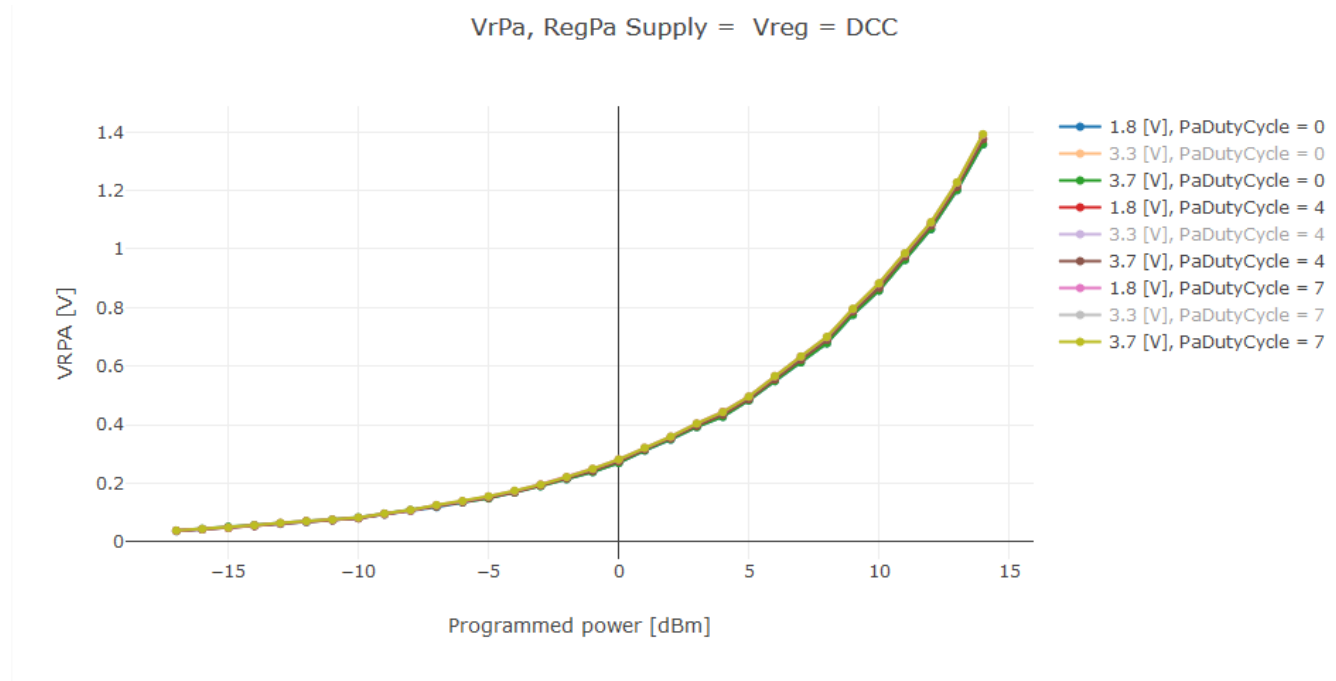


Figure 9-3: Low Power PA VR_PA Voltage vs TxPower

9.1.2 High Power PA

For maximum efficiency, the high power PA should be operated with a maximum VR_PA near 3.1V.

To get VR_PA = 3.1V the user should set TxPower = 22. At this setting:

- The PA can deliver up to 22dBm output power. The output power in this case varies when the battery voltage drops below 3.3V
- The actual maximum output power can be set according to the PaDutyCycle and PaHpSel
- If needed, the output power can be decremented in steps of 1dB from maximum by using TxPower < 22

The VR_PA variation over the programmed power TxPower for different supply voltages and duty cycle (PaDutyCycle) conditions is depicted in Figure 9-4: High Power PA VR_PA Voltage vs. TxPower below.

The internal regulator for VR_PA has 200mV of drop-out, which means VBAT must be 200mV higher than the VR_PA voltage in order to attain the corresponding output power.

For example: For Pout = +20dBm,

- VR_PA = 2.5V is required (brown curve)
- The high power PA can maintain Pout = +20dBm on the 2.7V < VBAT < 3.7V voltage range (2.5V + 200mV = 2.7V)
- Below 2.7V, the output power degrades as VBAT reduces

At 1.8V of supply voltage, the maximum VR_PA value is 1.6V (1.8V - 200mV), allowing therefore a +17dBm output power.

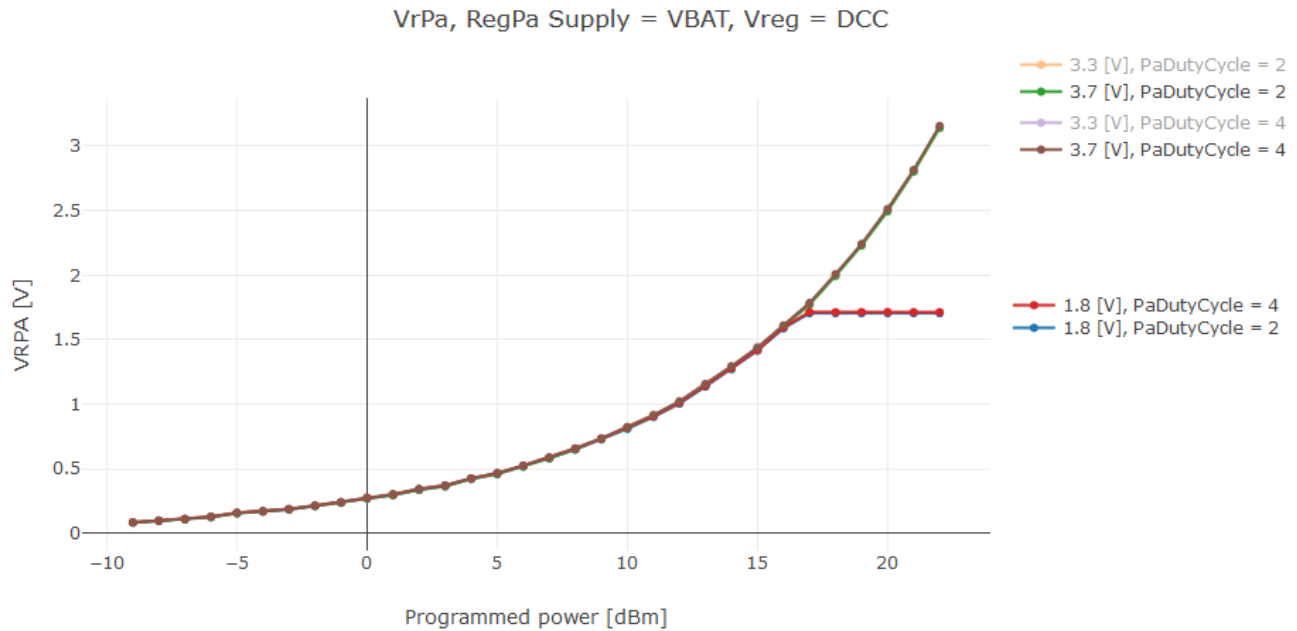


Figure 9-4: High Power PA VR_PA Voltage vs. TxPower

9.2 PA Output Power

As stated previously, two parameters do have an impact on the TX output power generated by each PA: the programmed power *TxPower* and the duty cycle *PaDutyCycle*. A third parameter, *PaHPSEL*, controls the size of the high power PA, and therefore has a direct impact on the high power PA output power.

To reach +22dBm output power, *PaHPSEL* must be set to 7. *PaHPSEL* has no impact on either the low power PA or the high frequency PA.

9.2.1 Low Power PA

Figure 9-5: Low Power PA Output Power vs. *TxPower* shows the output power of the low power PA with *TxPower* for different *PaDutyCycle* settings and over the supply voltage.

The supply voltage has no impact on the output power, since the low power PA is internally regulated. Only the *PaDutyCycle* has an influence on the output power. Therefore the plots for 1.8V, 3.3V and 3.7V are superimposed, and only the plots for 3.7V are visible.

For example:

- *TxPower* = 14 and *PaDutyCycle* = 0 gives +10dBm whatever the supply voltage (1.8V, 3.3V and 3.7V)
- *TxPower* = 14 and *PaDutyCycle* = 4 gives +14dBm whatever the supply voltage (1.8V, 3.3V and 3.7V)
- *TxPower* = 14 and *PaDutyCycle* = 7 gives +15dBm whatever the supply voltage (1.8V, 3.3V and 3.7V)

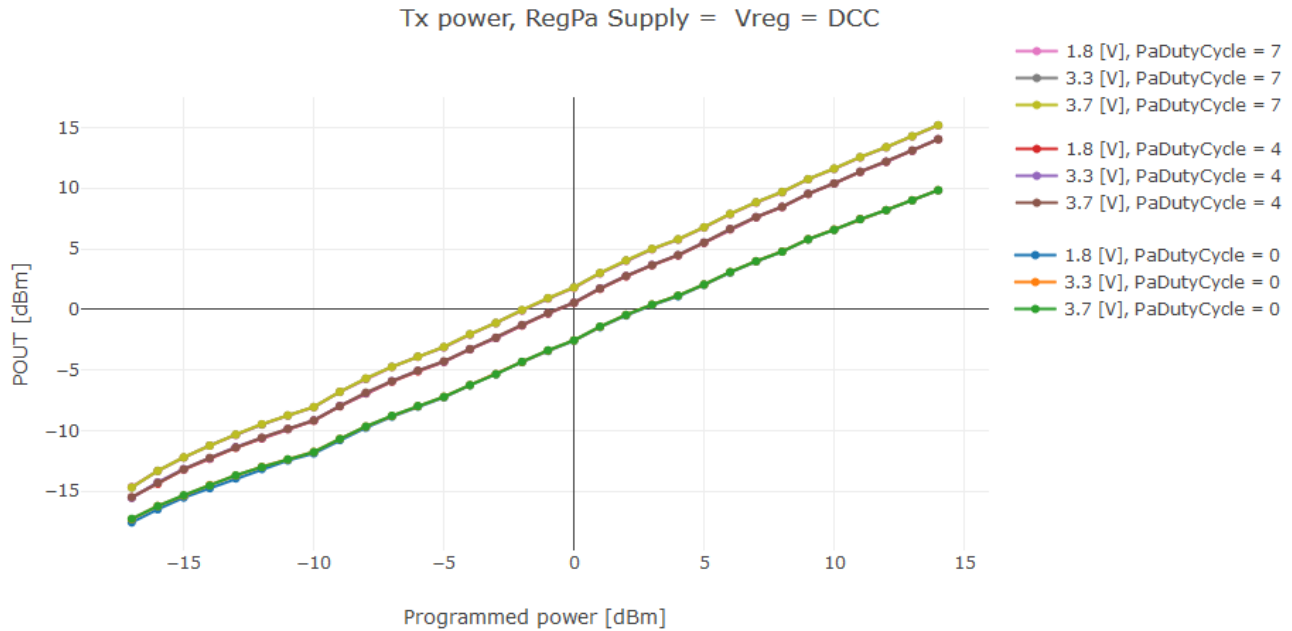


Figure 9-5: Low Power PA Output Power vs. *TxPower*

9.2.2 High Power PA

Figure 9-6: HP PA Output Power vs. TxPower shows the output power of the high power PA with TxPower for different PaDutyCycle settings and over the supply voltage.

For a given PaDutyCycle, the output power of the high power PA is maintained on a certain voltage range, and then decreases with VBAT. For example:

- For the +22dBm power setting, a VR_PA of ~3.1V is required (refer to Figure 9-4). Therefore, given the 200mV drop-out of the PA regulator, the +22dBm output power can only be obtained from a 3.3V to 3.7V supply voltage range.
- For +17dBm, VR_PA around 2V is required. Therefore the LR1120 output power will drop to +17dBm for the minimum supply voltage 1.8V.

Therefore, the 3.3V and 3.7V plots are then superimposed for a given PaDutyCycle, and only the plots for 3.7V are visible.

For a given supply voltage, increasing the PaDutyCycle increases the output power.

For example:

- For the +22dBm power setting at 3.3V, PaDutyCycle = 4 allows the high power PA to deliver +22dBm
- For the +22dBm power setting at 3.3V, PaDutyCycle = 2 allows the high power PA to deliver +20dBm

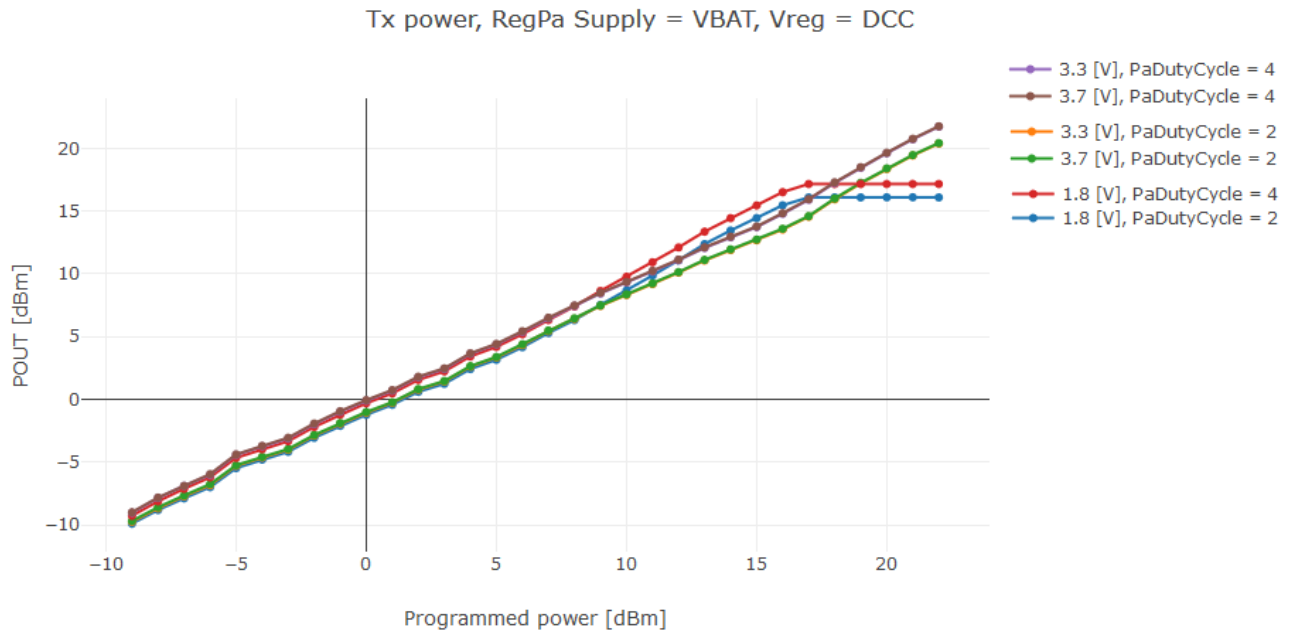


Figure 9-6: HP PA Output Power vs. TxPower

9.2.3 High Frequency PA

Figure 9-7: High Frequency PA Output Power vs. TxPower shows the output power of the high frequency PA with TxPower for different supply voltage values.

- The supply voltage has no impact on the output power, since the HF PA is internally regulated. Therefore the plots for 1.8V, 3.3V and 3.7V are superimposed, and only the plots for 3.7V are visible.

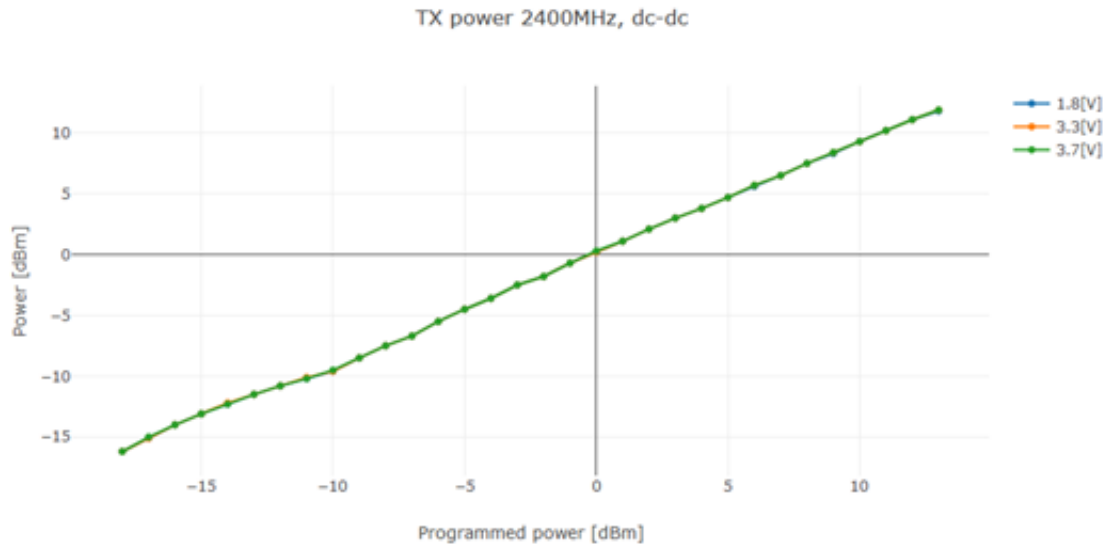


Figure 9-7: High Frequency PA Output Power vs. TxPower

9.3 PA Current Consumption

9.3.1 Low Power PA

Figure 9-8: IDDTX vs TxPower, Low Power PA, DC-DC Configuration shows the impact of the supply voltage for three *PaDutyCycle* settings (0, 4 and 7) in DC-DC configuration.

At a given supply voltage, a higher *PaDutyCycle* setting increases the device current consumption. At a given *PaDutyCycle* setting, the current consumption is optimum for a supply voltage equal or greater to 3.3V, therefore the plots for 3.3V and 3.7V are superimposed. A power supply of 1.8V is not as power efficient as 3.3V or more, resulting in a higher current consumption.

For example:

- For 3.7V, *PaDutyCycle* = 0 the current consumption is approx. 28mA, for *PaDutyCycle* = 4 approx. 47mA and for *PaDutyCycle* = 7 approx. 62mA
- For *PaDutyCycle* = 4, the current consumption is approx. 47mA for 3.3V and 3.7V, and approx. 49mA for 1.8V

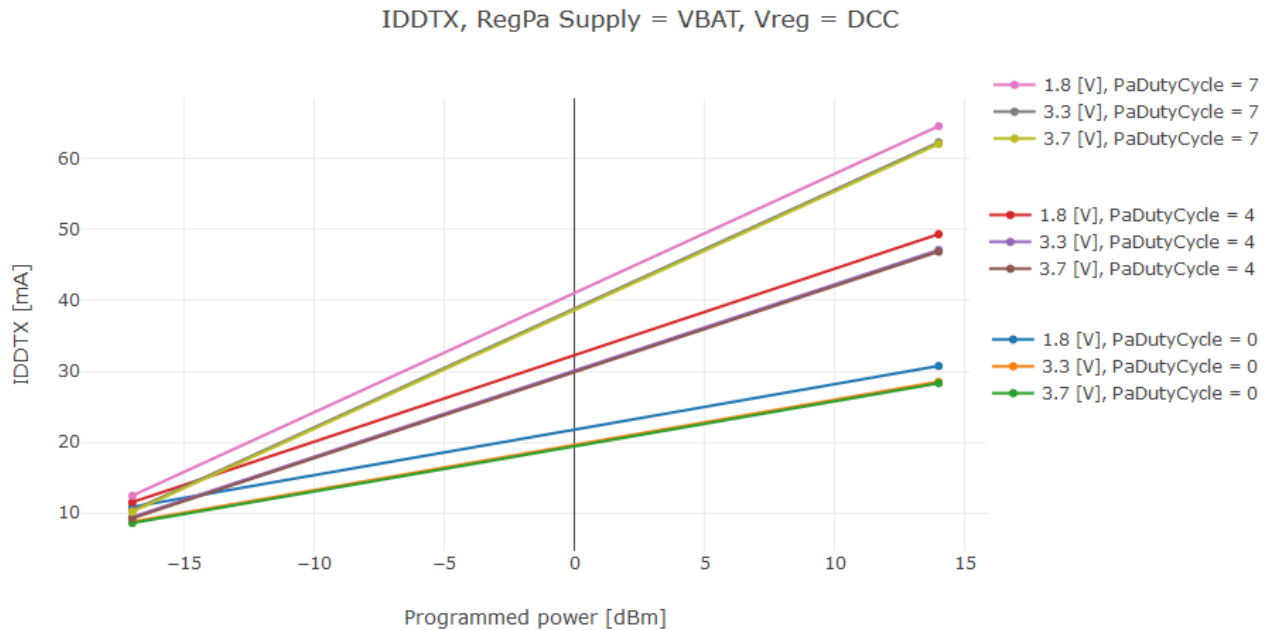


Figure 9-8: IDDTX vs TxPower, Low Power PA, DC-DC Configuration

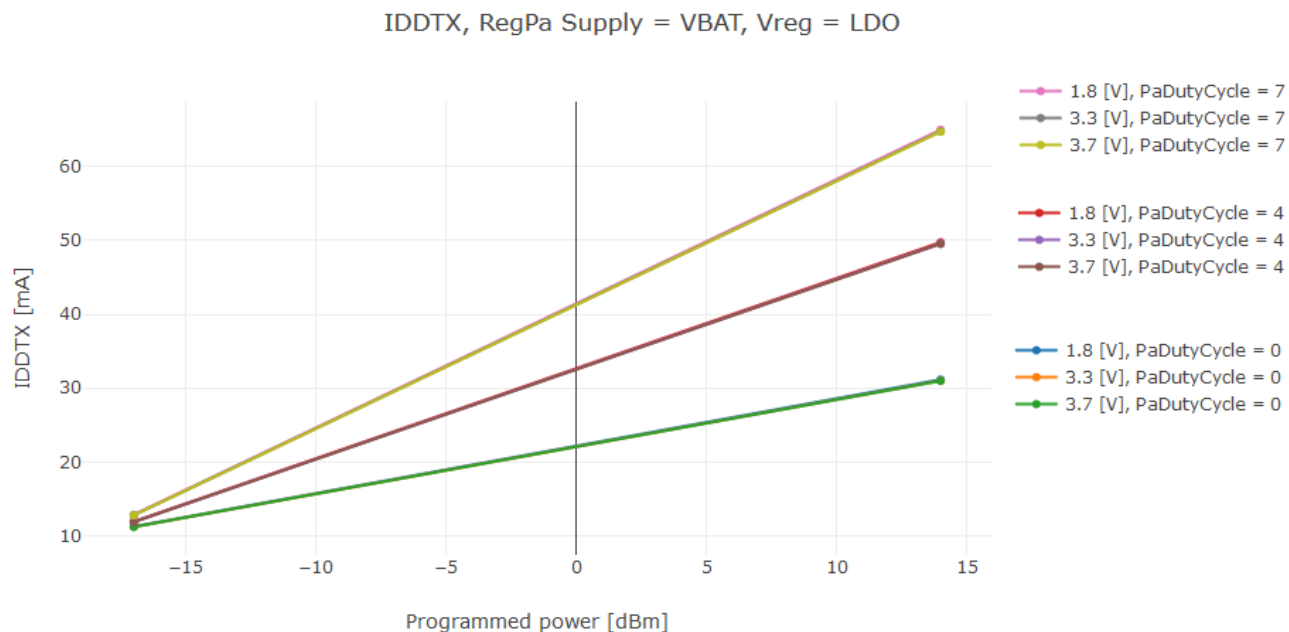


Figure 9-9: IDDTX vs TxPower, Low Power PA, LDO Configuration

Figure 9-9: IDDTX vs TxPower, Low Power PA, LDO Configuration shows the impact of the supply voltage for three *PaDutyCycle* settings (0, 4 and 7) in LDO configuration.

Similarly to the DC-DC configuration, we can see that, at a given supply voltage, a higher *PaDutyCycle* setting increases the device current consumption. However, the supply voltage has no influence on the current consumption at a given *PaDutyCycle* setting, which means that the plots for 1.8V, 3.3V, and 3.7V are superimposed.

Figure 9-8 and Figure 9-9 show that the power efficiency of the low power PA is maximized when the internal DC-DC regulator is used at, or above, 3.3V.

9.3.2 High Power PA

Figure 9-10: IDDTX vs TxPower, High Power PA, DC-DC Configuration and Figure 9-11: IDDTX vs TxPower, High Power PA, LDO Configuration shows the impact of the supply voltage for two *PaDutyCycle* settings (2 and 4) in both DC-DC and LDO configurations.

Similarly to the low power PA, at a given supply voltage a higher *PaDutyCycle* setting increases the device current consumption. However, at a given *PaDutyCycle* setting, the current consumption is stable with respect to the supply voltage, providing this latter is high enough to allow the generation of the VR_PA voltage required for the programmed power value *TxPower*.

For example:

- For 3.3V, the current consumption is approx. 98mA for *PaDutyCycle* = 2, and approx. 118mA for *PaDutyCycle* = 4
- For 1.8V, the current consumption is approx. 69mA for *PaDutyCycle* = 2, and approx. 81mA for *PaDutyCycle* = 4. This is due to the fact that at 1.8V supply voltage, the maximum VR_PA voltage is 1.6V, therefore a maximum output power of +17dBm

During the high power PA operation, the DC-DC supplies the analog and digital core of the devices, whereas the PA itself -the largest power consumption contributor- is supplied directly from VBAT. Therefore, there is no significant current consumption difference between the DC-DC or the LDO modes during the high power PA operation.

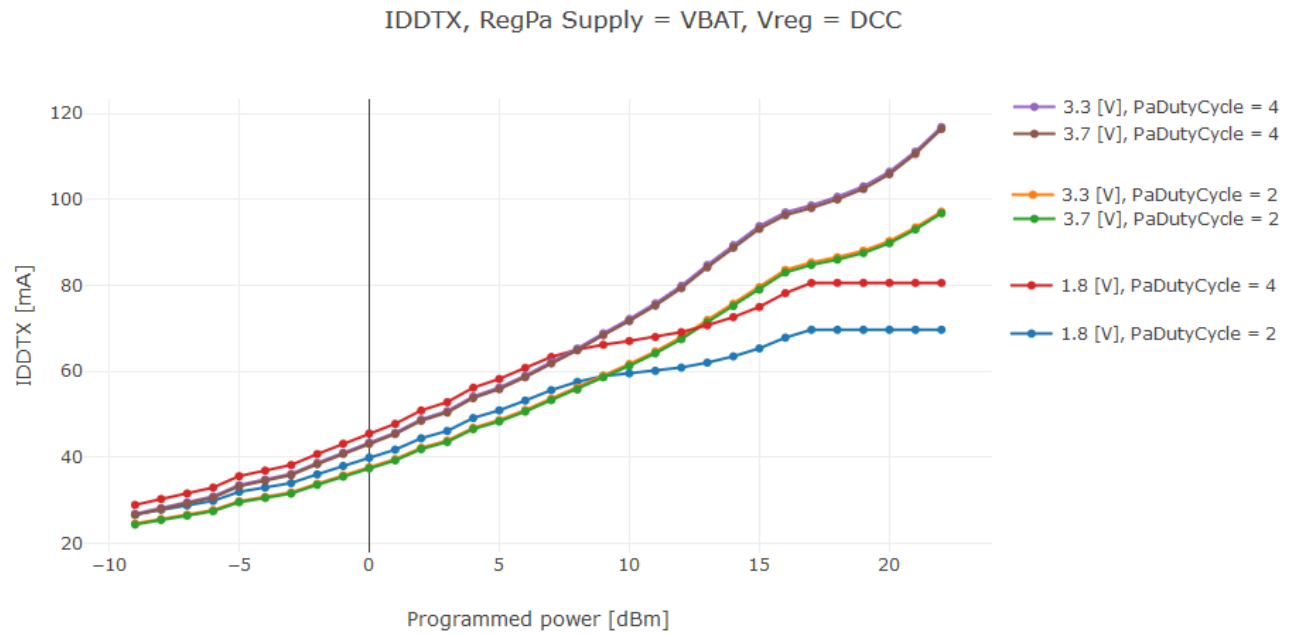


Figure 9-10: IDD TX vs TxPower, High Power PA, DC-DC Configuration

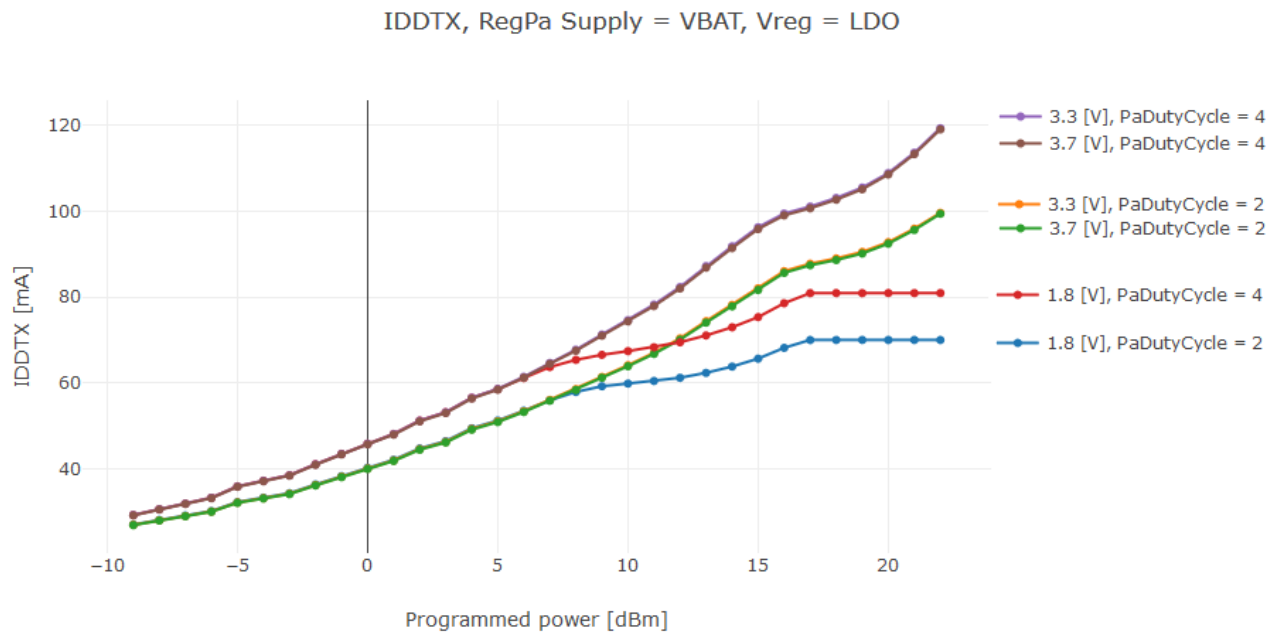


Figure 9-11: IDD TX vs TxPower, High Power PA, LDO Configuration

9.3.3 High Frequency PA

Figure 9-12 shows the impact of the TxPower setting for three supply voltages in a DC-DC configuration.

The current consumption is optimum for a supply voltage equal or greater to 3.3V, therefore the plots for 3.3V and 3.7V are superimposed. A power supply of 1.8V is not as power efficient as 3.3V or more, resulting in a higher current consumption.

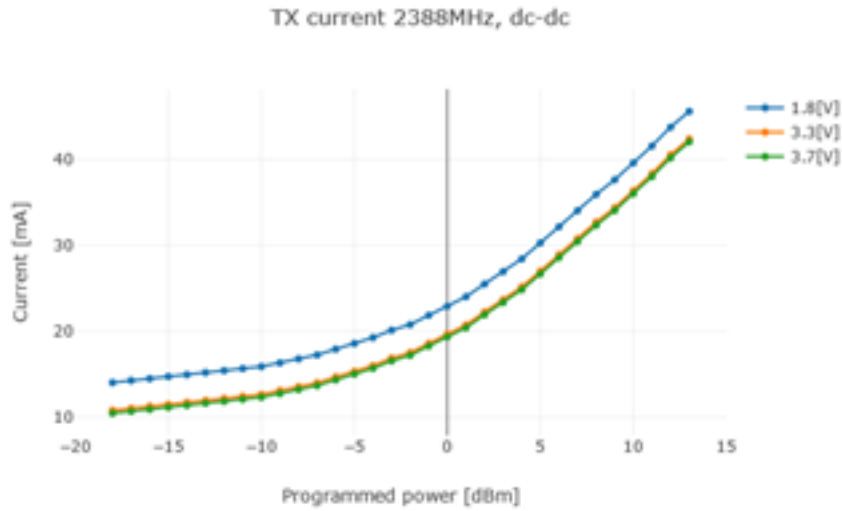


Figure 9-12: IDDTX vs TxPower, High Frequency PA, DC-DC Configuration

Figure 9-13: shows the impact of the TxPower setting for three supply voltages in LDO configuration.

The supply voltage has no influence on the current consumption, which means that the plots for 1.8V, 3.3V, and 3.7V are superimposed.

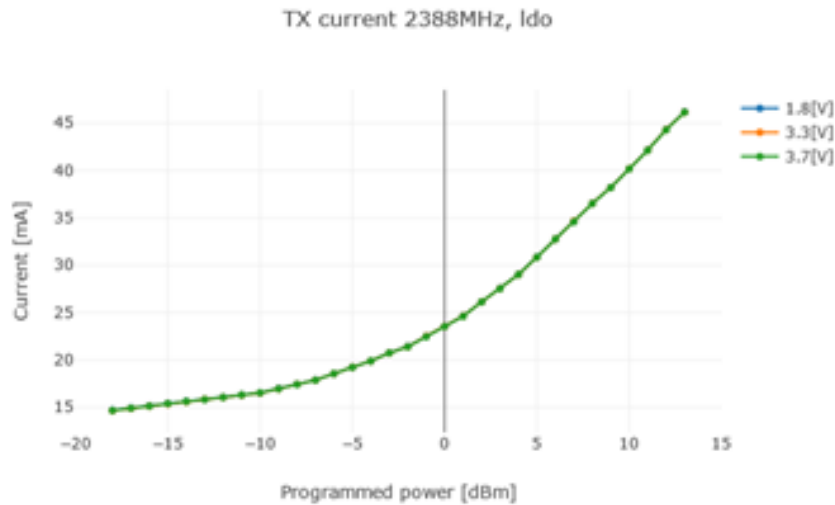


Figure 9-13: IDDTX vs TxPower, High Frequency PA, LDO Configuration

Figure 9-12 and Figure 9-13 show that the power efficiency of the High Frequency PA is maximized when the internal DC-DC regulator is used at, or above, 3.3V.

9.4 Impedance Matching Networks

The high power PA and low power PAs are available on the RFO_HP_LF and RFO_LP_LF pins respectively. They are connected to the antenna through a dedicated impedance matching network, which aims at presenting the optimized load at the output pins when loaded with 50Ohms at the antenna level.

The High Frequency PA is available on the RFIO-HF pin. It is internally matched at 50Ohms, reducing the need of external components for HF operation. Some components present on the RFIO_HF pin in the Semtech reference design are aimed at providing filtering to comply with the RF regulations.

9.4.1 Multi-Band Operation

It is possible to implement a multi-band configuration using a single impedance matching network, allowing the same set of SMD components to cover multiple sub-GHz ISM bands. The tables below show the optimal settings for the PA when using the Semtech matching network obtained on Semtech's reference design. The user can fine-tune the *PaDutyCycle* and *PaHpSel* according to their requirements of matching network, efficiency, output power, and harmonic emission. Please note that the effect of *PaDutyCycle* and *PaHpSel* on the efficiency, output power, and harmonic emission are PCB-design dependent, therefore the PA settings shown in [Table 9-1](#) and [Table 9-2](#) must be optimized on the user's application.

The matching network implementation proposed by Semtech is optimized for +22dBm and +15dBm for the higher ISM bands, i.e. a 868-928MHz operation.

Table 9-1: Optimized Settings for LP PA with Same Matching Network

Target Power	TxPower	PaSel	RegPASupply	PaDutyCycle	PaHPSEL
+15dBm	14	0	0	7	-
+14dBm	14	0	0	5	-
+10dBm	14	0	0	1	-

Table 9-2: Optimized Settings for HP PA with Same Matching Network

Target Power	TxPower	PaSel	RegPASupply	PaDutyCycle	PaHPSEL
+22dBm	22	1	1	4	7
+20dBm	22	1	1	3	7
+17dBm	22	1	1	1	5
+14dBm	22	1	1	1	3

Table 9-3: Optimized Settings for HF PA with the Same Matching Network

Target Power	TxPower	PaSel	RegPASupply	PaDutyCycle	PaHPSEL
+13dBm	13	2	0	0	0
+11dBm	12	2	0	3	0
+10dBm	11	2	0	4	0

9.4.2 RF Switch Implementation

The implementation examples hereafter show a combined high power PA and high efficiency PA operation, with the use of a 3-port RF switch SP3T. A single-band operation is also possible, the unused PA pin being left unconnected. In that case a 2-port RF switch SPDT would be necessary.

The RF switch implementation optimizes the impedance presented to the PA and the impedance presented to the LNA separately. Therefore one can optimize TX efficiency without compromising RX sensitivity.

The RF switch can be controlled either by the host controller, or by the LR1120 itself (pins DIO5, DIO6, DIO7, DIO8 and DIO10), using the `SetDioAsRfSwitch(...)` command.

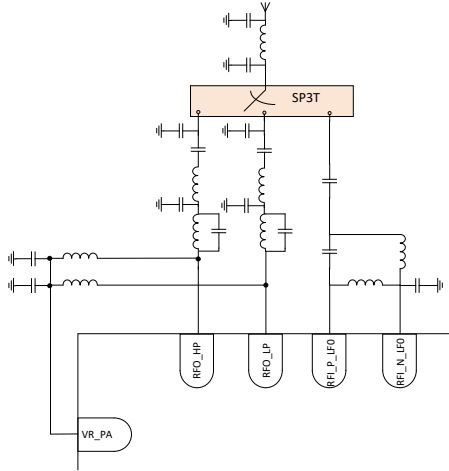


Figure 9-14: RF Switch, Double PA Operation

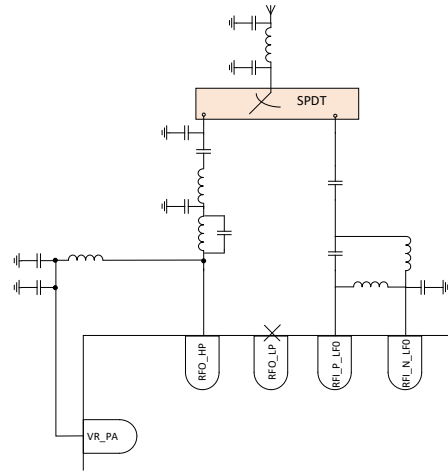


Figure 9-15: RF Switch, Single PA Operation (High Power PA Example)

9.4.3 Direct-Tie Implementation

In case of a cost-sensitive application, it is possible to get rid of the RF switch, and implement a so-called direct-tie implementation.

In such a configuration, the PA and the RX differential stages are connected as depicted in the figure hereafter. Please note that series capacitances are required between the PA and the RX stage in order to avoid damaging the LR1120 due to current flow in the RX stage.

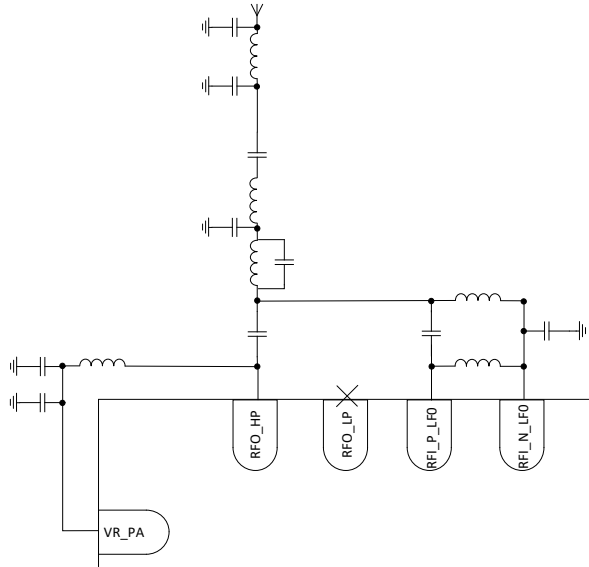


Figure 9-16: Single Tie Implementation: Only One PA Used (High Power PA Example)

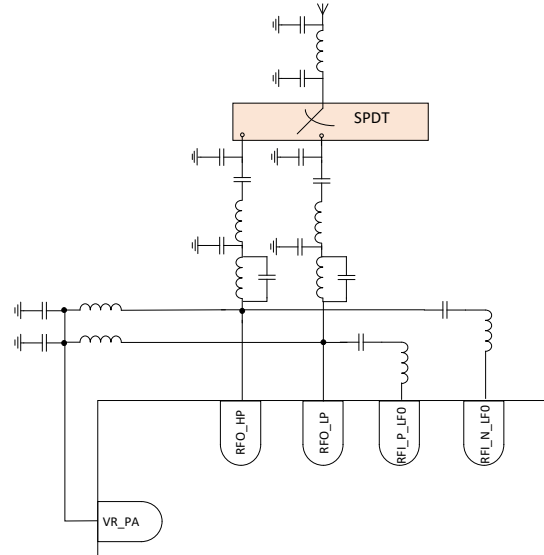


Figure 9-17: Single Tie Implementation: Both PAs Used (High Power PA Example)

Compared to the switched implementation, the direct-tie suffers a trade-off between TX efficiency and RX sensitivity. This is unavoidable because the transmitter and receiver require different optimal impedances, which may not be simultaneously feasible. In the case of a direct-tie, the user should expect a degradation of 2 ~ 3dB in RX sensitivity.

9.5 Commands

9.5.1 SetPaConfig

Command `SetPaConfig(...)` selects which PA to use and configures the supply of this PA.

Table 9-4: SetPaConfig Command

Byte	0	1	2	3	4	5
Data from Host	0x02	0x15	PaSel	RegPaSupply	PaDutyCycle	PaHPSEL
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

- *PaSel* selects the PA:
 - ♦ 0x00: Selects the low power PA
 - ♦ 0x01: Selects the high power PA
 - ♦ 0x02: Selects the high frequency PA
- *RegPaSupply* selects the PA power source:
 - ♦ 0x00: Powers the PA from the internal regulator
 - ♦ 0x01: Powers the PA from VBAT. The user must use *RegPaSupply* = 0x01 whenever *TxPower* > 14
- *PaDutyCycle* controls the duty cycle of each PA. Its default value is 0x04.

Table 9-5: PaDutyCycle Parameter Allowed Ranges

Low Power PA	High Power PA	High Frequency PA
For frequency $\geq 400\text{MHz}$, $0 \leq \text{PaDutyCycle} \leq 0x07$	At PaHpSel $\geq 0x05$, $0 \leq \text{PaDutyCycle} \leq 0x04$	$0 \leq \text{PaDutyCycle} \leq 0x04$
For frequency < 400MHz, $0 \leq \text{PaDutyCycle} \leq 0x04$	At PaHpSel $\leq 0x04$, $0 \leq \text{PaDutyCycle} \leq 0x07$	

- *PaHPSEL* controls the size of the high power PA.

9.5.2 SetTxParams

Command `SetTxParams(...)` sets the Tx Power and Ramp Time of the selected PA. `SetPaConfig(...)` must be sent prior to this command.

Table 9-6: SetTxParams Command

Byte	0	1	2	3
Data from Host	0x02	0x11	TxPower	RampTime
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:16)

- *TxPower* defines the output power in dBm in a range of:
 - ♦ - 17dBm (0xEF) to +14dBm (0x0E) by steps of 1dB if the low power PA is selected
 - ♦ - 9dBm (0xF7) to +22dBm (0x16) by steps of 1dB if the high power PA is selected
 - ♦ -18dBm (0xEE) to +13dBm (0x0F) by steps of 1dB if the high frequency PA is selected
 - ♦ If *TxPower* > +14dBm, the user must select the VBAT supply for the PA using the `SetPaConfig` command
- *RampTime* defines the PA power ramping time, which can be from 16 to 304μs according to the following table:

Table 9-7: RampTime Values

RampTime	Value	Ramp Time in μs
SET_RAMP_16U	0x00	16
SET_RAMP_32U	0x01	32
SET_RAMP_48U	0x02	48
SET_RAMP_64U	0x03	64
SET_RAMP_80U	0x04	80
SET_RAMP_96U	0x05	96
SET_RAMP_112U	0x06	112
SET_RAMP_128U	0x07	128
SET_RAMP_144U	0x08	144
SET_RAMP_160U	0x09	160
SET_RAMP_176U	0x0A	176
SET_RAMP_192U	0x0B	192
SET_RAMP_208U	0x0C	208
SET_RAMP_240U	0x0D	240
SET_RAMP_272U	0x0E	272
SET_RAMP_304U	0x0F	304

A *RampTime* of 48μs allows the best trade-off between a fast RF power establishment and minimum RF spurious emissions, therefore complying with radio standards. A 10us ramp time is not compliant with Australian or New-Zealand radio regulations.

10. Data Buffer

The transceiver is equipped with a 256-byte RAM data buffer which is accessible in all modes except sleep mode. This RAM area is fully customizable by the user and allows access to either data for transmission or from the last packet reception.

Principle Of Operation

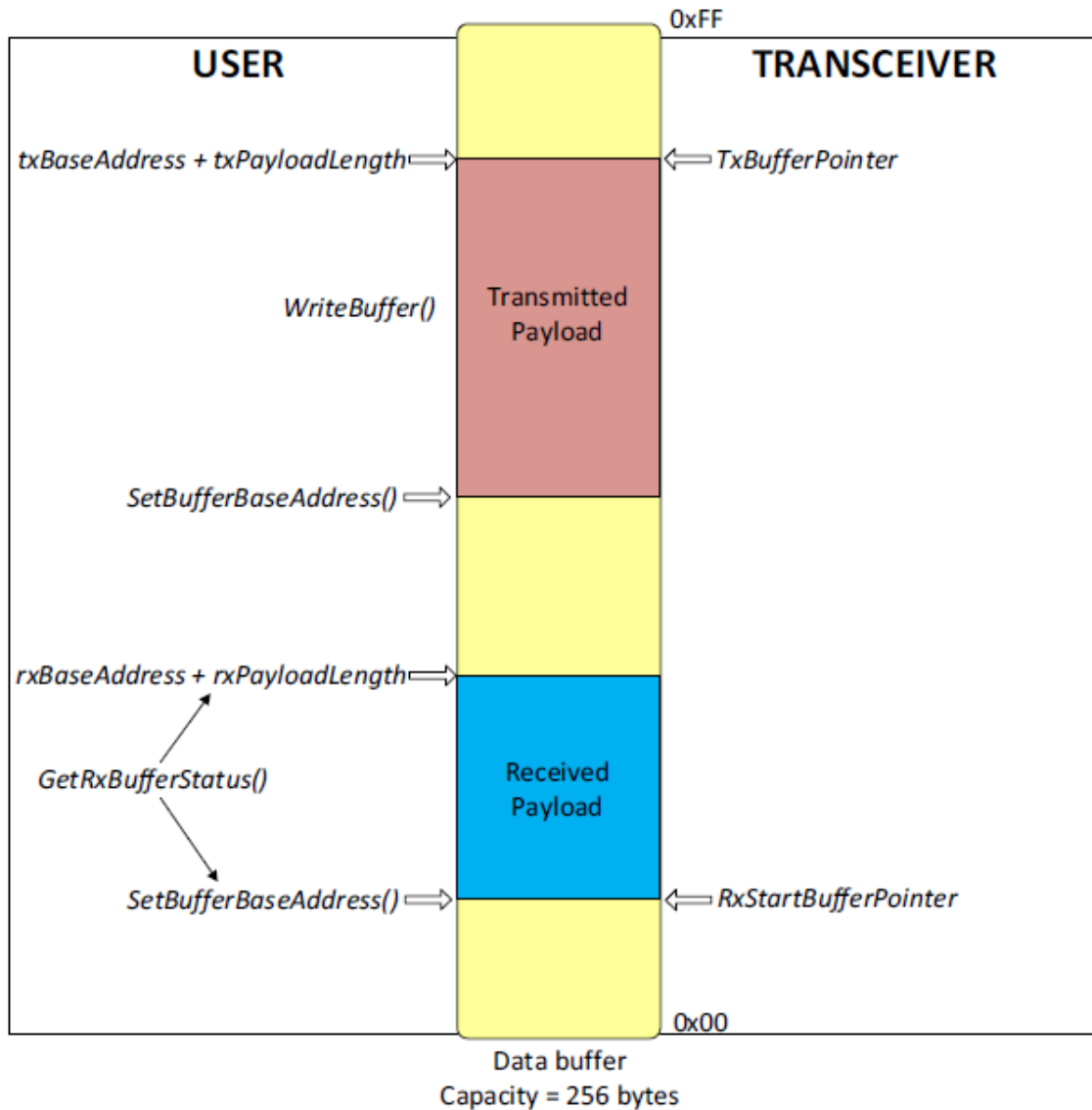


Figure 10-1: Data buffer operation

The data buffer can be configured to store both transmit and receive payloads.

10.1 Data Buffer In Receive Mode

In receive mode, *RxBASEADDR* specifies the buffer offset in memory at which the received packet payload data is written. The buffer offset of the last byte written in receive mode is then stored in *RxDATAPOINTER* which is initialized to the value of *RxBASEADDR* at the beginning of the reception.

The pointer to the first byte of the last packet received and the packet length can be read with command `GetRxbufferStatus`.

In single mode, *RxDATAPOINTER* is automatically initialized to *RxBASEADDR* each time the transceiver enters RX mode.

In continuous mode the pointer is incremented starting from the previous position.

10.2 Data Buffer In Transmit Mode

Upon each transition to transmit mode, *TxDATAPOINTER* is initialized to *TxBASEADDR* and is incremented each time a byte is sent over the air. This operation stops once the number of bytes sent equals the *Payloadlength* parameter as defined in the function `SetPacketParams`.

10.3 Using the Data Buffer

Both *RxBASEADDR* and *TxBASEADDR* are set using command `SetBufferBaseAddresses(...)`.

By default *RxBASEADDR* and *TxBASEADDR* are initialized at address 0x00.

Due to the contiguous nature of the data buffer, the base addresses for TX and RX are fully configurable across the 256-byte memory area. Each pointer can be set independently anywhere within the buffer. To exploit the maximum data buffer size in transmit or receive mode, the whole data buffer can be used in each mode by setting the base addresses *TxBASEADDR* and *RxBASEADDR* at the bottom of the memory (0x00).

The data buffer is cleared when the device is put into Sleep mode (implying no access). The data is retained in all other modes of operation.

The data buffer is accessed via commands `WriteBuffer` and `ReadBuffer`. In these functions the parameter offset defines the address pointer of the first data to be written or read. Offset zero defines the first position of the data buffer.

Before any read or write operation it is hence necessary to initialize this offset to the corresponding beginning of the buffer. Upon reading or writing to the data buffer the address pointer then increments automatically.

Two possibilities exist to obtain the offset value:

- Use the *RxBASEADDR* value since the user defines it before receiving a payload.
- Initialize the offset with the value of *RxStartBufferPointer* returned by `GetRxbufferStatus` command.

Note: All the received data is written to the data buffer even if the CRC is invalid, permitting user-defined post processing of corrupted data. When receiving, if the packet size exceeds the buffer memory allocated for the RX, it overwrites the transmit portion of the data buffer.

11. Wi-Fi Passive Scanning

The device provides device geolocation by scanning and processing of 802.11b/g/n Wi-Fi signals of opportunity in an energy efficient manner using three main concepts:

- ♦ Access Points MAC address extraction.
- ♦ Wi-Fi network SSID extraction.
- ♦ Country Code extraction.

The Wi-Fi types and Wi-Fi channels to scan are configurable:

- ♦ Wi-Fi types B and/or G/N:
 - ♦ When configuring the Wi-Fi signal to G/N, both 802.11g and 802.11n signals can be detected by the LR1120.
 - ♦ The device does not allow to scan for only 802.11g signals or only 802.11n signals.
- ♦ Wi-Fi channels:
 - ♦ From Channel ID 1 to Channel ID 14.

The maximal number of results to return is also configurable:

- ♦ From 1 to 32.
- ♦ This parameter can stop Wi-Fi passive scanning as soon as the required number of results is reached.

MAC addresses obtained during Wi-Fi scan can be resolved to a location using LoRa Cloud services or any other Wi-Fi solver provider.

11.1 Principle of Operation

The Wi-Fi passive scan entails the repetition of three steps:

- Preamble search
- Signal capture
- Signal demodulation

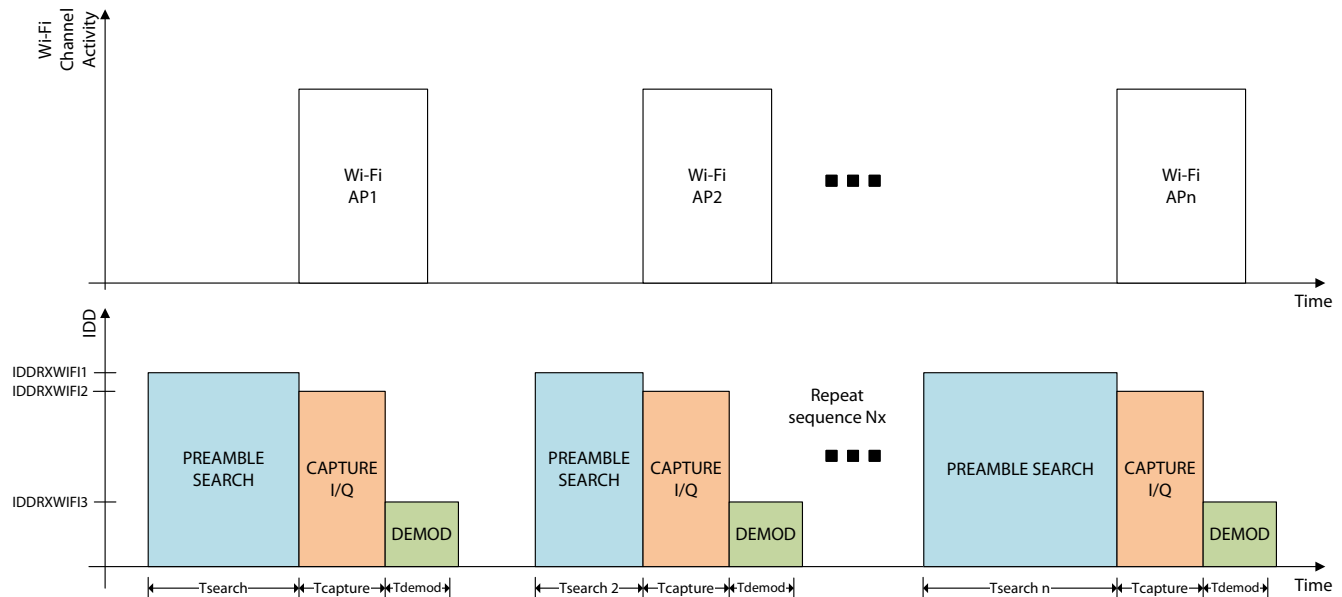


Figure 11-1: Wi-Fi Passive Scanning Sequence

11.1.1 Repetition Schemes

The device has two possible repetition schemes, which differ in the way they balance scan duration and power consumption:

- A maximal number of repetitions, used by `WifiScan` and `WifiCountryCode` commands:
 - ♦ Controls the maximal power consumed by the scan operation.
 - ♦ Configured by setting the maximal number of repetitions to execute per channel, per Wi-Fi type (*NbScanPerChan*).
- A maximal passive scan duration, used by `WifiScanTimeLimit` and `WifiCountryCodeTimeLimit` commands:
 - ♦ Controls the availability of the device for other tasks. Typically useful in the context of time slotted radio communications where a limited amount of time is available for Wi-Fi passive scanning before the next time slot.
 - ♦ Configured by setting the maximal duration of scan to spend per channel, per Wi-Fi type (*ScanTimePerChannel*).

11.1.2 Preamble Search Step

The duration of the preamble search step is configurable by the user. During this time, the radio is in reception mode configured for a Wi-Fi channel and a Wi-Fi signal type, waiting for a Wi-Fi preamble.

Note: When the LR1120 is configured to use a TCXO, the very first preamble search steps include the starting time of the TCXO. This happens only for the first preamble search as the TCXO is kept in running state until the end of the scanning.

When a Wi-Fi preamble is detected, the signal capture step is executed. If no Wi-Fi preamble has been detected by the radio during the configured duration, one of the following actions is automatically executed by the LR1120:

- Start another preamble search on the same Wi-Fi channel and same Wi-Fi type if:
 - ♦ Maximal number of repetitions has not been reached (for maximal number repetition) *NbScanPerChan*, or
 - ♦ Maximal duration has not been reached (for maximal duration repetition).
- Start another preamble search on next Wi-Fi channel, same Wi-Fi type if:
 - ♦ Maximal number of repetitions has been reached (for maximal number repetition), or
 - ♦ Maximal duration has been reached (for maximal duration repetition).
- Start preamble search on next Wi-Fi type, first channel, if:
 - ♦ All configured channels have been scanned for the current Wi-Fi type.
- Stop Wi-Fi scan if all channels have been scanned for all Wi-Fi types *NbMaxRes*.

11.1.3 Signal Capture Step

This step is executed as soon as the preamble search steps detected a Wi-Fi preamble. During this step, part of the Wi-Fi signal is captured and stored, only part of the Wi-Fi signal is stored, in order to reduce the power consumption. At the end of the signal capture step, the LR1120 automatically starts the signal demodulation step. There are two types of capture to define the length of the captured signal:

- Short capture:
 - ♦ Executed when the following acquisition modes are configured:
 - ♦ Beacon search mode (0x01)
 - ♦ Beacon and Packet search mode (0x02)
 - ♦ Reaches the lowest power consumption.
 - ♦ Capture duration is:
 - ♦ 500 microseconds in Wi-Fi type B
 - ♦ 100 microseconds in Wi-Fi type G/N
- Long capture:
 - ♦ Executed when:
 - ♦ Acquisition mode Full beacon search mode (0x04) is selected, or
 - ♦ `WifiCountryCode` or `WifiCountryCodeTimeLimit` is used.
 - ♦ Only available for Wi-Fi type B.
 - ♦ Extracts more information from the Wi-Fi signal, but increases power consumption
 - ♦ Capture time is 3 milliseconds

11.1.4 Signal Demodulation Step

During this step the device demodulates the captured Wi-Fi signal and extracts the result fields from it. The fields extracted depend on the acquisition mode.

For every captured signal, the extracted fields are stored in retention RAM memory.

The duration of this step depends on the capture type (short or long) and on the data rate of the signal received.

When this step terminates:

- If the configured number of results is not reached, the device automatically starts a new sequence of preamble search steps.
- If the configured number of results is reached, the device stops scanning.

11.2 Wi-Fi Commands

11.2.1 List of Wi-Fi Commands

Table 11-1: Summary Of Available Wi-Fi Commands

Command Name	Details
WifiScan	11.2.2 WifiScan
WifiScanTimeLimit	11.2.3 WifiScanTimeLimit
WifiCountryCode	11.2.4 WifiCountryCode
WifiCountryCodeTimeLimit	11.2.5 WifiCountryCodeTimeLimit
WifiGetNbResults	11.2.6 WifiGetNbResults
WifiReadResults	11.2.7 WifiReadResults
WifiResetCumulTimings	11.2.8 WifiResetCumulTimings
WifiReadCumulTimings	11.2.9 WifiReadCumulTimings
WifiGetNbCountryCodeResults	11.2.10 WifiGetNbCountryCodeResults
WifiReadCountryCodeResults	11.2.11 WifiReadCountryCodeResults
WifiCfgTimestampAPphone	11.2.12 WifiCfgTimestampAPphone
WifiReadVersion	11.2.13 WifiReadVersion

11.2.2 WifiScan

Captures Wi-Fi packets on the RFIO_HF pin. During Wi-Fi passive scanning, the BUSY signal is set high, indicating that the LR1120 is not ready to accept a command from the host. This can take a few hundred milliseconds, depending on the Wi-Fi passive scanning parameters. The BUSY signal returns to low when the Wi-Fi passive scanning procedure is complete.

If the *WifiScanDone* interrupt has been enabled, the IRQ signal goes high at the end of the Wi-Fi passive scanning process on the given channel mask for the given signal type.

Table 11-2: WifiScan Command

Byte	0	1	2	3	4	5	6	7	8	9	10
Data from Host	0x03	0x00	Signal Type	ChanMask (15:8)	ChanMask (7:0)	Acq Mode	NbMax Res	NbScan PerChan	Timeout (15:8)	Timeout (7:0)	AbortOn Timeout
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0	0	0	0	0

- *SignalType* defines the type of the 802.11 signal to be scanned:
 - ♦ 0x01: Wi-Fi 802.11b type
 - ♦ 0x02: Wi-Fi 802.11g type
 - ♦ 0x03: Wi-Fi 802.11n type
 - ♦ 0x04: All signals: Wi-Fi b, then Wi-Fi g/n on the same channel
 - ♦ 0x05-0xFF: RFU
- *ChanMask* defines which Wi-Fi channels to be scanned:
 - ♦ [0 0 Ch14 Ch13 Ch12 Ch11 Ch10 Ch9 Ch8 Ch7 Ch6 Ch5 Ch4 Ch3 Ch2 Ch1]
 - ♦ A channel bit set to 1 indicates that this channel must be scanned
- *AcqMode* indicates the *WifiScan* acquisition mode:
 - ♦ 0x01: Beacon search mode. Use only the Wi-Fi beacons to extract the MAC addresses.
 - ♦ 0x02: Beacon and Packet search mode. Use both the Wi-Fi beacons and Wi-Fi data packets to extract the MAC addresses.
 - ♦ 0x03: Full traffic mode. Recover all type/subtype packets. Same capture as AcquisitionMode 0x01 and 0x02.
 - ♦ 0x04: Full beacon mode. Scan for Beacons and Probe responses until Frame Check Sequence (FCS) field.
 - ♦ 0x05: SSID Beacon search mode. Capture and demodulate until the end of maximum possible SSIDs of a beacon or probe response. Fields until SSID are extracted and written in full results. Supported for Wi-Fi 802.11 b/g types only.
 - ♦ Other values are RFU.
- *NbMaxRes*: Maximum number of different MAC addresses allowed as a result for all scans on various channels and Wi-Fi types (must be inferior or equal to 32). If this number is reached, passive scanning is stopped. If a MAC address already present in the result structure is detected a second time with a different RSSI value, then the new result is ignored.
- *NbScanPerChan*: Number of Wi-Fi passive scans to be executed per channel (range: 1 to 255). If *NbMaxRes* results are found, scan is stopped.
- *Timeout*: 16-bit timeout of the Preamble Search mode in ms. Range: 1: 65535ms. For example, for a beacon period of 102.4ms, a 105ms timeout value can be set to ensure the *WifiScan* covers the whole beacon period.
- *AbortOnTimeout*: If set to 1, when a timeout preamble detect occurs, the passive scanning on this channel is aborted and the device jumps to the other channel to scan.

For example, the configuration (Scan Wi-Fi b / Channels 1, 6, 11 / Beacon and Packet search mode / 10 Maximum Results / 6 scans per channel / 70 ms timeout for Preamble Search mode / No abort on timeout) is coded as:

Opcode	SignalType	ChanMask	AcqMode	NbMaxRes	NbScanPerChan	Timeout	AbortOnTimeout
0x0300	0x01	0x0421	0x02	0x0A	0x06	0x0046	0x00

11.2.3 WifiScanTimeLimit

This API searches for Wi-Fi MAC addresses during a configurable maximal amount of time. The API is as follows:

Table 11-3: WifiScanTimeLimit Command

Byte	0	1	2	3	4	5	6	7	8	9	10
Data from Host	0x03	0x01	Signal Type	ChanMask (15:8)	ChanMask (7:0)	Acq Mode	NbMax Res	ScanTime PerChannel (15:8)	ScanTime PerChannel (7:0)	Timeout PerScan (15:8)	Timeout PerScan (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00	0x00	0x00

- *SignalType*: See definition in [11.2.2 WifiScan](#)
- *ChanMask*: See definition in [11.2.2 WifiScan](#)
- *AcqMode*: See definition in [11.2.2 WifiScan](#)
- *NbMaxRes*: See definition in [11.2.2 WifiScan](#)
- *ScanTimePerChannel*: Maximal duration of a scan on a single channel. Expressed in ms. Possible values go from 1ms to 65535ms included.
- *TimeoutPerScan*: Maximal duration of the wait for preamble. Expressed in ms. Possible values go from 0ms to 65535ms. Each time a preamble is detected and a capture is done, the counter of this timeout is restarted for the next scan. If set to 0ms, the LR1120 stays in preamble search for a maximal duration defined by *ScanTimePerChannel*.

The duration of a scan is (number of channels to scan) x (configured *ScanTimePerChannel*). However, this duration may be exceeded depending on the crystal drift of the clock source and on the instant the last Wi-Fi signal is detected by the device. Therefore the maximal duration of a Wi-Fi scan with this API is provided by the following equation:

$T_{max} = N_{channel} \times (1 + Xtal_precision) \text{ Timeout per Channel} + T_{offset}$, where:

- *Xtal_precision* depends on the clock source crystal. If clock source is 32kHz internal RC, then *Xtal_precision* = 1/100
- *T_offset* depends on the *SignalType* and the *AcqMode* selected for Wi-Fi B:
 - ♦ Wi-Fi b:
 - ♦ if Acquisition Mode != LR1120_WIFI_SCAN_MODE_FULL_BEACON: 2.31ms
 - ♦ if Acquisition Mode == LR1120_WIFI_SCAN_MODE_FULL_BEACON: 9.59ms
 - ♦ Wi-Fi g: 52.55ms

11.2.4 WifiCountryCode

This is the main function to extract Country code from Beacon or Probe Response. Only Wi-Fi b signals are searched.

Country code results are filtered to not have duplicates by comparing MAC addresses associated with the country code.

The number of parameters is tested and the range of each parameter is tested. If a range is not respected or a parameter is missing, a CMD_PERR status is returned. If CMD_FAIL is returned, it's due to radio configuration errors.

Table 11-4: WifiCountryCode Command

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x03	0x02	ChanMask (15:8)	ChanMask (7:0)	NbMaxRes	NbScan PerChannel	Timeout (15:8)	Timeout (7:0)	AbortOnTimeout
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0	0	0

- *ChanMask*: Mask of channels to scan. 14 bits from LSB:
 - ♦ [0 0 ch14 ch13 ch12 ch11 ch10 ch9 ch8 ch7 ch6 ch5 ch4 ch3 ch2 ch1]
 - ♦ Channel bit at 1 indicates that this channel must be scanned
- *NbMaxRes*: See definition in [11.2.2 WifiScan](#)
- *NbScanPerChannel*: See definition in [11.2.2 WifiScan](#)
- *Timeout*: See definition in [11.2.2 WifiScan](#)
- *AbortOnTimeout*: See definition in [11.2.2 WifiScan](#)

Example: [Table 11-5: WifiCountryCode Example](#) shows a search for:

- Country code channels 1, 6 and 11
- For a maximum of 20 results
- 15 scans per channel
- 105ms timeout for preamble detection phase
- Don't abort if timeout on preamble detection is triggered

Table 11-5: WifiCountryCode Example

Opcode	ChanMask	NbMaxRes	NbScanPerChannel	Timeout	AbortOnTimeout
0x03 0x02	0x04 0x21	0x14	0x0F	0x00 0x46	0x00

11.2.5 WifiCountryCodeTimeLimit

This API searches for Wi-Fi MAC addresses during a configurable maximal amount of time. The API is as follows:

Table 11-6: WifiCountryCodeTimeLimit Command

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x03	0x03	ChanMask (15:8)	ChanMask (7:0)	NbMaxRes (7:0)	ScanTime PerChannel (15:8)	ScanTime PerChannel (7:0)	Timeout PerScan (15:0)	Timeout PerScan (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00

- *ChanMask*: See definition in [11.2.2 WifiScan](#)
- *NbMaxRes*: See definition in [11.2.2 WifiScan](#)
- *ScanTimePerChannel*: See definition in [11.2.3 WifiScanTimeLimit](#)
- *TimeoutPerScan*: Maximal duration of the wait for preamble. Expressed in ms. Possible values go from 0ms to $2^{16} - 1$ ms (or 65535). Each time a preamble is detected and a capture is done, the counter of this timeout is restarted for the next scan. If set to 0ms, the LR1120 stays in preamble search for a maximal duration defined by *ScanTimePerChannel*.

The maximal duration of a scan is determined by the number of channels to scan, times the *ScanTimePerChannel* configured. However, this duration may be exceeded depending on the crystal drift of the clock source and on the instant the last Wi-Fi signal is detected by the device. Therefore the maximal duration of a Wi-Fi scan with this API is provided by the following equation:

$T_{max} = N_{channel} \times ((1 + Xtal_precision) \text{ Timeout per Channel} + T_{offset})$, where:

- *Xtal_precision* depends on the clock source crystal. If clock source is 32kHz internal RC, then $Xtal_precision = 1/100$.
- *T_offset* is always 9.59ms.

11.2.6 WifiGetNbResults

Gives the number of Wi-Fi Scanning results.

The number of results is returned on 8 bits and can be read at the next SPI transaction.

Table 11-7: WifiGetNbResults Command

Byte	0	1
Data from Host	0x03	0x05
Data to Host	Stat1	Stat2

Table 11-8: WifiGetNbResults Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	NbResults

11.2.7 WifiReadResults

Reads the byte stream containing a defined number of Wi-Fi Passive Scanning results from a given index, in the requested format. It is necessary to issue the command `WifiGetNbResults(...)` before this command. NOP bytes (0x00) must be issued to read back the results.

Table 11-9: WifiReadResults Command

Byte	0	1	2	3	4
Data from Host	0x03	0x06	Index	NbResults	Format
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)

Table 11-10: WifiReadResults Response

Byte	0	1	2	...	N+1
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	ResultsByte0	ResultsByte1	...	ResultsByteN

- *Index*: Index of Wi-Fi Passive Scanning results to read, from 0 to 31.
- *NbResults*: Number of Wi-Fi AP MAC Addresses to read, from 1 to 32.
- *Format*: Format of the Wi-Fi Passive Scanning results to read, depending on the previous acquisition mode (see [Table 11-23](#)):
 - ♦ 1: Basic Complete results format
 - ♦ 4: Basic MAC/Type/Channel Results format
 - ♦ Other values are RFU

For example, in order to read the results of a Wi-Fi Passive Scanning that returned 6 MAC Addresses in Basic MAC/Type/Channel Results format, the user shall send:

Table 11-11: Example to Read Basic Results of Passive Scan

Opcode	Index	NbResults	Format
0x0306	0x00	0x06	0x04

The result data is sent in a stream of $6 \times 9 = 54$ bytes.

The maximum number of bytes that can be read from one `WifiReadResults(...)` command is 1020 bytes. Therefore if the size to read is greater than 1020 bytes, the read operation must be separated into two requests.

11.2.8 WifiResetCumulTimings

Resets the Wi-Fi Passive Scanning cumulative timings (refer to [11.2.9 WifiReadCumulTimings](#)).

This command must be called prior to the executing the Wi-Fi Passive Scanning, in order to initialize the Wi-Fi Passive Scanning cumulative timings if those are to be read.

Table 11-12: WifiResetCumulTimings Command

Byte	0	1
Data from Host	0x03	0x07
Data to Host	Stat1	Stat2

11.2.9 WifiReadCumulTimings

Reads the Wi-Fi Passive Scanning cumulative timings, coded on 16 bytes, coded as in [Table 11-13: Wi-Fi Cumulative Timings Description](#). The Cumulative Timing represents the total time in the various modes during a `WifiScan(...)` command, therefore summed up for all Wi-Fi acquisitions, over the different `WifiScan(...)` parameters (Wi-Fi Types, Wi-Fi channels and so on). These timings are expressed in microseconds.

Table 11-13: Wi-Fi Cumulative Timings Description

Byte	0:3	4:7	8:11	12:15
Meaning	RFU	Total duration in preamble detection mode	Total duration in capture mode	Total duration in demodulation mode

This cumulative timing can be read regularly to compute the energy consumption of the device for Wi-Fi Passive Scanning operations. All 16 bytes shall be read. Cumulative timing must be reset by the host.

Table 11-14: WifiReadCumulTimings Command

Byte	0	1
Data from Host	0x03	0x08
Data to Host	Stat1	Stat2

Table 11-15: WifiReadCumulTimings Response

Byte	0	1	2	...	17
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	Byte 0	Byte 1	...	Last Byte

11.2.10 WifiGetNbCountryCodeResults

Returns the number of results after Country Code scanning execution by WifiCountryCode or WifiCountryCodeTimeLimit

Table 11-16: WifiGetNbCountryCodeResults Command

Byte	0	1
Data from Host	0x03	0x09
Data to Host	Stat1	Stat2

commands.

Table 11-17: WifiGetNbCountryCodeResults Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	NbResults

11.2.11 WifiReadCountryCodeResults

Reads the byte stream containing a defined number of Wi-Fi Passive Scanning Country Code results from a given index. It is necessary to issue the command `WifiGetNbCountryCodeResults(...)` before this command.

NOP bytes (0x00) must be issued to read back the results. The size of one Country Code result is 10 bytes.

Table 11-18: WifiReadCountryCodeResults Command

Byte	0	1	2	3
Data from Host	0x03	0x0A	Index	NbResults
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)

Table 11-19: WifiReadCountryCodeResults Response

Byte	0	1	2	----	N+1
Data from Host	0x00	0x00	0x00		0x00
Data to Host	Stat1	Byte0Res	Byte1Res		LastByteRes

- *Index*: Index of the Wi-Fi Passive Scanning Country Code result to start reading from, from 0 to 31.
- *NbResults*: Number of Wi-Fi Passive Scanning Country Code results to read, from 1 to 32.

Refer to [11.3.5 WifiCountryCode Result Format](#) for details concerning interpretation of Wi-Fi Passive Scanning Country Code results.

11.2.12 WifiCfgTimestampAPphone

Configures a timestamp threshold used to discriminate mobile access point from gateways.

Table 11-20: WifiCfgTimestampAPphone Command

Byte	0	1	2	3	4	5
Data from Host	0x03	0x0B	TimeStamp (31:24)	TimeStamp (23:16)	TimeStamp (15:8)	TimeStamp (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

- *TimeStamp*: Threshold timestamp value expressed in seconds. Default is 1 day. If the timestamp extracted from a beacon or probe response is greater than this limit, the field MAC validation from Channel information field of results indicates that the frame is probably from a gateway and not from a mobile device access point.

11.2.13 WifiReadVersion

Returns the internal Wi-Fi firmware version major and minor numbers.

Table 11-21: WifiReadVersion Command

Byte	0	1
Data from Host	0x03	0x20
Data to Host	Stat1	Stat2

Table 11-22: WifiReadVersion Response

Byte	0	1	2
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	VersionMajor	VersionMinor

11.3 Wi-Fi Results Formats

11.3.1 Wi-Fi Passive Scanning Result Formats

Various formats of Wi-Fi Passive Scanning Results are implemented, allowing the user to retrieve either:

- Basic MAC/Type/Channel results: The minimum set of geolocation information in order to optimize the application power consumption (Basic MAC/Type/Channel results format),
- Basic Complete results: The maximum amount of information available for the `WifiScan(...)` operation (Basic Complete results format).
- Extended Complete results: Basic Complete results with additional Information Elements.

Table 11-23: Wi-Fi Result Formats and Wi-Fi Scan Mode Relationship

Acquisition Mode value	Result Format value	Result format to read
0x01	0x01	Full Result (22 bytes per MAC address detected)
0x01	0x04	Basic MAC/Type/Channel Result (9 bytes per MAC address detected)
0x02	0x01	Full Result (22 bytes per MAC address detected)
0x02	0x04	Basic MAC/Type/Channel Result (9 bytes per MAC address detected)
0x04	0x01	Extended Complete Result (79 bytes per MAC address detected)

Other values are RFU.

11.3.2 Basic MAC/Type/Channel Result Format

The Basic MAC/Type/Channel Result Format is the result format returned by the LR1120 when reading results with format 0x04 after executing a Wi-Fi scan with acquisition mode 0x01 or 0x02.

This result structure is organized as a continuous series of MAC Address Basic MAC/Type/Channel Results, each coded on 9 bytes, defined in [Table 11-24: Basic Results Format per MAC Address](#) below. The maximum number of MAC Addresses reported is 32.

Table 11-24: Basic Results Format per MAC Address

Byte	0	1	2	3	4	5	6	7	8
Content	WifiType	ChannelInfo	RSSI	MAC6	MAC5	MAC4	MAC3	MAC2	MAC1

- *WifiType*: Coded on 8 bits:
 - ♦ Bits 0-1: Wi-Fi signal type:
 - ♦ 1: Wi-Fi b
 - ♦ 2: Wi-Fi g
 - ♦ 3: Wi-Fi n
 - ♦ Bits 2-7: *DatarateID*, coded as indicated in [Table 11-26: Wi-Fi DatarateID Field](#).
- *ChannelInfo*: coded on 8 bits:
 - ♦ Bits 0-3: *ChannelID*, coded as indicated in [Table 11-27: Wi-Fi ChannelID Field](#). *ChannelID* indicates the Wi-Fi channels configured for the scan.
 - ♦ Bits 4-5: *MacOrigin*, coded as indicated in [Table 11-28: Wi-Fi MacOrigin Field](#). *MacOrigin* indicates if the MAC address belongs to a gateway, to a phone, or if it is undetermined because MAC address is extracted from a packet.
 - ♦ Bit 6: *RssiValidation*: indicates if the signal comes from an access point (therefore reliable for localization) or from an end device:
 - ♦ 0: Access point.
 - ♦ 1: End device.
 - ♦ Bit 7: Reserved.
- *RSSI*: RSSI value of the signal captured, coded on 8 bits.
- *MAC*: MAC address of the Access Point, coded on 6 bytes:
 - ♦ MAC6:MAC5:MAC4:MAC3:MAC2:MAC1, from MSB to LSB.

11.3.3 Full Result Format

The Full Result Format is the result format returned by the LR1120 when reading results with format 0x01 after executing a Wi-Fi scan with acquisition mode 0x01 or 0x02.

The full result structure is organized as continuous series of MAC Address Basic Complete results, each MAC Address Full Result being coded on 22 bytes, defined in [Table 11-25: Basic Complete Results Format per MAC Address](#) below. The maximum number of MAC Addresses reported is 32.

Table 11-25: Basic Complete Results Format per MAC Address

Byte	0	1	2	3	4	5	6	7
Content	WifiType	Channel Info	RSSI	FrameCtrl	MAC6	MAC5	MAC4	MAC3

Byte	8	9	10	11	12	13	14	15
Content	MAC2	MAC1	PhiOffset (15:8)	PhiOffset (7:0)	Timestamp (63:56)	Timestamp (55:48)	Timestamp (47:40)	Timestamp (39:32)

Byte	16	17	18	19	20	21
Content	Timestamp (31:24)	Timestamp (23:16)	Timestamp (15:8)	Timestamp (7:0)	PeriodBeacon (15:8)	PeriodBeacon (8:0)

- *WifiType*: Coded on 8 bits:
 - ♦ Bits 0-1: Wi-Fi signal type:
 - ♦ 1: Wi-Fi b
 - ♦ 2: Wi-Fi g
 - ♦ 3: Wi-Fi n
 - ♦ Bits 2-7: *DatarateID*, coded as indicated in [Table 11-26: Wi-Fi DatarateID Field](#).
- *ChannelInfo*: coded on 8 bits:
 - ♦ Bits 0-3: *ChannelID*, coded as indicated in [Table 11-27: Wi-Fi ChannelID Field](#).
 - ♦ Bits 4-5: *MacOrigin*, coded as indicated in [Table 11-28: Wi-Fi MacOrigin Field](#).
 - ♦ Bit 6: *RssiValidation*, indicates if the signal comes from an access point (therefore reliable for localization) or from an end device:
 - ♦ 0: Access point
 - ♦ 1: End device
 - ♦ Bit 7: Reserved.
- *RSSI*: RSSI value of the signal captured, coded on 8 bits.
- *FrameCtrl*: 16-bit frame control, coded as indicated in [Table 11-29: Wi-Fi FrameCtl Field](#).
- *MAC*: MAC address of the Access Point, coded on 6 bytes:
 - ♦ MAC6:MAC5:MAC4:MAC3:MAC2:MAC1, from MSB to LSB.
- *PhiOffset*: Coded on 2 bytes. Used to compute frequency offset of the signal.
- *Timestamp*: Indicates the number of microseconds the AP is active, coded on 64 bits.
- *PeriodBeacon*: Beacon period expressed in Time Units. Must be multiplied for the time in ms.

Table 11-26: Wi-Fi DatarateID Field

DatarateID	Signal type	Modulation	Coding rate	Datarate (Mbps)
1	Wi-Fi b	DBPSK		1
2		DQPSK		2
3	Wi-Fi g	BPSK	1/2	6
4		BPSK	3/4	9
5		QPSK	1/2	12
6		QPSK	3/4	18
7		16-QAM	1/2	24
8		16-QAM	3/4	36
11		BPSK	1/2	6.5
12		QPSK	1/2	13
13	Wi-Fi n mixed mode	QPSK	3/4	19.5
14		16-QAM	1/2	26
15		16-QAM	3/4	39
19		BPSK	1/2	7.2
20		QPSK	1/2	14.4
21		QPSK	3/4	21.7
22		16-QAM	1/2	28.9
23		16-QAM	3/4	43.3

Table 11-27: Wi-Fi ChannelID Field

Channel ID	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Center Freq (MHz)	2412	2417	2422	2427	2432	2437	2442	2447	2452	2457	2462	2467	2472	2484

Table 11-28: Wi-Fi MacOrigin Field

MacOrigin Value	Meaning
1	MAC Address from a gateway
2	MAC Address from a phone
3	Undetermined

Table 11-29: Wi-Fi FrameCtl Field

Bit	(0:1)	(2:5)	6	7
Fields	Type	SubType	ToDS	FromDS

The *FrameCtl* field is transmitted LSB first, and therefore represented accordingly.

- *Type*: 00: Management Frame; 01: Control Frame; 10: Data Frame; 11: Reserved.
- *SubType*: Coded as indicated in [Table 11-30: FrameCtl SubType Values](#).
- *ToDS*: 1 indicates that the data frame is going from the client station (STA) to the Distribution System (DS).
- *FromDS*: 1 indicates that the data frame is going from the Distribution System (DS) to the client Station (STA).

Table 11-30: FrameCtl SubType Values

Type	Subtype Value	SubType	Type	Subtype Value	SubType	Type	Subtype Value	SubType
00	0000	Assoc req	01	0000	Reserved	10	0000	Data
	0001	Assoc res		0001	Reserved		0001	Data +CF-ACK
	0010	Reassoc req		0010	Reserved		0010	Data +CF-Poll
	0011	Reassoc res		0011	Reserved		0011	DATA+CF-ACK/Poll
	0100	Probe Req		0100	Reserved		0100	Null
	0101	Probe res		0101	Reserved		0101	CF-ACK
	0110	Reserved		0110	Reserved		0110	CF-Poll
	0111	Reserved		0111	Reserved		0111	CF-ACK /Poll
	1000	Beacon		1000	Block ACK Req		1000	Qos Data
	1001	Announcement		1001	Block Acq		1001	Qos + CF-ACK
	1010	Diassoc		1010	PS-Poll		1010	Qos + CF-Poll
	1011	Auth		1011	RTS		1011	Qos + CF-ACL/Poll
	1100	Deauth		1100	CTS		1100	Qos Null
	1101	Action		1101	ACK		1101	Reserved
	1110	Reserved		1110	CF-End		1110	Qos + CF-Poll
	1111	Reserved		1111	CF-END +CF-ACK		1111	Qos + CF-ACK

11.3.4 Extended Complete Result Format

The Extended Complete Result Format is the result format returned by the LR1120 when reading results with format 0x01 after executing a Wi-Fi scan with acquisition mode 0x04.

Table 11-31: Extended Basic Complete Results Format per MAC Address

Byte	0	1	2	3	4	5	6	7
Content	WifiType	Channel Info	RSSI	Rate	Service (15:8)	Service (7:0)	Length (15:8)	Length (7:0)

Byte	8	9	10	11	12	13	14	15
Content	FrameCtrl (15:8)	FrameCtrl (7:0)	MAC_5_0	MAC_4_0	MAC_3_0	MAC_2_0	MAC_1_0	MAC_0_0

Byte	16	17	18	19	20	21	22	23
Content	MAC_5_1	MAC_4_1	MAC_3_1	MAC_2_1	MAC_1_1	MAC_0_1	MAC_5_1	MAC_4_1

Byte	24	25	26	27	28	29	30	31
Content	MAC_3_1	MAC_2_1	MAC_1_1	MAC_0_1	Timestamp(63:56)	Timestamp(55:48)	Timestamp(47:40)	Timestamp(39:32)

Byte	32	33	34	35	36	37	38	39
Content	Timestamp(31:24)	Timestamp(23:16)	Timestamp(15:8)	Timestamp(7:0)	Period Beacon (15:8)	Period Beacon (7:0)	SeqCtrl [15:8]	SeqCtrl [7:0]

Byte	40	41	42	43	44	...	...	70
Content	SSID[0]	SSID[1]	SSID[2]	SSID[3]	SSID[4]	...	...	SSID[30]

Byte	71	72	73	74	75	76	77	78
Content	SSID[31]	Current Channel	Country Code[0]	Country Code[1]	IOReg	FCS CheckOk	PhiOffset (15:8)	PhiOffset (7:0)

- *WifiType*: (8 bits):
 - ♦ Bits 0-1: Wi-Fi signal type:
 - ♦ 1: Wi-Fi b
 - ♦ 2: Wi-Fi g
 - ♦ 3: Wi-Fi n
 - ♦ Bits 2-7: *DatarateID*, coded as indicated in [Table 11-26: Wi-Fi DatarateID Field](#)
- *ChannelInfo*: (8 bits):
 - ♦ Bits 0-3: *ChannelID*, coded as indicated in [Table 11-27: Wi-Fi ChannelID Field](#)
 - ♦ Bits 4-5: *MacOrigin*, coded as indicated in [Table 11-28: Wi-Fi MacOrigin Field](#)
 - ♦ Bit 6: *RssiValidation*, indicates if the signal comes from an access point (therefore reliable for localization) or from an end device:
 - ♦ 0: Access point
 - ♦ 1: End device
 - ♦ Bit 7: Reserved
- *RSSI*: (8 bits) RSSI value of the signal captured.
- *Rate*: (16 bits) the meaning depends on Wi-Fi type:
 - ♦ Wi-Fi b (data rate)
 - ♦ 0x0A: 1Mb/s
 - ♦ 0x14: 2Mb/s
 - ♦ Wi-Fi g (data rate)
 - ♦ 0x0D: 6Mb/s
 - ♦ 0x0F: 9Mb/s
 - ♦ 0x05: 12Mb/s
 - ♦ 0x07: 18Mb/s
 - ♦ 0x09: 24Mb/s
 - ♦ 0x0B: 36Mb/s
 - ♦ 0x01: 48Mb/s
 - ♦ 0x03: 54Mb/s
 - ♦ Wi-Fi n: Modulation and Coding Scheme index (from 0 to 7)
- *Service*: Refer to IEEE Std 802.11, 2016, Part 11: Wireless LAN MAC and PHY Spec.
- *Length*: Refer to IEEE Std 802.11, 2016, Part 11: Wireless LAN MAC and PHY Spec.
- *FrameCtrl*: (16 bits) Frame control, coded as indicated in [Table 11-29: Wi-Fi FrameCtl Field](#).
- *MAC_x_Y*: The x byte of the Y MAC Address.
- *Timestamp*: Indicates the number of microseconds the AP is active, coded on 64 bits.
- *PeriodBeacon* (2 bytes): The beacon period expressed in Time Unit (TU). 1 TU is 1024 microseconds.
- *SeqCtrl*: Refer to IEEE Std 802.11, 2016, Part 11: Wireless LAN MAC and PHY Spec.
- *SSID* 0 - 31 (32 bytes): Service Set Identifier
- *CurrentChannel*: (1 byte) Current channel of the Wi-Fi frame as reported by the frame, in decimal.
- *CountryCode* (2 bytes).
- *IOReg*: Refer to IEEE Std 802.11, 2016, Part 11: Wireless LAN MAC and PHY Spec.
- *FCSCheckOk*: Indicates if the FCS of the frame has been checked and the check is OK.
- *PhiOffset*: Coded on 2 bytes. Used to compute frequency offset of the signal.

11.3.5 WifiCountryCode Result Format

Table 11-32: WifiCountryCode Result Format Sent Over SPI (12 bytes)

Byte	0	1	2	3	4	5	6	7	8	9
Content	Country Code[0]	Country Code[1]	IOReg	Channel Info	MAC6	MAC5	MAC4	MAC3	MAC2	MAC1

- *CountryCode*: 2 char encoded in ASCII that represents the country code:
 - ♦ Ex: 'FR' -> France, 'US' -> United States
- *IOReg*: One character encoded in ASCII that represents if the AP is indoor or outdoor or anywhere:
 - ♦ Ex: 'I' -> Indoor, 'O' -> Outdoor, 'Space (ASCII)' -> Any
- *ChannelInfo*: coded on 8 bits:
 - ♦ Bits 0-3: *ChannelID*, coded as indicated in [Table 11-27: Wi-Fi ChannelID Field](#). *ChannelID* indicates the Wi-Fi channels configured for the scan.
 - ♦ Bits 4-5: *MacOrigin*, coded as indicated in [Table 11-28: Wi-Fi MacOrigin Field](#). *MacOrigin* indicates if the MAC address belongs to a gateway, to a phone, or if it is undetermined because MAC address is extracted from a packet.
 - ♦ Bit 6: *RssiValidation*: indicates if the signal comes from an access point (therefore reliable for localization) or from an end device:
 - ♦ 0: Access point.
 - ♦ 1: End device.
 - ♦ Bit 7: Reserved.
- *MAC6-1*: MAC AP associated results.

12. GNSS Scanning

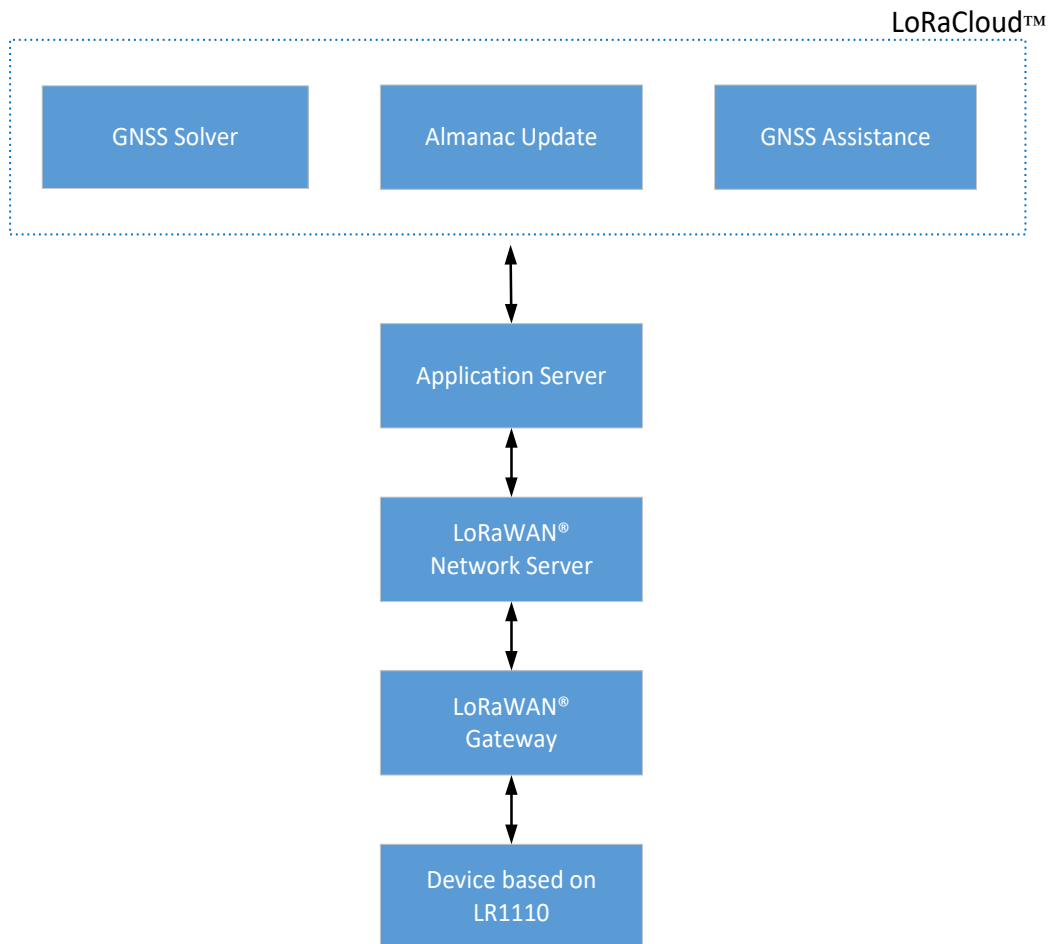


Figure 12-1: GNSS System Overview

12.1 GNSS Geolocation System Overview

The LR1120 features a GNSS receiver that allows a fast and energy efficient outdoor geolocation. The LR1120 GNSS Geolocation System achieves low energy geolocation by offloading time- and compute-intensive operations to back-end system components. In particular, the following three back-end system components are needed to operate the LR1120's GNSS Geolocation System:

- **GNSS Position Solving Component:** The LR1120 does not resolve the full position on-device. Instead, the measurements from GNSS signals are combined into a binary message (the NAV message) and expected to be sent via any communication channel to the GNSS Position Solver back-end component for final position calculation. This GNSS position solving component is required in all operation modes.
- **GNSS Almanac Update Component (required in assisted mode):** The LR1120 reduces the GNSS scanning time by taking into account coarse orbital parameters for different GNSS constellations (the Almanac parameters). In conjunction with a coarse time and position estimate, the LR1120 uses this information to optimize the search and acquisition of GNSS signals. Over time, the true satellite positions diverge from the fixed Almanac parameters, which requires them to be updated. This can be achieved by a back-end component which estimates the quality of the almanac image on device and issues updates when needed. Starting from LR1120 firmware 02.01, the Almanac data can be directly demodulated from the satellite signal.
- **GNSS Assistance Component (required in assisted mode):** In order to operate the GNSS Geolocation System in assisted mode, coarse estimates of time and position must be provided to the LR1120. This information can be obtained in a variety of ways including application-level knowledge. In LoRaWAN® the Application Layer Clock Synchronization protocol can retrieve assistance time information. The assistance position information can generally be derived from past position solutions.

LoRa Cloud™ offers these components in a single, easy to use, managed service as part of the Modem & Geolocation Services (MGS). Visit www.LoRacloud.com for more information.

Figure 12-1: GNSS System Overview shows the system components for a LoRaWAN®-enabled integration. In GNSS mode, the LR1120 searches for available GNSS signals and extracts the minimum set of satellite information needed for a position calculation. The GNSS satellite signal data (also referred as NAV message) is then transmitted via the LPWAN communication stack to a GNSS solver for the geolocation position calculation. If an update to the almanac parameters is needed, the Almanac Update Component schedules appropriate downlink messages. Starting from LR1120 firmware 02.01, the Almanac data can be directly demodulated from the satellite signal, removing the need to issuing downloads for the Almanac update process.

The LoRa Cloud solver allows both 2D and 3D solving. For a successful 3D solve, a minimum of 5 satellites is required for a Single constellation scan, and 6 satellites for a Double constellation scan. For a 2D solve, the minimum is 4 satellites in Single constellation, or 5 satellites in Double constellation. Moreover, a Double constellation scan requires at least two satellites in each constellation (for example, solving a scan of 5 GPS + 1 BeiDou SV will fail). Note that in Double constellation mode, if the LR1120 only detects one satellite in one of the two constellations, it generates a NAV message containing only one constellation.

Starting from LR1120 firmware 02.01, the device is able to demodulate the time information from the satellite signal, and the knowledge from a precise time reduces by one the number of satellites to be detected for a successful solve:

- For a successful 3D solve, a minimum of 4 satellites is required for a Single constellation scan, and 5 satellites for a Double constellation scan.
- For a successful 2D solve, the minimum is 3 satellites for a Single constellation scan, and 4 satellites for a Double constellation scan.

12.2 GNSS Principle of Operation

Two GNSS modes are implemented:

- GNSS autonomous mode does not require any assistance location or almanac data, and aims to detect strong satellite signals. Therefore it is suitable for outdoor conditions with good sky visibility.
- GNSS assisted mode allows the most efficient GNSS geolocation. Assistance information can build a list of the satellites in view at the current time and location, in order to reduce the GNSS satellites search space, and therefore optimize the time and energy spent geolocating. The assistance information is tailored to an LPWAN network, limiting the data sent, especially the downlink size and frequency. It consists of:
 - ♦ LR1120 approximate position
 - ♦ Current time
 - ♦ Up-to-date reduced size Almanac information (less than 3 months old)

Starting from LR1120 firmware 02.01, the device may run GNSS Scanning without needing assistance information provided to the LR1120:

- An integrated 2D solver determines locally the Assistance Position
- The GNSS time can be directly demodulated from the satellite signal, thus improving positioning accuracy through knowledge of a precise time base
- The LR1120 can also demodulate the Almanac data from the satellites" after "Two GNSS modes are implemented (...) Almanac information (less than 3 months old).

Starting from LR1120 firmware 02.01, the GNSS search mode does not have to be configured by the user, it is determined by the LR1120 firmware, based on the availability of the assistance information and on the actual detection of the visible satellites.

Determining assistance information autonomously allows continuous GNSS Scanning if there is scarce access to the network. In such cases, the GNSS scan results can also be stored by the host MCU for further geolocation resolution, and forwarded to the solver when network access is available, allowing to reconstruct the whole device's itinerary.

The LR1120 supports both GPS L1 and BeiDou B1 signals. It can perform either a single GNSS in any (or both) GPS and BeiDou constellations, or a dual GNSS BeiDou in any (or both) GPS and BeiDou constellations.

During the GNSS, the BUSY signal is set high, indicating that LR1120 is not ready to accept SPI transactions. BUSY returns to low when the procedure is complete. If the *GNSSDone* interrupt has been enabled, the IRQ signal goes high at the end of the GNSS process. Starting from LR1120 firmware 02.01, the GNSS scan can be interrupted.

To achieve the best GNSS sensitivity performance, an external LNA is necessary on the GNSS path. If the external GNSS LNA is controlled from the LR1120, the GNSS scan power consumption can be optimized.

A 32MHz TCXO and 32.768kHz clock source are mandatory for any GNSS operation.

A 32.768kHz clock source is optional for LR1120 firmware prior to version 02.01 for GNSS single Constellation and for GNSS Legacy scan mode.

Refer to sections [12.3.2 GnssSetConstellationToUse](#) and [12.3.5 GnssSetMode](#) for details.

The clock requirements associated with the GNSS feature of LoRa Edge devices are summarized in AN1200.74 LoRa Edge Clock Requirements.

Caution: Starting from LR1120 firmware version 02.01, the NAV message generated by GNSS Scan operations is NAV-3 format (all previous firmware versions used NAV format).

12.3 GNSS Commands

12.3.1 List of GNSS Commands

Table 12-1: Summary of Available GNSS Commands (Sheet 1 of 2)

Command Name	Details
GnssSetMode	12.3.5 GnssSetMode
GnssAlmanacUpdateFromSat ²	12.6.1 GnssAlmanacUpdateFromSat
GnssAssisted ¹	12.3.7 GnssAssisted
GnssAutonomous ¹	12.3.6 GnssAutonomous
GnssConfigAlmanacUpdatePeriod ²	12.6.3 GnssConfigAlmanacUpdatePeriod
GnssConfigDelayResetAP ²	12.3.17 GnssConfigDelayResetAP
GnssFetchTime ²	12.3.19 GnssFetchTime
GnssGetConsumption	12.5.6 GnssGetConsumption
GnssGetContextStatus	12.3.11 GnssGetContextStatus
GnssGetNbSvDetected	12.5.4 GnssGetNbSvDetected
GnssGetResultSize	12.5.2 GnssGetResultSize
GnssGetSvDetected	12.5.5 GnssGetSvDetected
GnssGetSvVisible	12.5.7 GnssGetSvVisible
GnssGetSvWarmStart ²	12.3.30 GnssGetSvWarmStart
GnssPushDmMsg ²	12.3.16 GnssPushDmMsg
GnssPushSolverMsg	12.3.15 GnssPushSolverMsg
GnssReadAlmanacStatus ²	12.6.2 GnssReadAlmanacStatus
GnssReadAlmanacUpdate	12.3.14 GnssReadAlmanacUpdate
GnssReadAlmanacUpdatePeriod ²	12.6.4 GnssReadAlmanacUpdatePeriod
GnssReadAssistancePosition	12.3.10 GnssReadAssistancePosition
GnssReadConstellationToUse	12.3.3 GnssReadConstellationToUse
GnssReadCumulTiming ²	12.3.25 GnssReadCumulTiming
GnssReadDelayResetAP ²	12.3.28 GnssReadDelayResetAP
GnssReadDemodStatus ²	12.3.24 GnssReadDemodStatus
GnssReadDopplerSolverRes ²	12.3.27 GnssReadDopplerSolverRes
GnssReadLastScanModeLaunched ²	12.3.18 GnssReadLastScanModeLaunched

Table 12-1: Summary of Available GNSS Commands (Sheet 2 of 2)

Command Name	Details
GnssReadResults	12.5.3 GnssReadResults
GnssReadSupportedConstellations	12.3.4 GnssReadSupportedConstellations
GnssReadTime ²	12.3.20 GnssReadTime
GnssReadVersion	12.3.12 GnssReadVersion
GnssReadWarmStartStatus ²	12.3.29 GnssReadWarmStartStatus
GnssReadWNRollover ²	12.3.23 GnssReadWNRollover
GnssResetPosition ²	12.3.22 GnssResetPosition
GnssResetTime ²	12.3.21 GnssResetTime
GnssScan ²	12.3.8 GnssScan
GnssSetAlmanacUpdate	12.3.13 GnssSetAlmanacUpdate
GnssSetAssistancePosition	12.3.9 GnssSetAssistancePosition
GnssSetConstellationToUse	12.3.2 GnssSetConstellationToUse
GnssSetTime ²	12.3.26 GnssSetTime
GnssWriteBitMaskSatActivated ²	12.3.31 GnssWriteBitMaskSatActivated

1. Not supported by Transceiver firmware revision 02.01 and beyond.
2. Supported from LR1120 firmware 02.01 and beyond.

12.3.2 GnssSetConstellationToUse

Command `SetGnssConstellationToUse(...)` configures the GNSS scanning for the selected constellation (GPS /BeiDou).

Table 12-2: GnssSetConstellationToUse Command

Byte	0	1	2
Data from Host	0x04	0x00	ConstellationBitMask (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *ConstellationBitMask*: Selection between GPS, or BeiDou, or both GPS and BeiDou.
 - ♦ Bit 0 = 1: GPS selected
 - ♦ Bit 1 = 1: BeiDou selected
 - ♦ Other values are RFU

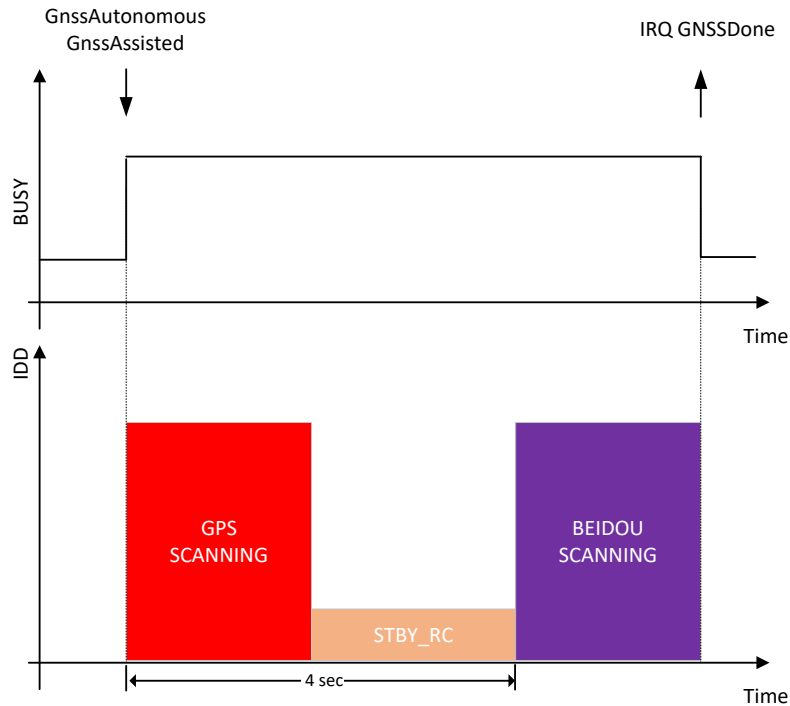


Figure 12-2: GNSS Dual Constellation Timing

If both GPS and BeiDou are selected, the GPS scanning is triggered immediately after the `GnssAutonomous()`, `GnssAssisted()` or `GnssScan(...)` command is issued by the Host MCU, followed by the BeiDou scanning. The BeiDou scanning is autonomously triggered by the LR1120 after a fixed delay, which requires a 32.768kHz clock source in order to respect this delay in the lowest possible power-consuming mode. This delay is exactly 4 seconds (with +/- 1ms accuracy) for LR1120 firmware versions up to 01.02, and is variable in steps of one second for versions 02.01 and beyond.

Between both scans, the LR1120 automatically returns in STBY_RC mode for minimum power consumption. The BUSY signal is set high until the end of the BeiDou scanning -including during the STBY_RC phase. At the end of the GNSS capture, the *GNSSDone* interruption is set high.

12.3.3 GnssReadConstellationToUse

Command `GnssReadConstellationToUse(...)` reads the selected con.stellation (GPS /BeiDou).
See [12.3.2 GnssSetConstellationToUse](#) for parameter descriptions.

Table 12-3: GnssReadConstellationToUse Command

Byte	0	1
Data from Host	0x04	0x01
Data to Host	Stat1	Stat2

Table 12-4: GnssReadConstellationToUse Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	ConstellationBitMask(7:0)

12.3.4 GnssReadSupportedConstellations

Command `GnssReadSupportedConstellations(...)` reads the supported con.stellations.
See [12.3.2 GnssSetConstellationToUse](#) for parameter descriptions.

See AN1200.74 LoRa Edge Clock Requirements for 32MHz TCXO and 32.768kHz clock considerations

Table 12-5: GnssReadSupportedConstellations Command

Byte	0	1
Data from Host	0x04	0x07
Data to Host	Stat1	Stat2

Table 12-6: GnssReadSupportedConstellations Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	ConstellationBitMask(7:0)

12.3.5 GnssSetMode

Command `GnssSetMode( . . . )` configures the GNSS for Legacy or Advanced scanning of the selected constellation (GPS or BeiDou).

Table 12-7: GnssSetMode Command

Byte	0	1	2
Data from Host	0x04	0x08	GnssMode
Data to Host	Stat1	Stat2	IrqStatus (31:24)

- *GnssMode*: Selection between Legacy or Advanced GNSS scanning.
 - ♦ 0x00: Legacy (single) scanning
 - ♦ 0x03: Advanced (multiple) scanning
 - ♦ Other values are RFU

During Advanced scanning, the LR1120 performs and averages multiple GNSS captures to increase the precision of the GNSS geolocation. Therefore, an Advanced scan will last longer and consume more energy than a Legacy scan, but the GNSS precision will be increased.

For LR1120 firmware versions 02.01 and beyond:

- The NAV message format is different than that of previous versions of firmware.
- An additional series of GNSS scans can be executed automatically after the Legacy or Advanced scan to extract time information from the satellites. Refer to [Section 12.3.8](#) (GnssScan).

12.3.6 GnssAutonomous

Command `GnssAutonomous( . . . )` captures GNSS signals in autonomous mode, for example in case no assistance information is available, or for fast indoor/outdoor detection. This command is in LR1120 firmware versions prior to 02.01. In LR1120 firmware versions 02.01 and beyond, it is replaced by the command `GnssScan( . . . )`.

Table 12-8: GnssAutonomous Command

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x04	0x09	Time (31:24)	Time (23:16)	Time (15:8)	Time (7:0)	Effort Mode	Result Mask	NbSvMax
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00

- *Time*: GPS time (GPST), in number of seconds elapsed since 6 January 1980 00:00:00. Hint: When converting from UTC to GPST, the UTC-GPST corresponding leap second offset must be taken into account.
- *EffortMode* = 0x00. Other values are RFU.
- *ResultMask*: bit mask indicating which information is added in the NAV message. This mask has a different meaning depending on the value of *GnssMode* (refer to [12.3.5 GnssSetMode](#)).
 - ♦ Bit 0:
 - ♦ *GnssMode* = 0: If set, includes pseudo-ranges in the NAV message: 19 bits per satellite. Warning: if this bit is not set, then the solver cannot compute the location.
 - ♦ *GnssMode* = 1: if set, include Doppler information in the NAV message.
 - ♦ Bit 1:
 - ♦ *GnssMode* = 0: If set, include the Doppler information in the NAV message: 15 bits per satellite. Note however that at least 5 satellites will have their Doppler reported in the NAV message independently of this configuration. The Doppler information is used by the solver to compute a coarse location of the device. It is advised to set this bit.
 - ♦ *GnssMode* = 1: If set, include the Doppler information in the NAV message, maximum of 14 satellites Doppler information per constellation used. Note however that at least 7 satellites will have their Doppler reported in the NAV message independently of this configuration.
 - ♦ Bit 2: If set, includes bit changes in the NAV message: 1 byte per satellite. This bit is currently not used by the solver so setting it does not bring any improvement.
- *NbSvMax* defines the maximum number of satellites wanted as a result of the `GnssAutonomous( . . . )`. If more satellites are detected during the scanning than *NbSvMax*, then the satellites with the highest C/N0 are returned.
 - ♦ If *NbSvMax* = 0, then all the detected satellites are returned.
 - ♦ Values *NbSvMax* = 1 and *NbSvMax* = 2 are not accepted.

Please note that calling this command resets the previous GNSS results, if any.

12.3.7 GnssAssisted

Command `GnssAssisted(...)` captures GNSS signals using assistance data (current time, approximate position, and Almanac information). This command is in LR1120 firmware versions prior to 02.01. In LR1120 firmware versions 02.01 and beyond, it is replaced by the command `GnssScan(...)`.

Table 12-9: GnssAssisted Command

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x04	0x0A	Time (31:24)	Time (23:16)	Time (15:8)	Time (7:0)	Effort Mode	ResultMask	NbSvMax
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00

- *Time*: GPS time (GPST), in number of seconds elapsed since 6 January 1980 00:00:00. Hint: When converting from UTC to GPST, the UTC-GPST corresponding leap second offset must be taken into account.
 - *EffortMode*
 - ♦ 0x00: Low Power mode. GNSS assisted scanning stops detection if no strong satellite is detected. Behaves as explained below:
 - Performs a strong satellite search on all visible SV. (A strong satellite is typically > 30dBHz).
 - If no SV are found during step 1, then stops.
 - If SV are found but the frequency error is too high, provides detected SV and raises "Doppler error" flag. The search window is reduced by the Doppler estimation done with SW found in step 1.
 - Performs a search on all visible SV (up to 12, + EGNOS/WAAS for GPS).
 - ♦ 0x01: Best Effort mode. GNSS assisted scanning continues detection even if no strong satellite is detected.
 - Performs a strong satellite search on all visible SV.
 - If SV are found in step 1, computes the right search window. Else, uses the default search window.
 - Performs a search (using the search window set in step 2) on all visible SV (up to 12, + EGNOS/WAAS for GPS - limited to 11 when double constellation).
 - *ResultMask*: Refer to [12.3.6 GnssAutonomous](#) for description.
 - *NbSvMax* defines the maximum number of satellites to detect.
 - ♦ If *NbSvMax* = 0, all the detected satellites are returned. Otherwise, only the *NbSvMax* satellites with higher C/N0 are returned.
 - ♦ Values *NbSvMax* = 1 and *NbSvMax* = 2 are not accepted
- Please note that calling this command resets the previous GNSS results, if any.

12.3.8 GnssScan

Command `GnssScan(...)` captures GNSS signals, independently of the availability of the assistance data (Assistance Position, Time and Almanac data). The duration and the sensitivity of the GNSS capture, through, will depend of the availability of the assistance data information, a GNSS scan being faster and with better sensitivity when the assistance data information (either network-provided, or determined by the LR1120) is available.

There are two types of GNSS scans:

- Cold start scan: Time or Approximate Position is not known. The scan searches for all the satellites in the constellation, and determines the Time (via Satellite signal demodulation) or Approximate Position (via 2D solving inside the chip), hence a longer execution time and increased power consumption, depending on the selected `EffortMode`.
- Assisted scan: Time and Assistance Position are both known, and the almanac data of the satellites is up-to-date (less than 3 months old for at least 1 satellite). The scan searches for only the 12 strongest visible satellites for each constellation, with the best sensitivity. The scan is much shorter for each constellation and power consumption is much reduced.

Refer to commands `GnssReadTime(...)` and `GnssGetDopplerSolverRes(...)` to determine respectively if the Time is known and if Approximate Position has been determined by the LR1120.

Table 12-10: GnssScan Command

Byte	0	1	2	3	4
Data from Host	0x04	0x0B	EffortMode	Result Mask	NbSvMax
Data to Host	Stat1	Stat2	IrqStatus(23:16)	IrqStatus(15:8)	IrqStatus(7:0)

- *EffortMode*:
 - ♦ 0x00: Low Effort mode. For both Legacy and Advanced scanning, the duration and the sensitivity of the GNSS capture depends on the availability of the assistance data information (time and position): a GNSS scan is faster and has better sensitivity when both assistance data Assistance Position and Time are available to the LR1120 (either network-provided, or determined by the LR1120).
 - ♦ 0x01: Middle Effort Mode.
 - ♦ Legacy scanning (scan mode 0, see `GnssSetMode`) triggers a fast GNSS scan with medium sensitivity if both assistance data Assistance Position and Time are available to the LR1120 (either network-provided, or self-provided), otherwise it returns “no time” or “no assisted position” errors.
 - ♦ Advanced scanning (see `GnssSetMode`) duration and the sensitivity of the GNSS capture depends on the availability of the assistance data information (time and position): a GNSS scan is faster and has better sensitivity when both assistance data Assistance Position and Time are available to the LR1120 (either network-provided, or determined by the LR1120).
 - ♦ 0x02: RFU

Table 12-11: Effort Mode Summary

Both Time and Assistance Position known		Time and/or Assistance Position unknown	
Effort mode 0	Effort mode 1	Effort mode 0	Effort mode 1
Scan mode 0 (Legacy)	Assisted scan with high sensi (GPS -141dBm, Beidou -137dBm)	Cold start scan with low sensi (GPS -134dBm, Beidou -131dBm)	“No time” or “no assisted position” error
Scan mode 3 (Advanced)	The scan stops if no strong SV is detected.		Cold start scan with mid sensi (GPS -137dBm, Beidou -134dBm)

- *ResultMask*: Bit mask indicating which information is added in the NAV message. This mask has a different meaning depending on the value of *GnssMode*.

Table 12-12: ResultMask Field Definition

Scan Mode	Bit 7-6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1-0
Scan mode 0 (Legacy)	RFU	0: Add Assistance Position in NAV 1: Do not add Assistance Position in NAV	0: Add Time in NAV 1: Do not add Time in NAV	No effect, no time demodulation	No effect, no bit change	Configure the Number of Doppler in the NAV message, see Table below
Scan mode 3 (Advanced)	RFU	0: Do not demodulate time 1: Demodulate time	0: No bit change 1: Add bit change	0: Do not demodulate time 1: Demodulate time	0: No bit change 1: Bit change	

Table 12-13: Number of Doppler in NAV Message Configuration

Bit 1	Bit 0	Behavior
0	0	Let LR1120 decide Doppler (0 or 7)
0	1	No Doppler
1	0	Maximum 7 Doppler
1	1	Maximum 14 Doppler

- *NbSvMax* defines the maximum number of satellites wanted as a result of the `GnssScan(...)`. If more satellites are detected during the scanning than *NbSvMax*, then the satellites with the highest C/N0 are returned.
 - ♦ If *NbSvMax* = 0, then all the detected satellites are returned.
 - ♦ Values *NbSvMax* = 1 and *NbSvMax* = 2 are not accepted.

Command `GnssScan(...)` switches from assisted scan to cold start scan if no satellites are detected after a certain period (configured by `GnssConfigDelayResetAP(...)`).

This command can be aborted by sending 0x0 on the SPI while the Busy pin is active, as described in [Section 12.4](#).

12.3.9 GnssSetAssistancePosition

Command `GnssSetAssistancePosition(...)` configures the approximate position for GNSS assisted mode. In LR1120 firmware versions 02.01 and beyond, the LR1120 uses integrated 2D solving to determine the Assistance Position, and this replaces the Assistance Position configured by the command `GnssSetAssistancePosition(...)`.

Table 12-14: GnssSetAssistancePosition Command

Byte	0	1	2	3	4	5
Data from Host	0x04	0x10	Latitude (15-8)	Latitude (7-0)	Longitude (15-8)	Longitude (7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

- Latitude:** Latitude, coded on 12 bits (resolution of 0.044°)
 $Latitude = \text{latitude in degrees (decimal value)} * 2048/90$
 For example, for 47.006° latitude: $47.006 * 2048/90 = 1070$ (rounded) = 0x042E
- Longitude:** Longitude, coded on 12 bits (resolution of 0.088°)
 $Longitude = \text{longitude in degrees (decimal value)} * 2048/180$
 For example, for 6.966° longitude: $6.966 * 2048/180 = 79$ (rounded) = 0x004F

12.3.10 GnssReadAssistancePosition

Command `GnssReadAssistancePosition(...)` reads the assistance position.

Table 12-15: GnssReadAssistancePosition Command

Byte	0	1
Data from Host	0x04	0x11
Data to Host	Stat1	Stat2

Table 12-16: GnssReadAssistancePosition Response

Byte	0	1	2	3	4
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	Latitude (15-8)	Latitude (7-0)	Longitude (15-8)	Longitude (7-0)

See [12.3.9 GnssSetAssistancePosition](#) for parameter description.

12.3.11 GnssGetContextStatus

Command `GnssGetContextStatus(...)` reads the GNSS context status.

Table 12-17: GnssGetContextStatus Command

Byte	0	1
Data from Host	0x04	0x16
Data to Host	Stat1	Stat2

Table 12-18: GnssGetContextStatus Response

Byte	0	1	2	3	4-7	8			9	
Data from Host	0x00	0x02	0x18	0x00	0x00	0x00			0x00	
Data to Host	Stat1	DestinationId	Opcode	GNSSFw Version	Global Almanac CRC	Error Code (7:4)	Almanac Update BitMask (3:1)	Freq Search Space (0)	Freq Search Space (7)	RFU (6:0)

- *GNSSFwVersion*: Firmware version.
- *GlobalAlmanacCRC* is the 32-bit CRC computed on all the flash memory content, on all 128 satellites. Per SV 21 bytes:
 - ♦ Byte 0: SvId
 - ♦ Bytes 1-2: AlmanacDate
 - ♦ Bytes 3-17: Almanac
 - ♦ Bytes 18-19: CaCodeGenerator
 - ♦ Byte 20: ModulationBitMask
 - ♦ Byte 21: ConstellationId
- *ErrorCode*
 - ♦ 0: No error
 - ♦ 1: Almanac too old
 - ♦ 2: Last Almanac update CRC mismatch
 - ♦ 3: Flash memory integrity error
 - ♦ 4: Almanac update not allowed (GPS and Beidou satellite can't be updated in a same request)
 - ♦ 5-15: RFU
- *AlmanacUpdateBitMask*:
 - ♦ Bit 0: GPS
 - ♦ Bit 1: Beidou
 - ♦ Bit 2: RFU
- *FreqSearchSpace*: (2-bit field)
 - ♦ 0: 250Hz
 - ♦ 1: 500Hz
 - ♦ 2: 1kHz
 - ♦ 3: 2kHz
 - ♦ Other bits are RFU.

12.3.12 GnssReadVersion

Command `GnssReadVersion(...)` returns the internal GNSS firmware version and almanac version..

Table 12-19: GnssReadVersion Command

Byte	0	1
Data from Host	0x04	0x06
Data to Host	Stat1	Stat2

Table 12-20: GnssReadVersion Response

Byte	0	1	2
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	Firmware Version	Almanac Version

12.3.13 GnssSetAlmanacUpdate

Command `GnssSetAlmanacUpdate(...)` configures the constellation almanac information to be updated..

Table 12-21: GnssSetAlmanacUpdate Command

Byte	0	1	2
Data from Host	0x04	0x02	AlmanacUpdateBitMask
Data to Host	Stat1	Stat2	IrqStatus(31-24)

- *AlmanacUpdateBitMask*: activation of GPS or BeiDou. By default, both constellations are activated.
 - ♦ Bit 0: GPS
 - ♦ Bit 1: BeiDou
 - ♦ Bit 2: RFU

12.3.14 GnssReadAlmanacUpdate

Command `GnssReadAlmanacUpdate(...)` reads the almanac update information (refer to [12.3.13 GnssSetAlmanacUpdate](#)).

Table 12-22: GnssReadAlmanacUpdate Command

Byte	0	1
Data from Host	0x04	0x03
Data to Host	Stat1	Stat2

Table 12-23: GnssReadAlmanacUpdate Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	AlmanacUpdateBitMask

12.3.15 GnssPushSolverMsg

Command `GnssPushSolverMsg(...)` pushes the messages from the GNSS solver to the LR1120 (assistance position update, and so on).

Table 12-24: GnssPushSolverMsg Command

Byte	0	1	2	n
Data from Host	0x04	0x14	Payload	...
Data to Host	Stat1	Stat2	IrqStatus(31-24)	...

- *Payload*: Byte stream to be forwarded by the Host MCU to the LR1120.

12.3.16 GnssPushDmMsg

Command `GnssPushDmMsg(...)` pushes the messages from the LoRaWAN network to the LR1120.

Table 12-25: GnssPushDmMsg Command

Byte	0	1	2	n
Data from Host	0x04	0x15	Payload	...
Data to Host	Stat1	Stat2	IrqStatus(31-24)	...

- *Payload*: Byte stream to be forwarded by the Host MCU to the LR1120.

12.3.17 GnssConfigDelayResetAP

Command `GnssConfigDelayResetAP ( . . . )` configures the delay (in seconds) after which the LR1120 will reset the Assistance Position and the GNSS scan switches from assisted scan to cold start scan if (both conditions):

- The time elapsed from last Assistance Position update is larger than this delay
- No SV is detected

Table 12-26: GnssConfigDelayResetAP Command

Byte	0	1	2	3	4
Data from Host	0x04	0x65	Delay (23:16)	Delay (15:8)	Delay (7:0)
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:16)	IrqStatus(15:8)

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.18 GnssReadLastScanModeLaunched

Command `GnssReadLastScanModeLaunched ( . . . )` returns the type of scan launched during the last scan:

Table 12-27: GnssReadLastScanModeLaunched Command

Byte	0	1
Data from Host	0x04	0x26
Data to Host	Stat1	Stat2

Table 12-28: GnssReadLastScanModeLaunched Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	LastScanMode

- *LastScanMode*:
 - ♦ 0-2: RFU
 - ♦ 3: Assisted mode (Time and Assisted Position known)
 - ♦ 4: Cold start mode (Time and Assisted Position unknown)
 - ♦ 5: Cold start mode (Time known and Assisted Position unknown)
 - ♦ 6: Fetch time or integrated 2D command launched
 - ♦ 7: Almanac update command launched without flash at the end
 - ♦ 8: Keep sync launched
 - ♦ 9: Almanac Update command launched with 1 constellation flashed at the end
 - ♦ 10: Almanac Update command launched with 2 constellation flashed at the end

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.19 GnssFetchTime

Command `GnssFetchTime(...)` configures the LR1120 to determine the time by demodulating the satellite signals:

Table 12-29: GnssFetchTime Command

Byte	0	1	2	3
Data from Host	0x04	0x32	EffortMode	Options
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:16)

- *EffortMode*: sensitivity of the demodulation
 - ♦ 0: Low sensitivity (GPS: - 134 dBm; Beidou: - 1301 dBm)
 - ♦ 1: Mid sensitivity (GPS: - 137 dBm; Beidou: - 134 dBm)
- Options:
 - ♦ 0: Search for a Time Of Week (TOW) in any subframe of any constellation (GPS or Beidou). This option can be used only after the Week Number (WN) is already demodulated.
 - ♦ 1: Search for TOW and WN (GPS or Beidou).
 - ♦ 2: Search for TOW and WN with rollover (Beidou) to have the WN rollover. Beidou constellation must be configured to use this option.

Add plot to explain WN and TOW + Rollover

This command is available in LR1120 firmware versions 02.01 and beyond.

This command can be aborted by sending 0x0 on the SPI while the Busy pin is active, as described in [Section 12.4](#).

12.3.20 GnssReadTime

Command `GnssReadTime(...)` has two purposes: it returns the GPS time, and also controls the precision of the 32kHz oscillator to optimize the GNSS power consumption. It must be called at least once every 36 hours to maintain a good accuracy of the 32 kHz oscillator.

This command requires a 32kHz clock to be available to the LR1120 (fed by the Host MCU, or 32kHz crystal attached to the LR1120), otherwise it returns an error.

Table 12-30: GnssReadTime Command

Byte	0	1
Data from Host	0x04	0x34
Data to Host	Stat1	Stat2

Table 12-31: GnssReadTime Response

Byte	0	1	2-5	6-7	8-11
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	ErrorCode	GPSTime	NbUs	TimeAccuracy

- *ErrorCode*:
 - ♦ 0: No error, WN, Tow is available
 - ♦ 1: No 32KHz clock available
 - ♦ 2: Time not available: Week Number or Time Of Week missing
- *GPSTime*:
 - ♦ If *ErrorCode*=0, *GPSTime*= the number of seconds from 06/Jan/1980 00:00:00, with modulo 1024 weeks, 4 bytes, unsigned, LSB = 1sec, MSB first
 - ♦ Otherwise *GPSTime*=0xFFFFFFFF
- *NbUs*:
 - ♦ If *ErrorCode*=0, then *NbUs*=time within one second, 3 bytes, unsigned, LSB = 1/16 us, MSB first
 - ♦ Otherwise *GPSTime*=0xFFFFFFFF
- *TimeAccuracy*:
 - ♦ If *ErrorCode*=0, then *TimeAccuracy*, 4 bytes, unsigned, LSB = 1/16 us, MSB first ???
 - ♦ Otherwise *TimeAccuracy*=0xFFFFFFFF

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.21 GnssResetTime

Command `GnssResetTime(...)` resets the time.

Table 12-32: GnssResetTime Command

Byte	0	1
Data from Host	0x04	0x35
Data to Host	Stat1	Stat2

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.22 GnssResetPosition

Command `GnssResetPosition(...)` resets the Assistance Position. The default position is (90, 0).

Table 12-33: GnssResetPosition Command

Byte	0	1
Data from Host	0x04	0x37
Data to Host	Stat1	Stat2

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.23 GnssReadWNRollover

Command `GnssReadWNRollover(...)` reads the number of GPS time week number rollover (every 1024 weeks). In the year 2023, this value is equal to 2. The Week Number rollover can be changed by the API `GnssSetTime` or `GnssFetchTime` or `GnssScan`.

Table 12-34: GnssReadWNRollover Command

Byte	0	1
Data from Host	0x04	0x67
Data to Host	Stat1	Stat2

Table 12-35: GnssReadWNRollover Response

Byte	0	1	2
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	WN status	WN rollover

- *WN status*:
 - ♦ 0: *WN rollover* never set (default value *WN rollover* = 2)
 - ♦ 1: *WN rollover* set by `GnssScan` (when internal time is not set and demodulation of time is done on BEIDOU), by `GnssSetTime` or by `GnssFetchTime` (option 2)
- *WN rollover*: Value of the current GPS *WN rollover*

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.24 GnssReadDemodStatus

This command can be called after each `GnssScan(...)`, `GnssAlmanacUpdateFromSat(...)` or `GnssFetchTime(...)` to obtain information about the execution of the scan.

Table 12-36: GnssReadDemodStatus Command

Byte	0	1
Data from Host	0x04	0x39
Data to Host	Stat1	Stat2

Table 12-37: GnssReadDemodStatus Response

Byte	0	1	2
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	DemodStatus	DemodInfo

DemodStatus:

- -21: Impossible to demodulate BDS almanac SV31-43
- -20: The satellite selected for demodulation lost
- -19: Almanac demod error
- -18: Wake up after preamble (demodulation launched too late to get time information from the satellite signal)
- -17: 20 ms real time failure
- -16: Wake up sync Failure
- -15: Week Number not validated
- -14: No activated satellite in satellite list at least for one constellation
- -13: Sleep time too long
- -12: Wrong Time Of Week demodulated
- -11: Preamble not validated
- -10: Demod disable
- -9: Demod extraction failure
- -8: No bit change found during start demod
- -7: No bit change found during advanced scan
- -6: No satellite found
- -5: Word sync lost
- -3: Not enough parity check found
- -2: Too many parity check found
- -1: No parity check found
- 0: word sync search not started
- 1: Word sync potentially found
- 2: Word sync found
- 3: Time Of Week found
- 4: Week Number found (Time Of Week as well)
- 5: Almanac found but no saved in retention memory
- 6: Half Almanac found saved in retention memory
- 7: All Almanac found saved in retention memory

DemodInfo:

- Word sync found:
 - ♦ - 0: Word synchronization not found
 - ♦ - 1: Word synchronization found
- First TOW found:
 - ♦ - 0: Time Of Week not found
 - ♦ - 1: Time Of Week found
- WN found:
 - ♦ - 0: Week Number not found
 - ♦ - 1: Week Number found
- WN demodulated:
 - ♦ - 0: Week Number not demodulated
 - ♦ - 1: Week Number demodulated
- Subframe 1/4/5 found
 - ♦ - 0: Subframe ID not found
 - ♦ - 1: Subframe ID found

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.25 GnssReadCumulTiming

This command can be called after each GNSS scan (`GnssScan(...)` / `GnssAlmanacUpdateFromSat(...)` / `GnssFetchTime(...)`) to retrieve the duration of the scan. This duration can be used to precisely compute the energy consumption of the scan. The value of time is in periods of 32KHz. To calculate this in seconds, the counter must be divided by 32768.

All 125 Bytes of the response must be read.

Table 12-38: GnssReadCumulTiming Command

Byte	0	1
Data from Host	0x04	0x4A
Data to Host	Stat1	Stat2

Table 12-39: GnssReadCumulTiming Response

Byte	0	1	2	...	125
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	Byte 0	Byte 1	...	Byte 124

Each sequence of bytes corresponds to the various GNSS Scan sequences, as shown in the power profile capture in the image below (performed with LR1120 EVK, with 32MHz TCXO and 32kHz crystal):

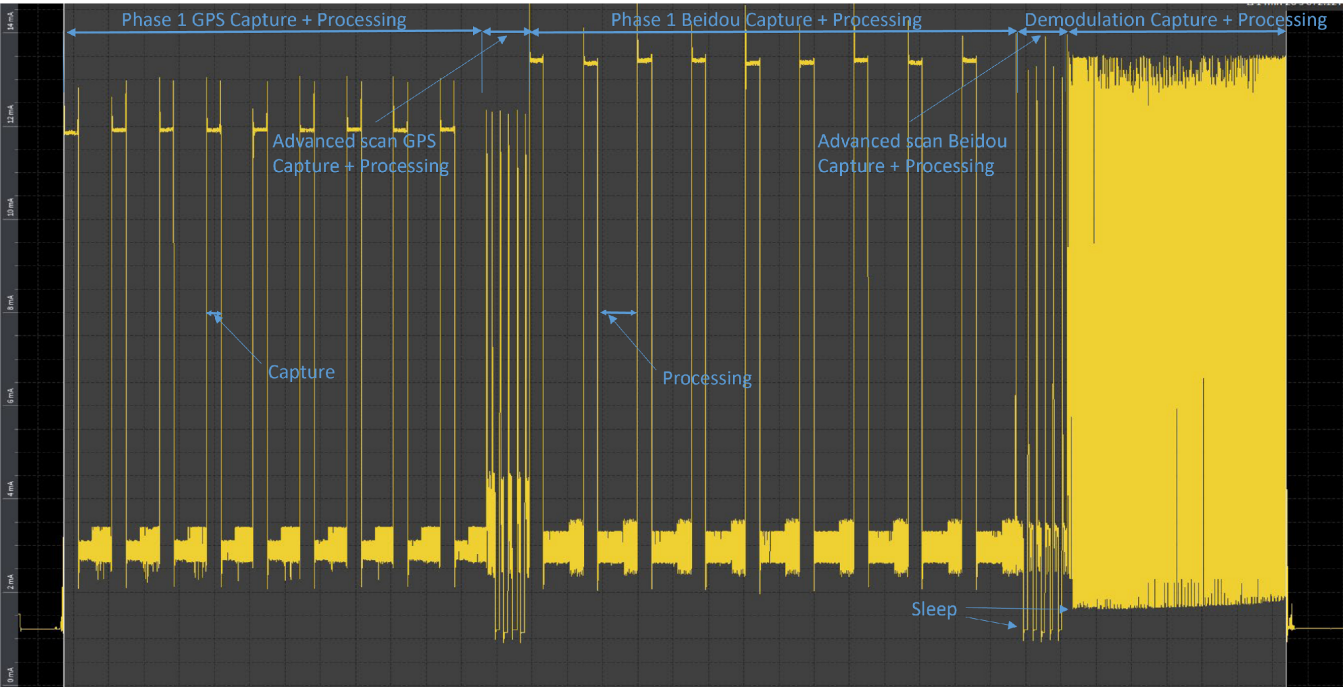


Figure 12-3: LR1120 Power Profile During GnssScan(...) Execution

- Byte 0-3: Init
- Byte 4-7: Phase 1 GPS Capture
- Byte 8-11: Phase 1 GPS Processing
- Byte 12-15: Advanced scan GPS Capture
- Byte 16-19: Advanced scan GPS Processing
- Byte 20-23: Advanced scan GPS sleep (32.768 kHz clock running)
- Byte 24-27: Phase 1 Beidou Capture
- Byte 28-31: Phase 1 Beidou Processing
- Byte 32-35: Advanced scan Beidou Capture
- Byte 36-39: Advanced scan Beidou Processing
- Byte 40-43: Advanced scan Beidou sleep (32.768 kHz clock running)
- Byte 44-47: Demod Capture
- Byte 48-51: Demod Processing
- Byte 52-55: Demod sleep (32.768kHz clock running)
- Byte 56-59: Demod sleep (32MHz clock running)
- Byte 60-63: Total GPS Capture
- Byte 64-67: Total GPS Processing
- Byte 68-71: Total GPS sleep (32.768kHz clock running)
- Byte 72-75: Total GPS sleep (32MHz clock running)
- Byte 76-79: Total GPS Duration
- Byte 80-83: Total Beidou Capture
- Byte 84-87: Total Beidou Processing
- Byte 88-91: Total Beidou sleep (32.768kHz clock running)
- Byte 92-95: Total Beidou sleep (32MHz clock running)
- Byte 96-99: Total Beidou Duration
- Byte 100-103: Total Capture Duration
- Byte 104-107: Total Processing Duration
- Byte 108-111: Total sleep (32.768kHz clock running) Duration
- Byte 112-115: Total sleep (32MHz clock running) Duration
- Byte 116-119: Total Scan Duration
- Byte 120-123: Last Capture size in 32.768kHz period ???
- Byte 124: Constellation demodulation ???

The energy consumption in micro Watt hours of each sequence (result Byte xx) can be calculated, knowing the current consumption of the sequence, using the following formula:

$$\text{Energy_Sequence (uWh)} = \text{Supply Voltage (V)} * \text{CurrentConsumption_Sequence} * \text{Byte xx} * 1000 / 3600;$$

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.26 GnssSetTime

Command `GnssSetTime(...)` allows the Host MCU to set the LR1120 GPS time.

Table 12-40: GnssSetTime Command

Byte	0	1	2-5	6-7
Data from Host	0x04	0x4B	GPSTime	TimeAccuracy
Data to Host	Stat1	Stat2	IrqStatus(31:0)	0x00

- *GPSTime*: Number of seconds since Jan, 6 1980 00:00:00
- *TimeAccuracy*: Time accuracy in milliseconds
 - ♦ If *TimeAccuracy*= 0, the LR1120 configures the time accuracy to > 24 secs

This command is available in LR1120 firmware versions 02.01 and beyond

12.3.27 GnssReadDopplerSolverRes

Command `GnssReadDopplerSolverRes(...)` allows to read the Assistance Position (calculated by the LR1120 using the internal 2D Solver during the execution of the commands `GnssScan (...)` or `GnssAlmanacUpdateFromSat (...)` or updated by the command `GnssComputeAssistancePosition(...)`.

All 18 Bytes of the response must be read.

Table 12-41: GnssReadDopplerSolverRes Command

Byte	0	1
Data from Host	0x04	0x4F
Data to Host	Stat1	Stat2

Table 12-42: GnssReadDopplerSolverRes Response

Byte	0	1	2	3-4	5-6	7-8	9-10	11-12	13-14	15-16	17-18
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	Error Code	NbSv Used	Latitude	Longitude	Accuracy	Xtal	Filtered Latitude	Filtered Longitude	Filtered Accuracy	Filtered Xtal

- *ErrorCode*:
 - ♦ 0: NoError
 - ♦ 1: ResidueHigh
 - ♦ 2: NotConverged
 - ♦ 3: NotEnoughSv
 - ♦ 4: IllMatrix
 - ♦ 5: Time error
 - ♦ 6: PartialAlmanacTooOldOrNotAvailable
 - ♦ 7: NotConsistentWithHistory
 - ♦ 8: AllAlmanacTooOld
- *NbSvUsed*: number of dopplers used for the oneshoot solving.
- *Latitude*
- *Longitude*
- *Accuracy*
- *Xtal*
- *FilteredLatitude*
- *FilteredLongitude*
- *FilteredAccuracy*
- *FilteredXtal*

If *ErrorCode* = NoError, all returned values should be used. Otherwise, *latitude* = 0xFFFF, *longitude* = 0xFFFF.

This command is available in LR1120 firmware versions 02.01 and beyond.

12.3.28 GnssReadDelayResetAP

Command `GnssReadDelayResetAP(...)` reads back the value of *Delay* (in seconds) before reset of the Assistance Position, as configured in `GnssConfigDelayResetAP (...)`.

Table 12-43: GnssReadDelayResetAP Command

Byte	0	1
Data from Host	0x04	0x53
Data to Host	Stat1	Stat2

Table 12-44: GnssReadDelayResetAP Response

Byte	0	1-3
Data from Host	0x00	0x00
Data to Host	Stat1	Delay

Delay: Number of seconds before reset of the Assistance Position, stored in 3 bytes.

12.3.29 GnssReadWarmStartStatus

Command `GnssReadWarmStartStatus(...)` reads the number of visible satellites and the time elapsed from the last update of the detected satellite list of this constellation.

Table 12-45: GnssReadWarmStartStatus Command

Byte	0	1	2
Data from Host	0x04	0x69	ConstellationBitMask
Data to Host	Stat1	Stat2	IrqStatus (31:24)

Table 12-46: GnssReadWarmStartStatus Response

Byte	0	1	2-5
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	NumVisibleSat	TimeElapsed

NumVisibleSat: Number of visible satellites in the selected constellation

- If = 0, either no satellites were found, or all visible satellites are unhealthy or deactivated for other reasons

TimeElapsed: Number of seconds since the last update of the detected satellite list in this constellation, coded on 4 bytes

- If = 0xFFFFFFFF the time was never set in the LR1120

12.3.30 GnssGetSvWarmStart

Command `GnssGetSvWarmStart(...)` returns the list of satellites for the next keep sync scan. The user must call `GnssReadWarmStartStatus(...)` to know how many satellites are in the list.

Table 12-47: GnssGetSvWarmStart Command

Byte	0	1	2
Data from Host	0x04	0x66	ConstellationBitMask
Data to Host	Stat1	Stat2	IrqStatus(31:24)

- *ConstellationBitMask*: Selection between GPS, or BeiDou, or both GPS and BeiDou
 - ♦ bit 0 = 1: GPS selected
 - ♦ bit 1 = 1: BeiDou selected
 - ♦ Other values are RFU. Only one constellation can be selected, otherwise a command error will be returned

Table 12-48: GnssGetSvWarmStart Response

Byte	0	1	2	...	N
Data from Host	0x00	0x00	0x00		0x00
Data to Host	Stat1	Sv0	Sv1		SvN

- *SvN*: All bytes must be read

12.3.31 GnssWriteBitMaskSatActivated

Command `GnssWriteBitMaskSatActivated(...)` configures the ability of the LR1120 to search for Almanacs for each satellite.

Table 12-49: GnssWriteBitMaskSatActivated Command For GPS

Byte	0	1	2	3-6
Data from Host	0x04	0x72	ConstellationBitMask	BitMaskActivated
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:0), 0x00

- *ConstellationBitMask*: Selection between GPS, or BeiDou, or both GPS and BeiDou
 - ♦ bit 0 = 1: GPS selected
 - ♦ bit 1 = 1: BeiDou selected
 - ♦ Other values are RFU. Only one constellation can be selected, otherwise a command error is returned
- *BitMaskActivated*: 32-bit bit-mask activated for satellites 1-32 (default value: 4294967295)

Table 12-50: GnssWriteBitMaskSatActivated Command For Beidou

Byte	0	1	2	3-6	7-10
Data from Host	0x04	0x72	ConstellationBitMask	BitMaskActivated(0)	BitMaskActivated(1)
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:0), 0x00	0x00

- *ConstellationBitMask*: Selection between GPS, or BeiDou, or both GPS and BeiDou
 - ♦ bit 0 = 1: GPS selected
 - ♦ bit 1 = 1: BeiDou selected
 - ♦ Other values are RFU. Only one constellation can be selected, otherwise a command error is returned
- *BitMaskActivated(0)*: 32-bit bit-mask activated for satellites 1 to 32 (default value: 3221012479)
- *BitMaskActivated(1)*: 32-bit bit-mask activated for satellites 33 to 63 (default value: 201328639)

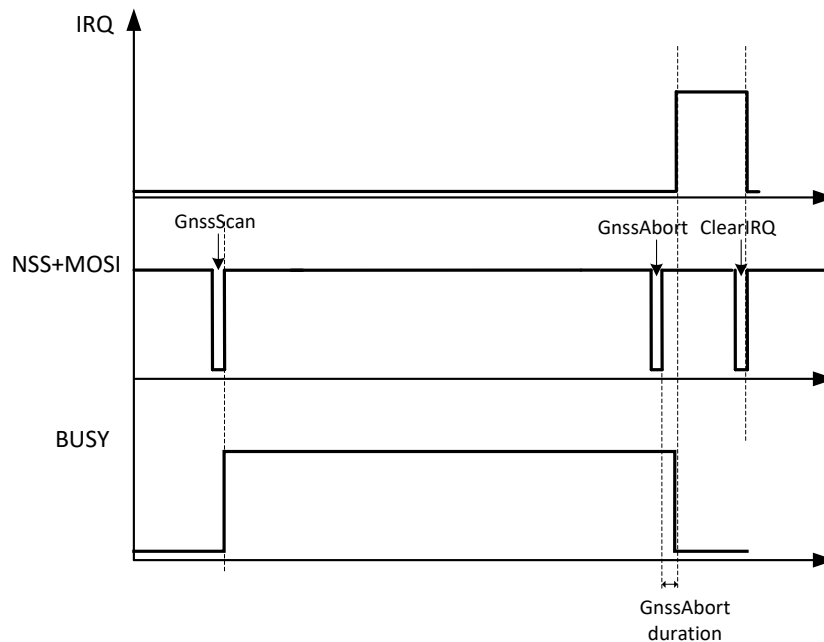
12.4 GNSS Scan Abort Procedure

Starting from LR1120 transceiver firmware 02.01, commands `GnssScan(...)`, `GnssAlmanacUpdateFromSat(...)`, and `GnssFetchTime(...)` can be aborted by sending 1 byte 0x0 on the SPI while the Busy pin is active.

The delay before the GNSS gets aborted is variable, since it depends on the current task executed by the internal MCU. The maximum delay is approximately 2.9s.

When an abort is detected during the scan, the IT Command abort is set instead of Interrupt *GnssScanDone*. The Host MCU must clear the corresponding IT pending bit.

Figure 12-4: GNSS Scan Abort Sequence



12.5 GNSS Scanning Results & Commands

GNSS scanning results are formatted in messages, of variable length depending on the number of satellites detected and on the *GnssMode*. The message destination can be either:

- Host (for status information)
- GNSS Position Solving component (for geolocation cloud calculation)
- GNSS Almanac Update component of Semtech LoRa Cloud

To read the GNSS scanning results, the result stream size must be determined first using command `GnssGetResultSize(...)`. Afterwards, the results can be read using command `GnssReadResult(...)`.

12.5.1 GNSS Scan Result Message Description

The message format is shown below. It is composed of a *DestinationID* field, followed by a *Payload* of variable length



Figure 12-5: GNSS Scan Result Message Format

- *DestinationID* = 0x00: Scan result status message to the Host (see [Section 12.5.1.1](#))
- *DestinationID* = 0x01: NAV message to the GNSS Solver
- *DestinationID* = 0x02: Almanac update message to the device management service

12.5.1.1 Scan Result Status Message

The scan result status messages to the host (*DestinationID* = 0x00) have a single byte *Payload*, coded as below:

- 0x00: OK
- 0x01: Command unexpected
- 0x02: Command not implemented
- 0x03: Command parameters invalid
- 0x04: Message Sanity check error
- 0x05: Scanning failed
- 0x06: No time
- 0x07: No satellite detected
- 0x08: Almanac too old
- 0x09: Almanac update fails due to CRC errors
- 0x0A: Almanac update fails due to flash integrity error
- 0x0B: Almanac update fails due to almanac date too old
- 0x0C: Almanac update not allowed (GPS and Beidou satellite can't be updated in a same request)
- 0x0D: Global Almanac CRC error
- 0x0E: Almanac version not supported
- All other values are RFU

These messages must not be transmitted to the GNSS solver.

12.5.2 GnssGetResultSize

Command `GnssGetResultSize(...)` reads the size in bytes of the byte stream containing the available GNSS results.

Table 12-51: GnssGetResultSize Command

Byte	0	1
Data from Host	0x04	0x0C
Data to Host	Stat1	Stat2

Table 12-52: GnssGetResultSize Response

Byte	0	1	2
Data from Host	0x00	0x00	0x00
Data to Host	Stat1	ResultSize (15:8)	ResultSize (7:0)

12.5.3 GnssReadResults

Command `GnssReadResults(...)` retrieves the last GNSS results.

Table 12-53: GnssReadResults Command

Byte	0	1
Data from Host	0x04	0x0D
Data to Host	Stat1	Stat2

Table 12-54: GnssReadResults Response

Byte	0	1	2	3	...N
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	ResultsByte1	ResultsByte2	ResultsByte3	ResultsByteN

12.5.4 GnssGetNbSvDetected

Command `GnssGetNbSvDetected(...)` retrieves the number of Satellite Vehicles detected during the last GNSS Scanning.

Table 12-55: GnssGetNbSvDetected Command

Byte	0	1
Data from Host	0x04	0x17
Data to Host	Stat1	Stat2

Table 12-56: GnssGetNbSvDetected Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	NbSv

12.5.5 GnssGetSvDetected

Command `GnssGetSvDetected(...)` retrieves the ID, the SNR and the Doppler of the Satellite Vehicles detected during the last GNSS Scanning.

Table 12-57: GnssGetSvDetected Command

Byte	0	1
Data from Host	0x04	0x18
Data to Host	Stat1	Stat2

Table 12-58: GnssGetSvDetected Response

Byte	0	1	2	3	4	5	...
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00	...
Data to Host	Stat1	SvId1	SNR1	Doppler1 (15-8)	Doppler1 (7-0)	SvId2	...

- *SvIdn*: Satellite Vehicle ID
- *SNRn*: Signal to Noise of the detected Satellite Vehicle, expressed in dB. To convert into C/N0, add 31dB
- *Dopplern*: Signed 16-bit value

12.5.6 GnssGetConsumption

Command `GnssGetConsumption(...)` reads the duration of the Radio capture and the CPU processing phases of the GNSS Scanning capture. These timings are expressed in microseconds.

This can be used to determine the GNSS Scanning power consumption.

Table 12-59: GnssGetConsumption Command

Byte	0	1
Data from Host	0x04	0x19
Data to Host	Stat1	Stat2

Table 12-60: GnssGetConsumption Response

Byte	0	1	2	3	4	5	6	7	8
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	CPUTime (31:24)	CPUTime (23:16)	CPUTime (15:8)	CPUTime (7:0)	Radio Time (31:24)	Radio Time (23:16)	Radio Time (15:8)	Radio Time (7:0)

12.5.7 GnssGetSvVisible

Command `GnssGetSvVisible(...)` returns the number of visible Space Vehicles in the sky, based on given assistance data (assistance position and time).

This command can be used to schedule a GNSS scan to target the maximum SV visibility.

Table 12-61: GnssGetSvVisible Command

Byte	0	1	2	3	4	5	6	7	8	9	10
Data from Host	0x04	0x1F	Time (31-24)	Time (23-16)	Time (15-8)	Time (7-0)	Latitude (15-8)	Latitude (7-0)	Longitude (15-8)	Longitude (7-0)	Constel lation
Data to Host	Stat1	Svid1	SNR1	Doppler 1 (15-8)	Doppler 1 (7-0)	Svid2					

Constellation: selected constellation

- 0 = GPS
- 1 = BeiDou

Refer to sections [12.3.7 GnssAssisted](#) and [12.3.9 GnssSetAssistancePosition](#) for the other parameters descriptions.

Table 12-62: GnssGetSvVisible Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	NbS

12.6 GNSS Almanac

The GNSS Almanac consists of information about the state of the entire GNSS satellite constellation and coarse data on every satellite's orbit. There is a specific almanac for each constellation. The almanac data is valid for up to 90 days.

The Almanac is used by the LR1120 in GNSS assisted mode. Use of the Almanac significantly optimizes the GNSS scanning duration: the LR1120 searches only for visible satellites within the user location and time, and therefore reduces the energy required for GNSS scanning.

The LR1120 is pre-programmed with the latest Almanac data at the date of the production test. Even if the Almanac data is valid for 90 days, it is advised to use the latest Almanac data for power optimization. The up-to-date Almanac is available from the Full Almanac image Download component of LoRa Cloud.

The whole Almanac data can be updated (see [12.6.5 GnssAlmanacFullUpdate](#)). The Almanac is entirely stored in flash memory, and is therefore kept after power off or Sleep mode without retention.

Starting from LR1120 firmware 02.01, the LR1120 can also demodulate the Almanac data from the satellite signal. For an implementation example, refer to the LoRa Edge geolocation reference code available on <https://github.com/Lora-net>.

12.6.1 GnssAlmanacUpdateFromSat

Command `GnssAlmanacUpdateFromSat(...)` launches a GNSS scan to download the Almanac parameters (broadcast from the satellite signal) to one page (subframe 4 and/or 5) per constellation.

This command must be sent at a precise time, matching the availability of the Almanac data in the satellite signal. The command `GnssReadAlmanacStatus()` can be used for this purpose. Refer to [Section 12.6.5](#) for the complete procedure to update the Almanac data from the satellites. Refer to Semtech LoRa Edge Tracker source code for an implementation example of the Almanac update procedure.

By default, the Almanac data is stored in the LR1120's RAM memory. It is written in flash memory when the Almanacs of more than 6 satellites are available, or when more than half of the Almanacs to update for one constellation are available.

This command can be aborted by sending 1 byte on the SPI while the Busy pin is active, as described in [Section 12.4](#).

Table 12-63: GnssAlmanacUpdateFromSat Command

Byte	0	1	2	3
Data from Host	0x04	0x55	Effort Mode	ConstellationBitMask(7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)

- *EffortMode*:
 - ♦ 0x00: Low effort mode, low sensitivity (typically GPS: -134dBm, Beidou. -131dBm)
 - ♦ 0x01: Middle effort mode, middle sensitivity (typically GPS: -137dBm, Beidou. -134dBm)
 EffortMode has no effect if a Warm start is launched. In this case the sensitivity of almanac demodulation is equivalent to the middle effort mode.
- *ConstellationBitMask*: Selection between GPS, or BeiDou, or both GPS and BeiDou.
 - ♦ Bit 0 = 1: GPS selected
 - ♦ Bit 1 = 1: BeiDou selected
 - ♦ Other values are RFU. Only one constellation can be selected, otherwise a command error will be returned.

12.6.2 GnssReadAlmanacStatus

Command `GnssReadAlmanacStatus(...)` provides information about the Almanacs required for GPS and Beidou satellite updates:

- The number of satellites to update and which ones. This information is updated when one SV's Almanac is demodulated and stored in the retention memory or flash.
- Information about the next direct Almanac to update for both constellations; by computing the current GPS time and remaining satellites to update, and returning the time duration between the current time and the direct next start subframe to capture to demodulate the appropriate Almanac in the list.
- Refer to Semtech LoRa Edge Tracker source code for an implementation example of the Almanac update procedure.

Table 12-64: GnssReadAlmanacStatus Command

Byte	0	1
Data from Host	0x04	0x57
Data to Host	Stat1	Stat2

Table 12-65: GnssReadAlmanacStatus Response

Byte	0	1	2	...	52
Data from Host	0x00	0x00	0x00		0x00
Data to Host	Stat1	Byte 0	Byte 1		Byte 52

All 53 Bytes must be read.

- Byte 0: GPS status:
 - ♦ -4: Internal time accuracy too low
 - ♦ -3: No time set
 - ♦ -2: Impossible to find next time
 - ♦ -1: No page id known
 - ♦ 0: No satellite to update
 - ♦ 1: At least 1 satellite must be updated
- Byte 1-4: Number of milliseconds before the next GPS satellite data subframe indicating where to get the new GPS Almanac
- Byte 5: Next number of GPS subframes to demodulate
- Byte 6: GPS satellite number that can be demodulated in next subframe 4
- Byte 7: GPS satellite number that can be demodulated in next subframe 5
- Byte 8: Next GPS subframe ID start: can be equal to 4, 5 or 0 depending if the first Almanac to download is in subframe 4 or 5, or if no next Almanac must be demodulated
- Byte 9: Total number of GPS satellites requiring Almanac update. Default value at device start is 32 (full Almanac update necessary)
- Byte 10-13: Bit mask (see [Table 12-66](#)) indicating which GPS satellite ID Almanac needs updating
 - ♦ 0: Satellite Almanac already updated
 - ♦ 1: Satellite Almanac to be updated
- Byte 14-17: Bit mask (see [Table 12-66](#)) indicating which GPS satellite ID is activated
 - ♦ 0: Satellite ID not activated
 - ♦ 1: Satellite ID activated
- Byte 18: Beidou status:
 - ♦ -4: Internal time accuracy too low

- ◆ -3: No time set
- ◆ -2: Impossible to find next time
- ◆ -1: No page id known
- ◆ 0: No satellite to update
- ◆ 1: At least 1 satellite must be updated
- Byte 19-22: Number of milliseconds before the next Beidou satellite data subframe indicating where to get the new Beidou Almanac
- Byte 23: Next number of Beidou subframes to demodulate
- Byte 24: Beidou satellite number that can be demodulated in next subframe 4
- Byte 25: Beidou satellite number that can be demodulated in next subframe 5
- Byte 26: Next GPS subframe ID start: can be equal to 4, 5 or 0 depending if the first Almanac to download is in subframe 4 or 5, or if no next Almanac must be demodulated
- Byte 27: Total number of Beidou satellites requiring Almanac update. Default value at device start is 34 (full Almanac update necessary)
- Byte 28-35: Bit mask (see [Table 12-67/ 12-68](#)) indicating which Beidou satellite ID Almanac needs updating
 - ◆ 0: Satellite Almanac already updated
 - ◆ 1: Satellite Almanac to be updated
- Byte 36-43: Bit mask (see [Table 12-67/ 12-68](#)) indicating which Beidou satellite ID is activated
 - ◆ 0: Satellite ID not activated
 - ◆ 1: Satellite ID activated
 - ◆ Bytes 36-39 default value = 3221012479 (satellites 15, 17, 18 and 31 inactivated)
 - ◆ Bytes 40-44 default value = 201328639 (satellites 33-43 and 59-61 activated)
- Byte 44-51: Bit mask (see [Table 12-67/ 12-68](#)) indicating which Beidou satellites do not broadcast the Almanac of satellite ID ≥ 31
 - ◆ Bit = 0:(default value) Satellite ID can be used to demodulate Almanac of satellite ID ≥ 31
 - ◆ Bit = 1: Satellite ID is blacklisted, it cannot be used to demodulate the Almanac of satellite ID ≥ 31
- Byte 52: Next Almanac ID. Pages 11-24 of subframe 5 are used to broadcast the Almanacs of satellites 31 to 63. These pages are time shared in 4 NAV messages of 12 minutes. When:
 - ◆ 0: No Almanac broadcast on pages 11-24
 - ◆ 1: Almanac of satellites 31-43 broadcast on pages 11-24
 - ◆ 2: Almanac of satellites 44-56 broadcast on pages 11-24
 - ◆ 3: Almanac of satellites 57-63 broadcast on pages 11-24
 - ◆ In the LR1120, only satellites 31 to 43 are downloaded, so these Almanacs can be captured

Table 12-66: GPS Bit Mapping

Bit	31	30	...	1	0
Satellite ID	32	31	...	2	1

Table 12-67: Beidou Bit Mapping (0)

Bit	31	30	...	1	0
Satellite ID	32	31	...	2	1

Table 12-68: Beidou Bit Mapping (1)

Bit	31	30	...	1	0
Satellite ID	64	63	...	34	33

12.6.3 GnssConfigAlmanacUpdatePeriod

Command `GnssConfigAlmanacUpdatePeriod(...)` configures the Almanac update period, i.e. the duration after which the application is notified that a new Almanac must be downloaded by the command `GnssReadAlmanacStatus(...)`.

Table 12-69: GnssConfigAlmanacUpdatePeriod Command

Byte	0	1	2	3	4-5
Data from Host	0x04	0x64	ConstellationBitMask	SvType	Period
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:16)	IrqStatus(15:0)

- *ConstellationBitMask*: Selection between GPS or BeiDou
 - ♦ Bit 0 = 1: GPS selected
 - ♦ Bit 1 = 1: BeiDou selected
 - ♦ Other values are RFU. Only one constellation can be selected, otherwise a command error is returned
- *SvType*:
 - ♦ 0: Configure period for MEO satellite
 - ♦ 1: Configure period for IGSO satellite
 - ♦ *SvType* has no impact on GPS configuration but must be lower or equal to 1
- *Period* (16-bit): A delta day is computed between the age of the Almanac in flash and the current day. This is compared with this *Period* to indicate to the application (during a `GnssReadAlmanacStatus`) if it must be downloaded. By default, Almanac update periods are configured as follows:
 - ♦ GPS: 31 days
 - ♦ Beidou MEO: 60 days
 - ♦ Beidou IGSO: 30 days

12.6.4 GnssReadAlmanacUpdatePeriod

Command `GnssReadAlmanacUpdatePeriod(...)` reads the Almanac update period of a constellation and a particular SvType.

Table 12-70: GnssReadAlmanacUpdatePeriod Command

Byte	0	1	2	3
Data from Host	0x04	0x64	ConstellationBitMask	SvType
Data to Host	Stat1	Stat2	IrqStatus(31:24)	IrqStatus(23:16)

- *ConstellationBitMask*: Selection between GPS, or BeiDou, or both GPS and BeiDou
 - ♦ bit 0 = 1: GPS selected
 - ♦ bit 1 = 1: BeiDou selected
 - ♦ Other values are RFU. Only one constellation can be selected, otherwise a command error will be returned
- SvType:
 - ♦ 0: Read period for MEO satellite
 - ♦ 1: Read period for IGSO satellite

Table 12-71: GnssReadAlmanacUpdatePeriod Response

Byte	0	1-2
Data from Host	0x00	0x00
Data to Host	Stat1	Period

- *Period*: (2 Bytes) Period in days before necessary Almanac Update. MSB first.

12.6.5 GnssAlmanacFullUpdate

The Almanac data for all the satellites can be updated using command `GnssAlmanacFullUpdate(...)`. Each constellation must be updated separately.

Table 12-72: GnssAlmanacFullUpdate Command

Byte	0	1	2: See Table 12-73 below
Data from Host	0x04	0x0E	AlmanacFullUpdatePayload
Data to Host	Stat1	Stat2	IrqStatus(31:24)

- *AlmanacFullUpdatePayload*: defined as in below:

Table 12-73: AlmanacFullUpdatePayload Parameter Format

Byte	(0:19)	(20:39)	(40:56)	...	(2560:2579) ¹
Field	AlmanacHeader	SV1 Almanac	SV2 Almanac	...	SV128 Almanac

1. In transmission, the frame cannot be longer than 512 bytes. In reception, the frame cannot be longer than 1020 bytes. See below for more detail.

- *AlmanacHeader*: defined as below:

Table 12-74: AlmanacHeader Parameter Format

Byte	0	(1:2)	(3:6)	(7:19)
Field	128	AlmanacDate	Global CRC	RFU

- Each *SVn Almanac* is a 20-byte structure, defined as below:

Table 12-75: SVn Almanac Parameter Format

Byte	0	(1:15)	(16:17)	18	19
Field	SV id	Almanac Content	CA code	Modulation bit mask	Constellation Id

The user must ensure that the list of almanacs and the list of satellites ids are coherent. The Almanac data must be provided in the same order as satellite ids.

The *AlmanacFullUpdatePayload* takes 20 bytes (Header) + 128 (number of SV) * 20 bytes = 2580 bytes.

The payload must be made up of complete Blocks, where a Block is 20 data bytes.

The maximum number of bytes that can be sent from the host MCU to the LR1120 is 512 bytes. Therefore, the Almanac Full Update must be handled in multiple SPI transactions. For example, the two following approaches are possible:

- Minimum memory overhead: The *AlmanacFullUpdatePayload* can be sent in 129 successive SPI transactions of blocks (20 bytes each).
- Minimum number of SPI transactions: The *AlmanacFullUpdatePayload* can be sent in 5 SPI transactions of 25 blocks (500 bytes), and a sixth SPI transaction of 4 blocks (80 bytes).

13. Ranging by Round Trip Time Of Flight (RTToF)

The LR1120 features an RTToF ranging engine that can estimate the distance between two LR1120 devices. It is based on time of flight measurements of dedicated LoRa modulated communication exchanges between the two devices.

Additional information on [ranging](#) using other LoRa products can be found in the following [references](#):

- AN1200.50: SX1280 Ranging Protocol
- AN1200.29 Introduction to Ranging with the SX1280
- AN1200.31 How to Perform Ranging Tests with the SX1280 Development Kit

13.1 RTToF Commands API Interface

Starting from transceiver firmware version 0x0201, five additional commands are supported to enable the use of the LR1120 RTToF ranging functionality. The user must configure and transmit the ranging packets according to the command order depicted in [Figure 8-1](#). The additional commands are as follows:

Table 13-1: Ranging Commands

Command	Opcode	Parameter	Description
SetRangingAddr	0x021C	<i>Addr, Checklength</i>	Sets the ranging id of the device and the number of bytes to be checked on a request
SetRangingReqAddr	0x021D	<i>ReqAddr</i>	Sets the ranging id for the request
GetRangingResult	0x021E	<i>Type</i>	Gets the ranging results (distance, RSSI)
SetRangingTxRxDelay	0x021F	<i>Delay</i>	Sets the TxRx delay for the ranging
SetRangingParameter	0x0228	<i>SymbNb</i>	Sets the number of synchronization symbols

13.2 SetRangingAddr

Command `SetRangingAddr(...)` sets the ranging ID for this slave.

Table 13-2: SetRangingAddr Command

Byte	0	1	2	3	4	5	6
Data from host	0x02	0x1C	Addr (31:24)	Addr (23:16)	Addr (15:8)	Addr (7:0)	CheckLength
Data to host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0

- *Addr*: 32 bits, default = 0x00000019
- *CheckLength*: Defines how many of the 4 address bytes are checked against the requested address sent by the Master. Default=4. Checked bytes are the LSB if $CheckLength < 4$.

13.3 SetRangingReqAddr

Command SetRangingReqAddr(...) sets the address that is requested by the Master in the ranging request.

This must match the receiving Slave's ranging address.

Table 13-3: SetRangingReqAddr Command

Byte	0	1	2	3	4	5
Data from host	0x02	0x1D	ReqAddr (31:24)	ReqAddr (23:16)	ReqAddr (15:8)	ReqAddr (7:0)
Data to host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

- *ReqAddr*: 32 bits, default 0x00000019

13.4 GetRangingResult

Command GetRangingResult(...) reads the results in the Master, based on the *Type*:

Table 13-4: GetRangingResult Request

Byte	0	1	2
Data from host	0x02	0x1E	Type
Data to host	Stat1	Stat2	IrqStatus (31:24)

- *Type*: Defines the type of result to be returned:
 - ♦ 0: last ranging distance result: distance measurement
 - ♦ 1: last ranging RSSI result: signal strength indication

Table 13-5: GetRangingResult Response

Byte	0	1	2	3	4
Data from host	0x00	0x00	0x00	0x00	0x00
Data to host	Stat1	Res (31:24)	Res (23:16)	Res (15:8)	Res (7:0)

- *Res*: Ranging results information depending upon the *Type*:
 - ♦ Ranging Distance *Res*(31:0): Distance (signed 24-bit), which is converted into meters as follows:
Round Trip Distance [m] = $Res * 3e8 / (2^{12} * BW)$, where BW is the LoRa bandwidth in Hz
 - ♦ Ranging RSSI *Res*(7:0): Received signal strength, linear, in dB units calculated as follows:
RSSI [dB] = $Res / 2$

13.5 SetRangingTxRxDelay

Command `SetRangingTxRxDelay(...)` sets the TxRx delay for the ranging calibration. The processing of the ranging response and request incurs a deterministic fixed delay which must be compensated for in order to provide an accurate range estimation. This fixed timing (distance) offset is removed by subtracting the TxRx delay offset calibration value. Note that the same value must be written in both Master and Slave.

Table 13-6: SetRangingTxRxDelay Command

Byte	0	1	2	3	4	5
Data from host	0x02	0x1F	Delay (31:24)	Delay (23:16)	Delay (15:8)	Delay (7:0)
Data to host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)

- *Delay*. The value depends on the LoRa BW/SF used (the following are typical values for Semtech EVKs and may differ on other PCB designs):

Table 13-7: Typical Delay Values for Semtech EVKs

BW	SF5	SF6	SF7	SF8	SF9	SF10	SF11	SF12
125	19035	19040	19040	19039	19036	19038	19036	19024
250	20235	20239	20238	20237	20236	20235	20236	20232
500	20149	20227	20258	20277	20286	20292	20295	20298

13.6 SetRangingParameter

Command `SetRangingParameter(...)` defines how many symbols are used during synchronization.

Table 13-8: SetRangingParameter Command

Byte	0	1	2	3
Data from host	0x02	0x28	Reserved	SymbNb
Data to host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)

- *Reserved*: 0
- *SymbNb*: The recommended value of 15 gives a good compromise between the accuracy of the measurement result and the time on air and energy consumption). Increasing the number of symbols can help increase the accuracy of the result at the expense of longer time on air.

14. Bluetooth® Low Energy Beacons Compatibility

The LR1120 embeds a transmitter that is compatible with Bluetooth® technology, to address indoor/outdoor localization use cases based on Bluetooth Low Energy beacons.

- The Bluetooth Low Energy beacons compatible data is detected by Bluetooth Low Energy beacons receivers to determine the device's location.
- The Bluetooth Low Energy beacons compatible parameters are fully configurable to permit transmission of any type of Bluetooth Low Energy beacon.

Note: There is no Bluetooth Low Energy beacons compatible stack or state machine implemented inside the LR1120, therefore the user must handle the transmission of the Bluetooth Low Energy compatible beacons in the 3 advertising channels at the correct advertising interval.

14.1 ConfigBleBeacon

Command `ConfigBleBeacon(...)` configures the radio to be Bluetooth Low Energy beacons compatible on the given channel: syncword 0x8E89BED6, 3 byte CRC (polynomial 0x100065B, seed 0x555555).

Note: This command does not send the packet. It must be sent with the normal `SetTx()` command.

Table 14-1: ConfigBleBeacon Command

Byte	0	1	2	3	4	5	N+3
Data from host	0x02	0x2E	Channel(7:0)	Payload[0]	Payload[1]	...	Payload[N]
Data to host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	...	0

- *Channel*: Allowed channel values are 37, 38, or 39. The channel determines the frequency and whitening seed settings.
- *Payload*: (optional). To send the same beacon on different channels, this command can be issued with only the *channel* as parameter.

14.2 BleBeaconSend

Command `BleBeaconSend(...)` configures the radio to be Bluetooth Low Energy beacons compatible on the given channel and sends the packet: syncword 0x8E89BED6, 3 byte CRC (polynomial 0x100065B, seed 0x555555).

Table 14-2: BleBeaconSend Command

Byte	0	1	2	3	4	5	N+3
Data from host	0x02	0x31	Channel(7:0)	Payload[0]	Payload[1]	...	Payload[N]
Data to host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	...	0

See [14.1 ConfigBleBeacon](#) for parameter descriptions.

15. Cryptographic Engine

15.1 Description

The Cryptographic Engine provides a dedicated hardware accelerator for AES-128 encryption based algorithms and dedicated flash and RAM memory to handle device parameters such as encryption keys, with no read access possible.

The Cryptographic Engine improves the power efficiency of cryptographic operations and reduces the code size of the software stack. Verifying the integrity of data such as the payload of downlink frames is important to guarantee a secure communication. The message integration check (MIC) uses the AES-CMAC algorithm to calculate a hash. Implementing the MIC calculation in software would jeopardize the confidentiality of the used key. The cryptographic engine provides a hardware implementation of the AES-CMAC to internally calculate and check the MIC.

The status of cryptographic operations can be checked by either polling the internal status register or using an interrupt service routine.

15.2 Cryptographic Keys Definition

The cryptographic keys are arranged into several groups, according to the function they serve, as shown in [Table 15-1: Cryptographic Keys Usage and Derivation](#). The table summarizes the allowed uses of the keys and if some of the keys can be derived from other keys.

Table 15-1: Cryptographic Keys Usage and Derivation

Group Name	Key Source/ Dest. Index	Key Name	Usage	Derivation From
Network	2	NwkKey	CryptoProcessJoinAccept() CryptoComputeAesCmac() CryptoDeriveKey() CryptoSetKey(...)	DKEY ¹
Application	3	AppKey	CryptoDeriveKey() CryptoSetKey(...)	DKEY ¹
LifeTimeEnc	4	JSEncKey	CryptoProcessJoinAccept() (Decryption) CryptoSetKey(...)	From Network & Application
LifeTimeInt	5	JSIntKey	CryptoProcessJoinAccept() (MIC Computation) CryptoComputeAesCmac() CryptoSetKey(...)	From Network & Application
GpTransport	6	GpKEKey0	CryptoDeriveKey(...) CryptoSetKey(...) Any multicast Key	From any other Gp Transport key or from Application Key
	7	GpKEKey1		
	8	GpKEKey2		
	9	GpKEKey3		
	10	GpKEKey4		
	11	GpKEKey5		

Table 15-1: Cryptographic Keys Usage and Derivation (Continued)

Group Name	Key Source/ Dest. Index	Key Name	Usage	Derivation From
Unicast	12	AppSKey	CryptoAesEncrypt01(...) CryptoComputeAesCmac() CryptoSetKey(...)	From Network & Application
	13	FNwkSIntKey		
	14	SNwkSIntKey		
	15	NwkSEncKey		
	16	RFU0		
	17	RFU1		
Multicast	18	McAppSKey0	CryptoAesEncrypt01(...) CryptoVerifyAesCmac(...) CryptoSetKey(...)	Only from GpTransport Key
	19	McAppSKey1		
	20	McAppSKey2		
	12	McAppSKey3		
	22	McNwkSKey0		
	23	McNwkSKey1		
	24	McNwkSKey2		
	25	McNwkSKey3		
General Purpose	26	GP0	CryptoAesEncrypt(...) CryptoAesDecrypt(...) CryptoSetKey(...)	Not Allowed
	27	GP1		

1. Built-in the device, derived upon DeriveRootKeysAndGetPin() command.

15.3 Commands

15.3.1 CESTatus

The Crypto Status byte *CEStatus* indicates the Crypto Engine state. It is returned after each command invoking the Crypto Engine.

CEStatus:

- 0: CRYPT_API_SUCCESS. The previous command was successful
- 1: CRYPT_API_FAIL_CMAC. MIC (first 4 bytes of the CMAC) comparison failed
- 2: RFU
- 3: CRYPT_API_INV_KEY_ID. Key/Param Source or Destination ID error
- 4: RFU
- 5: CRYPT_API_BUF_SIZE. Data buffer size is invalid. For `CryptoAesEncrypt(...)` the buffer size must be multiple of 16 bytes
- 6: CRYPT_API_ERROR. Any other error

15.3.2 CryptoSetKey

Command `CryptoSetKey(...)` sets a specific *Key* identified by *KeyID* into the Crypto Engine:

Table 15-2: CryptoSetKey Command

Byte	0	1	2	3	4	5	...	18
Data from Host	0x05	0x02	KeyID (7:0)	Key1	Key2	Key3	...	Key16
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	...	0x00

Table 15-3: CryptoSetKey Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	CEStatus

- *KeyID* goes from 2 to 27, as defined in [Table 15-1: Cryptographic Keys Usage and Derivation](#). Other values are reserved.
- *Key* is an array of bytes as defined in the FIPS-197. With the key K (2b7e1516 28aed2a6bf71588 09cf4f3c) provided in test vectors of the rfc4493, we then have *Key1* = 0x2b, *Key2* = 0x7e, *Key3* = 0x15, *Key4* = 0x16 thru to *Key16* = 0x3c.
- *CEStatus* is defined in section [CEStatus on page 180](#)

15.3.3 CryptoDeriveKey

Command `CryptoDeriveKey(...)` derives (encrypts) into a specific Key identified by *DstKeyID*, the input (including the LoRaWAN DevNonce) value provided, using a source key identified by *SrcKeyID*.

Table 15-4: CryptoDeriveKey Command

Byte	0	1	2	3	4	5	6	...	19
Data from Host	0x05	0x03	SrcKeyID (7:0)	DstKeyID (7:0)	Input[1:16]				
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	...	0x00

Table 15-5: CryptoDeriveKey Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	CEStatus

- *DstKeyID* and *SrcKeyID* for this function are defined in [Table 15-1: Cryptographic Keys Usage and Derivation](#)
 - ♦ *DstKeyID*: Destination Key ID. Valid values are from 4 to 25
 - ♦ *SrcKeyID*: Source Key IDs 2-3 and 6-11 are possible for this function
- *Input[1:16]* is an array of bytes. An example of its construction is given in [Section 16.3](#) and [Section 16.4](#)
- *CEStatus* is defined in section [CEStatus on page 180](#)

Note: At the end of the `CryptoDeriveKey()` process, the generated key is located in the dedicated Crypto Engine RAM, and can be stored in the flash memory using the `CryptoStoreToFlash()` command.

15.3.4 CryptoProcessJoinAccept

Command `CryptoProcessJoinAccept(...)` decrypts the join accept message (using AES-ECB encrypt as per LoRaWAN specification) on the Data and Header, and then verifies the MIC of the decrypted message.

The decrypted data is then provided, if the MIC verification is successful.

Table 15-6: CryptoProcessJoinAccept Command

Byte	0	1	2	3	4	5	...	N+4	N+5	...	N+4+M
Data from Host	0x05	0x04	DecKeyID (7:0)	VerKeyID (7:0)	LoRa WANVer (7:0)	Header1	...	HeaderN	Data1	...	DataM
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	...	0x00	0x00	...	0x00

Table 15-7: CryptoProcessJoinAccept Response

Byte	0	1	2	...	M+1
Data from Host	0x00	0x00	0x00	...	0x00
Data to Host	Stat1	CEStatus	Data1	...	DataM

- *DecKeyID* and *VerKeyID* are defined in [Table 15-1: Cryptographic Keys Usage and Derivation](#):
 - ♦ *DecKeyID*: Key used for decryption of the message
 - ♦ *VerKeyID*: Key used for MIC verification
- *LoRaWANVer*: Determines the expected *Header* size N: 1 byte (v1.0) or 12 bytes (v1.1)
 - ♦ *LoRaWANVer* = 0: LoRaWAN version 1.0.x, only MHDR (1 byte)
 - ♦ *LoRaWANVer* = 1: LoRaWAN version 1.1.x, SIntKey, JoinReqType | JoinEUI | DevNonce | MHDR
- *Header1 to HeaderN*: Header. N depends on *LoRaWANVer*
- *Data1 to DataM*: Data. Data size M is either 16 or 32 bytes. Data must include the encrypted MIC
- *CEStatus* is defined in section [CEStatus on page 180](#)

15.3.5 CryptoComputeAesCmac

Command `CryptoComputeAesCmac ( . . . )` computes the AES CMAC of the provided data using the specified Key and returns the MIC.

Table 15-8: CryptoComputeAesCmac Command

Byte	0	1	2	3	4	5	...	N+2
Data from Host	0x05	0x05	KeyID (7:0)	Data1	Data2	Data3	...	DataN
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	...	0x00

Table 15-9: CryptoComputeAesCmac Response

Byte	0	1	2	3	4	5
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	CEStatus	MIC1	MIC2	MIC3	MIC4

- *KeyID*: Specified Key ID, as defined in [Table 15-1: Cryptographic Keys Usage and Derivation](#); 2, 5, 12-17 are supported
- *Data1, Data2, ..., DataN*: Provided data, considered as byte buffers
- *CEStatus*: Defined in section [CEStatus on page 180](#)
- *MIC*: Message Integrity Check (first 4 bytes of the CMAC)

For example, when using the test vectors of the RFC4493 example 2, we would have:

- Message: 6BC1BEE2 2E409F96 E93D7E11 7393172A (N = 16)
- MIC: 070A16B4

Table 15-10: CryptoComputeAesCmac Command Example

Byte	0	1	2	3	4	5	...	18
Data from Host	0x05	0x05	KeyID (7:0)	0x6b	0xc1	0xbe	...	0x2a
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	...	0x00

Table 15-11: CryptoComputeAesCmac Response Example

Byte	0	1	2	3	4	5
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	CEStatus	0x07	0x0a	0x16	0xb4

15.3.6 CryptoVerifyAesCmac

Command `CryptoVerifyAesCmac(...)` computes the AES CMAC of the provided data using the specified Key, and compares the provided MIC with the actual calculated MIC (first 4 bytes of the CMAC).

Table 15-12: CryptoVerifyAesCmac Command

Byte	0	1	2	3	4	5	6	7	...	N+6
Data from Host	0x05	0x06	KeyID (7:0)	Expected MIC1	Expected MIC2	Expected MIC3	Expected MIC4	Data1	...	DataN
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	...	0x00

Table 15-13: CryptoVerifyAesCmac Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	CEStatus

- *KeyID*: Specified Key ID, as defined in [Table 15-1: Cryptographic Keys Usage and Derivation](#). 2, 5; 12-25 are possible
- *ExpectedMIC*: Provided MIC (first 4 bytes of the CMAC)
- *Data1, Data2 to DataN*: Provided data, considered as byte buffers
- *CEStatus*: Defined in section [CEStatus on page 180](#)

If the 2 MICs are identical, the command returns `CRYP_API_SUCCESS`, otherwise, `CRYP_API_FAIL_CMAC`.

15.3.7 CryptoAesEncrypt01

Command `CryptoAesEncrypt01(...)` encrypts the provided data using the specified Key and returns the encrypted data. It can't be used on key indexes 2-11, in order to prevent re-calculating the session keys.

Table 15-14: CryptoAesEncrypt01 Command

Byte	0	1	2	3	4	...	N+2
Data from Host	0x05	0x07	KeyID (7:0)	0x01	Data2	...	DataN
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	...	0x00

Table 15-15: CryptoAesEncrypt01 Response

Byte	0	1	2	...	N+1
Data from Host	0x00	0x00	0x00	...	0x00
Data to Host	Stat1	CEStatus	Encrypted Data1	...	Encrypted DataN

- *KeyID*: Specified Key ID, as defined in [Table 15-1: Cryptographic Keys Usage and Derivation](#). Valid values are from 12 to 25.
- *Data1, Data2 to DataN*: Provided data, considered as byte buffers
- *CEStatus*: Defined in section [CEStatus on page 180](#)
- *EncryptedData1, EncryptedData2,... , EncryptedDataN*: Encrypted data, considered as byte buffers

15.3.8 CryptoAesEncrypt

Command `CryptoAesEncrypt(...)` encrypts the provided data using the specified Key and returns it. It is to be used for generic, non-LoRaWAN cryptographic operations, where the Crypto Engine is used as a hardware accelerator, on key indexes 26 and 27 only (General Purpose keys).

Table 15-16: CryptoAesEncrypt Command

Byte	0	1	2	3	...	N+2
Data from Host	0x05	0x08	KeyID (7:0)	Data1	...	DataN
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	...	0x00

Table 15-17: CryptoAesEncrypt Response

Byte	0	1	2	...	N+1
Data from Host	0x00	0x00	0x00	...	0x00
Data to Host	Stat1	CEStatus	Encrypted Data1	...	Encrypted DataN

- *KeyID*: Specified Key ID, as defined in [Table 15-1: Cryptographic Keys Usage and Derivation](#). Valid values are from 26 to 27.
- *Data1, Data2 to DataN*: Provided data, considered as byte buffers
- *CEStatus*: Defined in section [CEStatus on page 180](#)
- *EncryptedData1, EncryptedData2 to EncryptedDataN*: Encrypted data, considered as byte buffers

15.3.9 CryptoAesDecrypt

Command `CryptoAesDecrypt(...)` decrypts the provided data using the specified Key and returns it, and can only be used on keys indexes 26-27, when the Crypto Engine is used as a stand-alone hardware accelerator for non-LoRaWAN security tasks.

Table 15-18: CryptoAesDecrypt Command

Byte	0	1	2	3	...	N+2
Data from Host	0x05	0x09	KeyID (7:0)	Data1	...	DataN
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	...	0x00

Table 15-19: CryptoAesDecrypt Response

Byte	0	1	2	...	N+1
Data from Host	0x00	0x00	0x00	...	0x00
Data to Host	Stat1	CEStatus	Decrypted Data1	...	Decrypted DataN

- *KeyID*: Specified Key ID, as defined in [Table 15-1: Cryptographic Keys Usage and Derivation](#). Valid values are from 0 to 27.
- *Data1, Data2 to DataN*: Provided data, considered as byte buffers
- *CEStatus*: Defined in section [CEStatus on page 180](#)
- *DecryptedData1, DecryptedData2 to DecryptedDataN*: Decrypted data, considered as byte buffers

15.3.10 CryptoStoreToFlash

Command `CryptoStoreToFlash(...)` makes the Crypto Engine store the data (Keys and Parameters) from RAM into flash memory.

Table 15-20: CryptoStoreToFlash Command

Byte	0	1
Data from Host	0x05	0x0A
Data to Host	Stat1	Stat2

Table 15-21: CryptoAesDecrypt Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	CEStatus

- *CEStatus*: Defined in section [CEStatus on page 180](#)

15.3.11 CryptoRestoreFromFlash

Command `CryptoRestoreFromFlash(...)` makes the Crypto Engine restore the data (Keys and Parameters) from flash memory into RAM.

Table 15-22: CryptoRestoreFromFlash Command

Byte	0	1
Data from Host	0x05	0x0B
Data to Host	Stat1	Stat2

Table 15-23: CryptoRestoreFromFlash Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	CEStatus

- *CEStatus*: Defined in section [CEStatus on page 180](#)

15.3.12 CryptoSetParam

Command `CryptoSetParam()` sets a specific Parameter into the Crypto Engine RAM.

Table 15-24: CryptoSetParam Command

Byte	0	1	2	3	4	5	6
Data from Host	0x05	0x0D	ParamID(7:0)	Data(31:24)	Data(23:16)	Data(15:8)	Data(7:0)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00

Table 15-25: CryptoSetParam Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	CEStatus

- *ParamID*: Parameter ID, 0 to 119
- *Data*: Parameter Data
- *CEStatus*: Defined in [CEStatus on page 180](#)

15.3.13 CryptoGetParam

Command `CryptoGetParam()` gets a specific Parameter from the Crypto Engine RAM.

Table 15-26: CryptoGetParam Command

Byte	0	1	2
Data from Host	0x05	0x0E	ParamID(7:0)
Data to Host	Stat1	Stat2	0x00

Table 15-27: CryptoGetParam Response

Byte	0	1	2	3	4	5
Data from Host	0x00	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	CEStatus	Data(31:24)	Data(23:16)	Data(15:8)	Data(7:0)

- *ParamID*: Parameter ID, values 0 to 119
- *Data*: Parameter Data
- *CEStatus*: Defined in [CEStatus on page 180](#)

15.3.14 CryptoCheckEncryptedFirmwareImage

Command `CryptoCheckEncryptedFirmwareImage()` adds a chunk to the encrypted FW image to be checked.

It must be called several times until the complete image is sent to the chip:

- It only takes 64 chunks of 32-bit data (overall 256 bytes + command and opcode) in one go, except for the last segment which may be shorter
- The first offset is 0
- The BUSY pin is released by the LR1120 when it is ready to accept another chunk of data

Once the firmware image has been sent to the chip, you can call the `CryptoCheckFirmwareImageResult()` command to check the result. These commands are useful to check the validity of a firmware before flashing it.

Table 15-28: CryptoCheckEncryptedFirmwareImage Command

Byte	0	1	2	3	4	5	6	7	8	9
Data from host	0x05	0x0F	Offset (31:24)	Offset (23:16)	Offset (15:8)	Offset (7:0)	Data1 (31:24)	Data1 (23:16)	Data1 (15:8)	Data1 (7:0)
Data to host	Stat1	Stat2	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00

Byte	10	11	...	4*N+5
Data from host	Data2 (31:24)	Data2 (23:16)	...	DataN (7:0)
Data to host	0x00	0x00	...	0x00

Offset: Must be a multiple of 4 (bytes)

Data: Length must be a multiple of 4. Max length = 256 bytes

Note: N = 64 for all chunks of data, except the last one which may be shorter

15.3.15 CryptoCheckEncryptedFirmwareImageResult

Command `CryptoCheckEncryptedFirmwareImageResult()` gets the result of the check.

Table 15-29: CryptoCheckEncryptedFirmwareImageResult Command

Byte	0	1
Data from Host	0x05	0x10
Data to Host	Stat1	Stat2

Table 15-30: CryptoCheckEncryptedFirmwareImageResult Response

Byte	0	1
Data from Host	0x00	0x00
Data to Host	Stat1	Result

- *Result:*
 - ♦ 1: Firmware verification success
 - ♦ 0: Firmware verification error

16. LR1120 Provisioning

16.1 Description

The LR1120 is pre-provisioned during Semtech's production test flow with two identifiers, which can be used to identify devices on LoRaWAN® networks. For more information, please refer to the LoRa Alliance® website: <https://LoRa-alliance.org/>

- The ChipEui number is globally unique, and identifies the device.
- The SemtechJoinEui is re-used on a set of Semtech devices.
- A unique Device Key, DKEY, is also pre-provisioned.
- With these numbers, a Device PIN is calculated upon execution of the `DeriveRootKeysAndGetPin(...)` command. This PIN is necessary to claim the device on the LoRa Cloud™ JoinServer services. For more information, please refer to the LoRa Cloud website: https://www.LoRacloud.com/portal/join_service.

All these unique identifiers are stored in the device's persistent memory. They are pre-configured by Semtech to ease the LoRaWAN implementation and access LoRa Cloud Join Server services, but can also be ignored by the user.

16.2 Provisioning Commands

16.2.1 GetChipEui

Command `GetChipEui(...)` reads the LR1120's pre-provisioned *ChipEui*. It is a globally-unique number, assigned by Semtech in production, using one of Semtech's IEEE assigned EUIs. In the standard use-case, the *ChipEui* is used to derive the two LoRaWAN root keys (*AppKey*, *NwkKey*), and should also be used as *DevEui* (in the LoRaWAN definition) to generate the Join Request (which itself transports the *DevEui*).

Table 16-1: GetChipEui Command

Byte	0	1
Data from Host	0x01	0x25
Data to Host	Stat1	Stat2

Table 16-2: GetChipEui Response

Byte	0	1	...	8
Data from Host	0x00	0x00	...	0x00
Data to Host	Stat1	ChipEui(63:56)	...	ChipEui(7:0)

- *ChipEui* is coded on 8 bytes, in big endian format.

16.2.2 GetSemtechJoinEui

Command `GetSemtechJoinEui( . . . )` reads LR1120's pre-programmed JoinEui, installed in production by Semtech.

The two LoRaWAN root keys (*AppKey*, *NwkKey*) in the device are derived from this JoinEui, amongst other numbers.

On top, in the standard use-case, the user should use `SemtechJoinEui` as the LoRaWAN JoinEui field to build the Join Request frame (which itself transports the JoinEui).

Table 16-3: GetSemtechJoinEui Command

Byte	0	1
Data from Host	0x01	0x26
Data to Host	Stat1	Stat2

Table 16-4: GetSemtechJoinEui Response

Byte	0	1	...	8
Data from Host	0x00	0x00	...	0x00
Data to Host	Stat1	SemtechJoinEui(63:56)		SemtechJoinEui(7:0)

SemtechJoinEui is coded on 8 bytes, in big endian format.

16.2.3 DeriveRootKeysAndGetPin

Command `DeriveRootKeysAndGetPin( . . . )` derives the *AppKey* and *NwkKey* root keys, and calculates the corresponding PIN, required to claim a device on the Semtech Join Server. It is a very versatile function with a standard use and a more advanced use, as described in the coming sections:

Three use-cases are possible:

- Standard: Uses the pre-provisioned ChipEui, DevEui and use the Semtech Join Server.
- Advanced: The DevEui and/or the JoinEui can be personalized, whilst still using the Join Server.
- Alternate: AppKey and NwkKey are forced by the user, and the Join Server can't be used.

16.2.3.1 Standard Use

Table 16-5: DeriveRootKeysAndGetPin Command (Standard)

Byte	0	1
Data from Host	0x01	0x27
Data to Host	Stat1	Stat2

Table 16-6: DeriveRootKeysAndGetPin Response (Standard)

Byte	0	1	2	3	4
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	PIN(31:24)	PIN(23:16)	PIN(15:8)	PIN(7:0)

PIN: Coded on 4 bytes, in big endian format

In the standard use-case, the ChipEui is used as LoRaWAN DevEui, SemtechJoinEui is used as LoRaWAN JoinEui, and therefore no specific action should be taken. The host should:

1. Call the `DeriveRootKeysAndGetPin()` command with no argument.
2. Read the SemtechJoinEui (`GetSemtechJoinEui()` command) and assign it to a JoinEui variable.
3. Read the ChipEui (`GetChipEui()` command) and assign it to a DevEui variable.
4. Execute the Join procedure using the elements that have just been read.
5. At the reception of a valid Join Answer, all lifetime and session keys can be derived according to the required LoRaWAN Standard.

The device must also be claimed on the Semtech Join Server for the Join Request to be accepted.

16.2.3.2 Advanced Use

The LR1120 supports a user-defined DevEui and / or a dedicated JoinEui, both different from the ChipEui and SemtechJoinEui onboard the LR1120. If the derivation scheme defined by the Semtech Join Server service is re-used, the function call to `DeriveRootKeysAndGetPin()` should be done as follows.

Table 16-7: DeriveRootKeysAndGetPin Command (Advanced)

Byte	0	1	2	3	4	5	6:9	10:17	18
Data from Host	0x01	0x27	DevEui (63:0)					JoinEui	RFU (0x00)
Data to Host	Stat1	Stat2	IrqStatus (31:24)	IrqStatus (23:16)	IrqStatus (15:8)	IrqStatus (7:0)	0x00	0x00	0x00

Table 16-8: DeriveRootKeysAndGetPin Response (Advanced)

Byte	0	1	2	3	4
Data from Host	0x00	0x00	0x00	0x00	0x00
Data to Host	Stat1	PIN(31:24)	PIN(23:16)	PIN(15:8)	PIN(7:0)

- *PIN*: Coded on 4 Bytes, in big endian format.

In the advanced use-case, a user-specific DevEui and / or JoinEui can be used. The host should:

1. Call the `DeriveRootKeysAndGetPin()` command with your own DevEui and/or JoinEui, get back the PIN.
2. Execute the Join procedure using DevEui and JoinEui (i.e., not ChipEui and SemtechJoinEui).
3. At the reception of a valid Join Answer, all lifetime and session keys can be derived according to the required LoRaWAN Standard.

The device must also be claimed for the Semtech Join Server for the Join Request to be accepted.

16.2.3.3 Alternate Use

The LR1120 can be used outside of the Join Server service and its built-in root key derivation schemes. In this scenario, the PIN concept becomes irrelevant, and the user can provision his own *NwkKey* and *AppKey*, but still leverage the Crypto Engine for any of the authentication, signature and encryption tasks, for improved security. Note that, in this scenario, the `DeriveRootKeysAndGetPin(...)` command MUST NOT be used, as it would overwrite the root keys personalized in the device by the user with the `CryptoSetKey(...)` command. The host should:

1. Use `CryptoSetKey(...)` to personalize NwkKey and AppKey (according to the LoRaWAN version of interest).
2. Execute the Join procedure using DevEui and JoinEui of the user's choosing (ChipEui and SemtechJoinEui may be re-used for that matter).
3. Get the *DevAddr* and derive the session keys with the acceptance of the Join Response.

16.3 Crypto Engine Use With LoRaWAN V1.1.x

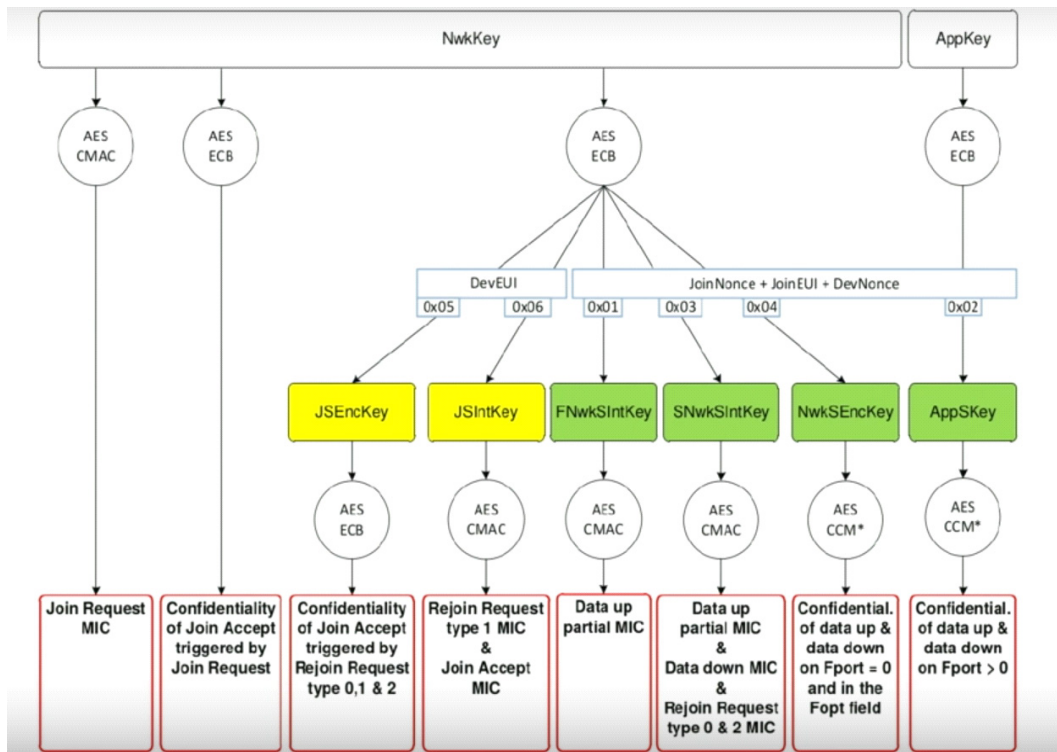


Figure 16-1: Key Derivation Scheme for LoRaWAN 1.1.x

All the keys required to sign, authenticate, or encrypt Join traffic, as well as the session keys, can be derived from the provided root keys (*AppKey*, *NwkKey*), following the derivation scheme described in Figure 16-1. The arguments of the `CryptoDeriveKey(...)` function should be computed by the host controller, and supplied to the function as its *Input[1:16]* argument, with appropriate padding bytes when applicable. For instance, when computing *FNwkSIntKey*, the command should be used as follows:

```
FNwkSIntKey = CryptoDeriveKey(NwkKey, 0x01 | JoinNonce | NetID | DevNonce | pad16)
```

Where *pad₁₆* is the required number of 0x00 bytes to expand to a 16-byte number.

16.4 Crypto Engine Use With LoRaWAN V1.0.x

The key derivation scheme is available from the LoRaWAN specification:

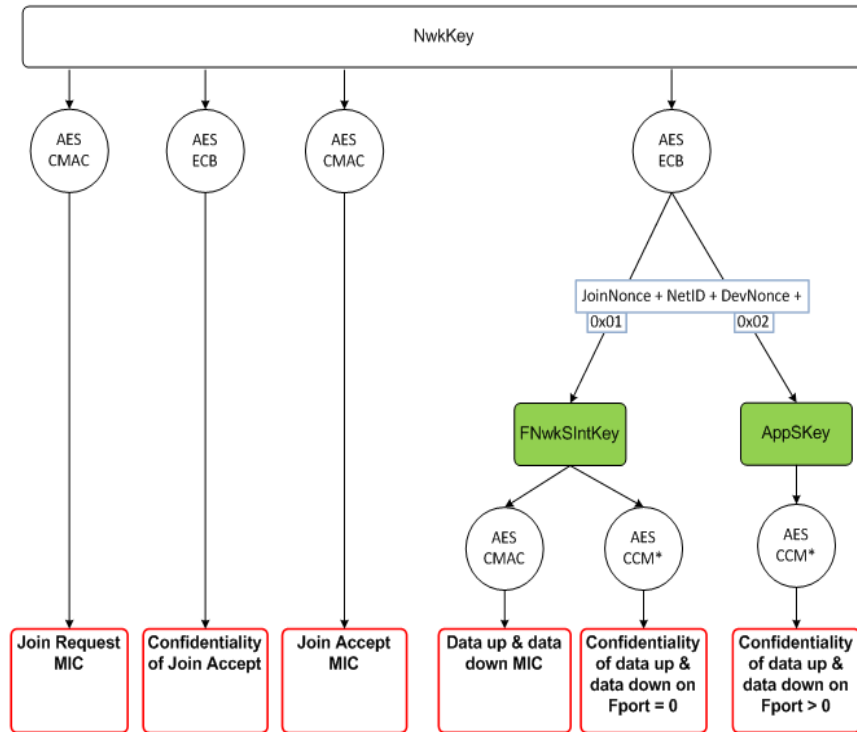


Figure 16-2: Key Derivation Scheme for LoRaWAN 1.0.x

Although the crypto engine of the LR1120 follows the key naming convention of the LoRaWAN 1.1.x Standard, it can be used for LoRaWAN 1.0.x. The correspondence table is delivered below:

Table 16-9: LoRaWAN 1.0.x vs. 1.1.x Security Correspondence Table

1.0.x	1.1.x
LoRaWAN_DEVICE_EUI	LoRaWAN_DEVICE_EUI
LoRaWAN_APP_EUI	LoRaWAN_JOIN_EUI
LoRaWAN_GEN_APP_KEY ¹	LoRaWAN_APP_KEY ¹
LoRaWAN_APP_KEY	LoRaWAN_NWK_KEY
LoRaWAN_NWK_S_KEY	LoRaWAN_F_NWK_S_INT_KEY
LoRaWAN_NWK_S_KEY	LoRaWAN_S_NWK_S_INT_KEY
LoRaWAN_NWK_S_KEY	LoRaWAN_NWK_S_ENC_KEY
LoRaWAN_APP_S_KEY	LoRaWAN_APP_S_KEY

1. The *GenAppKey* can be used as mother key to derive multicast-specific key material in LoRaWAN 1.0.x, whereas *AppKey* is used in LoRaWAN 1.1.x

17. Test Commands

Several LR1120 test commands allow an easy configuration of the device for regulatory ETSI or FCC compliance.

17.1 Regulatory Overview

This section only describes the RF modes necessary for ETSI and FCC regulatory testing. Please refer to the ETSI and FCC documents for a detailed test description and for the test limits indication.

17.1.1 ETSI

The EN 300 220 standards describe 4 test signals which the EUT (Equipment Under Test) must be able to transmit for CE certification. These test signals are listed in the table hereafter, with the operating mode correspondence for the LR1120.

Table 17-1: ETSI Test Signals

Test Signal	Description	LR1120 Operation
D-M1	Unmodulated carrier	TX CW mode (SetTxCw(...) command)
D-M2	Continuously modulated signal with the greatest occupied RF bandwidth	Continuous modulation (SetTxInfinitePreamble(...) command)
D-M2a	Same as D-M2 signal, but not continuous	RF packet transmission: LoRa SF12, BW500, 50% duty cycle
D-M3	Normal operating mode of the EUT in the application	Operation as in the application

The user should be able to modify the operating frequency, output power, and modulation parameters for the ETSI tests. The user should also be able to receive the incoming RF packets for any configuration (frequency, modulation parameters), and to determine a PER (Packet Error Rate) indication of the receive quality.

All this can be done using the regular LR1120 radio commands.

17.1.2 FCC

The FCC part 15.247 is applicable to frequency hopping and digitally modulated systems. For those tests, only an unmodulated carrier (TX CW) and a regular packet transmission are required.

The user should be able to modify the operating frequency, output power, and modulation parameters for the FCC tests. This can be done using the regular LR1120 radio commands.

17.2 Commands

17.2.1 SetTxCw

Command SetTxCw (...) sets the device in TX continuous wave mode (unmodulated carrier).

Table 17-2: SetTxCw Command

Byte	0	1
Data from Host	0x02	0x19
Data to Host	Stat1	Stat2

This command immediately sets the device in TX CW mode. Therefore, the operating frequency and the PA configuration commands (including the RF output power) have to be called prior to this command.

17.2.2 SetTxInfinitePreamble

Command SetTxInfinitePreamble(...) transmits an infinite preamble sequence.

Table 17-3: SetTxInfinitePreamble Command

Byte	0	1
Data from Host	0x02	0x1A
Data to Host	Stat1	Stat2

This command immediately starts transmission of the infinite preamble sequence. Therefore, the operating frequency, the PA configuration commands (including the RF output power) and the packet type have to be called prior to this command.

18. Known Limitations

Unexpectedly High Adjacent Channel Power

When the chip wakes up from sleep mode with retention, a parameter is not reconfigured properly. This misconfiguration can lead to an unexpectedly high adjacent channel power in all subsequent transmissions impacting the device regulatory compliance.

The issue appears only in LoRa modulation, for all bandwidths except 500kHz.

Affects LR1120 firmware version 0x0101.

Workaround

The workaround is to reset bit 30 in the register 0x00F30054 when the chip wakes up from sleep mode with retention.

This workaround does not work when LR11XX_RADIO_MODE_SLEEP is configured with lr11xx_radio_auto_tx_rx and the chip is set to Rx mode, because the workaround cannot be applied before the subsequent transmission (which is automatically launched by the chip after waking up from sleep mode with retention).

19. List Of Commands

For a general description of SPI Read and Write commands processing, see sections [3.2 Read Commands](#) and [3.1 Write Commands](#).

19.1 Register / Memory Access Operations

Table 19-1: Register / Memory Access Operations

Command	Opcode	Parameters	Description
WriteRegMem32	0x0105	Addr(4) Data(256)	Writes data at given register/memory address. Address must be 32-bit aligned and data length must be a multiple of 4
ReadRegMem32	0x0106	Addr(4) Len(1)	Reads data at given register/memory address. Address must be 32-bit aligned and data length < 65
WriteBuffer8	0x0109	Data(255)	Writes data to radio TX buffer (up to 255 bytes)
ReadBuffer8	0x010A	Offset(1) Len(1)	Reads data from radio RX buffer
ClearRxBuffer	0x010B	---	Clears all data from radio RX buffer
WriteRegMemMask32	0x010C	Addr(4) Mask(4) Data(4)	Reads-Modifies-Writes data at given register/memory address. Address must be 32-bit aligned.

19.2 System Configuration / Status Operations

Table 19-2: System Configuration / Status Operations (Sheet 1 of 2)

Command	Opcode	Parameters	Description
GetStatus	0x0100	---	Returns status of device
GetVersion	0x0101	---	Returns version of firmware
GetErrors	0x010D	---	Returns error status of device
ClearErrors	0x010E	---	Clears error bits in error status
Calibrate	0x010F	CalibParams(1)	Calibrates requested blocks according to parameter
SetRegMode	0x0110	RegMode(1)	Sets if DC-DC may be enabled for XOSC, FS, RX or TX mode, 0: LDO, 1: DC-DC
CalibImage	0x0111	Freq1(1) Freq2(1)	Launches an image calibration: Freq1, Freq2, in 4MHz steps
SetDioAsRfSwitch	0x0112	RfswEnable(1) RfswStbyCfg(1) RfswRxCfg(1) RfswTxCfg(1) RfswTxHPCfg(1) RfswTxHfCfg(1) RfswGnssCfg(1) RfswWifiCfg(1)	Sets up RFSWx output configurations for each radio mode
SetDioIrqParams	0x0113	Irq1ToEn(4) Irq2ToEn2(4)	Configures IRQs to output on IRQ pin(s)
ClearIrq	0x0114	IrqToClear(4)	Clears pending IRQs
ConfigLfClock	0x0116	LfClockSetup(1)	Configures the used LF clock
SetTcxoMode	0x0117	RegTcxoTune(1) Delay(3)	Configures device for a connected TCXO
Reboot	0x0118	StayInBootloader(1)	Reboots (SW reset) the device
GetVbat	0x0119	---	Gets VBAT voltage
GetTemp	0x011A	---	Gets temperature
SetSleep	0x011B	SleepConfig(1) SleepTime(4)	Sets chip into SLEEP mode
SetStandby	0x011C	StdbbyConfig(1)	0: RC, 1: XOSC Sets chip into RC or XOSC mode
SetFs	0x011D	---	Sets chip into FS mode
GetRandomNumber	0x0120	---	Gets a 32-bit random number
EraseInfoPage	0x0121	InfoPage(1)	Erases the Info Page data

Table 19-2: System Configuration / Status Operations (Sheet 2 of 2)

Command	Opcode	Parameters	Description
WriteInfoPage	0x0122	InfoPage(1) IndoWordAddr(2) dataN(4)	Writes a 32-bit block of the Info Page
ReadInfoPage	0x0123	dataN(1)	Reads a 32-bit block of the Info Page
GetChipEui	0x0125	---	Returns the 8-byte factory DeviceEui
GetSemtechJoinEui	0x0126	---	Returns the 8-byte factory JoinEui
DeriveRootKeysAndGetPin	0x0127	---	Returns the 4-byte PIN which can be used to register the device with LoRa Cloud Services
EnableSpiCrc	0x0128	Enable(1) CRC(1)	Enables / disables 8-bit CRC on SPI interface
DriveDiosInSleepMode	0x012A	Enable(1)	Enables / disables pull-up/down on configured RF switch and IRQ DIOs for sleep modes

19.3 Radio Configuration / Status Operations

Table 19-3: Radio Configuration / Status Operations (Sheet 1 of 3)

Command	Opcode	Parameters	Description
ResetStats	0x0200	---	Resets RX statistics
GetStats	0x0201	---	Gets RX statistics
GetPacketType	0x0202	---	Gets current radio protocol
GetRxBufferStatus	0x0203	---	Gets RS buffer status
GetPacketStatus	0x0204	---	Gets RX packet status
GetRssiInst	0x0205	---	Gets instantaneous RSSI
SetGfskSyncWord	0x0206	Syncword(8)	Sets the 64-bit (G)FSK syncword
SetLoRaPublicNetwork	0x0208	PublicNetwork(1)	Changes LoRa sync word for private or public network
SetRx	0x0209	RxTimeout(3)	Sets chip into RX mode
SetTx	0x020A	TxTimeout(3)	Sets chip into TX mode
SetRfFrequency	0x020B	RfFreq(4)	Sets PLL frequency
AutoTxRx	0x020C	Delay(3) IntermediaryMode(1) Timeout2(3)	Activates or deactivates the auto TX auto RX mode
SetCadParams	0x020D	SymbolNum(1) DetPeak(1) DetMin(1) CadExitMode(1) Timeout(3)	Configures LoRa CAD mode
SetPacketType	0x020E	PacketType(1)	Defines radio protocol
SetModulationParams	0x020F	Params ¹	Configures modulation parameters ¹
SetPacketParams	0x0210	Params ¹	Configures packet parameters ¹
SetTxParams	0x0211	TxPower(1) RampTime(1)	Sets TX power and ramp time
SetPacketAdrs	0x0212	NodeAddr(1) BroadcastAddr(1)	Sets the Node address and the broadcast address for the (G)FSK packets
SetRxTxFallbackMode	0x0213	Fall-back mode(1)	Defines into which mode the chip goes after a TX / RX done

Table 19-3: Radio Configuration / Status Operations (Sheet 2 of 3)

Command	Opcode	Parameters	Description
SetRxDutyCycle	0x0214	RxPeriod(3) SleepPeriod(3) Mode(1)	Starts RX Duty Cycle mode
SetPaConfig	0x0215	PaSel(1) RegPaSupply(1) PaDutyCycle(1) PaHpSel(1)	Configures PA settings
StopTimeoutOnPreamble	0x0217	StopOnPreamble(1)	Stops RX time-out on 0: Syncword/Header (default) or 1: preamble detection
SetCad	0x0218	---	Sets chip into RX CAD mode (LoRa)
SetTxCw	0x0219	---	Sets chip into TX mode with infinite carrier wave
SetTxInfinitePreamble	0x021A	---	Sets chip into TX mode with infinite preamble
SetLoRaSynchTimeout	0x021B	SymbolNum(1)	Configures LoRa modem to issue a time-out after exactly SymbolNum symbols
SetRangingAddr	0x021C	Addr(4), Checklength(1)	Sets the ranging id of the device and the number of bytes to be checked on a request
SetRangingReqAddr	0x021D	ReqAddr(4)	Sets the ranging id for the request
GetRangingResult	0x021E	Type(1)	Gets the ranging results (distance, RSSI)
SetRangingTxRxDelay	0x021F	Delay(4)	Sets the TxRx delay for the ranging
SetGfskCrcParams	0x0224	InitValue(4), Poly(4)	Sets parameters for CRC polynomial
SetGfskWhitParams	0x0225	Seed(2)	Sets parameters for whitening
SetRxBoosted	0x0227	RxBoosted(1)	Sets RX to boosted mode
SetRangingParameter	0x0228	SymbNb(1)	Sets the number of synchronization symbols
SetRssiCalibration	0x0229	TuneGxx(4) TuneG13hpx(3) TuneGxx(1) TuneG13hp7(1) GainOffset(2)	Sets gain offset for on-chip power estimation
SetLoRaSyncWord	0x022B	Syncword(1)	Sets LoRaSyncWord

Table 19-3: Radio Configuration / Status Operations (Sheet 3 of 3)

Command	Opcode	Parameters	Description
LrFhssBuildFrame	0x022C	HeaderCount(1) CR(1) ModType(1) Grid(1) Hopping(1) BW(1), HopSequence(2) DeviceOffset(1) Payload(n)	Encodes LR-FHSS frame, sets up hopping table
LrFhssSetSyncWord	0x022D	Syncword(4)	Sets LR-FHSS Syncword
ConfigBleBeacon	0x022E	Channel(1), Payload(N+3)	Configures radio to be Bluetooth Low Energy beaconing compatible
GetLoRaRxHeaderInfos	0x0230	---	Gets last packet header info
BleBeaconSend	0x0231	Channel(1), Payload(N+3)	Configures radio to be Bluetooth Low Energy beaconing compatible and sends packet

1. Parameters are modem/packet dependant.

19.4 Wi-Fi Configuration / Status Operations

Table 19-4: Wi-Fi Scanning Configuration / Status Operations

Command	Opcode	Parameters	Description
WifiScan	0x0300	SignalType(1) ChanMask(2) AcqMode(1) NbMaxRes(1) NbScanPerChan(1) Timeout(2) AbortOnTimeout(1)	Launches a Wi-Fi passive scan
WifiScanTimeLimit	0x0301	SignalType(1) ChanMask(2) AcqMode(1) NbMaxRes(1) ScanTimePerChannel(2) TimeoutPerScan(2)	Searches for Wi-Fi MAC address
WifiCountryCode	0x0302	ChanMask(2) NbMaxRes(1) NbScanPerChan(1) Timeout(2) AbortOnTimeout(1)	Extracts Country code from Beacon or Probe Response
WifiCountryCodeTimeLimit	0x0303	ChanMask(2) NbMaxRes(1) ScanTimePerChannel(2) TimeoutPerScan(2)	Searches for Wi-Fi MAC addresses during a configurable maximal amount of time
WifiGetNbResults	0x0305	---	Gets the number of passive scanning results
WifiReadResults	0x0306	Index(1), NbResults(1) Format(1)	Returns Wi-Fi results
WifiResetCumulTimings	0x0307	---	Initializes cumulative times per phases for power consumption measurements
WifiReadCumulTimings	0x0308	---	Returns cumulative time per phase for power consumption measurements
WifiGetNbCountryCodeResults	0x0309	---	Returns number of results after Country Code scanning execution
WifiReadCountryCodeResults	0x030A	Index(1) NbResults(1)	Reads byte stream containing defined number of Wi-Fi Passive Scanning Country Code results from a given index
WifiCfgTimestampAPphone	0x030B	Timestamp(31:0)	Configures time-stamp threshold used to discriminate mobile access point from gateways
WifiReadVersion	0x0320	---	Returns internal Wi-Fi firmware version major and minor numbers

19.5 GNSS Configuration / Status Operations

Table 19-5: GNSS Scanning Configuration / Status Operations (Sheet 1 of 2)

Command	Opcode	Parameters	Description
GnssSetConstellationToUse	0x0400	ConstellationBitMask(1)	Sets GNSS constellation to use for GNSS scan
GnssReadConstellationToUse	0x0401	-	Reads GNSS constellation to use for GNSS scan
GnssSetAlmanacUpdate	0x0402	AlmanacUpdateBitMask(1)	Configures constellation almanac information to be updated
GnssReadAlmanacUpdate	0x0403		Reads almanac update information
GnssReadVersion	0x0406		Reads internal GNSS firmware and almanac versions
GnssReadSupportedConstellations	0x0407	-	Reads supported GNSS constellations
GnssSetMode	0x0408	GnssMode(1)	Configures GNSS scan as single or dual capture
GnssAutonomous	0x0409	Time(4), EffortMode(1) ResultMask(1), NbSvMax(1)	Triggers GNSS autonomous scanning
GnssAssisted	0x040A	Time(4), EffortMode(1) ResultMask(1), NbSvMax(1)	Triggers GNSS assisted scanning
GnssScan	0x040B	EffortMode(1) Result Mask(1) NbSvMax(1)	Returns GNSS signals
GnssGetResultSize	0x040C	-	Returns the results payload size
GnssReadResults	0x040D	-	Returns the results payload byte stream
GnssAlmanacFullUpdate	0x040E	AlmanacFullUpdatePayload(2580)	Updates all the Almanac Data
GnssAlmanacFullUpdate	0x040E	AlmanacFullUpdatePayload(2580)	Updates all the Almanac Data
GnssSetAssistancePosition	0x0410	Latitude(2), Longitude(2)	Configures approx position for GNSS assisted mode
GnssReadAssistancePosition	0x0411	-	Reads the assistance position
GnssPushSolverMsg	0x0414	Payload(n)	Pushes messages from GNSS solver to LR1120
GnssPushDmMsg	0x0415	Payload(n)	Pushes messages from LoRaWAN network to LR1120
GnssGetContextStatus	0x0416	-	Reads the context status
GnssGetNbSvDetected	0x0417	-	Returns number of SV detected during last GNSS scan
GnssGetSvDetected	0x0418	-	Returns list of SV detected during the last GNSS scan, with their C/N0
GnssGetConsumption	0x0419	-	Returns radio capture and CPI processing duration of the last GNSS scan

Table 19-5: GNSS Scanning Configuration / Status Operations (Sheet 2 of 2)

Command	Opcode	Parameters	Description
GnssGetSvVisible	0x041F	Time(4), Latitude(2) Longitude(2) Constellation(1)	Returns number of visible SVs
GnssGetSvVisible	0x041F	Time(4), Latitude(2) Longitude(2) Constellation(1)	Returns number of visible SVs
GnssReadLastScanModeLaunched	0x0426	-	Returns the type of scan launched during last scan
GnssFetchTime	0x0432	EffortMode(1) Options(1)	Configures LR1120 to determine time by demodulating satellite signals
GnssReadTime	0x0434	-	Returns GPS time
GnssResetTime	0x0435	-	Resets time
GnssResetPosition	0x0437	-	Resets Assistance Position
GnssReadDemodStatus	0x0439	-	Returns scan execution information
GnssReadCumulTiming	0x044A	-	Returns scan duration
GnssSetTime	0x044B	GPS Time(4) Time Accuracy(2)	Allows Host MCU to set LR1120 GPS time
GnssReadDopplerSolverRes	0x044F	-	Returns Assistance Position
GnssReadDelayResetAP	0x0453	-	Reads delay after which LR1120 resets Assistance Position and GNSS scan switches to cold start scan
GnssAlmanacUpdateFromSat	0x0455	EffortMode(1), ConstellationBitMask(1)	Launches a GNSS scan to download Almanac parameters in one page by one constellation
GnssReadAlmanacStatus	0x0457	-	Returns how many and which satellites to update
GnssConfigAlmanacUpdatePeriod	0x0463	ConstellationBitMask(1) SvType(1), Period(2)	Configures the Almanac update period
GnssReadAlmanacUpdatePeriod	0x0464	ConstellationBitMask(1) SvType(1)	Reads the Almanac update period
GnssConfigDelayResetAP	0x0465	Delay(3)	Configures delay after which LR1120 resets Assistance Position and GNSS scan switches from assisted to cold start scan
GnssGetSvWarmStart	0x0466	ConstellationBitMask(1)	Returns the list of satellites for the next scan
GnssReadWNRollover	0x0467	-	Reads number of GPS time Week Number rollover
GnssReadWarmStartStatus	0x0469	ConstellationBitMask(1)	Reads number of visible satellites and time since last update of satellite list of this constellation
GnssWriteBitMaskSatActivated	0x0472	ConstellationBitMask(1) BitMaskActivated(4)	Configures the LR1120 to search for Almanacs for each satellite

19.6 CryptoElement Configuration / Status Operations

Table 19-6: CryptoElement Configuration / Status Operations

Command	Opcode	Parameters	Description
CryptoSetKey	0x0502	KeyID(1) Key(2)	Sets a Key identified by KeyID into Crypto Engine
CryptoDeriveKey	0x0503	SrcKeyID(1) DstKeyID(1) Input(2)	Derives and stores a key
CryptoProcessJoinAccept	0x0504	DecKeyID(1) VerKeyID(1) LoRaWANVer(1) Header(1 or 12) Data(16 or 32)	Processes a join accept message: decrypts full message (data+header) verifies MIC on the message, and if OK, provides decrypted message.
CryptoComputeAesCmac	0x0505	KeyID(1) Data(256)	Computes CMAC, returns MIC using specified Key.
CryptoVerifyAesCmac	0x0506	KeyID(1) ExpectedMIC(4) Data(256)	Verifies computed CMAC (compare calculated MIC with expected MIC)
CryptoAesEncrypt01	0x0507	KeyID(1) Data(256)	Encrypts data using specified Key
CryptoAesEncrypt	0x0508	KeyID(1) Data(256)	Encrypts data using specified Key
CryptoAesDecrypt	0x0509	KeyID(1) Data(256)	Decrypts data using specified Key
CryptoStoreToFlash	0x050A	---	Stores all Keys (and Parameters) to flash
CryptoRestoreFromFlash	0x050B	---	Restores all Keys (and Parameters) from flash
CryptoSetParam	0x050D	ParamID(1), Data(4)	Sets a parameter in RAM
CryptoGetParam	0x050E	ParamID(1)	Gets a parameter from RAM
CryptoCheckEncryptedFirmwareImage	0x050F	Offset(31:0) Data[1 to N]	Adds a chunk to the encrypted firmware image to be checked
CryptoCheckEncryptedFirmwareImage Result	0x0510	-	Gets the result of the check

19.7 Bootloader Commands

Table 19-7: Bootloader Commands¹

Command	Opcode	Parameters	Description
EraseFlash	0x8000	---	Erases content of program memory
WriteFlashEncrypted	0x8003	Offset(4) Data(128)	Writes a new encrypted firmware image
Reboot	0x8005	StayInBootloader(1)	Reboots (SW reset) device from bootloader mode
GetPIN	0x800B	-	Reads the 4-byte PIN of the device (little endian)
GetChipEui	0x800C	-	Reads the 8-byte factory ChipEui of the device (big endian)
GetJoinEui	0x800D	-	Reads the 8-byte factory JoinEui of the device (big endian)

1. See AN1200.57 LR1110 Program Memory Update for usage.

20. Revision History

The following table details the versions of the User Manual documentation issued, and the corresponding LR1120 versions supported (Use Case and FW Major.FW Minor), as returned by command `GetVersion(...)`.

Table 20-1: Revision History (Sheet 1 of 3)

User Manual Version	ECO	Date	Applicable to	Changes
1.0	061060	Apr 2022	Use Case: 02 FW Version: 01 or later	First Release
1.1	062816	Jul 2022	Use Case: 02 FW Version: 01 or later	- Added clock source information to Section 6.2.1 32.768kHz Clock - Added ref to AN1200.59 in Section 6.2.3 - Added GNSS information to Section 11.2 - Added Sections 11.3.4.1 & 11.3.4.2
1.2	066719	May 2023	Use Case: 02 FW Version: 01 or later	Added: - GnssPushDmMsg - GnssPushSolverMsg - EraseInfoPage, ReadInfoPage, WriteInfoPage - Section 2.5 InfoPage - In Sleep mode section, “all DIOs are in Hi-Z mode” - GNSS information to Section 11.1 and 11.2 about GNSS Dual Constellation and GNSS Advanced scan mode - References to AN1200.74 LoRa Edge Clock Requirements and AN1200.59 - Section 11.2 info about external LNA - Return code CMD_PERR for ReadInfoPage command Modified: - GnssReadAlmanacUpdate description - Section 6.2.1 32.768kHz Clock - SetRssiCalibration parameters are in dB not dBm Note 1 in List of Bootloader commands - Note 1 in List of Bootloader commands - SetModulationParams with Note 1 - SetGfskSyncWord: explanation for syncword not bit-aligned - Section 9.1 and 9.4.1 improved - Tables 9-1, 9-2, 9-3, & 9-5 - Table 2-15 Mode Transitions and Timings

Table 20-1: Revision History (Sheet 2 of 3)

User Manual Version	ECO	Date	Applicable to	Changes
1.3	067386	June 2023	Use Case: 02 FW Version: 01.02 or later	Added • Section 13. Ranging by Round Trip Time Of Flight (RTToF) • Section 14. Bluetooth® Low Energy Beaconsing Compatibility • Section 8.8 Sigfox Modulation • Section 6.3.2 TCXO supply voltage precision • CryptoCheckEncryptedFirmwareImage() • CryptoCheckEncryptedFirmwareImageResult() • Information to Section 8.10 RSSI Functionality • Bit 27 to Table 4-2: IrqToEnable Interruption Mapping Modified • Section 7.1 phrase about SX126x compatibility • GFSK SetModulationParams BitRate & RC definitions • LoRa SetModulationParams long interleaver description • GetPacketStatus SnrPkt definition • Table 2-8: Sleep Mode Summary • Figure 9-14: RF Switch, Double PA Operation • Section 12.3.11 ErrorCode definition • Section 8.1.1Section 8.1.2 definitions

Table 20-1: Revision History (Sheet 3 of 3)

User Manual Version	ECO	Date	Applicable to	Changes
2.0	069612	Dec 2023	Use Case: 02 FW Version 02.01	Modified: Section 2.3.1 Added shipping FW version Section 6.2.1 External 32.768kHz signal amplitude on DIO11 Section 8.5.2 SyncWordLen alignment Section 9.5.2 10us ramp time limitation Section 12.1 Info pertaining to FW 02.01 Section 12.2 32.768kHz clock source no longer mandatory Section 12.3.1 GnssAutonomous() and GnssAssisted () no longer supported Section 12.3.2 Fixed delay change Section 12.3.5 Nav message format change Section 12.3.6 & Section 12.3.7 Replaced by GnssScan Section 12.3.9 Integrated 2D solving Section 12.6 Moved the Almanac commands to this section Added: Section 5.1 & Section 5.2 Section 8.4.5 Special Settings for BRf= 0.6kbps and 1.2kbps Section 12.3.8 GnssScan and the following GNSS commands: GnssConfigDelayResetAP, GnssReadLastScanModeLaunched, GnssFetchTime, GnssReadTime, GnssResetTime, GnssResetPosition, GnssReadWNRollover, GnssReadDemodStatus, GnssReadCumulTiming, GnssSetTime, GnssReadDopplerSolverRes, GnssReadDelayResetAP, GnssAlmanacUpdateFromSat, GnssReadWarmStartStatus, GnssReadAlmanacStatus, GnssConfigAlmanacUpdatePeriod, GnssReadAlmanacUpdatePeriod, GnssGetSvWarmStart, GnssWriteBitMaskSatActivated

Glossary

List of Acronyms and their Meaning (Sheet 1 of 3)

Acronym	Meaning
ADC	Analog-to-Digital Converter
AP	Wi-Fi Access Point
API	Application Programming Interface
β	Modulation Index
BR	Bit Rate
BT	Bandwidth-Time bit period product
BW	BandWidth
BWF	BandWidth of the (G)FSK modem
BWL	BandWidth of the LoRa® Modem
CAD	Channel Activity Detection
CPHA	Clock Phase
CR	Coding Rate
CRC	Cyclical Redundancy Check
CW	Continuous Wave
DC-DC	Direct Current to Direct Current Converter
DIO	Digital Input / Output
DS	Distribution System
DSB	Double Side Band
DSP	Digital Signal Processing
ECO	Engineering Change Order
Fdev	Frequency Deviation
FIFO	First In First Out
FS	Frequency Synthesis
FSK	Frequency Shift Keying
IF	Intermediate Frequencies
IRQ	Interrupt Request
LDO	Low-Dropout
LDRO	Low Data Rate Optimization
LFSR	Linear-Feedback Shift Register

List of Acronyms and their Meaning (Sheet 2 of 3)

Acronym	Meaning
LNA	Low-Noise Amplifier
LoRa®	Long Range Communication the LoRa® Mark is a registered trademark of the Semtech Corporation
LR-FHSS	Long Range Frequency Hopping Spread Spectrum
LSB	Least Significant Bit
MAC	Wi-Fi Media Access Control
MIC	Message Integrity Code
MISO	Master Input Slave Output
MOSI	Master Output Slave Input
MSB	Most Significant Bit
MSK	Minimum-Shift Keying
NOP	No Operation (0x00)
NRZ	Non-Return-to-Zero
NSS	Slave Select active low
OCp	Over Current Protection
PA	Power Amplifier
PER	Packet Error Rate
PHY	Physical Layer
PID	Product Identification
PLCP	Wi-Fi Physical Layer Conformance Procedure
PLL	Phase-Locked Loop
POR	Power On Reset
PSDU	Wi-Fi PLCP Service Data Unit
RFO	Radio Frequency Output
RFU	Reserved for Future Use
RTC	Real-Time Clock
SCK	Serial Clock
SF	Spreading Factor
SN	Sequence Number
SNR	Signal to Noise Ratio
SPI	Serial Peripheral Interface
STA	Wi-Fi Client Station

List of Acronyms and their Meaning (Sheet 3 of 3)

Acronym	Meaning
STDBY	Standby
TCXO	Temperature-Compensated Crystal Oscillator
XOSC	Crystal Oscillator



Important Notice

Information relating to this product and the application or design described herein is believed to be reliable, however such information is provided as a guide only and Semtech assumes no liability for any errors in this document, or for the application or design described herein.

Semtech reserves the right to make changes to the product or this document at any time without notice. Buyers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. Semtech warrants performance of its products to the specifications applicable at the time of sale, and all sales are made in accordance with Semtech's standard terms and conditions of sale.

SEMTECH PRODUCTS ARE NOT DESIGNED, INTENDED, AUTHORIZED OR WARRANTED TO BE SUITABLE FOR USE IN LIFE-SUPPORT APPLICATIONS, DEVICES OR SYSTEMS, OR IN NUCLEAR APPLICATIONS IN WHICH THE FAILURE COULD BE REASONABLY EXPECTED TO RESULT IN PERSONAL INJURY, LOSS OF LIFE OR SEVERE PROPERTY OR ENVIRONMENTAL DAMAGE. INCLUSION OF SEMTECH PRODUCTS IN SUCH APPLICATIONS IS UNDERSTOOD TO BE

UNDERTAKEN SOLELY AT THE CUSTOMER'S OWN RISK. Should a customer purchase or use Semtech products for any such unauthorized application, the customer shall indemnify and hold Semtech and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs damages and attorney fees which could arise.

The Semtech name and logo are registered trademarks of the Semtech Corporation. All other trademarks and trade names mentioned may be marks and names of Semtech or their respective companies. Semtech reserves the right to make changes to, or discontinue any products described in this document without further notice. Semtech makes no warranty, representation or guarantee, express or implied, regarding the suitability of its products for any particular purpose. All rights reserved.

© Semtech 2023